

FUJINET

VIDEO COMPUTER SYSTEM



PROGRAMMER'S HANDBOOK

MODEL CX2600 · FUJINET CARTRIDGE



A FujiNet Project Publication

FUJINET PROJECT · fujinet.online · github.com/FujiNetWIFI

FujiNet Video Computer System Programmer's Handbook. First edition, 2026.

Typeset after the 1977 *Video Computer System Owner's Manual* as a tribute. FujiNet is an independent project and is not affiliated with, endorsed by, or sponsored by Atari. "Atari" and "Video Computer System" are trademarks of their owner and are used here to identify the console this handbook is about.

Every address, register, command and code fragment in this handbook is transcribed from the live project sources and verified in the FujiNet MAME cartridge model against a running FujiNet adapter; the sources and their commits are listed in the Parts List at the back.

Text and diagrams © 2026 the FujiNet Project, CC BY-SA 4.0. Program listings carry the licence of their repositories. batari Basic is © its authors and is distributed under its own licence.

CONTENTS

1 Unpack the Cartridge	1
What you should have	1
How this handbook is organised	1
2 Install the Tools	2
Macroassembler AS	2
batari Basic and dasm	2
MAME with the FujiNet cartridge	2
fujinet-pc	3
Running a program	3
3 The Cartridge Window	4
Two halves	4
The image	4
Reset survival	5
The static check	5
4 How the Mailbox Works	7
A connector with no write line	7
Arm, then commit	7
The registers	7
The TX stream	8
Launching, and the sequence number	8
The status cells	9
The reply window and its slices	10
The arming gate	10
The one rule that will bite	10
Why the scan can be a proof	11
The recipe	11
5 The Text Display	12
Six planes	12
Composing a row	12
The font	13
The kernel	14
The pixel that cannot be drawn	15
What the cartridge can draw for you	15
batari Basic	16
6 Living in 128 Bytes	17
Nothing is copied	17
The cartridge holds the strings	17
The cartridge does the loops	17
The zero page, three ways	18
The stack pointer is a register	18
The swap stub	18
A byte costs a bank	19
What survives what	19
7 The 6502 Library	20
The transport	20
Text	21
Payloads and the path buffers	21
Boot	22
Input	22
The display	22
The dasm twin	22
8 batari Basic on the Mailbox	23

The 2K rule	23
The names	23
Where the variables go	24
The helpers	24
Showing text	25
The budget	26
Building	26
9 First Contact	28
Build and run	28
The reply	29
The assembly program	29
The BASIC program	30
10 The Network Device	32
The lifecycle	32
netget in assembly	33
netget in BASIC	35
JSON	36
11 Directories, Mount and Boot	37
Hosts and directories	37
Mounting and booting	40
The mappers	43
12 Keys, Clock and Utilities	45
Appkeys	45
The clock	47
Entropy, hashing, encoding, QR codes	48
13 Banks and the Tail	50
A switch is a jump	50
The cold stub	50
Every bank carries its own library	51
Building a banked image	51
batari Basic	52
14 The Playfield Board	53
Two boards in forty bits	53
The tables	53
Composing them	54
The kernel	54
Four seats	55
Trying it without a network	56
15 Command Reference: The Fuji Device	57
WiFi and the adapter	57
Hosts, slots and mounting	60
Directories	64
Files, prefixes and paths	66
Appkeys	66
Utilities	68
Answered with a NAK on this build	69
16 Command Reference: The Network Device	70
The lifecycle five	70
Structured channels, seek and line discipline	73
Filesystems, sockets and credentials	75
Answered with a NAK on this build	75
17 Command Reference: Disk, Printer, Clock, Modem	76
Disk — devices \$31—\$38	76
Printer — device \$40	77
Clock — device \$45	77

Modem — device \$50	78
18 Command Reference: The Cartridge Itself	80
Registers	80
One-shot operations	80
The path buffers	81
The blit port	81
The card transform	83
The boot doorbells	83
19 Battleship	84
The screen	84
Controls	85
Seven banks and a tail	86
Where to carry on	86
The zero page	87
The transport	87
The URL	89
The state record	89
The game bank	91
Every frame is 262 lines	92
The composer	92
The lobby, the name, the menu, the placement	93
Sound	94
How it is checked	95
Traps paid for here	95
20 CONFIG	97
Seven screens, seven banks	97
Almost nothing lives in console RAM	98
Two blits it could not do without	98
Building it	99
21 Things That Cost Real Time	100
The milestones	100
Designing the mailbox	100
The display	101
The first programs	101
The MAME bench	102
This handbook's own	102
22 Trouble Shooting Checklist	103
Building	103
The mailbox	103
The display	104
The network	104
Booting	105
The MAME bench	105
23 Parts List	106
The cartridge	106
The adapter	106
The programs	106
The tools	106
Further reading	106
A Mailbox Quick Reference	108
The window	108
Status cells	108
Registers and one-shots	108
Path operations and blits	109
The transaction	109
The library's zero page	109

B Error and Status Codes	110
C Program Listings	111
D Battleship Listing	145

The FujiNet cartridge for the Atari Video Computer System is a network adapter that plugs into the Game Program slot. Inside the shell are two computers. An RP2040 microcontroller sits on the cartridge edge and does what a cartridge does: it answers the console's every fetch within the 838 nanoseconds the 6507 allows, and serves a 4K window of memory. Behind it, over a USB cable, an ESP32-S3 running the FujiNet firmware carries the WiFi radio, the SD card, the network protocols and everything else a FujiNet does on every other computer it has been built for.

The console never sees the ESP32-S3. It sees sixteen pages of cartridge memory, and this handbook is about what is in them.

What you should have

- A FujiNet cartridge, or the FujiNet cartridge model grafted into MAME, which is what every program in this handbook was verified on.
- A FujiNet adapter to talk to: an ESP32-S3 running the `fujiversal-rs232` firmware, or `fujinet-pc-rs232` on the same machine as MAME, listening on bus-over-IP.
- Macroassembler AS, for the assembly language programs.
- batari Basic and dasm, for the BASIC programs.
- This handbook.

NOTE: There is no reset line on the Video Computer System's cartridge connector, and the console has no interrupts. Both facts shape everything that follows.

How this handbook is organised

Sections 3 through 8 are the machine: the window, the mailbox, the display, the memory, and the two languages. Sections 9 through 14 are programs, each in 6502 assembly and in batari Basic. Sections 15 through 18 are the command reference: every command the adapter answers, and every operation the cartridge itself performs. Section 19 walks through Battleship, a complete networked game, bank by bank. The rest is what went wrong on the way, and how to tell when it is going wrong for you.

Everything in this handbook was built and run on a Linux machine with the tools below. Nothing here needs a real cartridge: the FujiNet cartridge model in MAME is bit-faithful to the RP2040 firmware's bus behaviour, and it talks to a real FujiNet, `fujinet-pc`, over a socket. A program that works here works on the cartridge, with the one exception the handbook flags where it matters: emulation answers inside the store, and hardware answers a little later.

Macroassembler AS

The assembly programs are written for Macroassembler AS (`asl` and `p2bin`), the assembler every cartridge bring-up in the FujiNet family shares. It assembles 6502 code, produces a listing, and `p2bin` turns its output into a raw image over an address range:

```
asl -q -L -i ../common netget.asm
p2bin netget.p build/netget.bin -r '$1000-$1FFF' -l 0
```

The handbook's assembly sources are in `listings/asm/`, the library they include in `listings/common/`, and `listings/asm/build.sh` runs the whole flow for every program: the equate check, the assembler, the image, the claim, and the static check described in Section 3.

batari Basic and dasm

batari Basic compiles BASIC to 6502 assembly and hands it to `dasm`. Install batari Basic from its repository, and `dasm` from github.com/dasm-assembler/dasm (any 2.20 release; make and copy `bin/dasm` somewhere on your PATH). batari Basic finds its own files through the `BB` environment variable, and its native build script is `2600basic.native.sh`:

```
export BB=$HOME/Workspace/batari-Basic
$BB/2600basic.native.sh hello.bas
```

The handbook's BASIC sources are in `listings/bas/`, and `listings/bas/build.sh` does the extra work a FujiNet client needs, which Section 8 explains: it prepends the constants, compiles, appends the fixed half of the cartridge image, and runs the static check.

MAME with the FujiNet cartridge

The cartridge bring-up carries a MAME cartridge device that is the RP2040 firmware's own `fujimail.c` and `vcs_render.c`, compiled into MAME. Graft it into a MAME source tree and build:

```
~/Workspace/fn-2600/pico/atari-2600/emu/apply.sh ~/Workspace/mame
make -C ~/Workspace/mame -j$(nproc) NOWERROR=1 REGENIE=1
```

The device is the `fujinet` cartridge slot option. It reaches the adapter through bus-over-IP, on `127.0.0.1:9995` unless `FUJINET_TCP` says otherwise.

IMPORTANT: MAME must run from its own source tree, or its `-autoboot_script` option is silently ignored. Where there is no display, `SDL_VIDEODRIVER=dummy` must be set, or MAME dies before it reads `-video none`. The handbook's `emu/run.sh` does both.

fujinet-pc

`fujinet-pc-rs232` is the FujiNet firmware built to run on a PC. Its bus-over-IP listener takes one client, so a MAME left running starves the next one and the symptom is a hang, not an error. `emu/run.sh` kills any stray MAME before it starts another.

```
cd ~/Workspace/fujinet-pc-rs232/build/dist
./run-fujinet
```

Its `sd/` directory is the adapter's SD card, host slot 0 in the programs here, and `SD/FujiNet/` holds the appkeys.

Running a program

```
emu/run.sh listings/bas/build/hello.bin      # in a window
emu/run.sh listings/bas/build/hello.bin frames # headless, with a harness
```

The harnesses in `emu/` are how every program in this handbook was checked, and they are described with the programs. Three are worth knowing from the start: `probe.lua` says whether a cartridge is answering and honoured the claim; `txn.lua` reports the last transaction's status cells and the reply; `frames.lua` taps `VSYNC` and reports any frame that is not 262 lines, which is the only honest test of a display kernel.

The 6507 has thirteen address lines. Everything above $\$0FFF$ selects the cartridge, and since A13 to A15 do not exist, $\$F000$ and $\$1000$ are the same place. The cartridge has sixteen pages of 256 bytes, total, for everything: the program, the text it shows, the replies it reads, and the two pages it writes into. This section is the map.

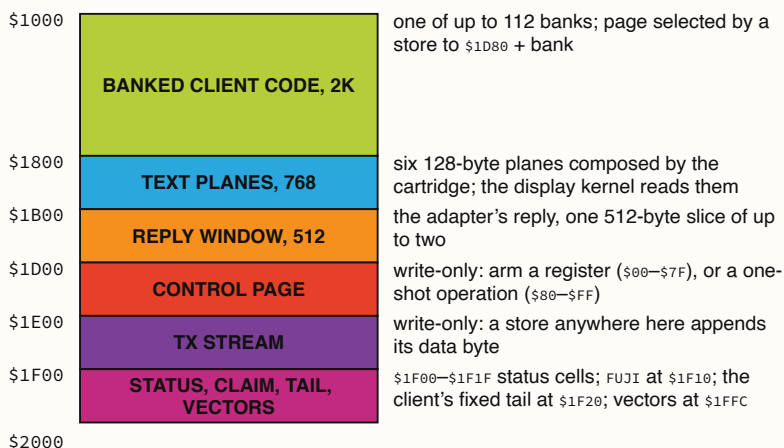


Figure 1. The cartridge window. The low 2K is banked; the high 2K is the mailbox in its entirety.

Two halves

The low half, $\$1000$ – $\$17FF$, is the program. It is a 2K bank, and the cartridge can serve any one of up to 112 of them there; a store to $\$1D80$ plus the bank number switches, and the switch is complete before the console's next fetch. A single-bank program simply never switches.

The high half, $\$1800$ – $\$1FFF$, belongs to the mailbox. Nothing a program stores in that range of its image is ever served, except the last 256 bytes: the status page, where the cartridge paints its cells in the low part and serves the program's own bytes in the rest. That is where the claim, the fixed tail and the vectors live, and it is the one region every bank sees at the same address.

The image

A client image is N banks of 2K followed by the 2K fixed half, so its size is $(N+1) \times 2048$ bytes. MAME's cartridge loader accepts only certain sizes, so N is 1, 3, 7 or 15: a 4K, 8K, 16K or 32K image. A program that needs five banks pads to seven.

The fixed half is the last 2K of the file whatever the size, and only its top 256 bytes matter. Four of those are the claim:

IMPORTANT: An image carrying the four bytes `FUJI` at `$1F10` promises that it is a FujiNet client, and the cartridge keeps the mailbox alive after it boots. An image without them is a game, and the mailbox goes dead for the session. The build scripts stamp the claim at file offset (size – 2048) + `$710`.

Offset in the fixed half	What is there
<code>\$700–\$70F</code>	painted by the cartridge: <code>ACKSEQ</code> , <code>STATUS</code> , <code>ERR</code> and the rest (Section 4)
<code>\$710–\$713</code>	the claim, <code>FUJI</code>
<code>\$714–\$716</code>	the header: bank count, cell height, layout revision
<code>\$717–\$719</code>	painted by the cartridge: the path buffer length and the blit generation
<code>\$720–\$7FB</code>	the fixed tail: 220 bytes of the client's code that every bank must reach at the same address — the cold stub, the bank trampoline
<code>\$7FC–\$7FF</code>	the <code>RESET</code> and <code>BRK</code> vectors

Reset survival

The cartridge port has no reset line, and the console's `RESET` switch is a bit in a `RIOT` register that the cartridge never sees. A console restart therefore begins with whatever bank was last selected still mapped, and RAM as it was. Two rules follow, and every client in this handbook obeys them:

- The `RESET` vector points into the fixed half, at a cold stub that reselects bank 0 before it jumps to the program. A vector into a bank would reset into the wrong code from any other bank.
- The next sequence number comes from the cartridge, never from a counter in RAM. Section 4 explains why.

The static check

Because the 6507 has no interrupts, every access a running program makes comes from its own instruction stream, and a scan of the image can prove things an emulator can only sample. `tools/checkrom.py` in the cartridge tree fails the build when:

- the image is not $(N+1) \times 2048$ bytes, or not a size MAME will load;
- the claim is missing;
- the `RESET` vector does not point into the fixed half for a banked image, or anywhere in the window for a flat one;
- any of `INC`, `DEC`, `ASL`, `LSR`, `ROL`, `ROR` with an absolute or absolute,X operand reaches `$1D00–$1EFF`;
- an indirect `JMP` has a vector on a page boundary (the 6502's own bug);
- an indirect store, `STA (zp),Y` or `STA (zp,X)`, appears anywhere at all, since the scan cannot know what a zero-page pointer holds and an indirect store performs a read at the address it writes.

Every bank is scanned, because every bank is entered at `$1000` and every bank is code.

NOTE: batari Basic's own kernel contains two indirect stores, aimed at RAM and the TIA. The BASIC build script vouches for those two by address, below the program's own code, and holds the program to the rule.

The mailbox is the cartridge's half of a conversation. The console writes a request into it; the cartridge carries the request over USB to the adapter, waits for the answer, and paints the answer into memory the console can read. Every FujiNet cartridge in the family works this way. What makes the Video Computer System's different is the connector.

A connector with no write line

The cartridge edge carries thirteen address lines, eight data lines, +5V and ground. There is no read/write line, no clock, and no chip select beyond A12. A cartridge sees an address and nothing else.

And yet the 6507 does write. During a store it drives the data bus, and the cartridge can see that bus. So the cartridge declares two pages write-only, never drives them, and recovers the stored byte by watching the address settle and keeping the second-to-last sample of the data lines:

```
while (ADDR_IN == addr) { data_prev = data; data = DATA_IN; }
append(data_prev);
```

This is not new. It is how every PlusROM game talks to its cartridge, and how every Superchip cartridge implements its 128 bytes of RAM. The console side of it is simply a store: `STA $1E00` appends a byte to the request, and the cartridge's bus loop is fast enough — 250 MHz against an 838-nanosecond bus cycle — to watch it happen.

IMPORTANT: A read of a write-only page is bit-identical to a write of it. A stray `LDA $1D10` decodes as a register access carrying whatever the floating bus held. Nothing on the console can tell the cartridge otherwise, so the protocol is shaped to survive it.

Arm, then commit

Because a stray read looks like a write, a register is never written in one store. It takes two:

```
lda    #value
sta    FNRSEL+n    ; arm register n; this store's data is ignored
sta    FNCMT       ; commit: register n = value, and disarm
```

An arm carries no value. A commit with nothing armed is discarded. One stray access therefore changes nothing: it either arms a register that is never committed, or commits a value into nothing. On the Channel F the same pair is a matter of protocol shape; here it is the defence. The library's `FNRW` routine is exactly those two stores, and the whole transport is built on it.

The registers

The arm addresses are `$1D00` plus the register number. Only the first `$80` bytes of the control page arm registers; the rest are one-shot operations, which Section 18 lists.

Reg	Name	Meaning
\$00	DEVICE	the FujiBus device id: \$70 for the FUJI device, \$71 for N1; \$45 for the clock
\$01	CMD	the command byte for that device
\$02	NPARAM	how many parameters the TX stream begins with
\$05	DATA_RST	any value: rewind the TX stream's write pointer
\$06	RXSLICE	which 512-byte slice of the reply the window shows
\$10	SEQ	nonzero and not the current ACKSEQ: launch the transaction
\$11	BOOTLOCK	\$B5: arm the ROM swap, once an image is staged
\$12, \$13	BOOTSEL	\$B5 then \$4A, as consecutive writes: reboot the cartridge's RP2040 into its USB bootloader

Register numbers \$00 to \$13 are the same on every cartridge in the family, so the firmware's `fujimail.c` compiles for this console byte-identically.

The TX stream

Parameters and payload go through the TX page. A store anywhere in \$1E00–\$1EFF appends its data byte; the address low byte does not matter, which is what lets `STA $1E00,X` work for any X. The stream has a fixed shape:

NPARAM ×	size 1 2 4	value, LE	... then the raw payload
----------	------------	-----------	--------------------------

Each parameter is a size byte, then that many value bytes little-endian. After the last parameter comes the payload, as raw bytes. The stream holds up to 320 bytes, which is more than any command needs; the two 256-byte path payloads are the largest.

NOTE: SET_DIRECTORY_POSITION takes one parameter of two bytes, not two parameters of one byte, and getting that wrong produces a NAK with nothing to point at. The size byte is not decoration: it is how the adapter knows the shape.

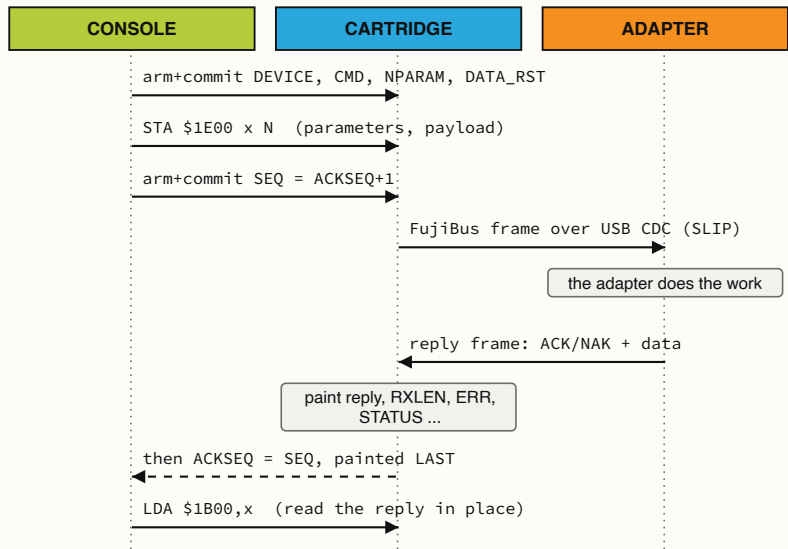
Launching, and the sequence number

A transaction runs when the SEQ register is committed with a value that is nonzero and different from the cartridge's current ACKSEQ. The value to use is always `ACKSEQ + 1`, wrapping from 255 to 1; zero is reserved as “never used”.

CAUTION: Derive the next sequence number from the cartridge's own ACKSEQ cell at \$1F00, never from a counter in RAM. A console RESET restarts the program and re-zeroes its variables but does not reset the cartridge. A program-local counter would restart at 1, collide with a sequence already answered, and every later transaction would be dropped in silence. Every port in this family learned this the hard way.

While the transaction runs, the cartridge frames the request as a FujiBus packet, sends it over USB to the adapter, and waits up to five seconds for the

reply (sixty for MOUNT_IMAGE and COPY_FILE, which take as long as the network does). Then it paints, in this order: the reply into the window, the reply command (\$06 ACK or \$15 NAK), the error code, the reply length, the link status, and last of all ACKSEQ. That single store is the whole interlock: by the time the console can see ACKSEQ match, everything behind it is already in place.



The console's side of the wait is a loop comparing \$1F00 with the sequence it committed. In emulation the whole round trip happens inside the commit store, so the first comparison matches; on the cartridge it takes as long as the network does, and a program that wants to keep a picture up draws frames while it waits.

The status cells

The cartridge paints these at the bottom of the status page. All of them are plain reads.

Cell	Name	Meaning
\$1F00	ACKSEQ	echoes SEQ when the reply is ready
\$1F01	STATUS	bit 0: the USB link to the adapter is up; bit 1: busy
\$1F02	ERR	the transport's verdict: 0 ok, 1 no link, 2 timeout, 3 bad frame, 4 too big
\$1F03	REPLY_CMD	\$06 if the adapter said ACK, \$15 for NAK
\$1F04, \$1F05	RXLEN	the reply's length, little-endian
\$1F06	BOOT_STATE	0 idle, 1 transferring, 2 ready, \$80 failed
\$1F07	BOOT_PCT	0–100 while an image is arriving
\$1F08	BOOT_ERR	1 too big, 2 truncated, 3 no mapper, 4 store busy
\$1F09, \$1F0A	MAGIC	F, N: a cartridge is answering
\$1F0B	PROTO_VER	2

\$1F0C	SLICE_ECHO	the slice number, painted last after a slice change
\$1F0D	TEXTGEN	the last text row rendered, plus a bit that toggles each time
\$1F0E	BANK	the bank currently served
\$1F0F	FLAGS	bit 1: the claim was honoured
\$1F17, \$1F18	PATHLEN	how many bytes the active path buffer holds
\$1F19	BLITGEN	bumped after every blit has landed on the planes

A NAK is not a transport error: ERR stays 0 and REPLY_CMD carries \$15. The library's FNACK turns that into its own code, \$EE. A reply longer than 1024 bytes is truncated, not refused.

The reply window and its slices

The reply lands in \$1B00–\$1CFF, 512 bytes at a time. A reply of up to 1024 bytes is kept whole by the cartridge in two slices, and RXSLICE selects which one the window shows; after a slice change, SLICE_ECHO is painted last and a program polls it before trusting the bytes. In practice almost nothing needs the second slice: the largest reply any program in this handbook reads is Battleship's four-player game record, 509 bytes, and the window is 512 so that it never pages at all.

The window is repainted only by a SEQ commit or a slice select. Between those it is stable, and that stability is a resource: a program can stream bytes straight out of the reply window into the TX page while building its next request, or into a text row, or into a path buffer, with no copy through RAM. On a console with 128 bytes of RAM this is not an optimisation. It is the only way a directory browser fits.

The arming gate

Nothing on the control page decodes until two stores arrive in order with two particular values: \$B5 to \$1DFC, then \$4A to \$1DFD. A BIOS probing cartridge space at power-on cannot produce that pair, and until it arrives the write-only pages are tri-stated and inert. Every client opens the gate first thing; the fixed tail's cold stub does it too, because the bank select that follows is a control-page operation. Opening a gate that is already open does nothing.

The one rule that will bite

CAUTION: Never use a read-modify-write instruction on the control page or the TX page. INC, DEC, ASL, LSR, ROL and ROR on an absolute address are three bus cycles at that address: a read, a write of the old value, and a write of the new one. The cartridge's "last value before the address changed" has no defined answer inside that burst, and it has no clock pin with which to do better. Only STA, STX and STY may target \$1D00–\$1EFF, and the build fails on anything else.

In batari Basic the same trap is one keystroke away: FNTX = FNTX + 1 compiles to an INC on the TX page. Compute in a variable, then assign.

Why the scan can be a proof

The 6507 has no interrupts. MAME's own model of it says so: no NMI, no SO, no SYNC, and nothing on the console is wired to /IRQ. Only `BRK` reaches the vector at `$1FFE`. Every stray access on this console therefore comes from the program's own instruction stream, never from something that happened to fire mid-transaction, which is why the static check in Section 3 can be complete where the ColecoVision's, with a vblank NMI landing in the middle of every transaction, could only be a heuristic.

The recipe

Every transaction in this handbook is the same six steps. Here they are once, in both languages, for `GET_ADAPTERCONFIG_EXTENDED` — device `$70`, command `$C4`, no parameters — whose 240-byte reply carries the adapter's SSID, IP address and firmware version.

6502 ASSEMBLY

```
jsr    FNARM           ; open the gate (once)
lda    #FNDEVF         ; 1. the device...
sta    FNDEV
lda    #FNCADPX        ; 2. ...the command...
sta    FNCMD
lda    #0              ; 3. ...and the parameter count
sta    FNNPR
jsr    FNBEG           ; commit all three, rewind TX
                        ; 4. (parameters and payload: none)
jsr    FNGO            ; 5. SEQ = ACKSEQ+1, wait; A = 0 if answered
bne    FAILED
jsr    FNACK           ; 6. ACK or NAK?
bne    FAILED
lda    FNRPLY+0        ; the SSID starts at reply offset 0
```

BATARI BASIC

```
FNH_ARM1 = FNAM1 : rem open the gate (once)
FNH_ARM2 = FNAM2
FNA_DEV = 0 : rem 1. the device: arm, then commit
FNCMT = FNDEVF
FNA_CMD = 0 : rem 2. the command
FNCMT = FNCADPX
FNA_NPAR = 0 : rem 3. the parameter count
FNCMT = 0
FNA_DRST = 0 : rem rewind the TX stream
FNCMT = 0
rem 4. (parameters and payload: none)
fnseq = FNACKS + 1 : rem 5. launch: SEQ = ACKSEQ + 1, wrapping
if fnseq = 0 then fnseq = 1
FNA_SEQ = 0
FNCMT = fnseq
wait
drawscreen
if FNACKS <> fnseq then goto wait
if FNERR <> 0 then goto fail
if FNRCMD <> 6 then goto fail : rem 6. ACK or NAK?
ch = FNRPLY[0] : rem the SSID starts at reply offset 0
```

The BASIC version draws a frame per poll, so the picture stays up while the adapter works; the assembly version, as written, spins with the screen blanked, which is what the first-contact program in Section 9 does before it turns the display on. Battleship in Section 19 shows the assembly way of drawing frames while waiting.

The Video Computer System has no framebuffer and no character generator. The processor races the beam, and every scanline of every picture is built by hand, in the forty-odd cycles the kernel has before the next one begins. On every other console in the FujiNet family, “draw a string” is a memory write. Here it would be a kernel, and a kernel the console has neither the RAM nor the cycles to feed with glyphs.

So the cartridge draws. It keeps the font, renders ASCII into bytes already in the exact shape a 48-pixel player kernel wants, and publishes them in the window. The console’s part is to stream those bytes into two TIA registers on a cycle-exact schedule, and that part is written once, in `fujidisp.inc`, and shared by every client.

Six planes

Text is twelve columns by twenty-one rows. A glyph is three pixels wide and five tall in a four-by-six cell; twelve cells are the 48 pixels that two players with three copies each can cover. Each row of the screen is six scanlines, and each scanline of the block needs six bytes, one per GRP write. The cartridge keeps those six bytes in six separate planes, 128 bytes each, indexed by the absolute scanline:

Plane	Address	Write	Text columns
0	\$1800	1st GRP0	0–1
1	\$1880	1st GRP1	2–3
2	\$1900	2nd GRP0	4–5
3	\$1980	2nd GRP1	6–7
4	\$1A00	3rd GRP0	8–9
5	\$1A80	3rd GRP1	10–11

The index is $Y = \text{row} \times 6 + \text{line}$, and that is the whole trick. Because a plane is 128-byte aligned and Y is always below 128, the address arithmetic can never carry: `LDA $1800,Y` is always exactly four cycles, never five. A kernel that spends an unpredictable number of cycles does not draw, it tears, so the alignment is not tidiness but the difference between a display and a mess. It also means the kernel needs no zero-page pointers and no per-row setup: Y simply counts from 0 to 125 down the whole screen, and the rows are implicit in the data.

Within a byte, bit 7 is the leftmost pixel. The left column of the pair occupies bits 7 to 5 with bit 4 as its inter-character gap; the right column bits 3 to 1 with bit 0 as its gap.

7	6	5	4 gap	3	2	1	0 gap
---	---	---	-------	---	---	---	-------

Composing a row

A program never touches the planes to write text. It tells the cartridge, through three one-shot operations on the control page:

Store to	Data	Effect
\$1DF0	row 0–20	begin composing that row; the cursor is at column 0
\$1DF1	a character	append it; past twelve, further characters are dropped
\$1DF2	0	render the composed row into the planes

The render happens on the cartridge's other core, so it is not instantaneous on hardware: a program that begins the next row's characters at once hands the render a row that is already being overwritten. The cell TEXTGEN at \$1F0D holds the row last rendered in its low bits and flips its top bit every time, and a careful program waits for it to change — bounded, since emulation renders inside the store and the cell has already changed by the time the wait begins.

6502 ASSEMBLY

```

; FNROWA / FNCHR / FNENDR are the library's names for the three stores.
    lda    #3
    jsr    FNROWA          ; row 3
    lda    #'H'
    sta    FNRSEL+FH_TCHR
    lda    #'I'
    sta    FNRSEL+FH_TCHR
    ldx    FNBTXG           ; the generation before...
    jsr    FNENDR          ; ...render...
    ldy    #0
WAITR: cpx    FNBTXG        ; ...and until it changes
       bne    DONE
       dey
       bne    WAITR
DONE:

```

BATARI BASIC

```

FNH_TROW = 3
FNH_TCHR = 72
FNH_TCHR = 73
gen = FNBTXG
FNH_TEND = 0
for l = 0 to 100
  if FNBTXG <> gen then goto done
next
done

```

Anything the reply window holds can go straight into a row the same way, byte by byte, without ever passing through RAM: the library's FNRRL renders up to Y bytes of the reply at offset X into row A, stopping at a NUL. That is how every program in this handbook shows what the adapter said.

The font

The font is the cartridge's, generated from one Python table in the bring-up. It covers \$20 to \$7F; lowercase folds to uppercase, and anything else renders as ?. At three by five pixels several pairs of characters would be identical, and three of them were redrawn: o is rounded against o's square, s is curved against s's flat top, l is half-width against c. The only remaining collision is ? against DEL, which is deliberate.



Figure 2. The bring-up's first screen: the whole font, baked into the planes of a plain 4K image and shown by the kernel — as MAME drew it, and as the cartridge's font renders the same `screen.txt`. Every one of its 756 plane bytes was byte-compared against the renderer.

The kernel

`fujidisp.inc` is two routines. `DINIT` sets the TIA up once: two players, three copies each eight pixels apart (`NUSIZ = $03`) so that P0 draws groups 0, 2 and 4 and P1 groups 1, 3 and 5, interleaving into 48 contiguous pixels; vertical delay on both, which is what makes the six-write sequence land in the right copies; and the positioning, transcribed from batari Basic's kernel: sixteen `NOPS` and one three-cycle store so that `RESP0` lands on cycle 38, `RESP1` three cycles later, `HMP0` of `$F0` to pull player 0 right one pixel, and an early `HMOVE` on the line after.

IMPORTANT: The four positioning constants are the coordinates of a one-pixel-wide window and must not be retimed. The six copies are 2.67 cycles apart and the kernel's stores are three, so where the block sits decides whether each update lands between copies or inside one. Every wrong setting still looks like text. They were found by decoding the raster back into plane bytes and comparing, not by eye.

`DLOOP` draws frames forever: three lines of `VSYNC`, 37 of `VBLANK` during which it calls the client's `APPVBL` hook, 33 blank lines, the 126 lines of text, 33 more, and 30 of overscan — 262 in all. The text lines are the body below, 53 of the 76 cycles a line has:

```
KERN:  sta    WSYNC
        lda    TP0,y      ; 4
        sta    GRP0      ; 3    -> 7    new GRP0 = group 0
        lda    TP1,y      ; 4    -> 11
        sta    GRP1      ; 3    -> 14    displays group 0
        lda    TP2,y      ; 4    -> 18
        sta    GRP0      ; 3    -> 21    displays group 1
```

```

lda    TP4,y      ; 4  -> 25
tax
txs      ; 2  -> 27
lda    TP3,y      ; 2  -> 29    park group 4 in S
tax      ; 4  -> 33
lda    TP5,y      ; 2  -> 35
tax      ; 4  -> 39
stx     GRP1      ; 3  -> 42    displays group 2
tsx     GRP1      ; 2  -> 44    recover group 4
stx     GRP0      ; 3  -> 47    displays group 3
sta     GRP1      ; 3  -> 50    displays group 4
sty     GRP0      ; 3  -> 53    displays group 5; value unused
iny
cpy     #TLINES
bne     KERN

```

Three things in it are not style. The stack pointer is a register: the four late writes drift a full cycle across the block, recovering a byte from zero page costs three cycles and `TSX` costs two, and that one cycle is the whole difference between a schedule that closes and one that does not. The seventh write, `STY GRP0`, exists because with vertical delay six data bytes need seven writes, and its value is a don't-care, so the scanline counter serves. And nothing between the `TXS` and the restore may touch the stack: no `JSR`, and the 6507 has no interrupts to worry about.

After the last line both players are blanked with three writes, not two: with `VDELP` set, a write to `GRP0` reloads only the delayed `GRP1` and vice versa. The obvious-looking alternative, toggling `VDELP` off and on, restores the stale delayed registers, and the blank lines below the text then redraw the last kernel line. That was one of the three bugs the bring-up's raster comparison found; the missing seventh write and the `LDX`-versus-`TSX` cycle were the others.

The pixel that cannot be drawn

The 48-pixel block is six player copies and the kernel's seven writes; the four late ones span 33 pixels, while the distance from the end of group 0 to the start of group 5 is 32. No position serves all 48, so every position loses exactly one pixel, and the least bad one to lose is pixel 7 — bit 0 of plane 0, which the text renderer already spends as column 1's inter-character gap. Text never notices. Anything else drawn through the planes, such as the card art of Section 18, has to know.

What the cartridge can draw for you

Beyond text rows, the cartridge's blit port moves and transforms bytes into the planes with six stores — source, destination, count and a transform — so that a screenful that the console has neither the RAM nor the raster time to build itself costs it 24 cycles. Section 18 lists all fourteen transforms. Four are text: a raw copy, a text row composed straight from the reply window, a single cell changed in place (which is how a list moves its cursor without recomposing the row), and a row rendered from a cartridge path buffer, which is how a program sees what it has typed into a buffer it cannot read back.

NOTE: A blit is a single-slot request on the RP2040: the bus core hands it to the other core and there is no queue. Fire two in a row and the second overwrites the first before it has run — on hardware. In emulation the blit runs inside the store and nothing is lost. Poll `BLITGEN` at `$1F19` for a change before the next one.

batari Basic

A BASIC program keeps batari Basic's own kernel for its playfield and sprites, and shows the cartridge's text through a minikernel, `fujitext.asm`, that draws a band of rows from the planes between the playfield and the score. It is the body above, verbatim, with the positioning folded in. Section 8 explains how many rows a frame can afford.

The console has 128 bytes of RAM, at \$80 to \$FF. The stack is not separate: \$0100-\$01FF mirrors into the same 128 bytes, so the stack pointer starting at \$FF means the stack grows down from the top of the only RAM there is. A path is 256 bytes. A directory entry is 30. A player's record in a game is 115. None of it fits, and none of it has to, because the cartridge is on the other side of the bus with all the memory in the world.

This section is the discipline that follows from that, and it is the discipline every program in this handbook is written under.

Nothing is copied

The reply window is stable between transactions. So a program does not copy a reply into RAM to use it; it uses it where it lies:

- **Reply to row.** `FNRRPL` streams up to twelve bytes of the reply, at any offset, into a text row. The name of a file, the SSID, a line of a page: cartridge to cartridge, through one byte of the console at a time in the accumulator.
- **Reply to request.** `FNPRPL` appends bytes of the reply straight into the TX stream of the next request. A directory browser that reads an entry's name and then mounts it never holds the name.
- **Reply to path buffer.** `FNWRPL` appends bytes of the reply to one of the cartridge's path buffers, where they wait for the transaction that needs them. Battleship's table id and player name go this way, from the server's reply into every later URL, without ever landing in RAM.

The rule that makes these possible: the window is repainted only by a `SEQ` commit or a slice select. Read what you need before you launch the next transaction, or read it while you build the next transaction, but never after.

The cartridge holds the strings

Four 256-byte path buffers live in the cartridge, selected by one-shot operations and edited a character at a time. A program appends characters, pops the last component, pops one character, resets, and asks the cartridge to emit the buffer into the TX stream — NUL-padded to exactly 256 bytes, which is what `OPEN_DIRECTORY` and `SET_DEVICE_FULLPATH` read, or raw, which is what a URL wants. The buffers survive a console reset, because nothing on the cartridge sees the reset; a program that assumes an empty buffer at startup is wrong, and resets it.

The console cannot read a buffer back: the pages it is written through are write-only. A blit renders the buffer into a text row, which is how the on-screen keyboard in `CONFIG` shows what has been typed.

`CONFIG` needs all four at once — the working directory, the filter, a pending copy's source path and the keyboard's edit scratch — and the cartridge is the only place they could be.

The cartridge does the loops

A Battleship game field is 100 cells in the reply, and turning it into ten rows of text is a hundred reads, a hundred compares and a 16-bit cursor, in a bank that has under a thousand bytes to spare, into RAM that does not exist. The blit port

does it in six stores. Whenever the work is “take these bytes from the reply and put them on the screen”, look for the transform first; Section 18 lists them.

The zero page, three ways

Every program divides the 128 bytes differently, and the three divisions below are the ones this handbook uses.

Cells	The library clients	Battleship	batari Basic
\$80—\$8B	the transport: FNSEQ \$80, the kernel's PAD3 \$81 and SAVSP \$82, FNCNT \$83, FNDEV/ FNCMD/FNNPR \$86—\$88, FNTMO \$89, FNPTRL/H \$8A—\$8B	the same cells, plus BSIDX and BSTMP in the holes	the kernel: sprite positions and pointers, heights, score, temp1—temp6
\$8C—\$A7	FNPCNT \$8E; the rest free — but this is where the swap stub lands, so a client that boots keeps nothing here it needs afterwards	the client's own cells: request, error, steps, the settle loop, cursor	the kernel, then rand and scorecolor
\$A8—\$BF	INCUR/INPREV \$A8—\$A9; free	cursor, edge memory, clock, ships, composer flags	the RAM playfield, var0—var47 (to \$D3)
\$C0—\$FF	the stack	the stack, which must stay above \$C0	a—z at \$D4—\$ED, temp7, playfieldpos, aux1—aux6 at \$F0—\$F5, the stack from \$F6

The library's cells were chosen so that a client can put everything of its own above \$8C. Battleship's map is in Section 19, with every alias and the reason for it. batari Basic's leaves a program 26 named variables and, if it draws no playfield, 48 more; its stack is ten bytes, which is five nested gosubs.

The stack pointer is a register

The text kernel parks a graphics byte in the stack pointer for two cycles because the schedule does not close otherwise, and restores it before anything can push. The rule that makes that safe is that nothing between the TXS and the restore touches the stack, and on this console nothing can: there are no interrupts. A JSR into a routine that does this is fine, because its return address is pushed before the trick begins and nothing pushes again until it ends.

The swap stub

The one thing that must run from RAM is the boot. The swap replaces every byte of the window, including the code that triggered it, so the thirty-byte stub that strobes the swap and jumps through the new vector is copied to \$80 and run there. The stack and the stub share the same 128 bytes; a program that boots keeps its stack pointer above the stub, and the library's copier checks that rather than corrupting itself.

A byte costs a bank

In a banked program each bank carries its own copy of the routines it calls, because the high 2K is the mailbox and the fixed tail is 220 bytes. A byte added to a shared include is therefore a byte in every bank that includes it. Battleship splits its shared code behind feature flags for exactly that reason, and numbers its sound cues so that each bank's are a prefix of the list. Section 19 has the arithmetic.

What survives what

State	Console RESET	Power off
console RAM	survives (nothing clears it)	lost
the selected bank	survives — the cold stub reselects bank 0	bank 0
ACKSEQ and the mailbox's sequence	survives — derive SEQ from it	restarts at 0
the reply window, the text planes	survive	lost
the four path buffers	survive — reset them	empty
the arming gate	stays open	closed until the pair arrives

NOTE: The RESET switch restarts the 6507. It does not tell the cartridge anything, and a client decides what it means: Battleship uses it for a menu, and only a real 6507 restart, which no switch on the console produces during play, reaches the cold stub.

The cartridge bring-up ships a client library in three files, and every assembly program in this handbook is built on them verbatim. They are in `listings/common/`, and printed in full in Appendix C.

File	What it is
<code>vcs.inc</code>	the TIA and RIOT registers, written from the hardware map (the usual <code>vcs.h</code> is a dasm companion file and may not be redistributed)
<code>fujinet.inc</code>	the mailbox equates: every cell, register, hotspot, device, command and error code, and the zero-page cells the library uses
<code>fujilib.inc</code>	the transport, the text and path helpers, the swap stub, the input scanner
<code>fujidisp.inc</code>	the display: <code>DINIT</code> and <code>DLOOP</code>
<code>netdefs.inc</code> , <code>devdefs.inc</code>	this handbook's additions: the N: device, the other devices, and the FUJI commands the testrom did not need

IMPORTANT: Equates and code go in different files, and the order is not optional. `fujinet.inc` is equates and is included before the `ORG`; `fujilib.inc` and `fujidisp.inc` are code and are included inside it. Including the code half early assembles the whole transport at `$0000`, and every `JSR` to it becomes `20 00 00` — which assembles cleanly and fails at run time.

The equates are hand-mirrored from the firmware's `fuji_mailbox.h` and will drift if nothing checks them; the bring-up's `checkdefs.py` does, before anything assembles, and the handbook's build runs it. The bug it exists for: a control page one page off comes up, draws a screen, and never arms the mailbox.

The transport

Routine	What it does	In / out
<code>FNrw</code>	register <code>X = A</code> : the arm and the commit. The whole transport is built on it.	A value, X reg
<code>FNarm</code>	open the decode gate: <code>\$B5</code> then <code>\$4A</code>	—
<code>FNchk</code>	Z set if a cartridge is answering (F, N at <code>\$1F09</code>)	Z
<code>FNbeg</code>	begin the transaction described by <code>FNDEV</code> , <code>FNCMD</code> and <code>FNNPR</code> ; rewinds the TX stream last so parameters can follow at once; sets the timeout	zero page
<code>FNtxb</code>	append A to the TX stream	A
<code>FNpb</code>	append a one-byte parameter: the size byte 1, then A	A (preserved)
<code>FNpw</code>	append a two-byte parameter: 2, then A low and X high	A, X
<code>FNgo</code>	<code>SEQ = ACKSEQ + 1</code> , commit, spin until <code>ACKSEQ</code> matches or about nine seconds pass; A = 0 if the	A

	transaction completed, else the ERR cell or \$FF for no answer	
FNACK	after a completed transaction: A = 0 for ACK, \$EE for NAK	A

FNGO spins with nothing else happening, which is right for a program that talks before it turns the display on and wrong for one that wants to keep a picture up. Battleship's NPGO is the same launch with a frame drawn per poll, and draws one frame before its first look at the acknowledgement: in emulation the cartridge answers inside the commit, and without that frame a whole transaction and the next one's setup piled into one overscan.

Text

Routine	What it does	In
FNROWA	begin composing text row A	A
FNENDR	render the composed row	—
FNRRPL	render up to Y bytes of the reply at offset X into row A, stopping at NUL; zero-copy, and the reason most screens in this handbook cost no RAM	A, X, Y
FNRSTR	render the NUL-terminated string at FNPTL/H into row A, at most twelve characters	A, pointer
FNHEX	append A as two hex digits to the row being composed	A

FNRRPL's offset is one byte, so it reaches the first 256 bytes of the reply, which is every field of the adapter configuration and every entry of a directory. Battleship reaches player records past 256 with a pointer.

Payloads and the path buffers

Routine	What it does	In
FNPATH	append the string at FNPTL/H to the TX stream, NUL-padded to exactly 256 bytes	pointer
FNPBEG, FNPCH, FNPSTR, FNPRPL, FNPEND	build a 256-byte payload a piece at a time: start, one character, a string, bytes from the reply, and pad to 256	various
FNWRST, FNWCH, FNWPOP	the cartridge path buffer: empty it, append A, drop the last component	A
FNWTX, FNPWD	emit the path buffer into the TX stream: padded to 256, or raw with FNPCNT advanced by its length	—
FNWSTR, FNWRPL	append a string, or bytes of the reply, to the path buffer	pointer / X, Y
FNEOF	Z set if the reply is the \$7F,\$7F end-of-directory marker	Z

CAUTION: `OPEN_DIRECTORY` and `SET_DEVICE_FULLPATH` read exactly 256 bytes, and a short payload fails the server's read. Every path payload goes through `FNPATH`, `FNPEND` or `FNWTX`, which pad. And reading past the end of a directory poisons the session: a fujinet-pc SD host returns `..` over and over, and every later `SET_DIRECTORY_POSITION` is refused. Check `FNEOF` after every entry.

Boot

`FNBLK` arms the swap by committing `$B5` to the `BOOTLOCK` register. `FNSWAP` copies the stub into `$80` and jumps to it; it does not return. Section 11 has the sequence, and the three stores in the stub that are easy to leave out.

Input

`INSCAN` folds joystick 0 and the console switches into one active-high byte and returns only what is newly pressed, so a held stick steps once: `UP $01`, `DOWN $02`, `LEFT $04`, `RIGHT $08`, `FIRE $10`, `SELECT $20`, `RESET $40`. It keeps its two cells at `$A8` and `$A9`, above the swap stub's landing ground.

The display

`DINIT` sets the TIA up and positions the players; `DLOOP` draws frames forever and calls the client's `APPVBL` once per frame during the vertical blank, with the screen off and the raster nowhere near the text. Input scanning belongs there; so does a transaction, with the caveat that a round trip takes far longer than a frame and the picture will stall for it. A client defines two zero-page cells for the kernel, `PAD3` and `SAVSP`, and an `APPVBL`, which may be a bare `RTS`.

The dasm twin

batari Basic assembles through `dasm`, and so does a good deal of the 2600 world. The same library exists in `dasm` syntax in `listings/dasm/`, generated from the AS sources by `tools/as2dasm.py` and checked against the committed copy on every build, so that the two can never drift. The translation is mechanical: labels lose their colons, `DB` and `DW` become `.byte` and `.word`, `REPT` becomes `REPEAT`, `LSR A` becomes `LSR`, the low and high byte idioms become `<` and `>`, and the library's zero-page equates are wrapped in `IFNCONST` so that a program can place them on cells of its own choosing before it includes the header — which is what a batari Basic program has to do, since the library's cells are where batari Basic keeps its kernel. The translated first-contact program assembles to a working client, and was run against the adapter to prove it.

batari Basic compiles BASIC into 6502 assembly around a kernel of its own: a playfield, two sprites, two missiles, a ball and a score. Everything the previous sections said about the mailbox is true for it too, because the mailbox is only memory, and BASIC can read and write memory. What is different is the shape of the program the compiler produces, and this section is about fitting that shape into the cartridge.

The 2K rule

A batari Basic program is normally 4K, with its kernel at the bottom, its code in the middle, and its score font and vectors at the top — at \$FF9C and \$FFFC, which on this cartridge is the status page, the claim and the fixed tail. That cannot work. What can is `set romsize 2k`:

- the image is exactly 2048 bytes, from \$F000 to \$F7FF, which is the cartridge's bank 0;
- the program's entry point, `start`, is the first thing after the header, at \$F000, which is \$1000;
- nothing the compiler emits reaches \$F800 or above.

The build script appends the other half: a 2K fixed half from `fujitail.asm` that carries the claim at \$1F10, a cold stub at \$1F20 that opens the gate, reselects bank 0 and jumps to \$1000, and the vectors pointing at that stub. The result is a 4096-byte image the cartridge serves as bank 0 plus the fixed half, exactly like an assembly client.

IMPORTANT: batari Basic's own bank switching — bank 2, `goto label bank 3` — cannot be used. Its trampolines and per-bank vectors live in the top 256 bytes of every bank, which on this cartridge is the mailbox. A batari Basic FujiNet client is one 2K bank, and this section ends with what that buys.

The names

batari Basic emits any name it does not know verbatim into the assembly, and `dasm` resolves it. So an assembler header of equates gives BASIC the mailbox under the same names assembly uses: `include fujinet.h` on the first lines of the program, then `fujibas.h`, which adds a name for every register's arm address and every one-shot hotspot, because BASIC cannot write

`FNRSEL+FH_TROW`.

BASIC	Compiles to
<code>FNH_TROW = 3</code>	<code>LDA #3 / STA \$1DF0</code>
<code>FNCMT = fnseq</code>	<code>LDA fnseq / STA \$1DFF</code>
<code>ch = FNRPLY[i]</code>	<code>LDX i / LDA \$1B00,X / STA ch</code>
<code>if FNACKS <> fnseq then goto wait</code>	<code>LDA \$1F00 / CMP fnseq / BNE .wait</code>
<code>FNTX = url[l]</code>	<code>LDX l / LDA url,X / STA \$1E00</code>

An assignment to a mailbox name is a store; a mailbox name on the right is a load. Two rules come with that:

CAUTION: A name used as a value must be a batari Basic `const`, or the compiler treats it as memory: `FNCMT = FNDEVF` with `FNDEVF` known only to `dasm` compiles to `LDA $0070`, a load from the TIA, and the transaction goes out addressed to nobody. The build script prepends `fujiconst.bas`, generated from the headers, so every device id, command byte, register number, hotspot offset, blit kind, path operation and error code is a `const`. Addresses — the window, the status cells — are deliberately not, so they stay memory.

CAUTION: Never `FNTX = FNTX + 1`, or any other arithmetic in place on a mailbox name: batari Basic compiles it to `INC` on the TX page, the one instruction the cartridge cannot sample. Compute in a variable, then assign. The static check catches it, after the fact.

And a small one: do not `dim` a name onto a single letter you also use, and do not name a `dim c` or `i`; `a` to `z` are batari Basic's own variables, and redefining one is an assembler error about a value mismatch.

Where the variables go

batari Basic's kernel owns `$80–$A3`; the RAM playfield is `var0–var47` at `$A4–$D3`; `a–z` are `$D4–$ED`; `aux1–aux6` at `$F0–$F5` are free unless the program uses playfield colours, heights, lives or the playfield score; and the stack is `$F6–$FF`, ten bytes, which is five nested `gosubs`. A FujiNet program needs about a dozen variables — the sequence it is waiting for, an error, the device, command and count for the transaction helper, a generation for the render wait, and its own cursors — and `a` to `z` hold them comfortably. The transport's state lives in the cartridge.

The helpers

Every BASIC program in this handbook carries the same three subroutines at its end. They are the whole transport.

```
rem fnbeg: begin a transaction from dev / cmd / npar
fnbeg
  FNA_DEV = 0
  FNCMT = dev
  FNA_CMD = 0
  FNCMT = cmd
  FNA_NPAR = 0
  FNCMT = npar
  FNA_DRST = 0
  FNCMT = 0
  return
rem fngo: launch (SEQ = ACKSEQ + 1), draw frames until the cartridge
rem answers; err = 0 on ACK, the transport's code, or 255
fngo
  fnseq = FNACKS + 1
  if fnseq = 0 then fnseq = 1
  FNA_SEQ = 0
  FNCMT = fnseq
  tries = 0
fngowait
  drawscreen
```

```

if FNACKS = fnseq then goto fngodone
tries = tries + 1
if tries <> 0 then goto fngowait
err = 255
return
fngodone
err = FNERR
if err <> 0 then return
if FNRCMD <> 6 then err = 255
return
rem fnend: render the composed row and wait for it to land
fnend
gen = FNBTXG
FNH_TEND = 0
for l = 0 to 100
if FNBTXG <> gen then return
next
return

```

Parameters and payload go between `gosub fnbeg` and `gosub fngo`, as stores to `FNTX`: the size byte, then the value. `fngo` draws a frame per poll, so the picture stays up and the joystick stays readable while the adapter works, and it gives up after 255 frames — about four seconds, less than the cartridge's own five-second budget, so a real timeout is reported by the cartridge as its error and not guessed at here.

Showing text

The program's picture is batari Basic's: its playfield and sprites, drawn by its kernel. The cartridge's text comes through `fujitext.asm`, a minikernel that draws a band of rows from the planes between the playfield and the score, in the slot batari Basic's standard kernel offers one:

```

include fujitext.asm
const FT_ROW0 = 18      ' the first text row shown, 0-20
const FT_ROWS = 3      ' how many
const pfrowheight = 6  ' see the table below

```

The band costs $3 + 6 \times \text{FT_ROWS}$ scanlines, and batari Basic's frame is timer-locked: those lines come out of the time the program has between frames unless the playfield gives them back. `pfrowheight` shortens the playfield's rows. The table is what `frames.lua` measured, and the only honest way to know whether a layout fits is to measure yours the same way.

<code>pfrowheight</code>	<code>score</code>	rows that fit	note
8 (default)	yes	3	176-line playfield; the frame has 45 lines to spare and the score takes ten
6	yes	10	144-line playfield
6	<code>const noscore = 1</code>	13	the directory lister uses 12, for one row of slack
5 or less	—	none	a playfield under 120 lines wraps the overscan timer and the frame breaks

Composing text costs time too: a row of twelve stores and the render wait is about a line and a half, so a program that composes several rows in one go between two `drawscreen`s produces one long frame, which the set shows as a

roll. Compose a row per frame, as the programs here do, or accept one long frame on a screen change, as Battleship does.

The budget

After the standard kernel there are about 800 bytes for the program. That is enough for a screen and a few transactions, and the numbers below are what each program in this handbook had left.

Program	Bytes free	Note
blank.bas	829	the frame loop and nothing else
hello.bas	483	three rows of text
clock.bas	643	two transactions, four rows
appkey.bas	454	five transactions, four rows
boot.bas	428	three transactions, the swap stub in <code>asm</code>
netget.bas	—	five transactions, a settle loop, five rows: over by five bytes with the stock modules, and comfortable with the trimmed list below
dir.bas	576	a listing of twelve rows, with the cursor, on the trimmed list

`listings/bas/default.inc` is batari Basic's module list without `pf_drawing.asm` and `pf_scrolling.asm`, which are the `pfpixel`, `pfhline`, `pfvline` and `pfscroll` statements and about 335 bytes; batari Basic uses a `default.inc` in the build directory in preference to its own. A program that draws its playfield from a `playfield:` block does not need them. `const noscore = 1` removes the score kernel and its font for about a hundred more, and its ten scanlines.

Building

```
cd listings/bas
./build.sh hello           # -> build/hello.bin, 4096 bytes
```

The script prepends `fujiconst.bas`, compiles in `build/` where the headers, the minikernel and `default.inc` are reachable, checks that the result is 2048 bytes, appends the fixed half, and runs the cartridge's static check. batari Basic's own kernel contains two indirect stores, aimed at RAM and the TIA; the script vouches for those two, by address, and fails on anything at or above the program's own `game` label. Then it prints how many bytes are left. Run the result with `emu/run.sh`, exactly like an assembly client.



Figure 3. `hello.bas`: batari Basic's playfield above, three rows of the cartridge's text below, through the minikernel.

One transaction, and the adapter's name, address and firmware version on the screen. The assembly program is the bring-up's own `fujitest.asm`, unchanged; the BASIC program is `text.bas`. Both ask the FUJI device for `GET_ADAPTERCONFIG_EXTENDED`, whose 240-byte reply is the whole adapter configuration, and both put three of its fields into text rows straight from the reply window.

Build and run

- A. Build: `cd listings/asm && ./build.sh fujitest`. The script checks the equates, assembles, makes a 4096-byte image, stamps the claim and runs the static check.
- B. Start `fujinet-pc` if it is not running, and run the image: `emu/run.sh listings/asm/build/fujitest.bin`.
- C. The screen below appears after the round trip. In emulation that is immediate; on the cartridge it is a blank frame or two, because this program talks before it turns the display on.
- D. Press the console's RESET switch. The program restarts, runs the transaction again, and the last row shows `02`: the cartridge kept its sequence across the reset, and the program derived the next one from it.
- E. Headless: `emu/run.sh listings/asm/build/fujitest.bin txn` prints the status cells and the reply's text fields and says `PASS`.



```
FUJINET 2600
SSID
DUMMY CAFE
IP
127.0.0.1
VERSION
01.6-2A9E2C2
RXSEQ
01
```

Figure 4. `fujitest.asm`. The last row is the sequence number, the evidence that the cartridge and not the program remembers it.

The reply

GET_ADAPTERCONFIG_EXTENDED needs no parameters and answers with one packed record. The text fields at the end are the ones to show; the binary ones in the middle are for programs that need to compute with an address.

Offset	Size	Field
0	33	ssid, NUL-terminated
33	64	hostname
97	4 × 4	localIP, gateway, netmask, dnsIP, binary
113	6 + 6	mac, bssid, binary
125	15	fn_version, text
140	16 × 4	sLocalIP, sGateway, sNetmask, sDnsIP, dotted text
204	18 × 2	sMacAddress, sBssid, text

Every field a program wants is below offset 256, which is why the library’s one-byte reply offset reaches all of them.

The assembly program

The cold start clears the TIA and RAM together — \$00–\$7F is the TIA and \$80–\$FF is RAM, so one loop from \$FF down does both — blanks the screen, opens the gate and checks that a cartridge is answering. Then the transaction, exactly the recipe of Section 4:

FUJITEST.ASM: THE TRANSACTION

```
GOTCART:
    lda    #FNDEVF
    sta    FNDEV
    lda    #FNCADPX
    sta    FNCMD
    lda    #0
    sta    FNNPR
    jsr    FNBEG
    jsr    FNGO
    sta    APERR
    cmp    #FNEOK
    bne    SHOW
    jsr    FNACK
    sta    APERR
```

The reply is rendered with FNRSTR for the labels, which live in ROM, and FNRRPL for the fields, which live in the reply window and never touch RAM:

FUJITEST.ASM: THE SSID ROW

```
SHOWOK: lda    #(TSSID)&$FF
          sta    FNPTL
          lda    #(TSSID)>>8
          sta    FNPTRH
          lda    #2
          jsr    FNRSTR
          lda    #3                ; row
          ldx    #AC_SSID          ; reply offset
          ldy    #FNTCOL          ; at most a rowful
          jsr    FNRRPL
```

The last row is the sequence number, rendered in hex from the cartridge’s ACKSEQ cell:

FUJITEST.ASM: THE EVIDENCE

```
; The sequence number, in hex. This is the rung's real evidence: after a
; console RESET it must read 02, not 01.
    lda    # (TSEQ) & $FF
    sta    FNPTRL
    lda    # (TSEQ) >> 8
    sta    FNPTRH
    lda    # 11
    jsr    FNRSTR
    lda    # 12
    jsr    FNROWA
    lda    FNACKS
    jsr    FNHEX
    jsr    FNENDR
```

Then the display is turned on and the program draws frames forever. Its APPVBL is a bare RTS: nothing happens per frame.

The BASIC program

text.bas does the same in BASIC, with batari Basic's playfield above and three rows of the cartridge's text below. The transaction is the recipe from Section 4 written out; the rows are one subroutine, called three times with a different reply offset, and each call is followed by a frame so that no frame runs long:

TEXT.BAS: THE TRANSACTION AND THE ROWS

```
FNH_ARM1 = FNAM1
FNH_ARM2 = FNAM2
FNA_DEV = 0
FNCMT = FNDEVF
FNA_CMD = 0
FNCMT = FNCADPX
FNA_NPAR = 0
FNCMT = 0
FNA_DRST = 0
FNCMT = 0
fnseq = FNACKS + 1
if fnseq = 0 then fnseq = 1
FNA_SEQ = 0
FNCMT = fnseq
wait
drawscreen
if FNACKS <> fnseq then goto wait
```

TEXT.BAS: A ROW FROM THE REPLY

```
src = 0 : FNH_TROW = 18 : gosub putrow
src = 140 : FNH_TROW = 19 : gosub putrow
src = 125 : FNH_TROW = 20 : gosub putrow
main
drawscreen
goto main
rem putrow: stream up to 12 bytes of the reply at src into the row begun by
rem the caller, stopping at NUL, then render and wait for the render.
putrow
for i = 0 to 11
ch = FNRPLY[src]
if ch = 0 then goto putend
FNH_TCHR = ch
src = src + 1
next
putend
gen = FNBTXG
FNH_TEND = 0
for i = 0 to 100
```

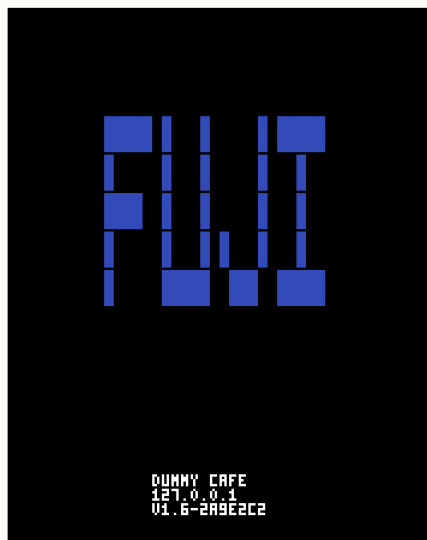


Figure 5. `text.bas`: the same three fields, through the minikernel.

`arm.bas`, in the same directory, is the smallest version: it runs the transaction and turns the screen green if the transport reported no error, red otherwise. It is the program to try first when nothing else works.

The N: device is the FujiNet's reason to exist. It opens a URL — HTTP and HTTPS, TCP and UDP, TNFS, FTP, SMB, NFS, SSH, TELNET — and hands the bytes back through the reply window, 512 at a time. Devices \$71 to \$78 are eight independent connections; every program in this handbook uses the first. The protocols, the devicespec grammar and each scheme's behaviour are the subject of *The FujiNet Network Protocol Handbook*; this section is the five commands that carry all of them, and a program that fetches a page.

The lifecycle

Command	Params	What it does
OPEN 'O'	mode, translation; payload: the devicespec	connects. Mode 4 is read, which on HTTP is GET; 8 write (PUT); 12 read/write, which on HTTP is GET with headers allowed; 13 POST. Translation 0 sends bytes as they are.
STATUS 'S'	two zero bytes	answers four bytes: bytes waiting, low and high; connected; and the device status, 1 for success.
READ 'R'	one two-byte count	answers exactly that many bytes in the reply window.
WRITE 'W'	one two-byte count; payload: the bytes	sends them.
CLOSE 'C'	—	hangs up.

Five rules, each learned by a port in this family and each written into `netget`:

- The adapter reports STATUS as soon as **some** of the response has arrived. Take readings until two agree and the count is not zero; the Channel F port once read a plausible header followed by forty bytes of nothing.
- The fourth STATUS byte is the only place an HTTP error shows. The GET is deferred until the first STATUS, so a bad URL is a status of not-1, not a failed OPEN.
- Cap the READ at what you can show, and at 512, which is one slice of the window; the largest thing any program here reads is 240 bytes, twenty rows.
- CLOSE belongs at the start of the next request, never right after a READ: every screen here renders straight out of the reply window, and closing wipes the thing being displayed. `netget` closes after it has composed its rows, which is the same thing.
- Compare byte counts through the carry, never the sign bit. 181 minus 1 comes out negative on a signed compare, and it did.

netget in assembly

netget.asm opens N:http://127.0.0.1:8765/hello.txt — a small text file served by python3 -m http.server from listings/fixtures/, so that the run is reproducible; change the string and it fetches anything — and puts the page on the screen, a line per text row.

NETGET.ASM: OPEN

```
GOTCART:    lda    #NETDEV
            sta    FNDEV
            lda    #NCOPEN
            sta    FNCMD
            lda    #2
            sta    FNNPR
            jsr    FNBEG
            lda    #NMHTPG
            jsr    FNPB
            lda    #NTRNONE
            jsr    FNPB
            ldy    #0
NURL:       lda    TURL,y
            beq    NURLE
            sta    FNTX
            iny
            bne    NURL
NURLE:     jsr    NGOACK
```

The devicespec goes into the TX stream a byte at a time from ROM, and the string's NUL is not sent: the cartridge counts the payload. Then STATUS until the count settles:

NETGET.ASM: STATUS UNTIL IT SETTLES

```
NSTAT:     lda    #NETDEV
            sta    FNDEV
            lda    #NCSTAT
            sta    FNCMD
            lda    #2
            sta    FNNPR
            jsr    FNBEG
            lda    #0
            jsr    FNPB
            lda    #0
            jsr    FNPB
            jsr    NGOACK
            lda    FNRPLY+NSAVLO
            sta    NAVLO
            lda    FNRPLY+NSAVHI
            sta    NAVHI
            cmp    NPRHI
            bne    NSAGN
            lda    NAVLO
            cmp    NPRLO
            bne    NSAGN
            ora    NAVHI
            bne    NREAD                ; settled, and there is something to read
NSAGN:     lda    NAVLO
            sta    NPRLO
            lda    NAVHI
            sta    NPRHI
            inc    NTRY
            lda    NTRY
            cmp    #NTRIES
            bcc    NSWAIT
            lda    #FNEWAIT
            sta    APERR
            jmp    SHOW
NSWAIT:    ldy    #3                ; about three frames of scanlines
NSW1:     ldx    #0
```

```

NSW2:   sta   WSYNC
        dex
        bne   NSW2
        dey
        bne   NSW1
        jmp

```

The READ's count is one parameter of two bytes, through `FNPW`, capped at 228, and the rows are composed straight out of the reply: a linefeed ends a row, a carriage return is skipped, and a thirteenth character on a line is lost.

NETGET.ASM: READ, AND A ROW PER LINE

```

NREAD:  lda   NAVHI
        bne   NRCAP
        lda   NAVLO
        cmp   #NMAX+1
        bcc   NRLEN
NRCAP:  lda   #NMAX
NRLEN:  sta   NAVLO
        lda   #0
        sta   NAVHI
        lda   #NETDEV
        sta   FNDEV
        lda   #NCREAD
        sta   FNCMD
        lda   #1
        sta   FNNPR
        jsr   FNBEG
        lda   NAVLO
        ldx   #0
        jsr   FNPW
        jsr   NGOACK

; ----- show it: a text row per line -----
        lda   #0
        sta   NSRC
        lda   #NROW0
        sta   NROW
NXROW:  lda   NROW
        cmp   #21
        bcs   NSHOWN
        jsr   FNROWA
        lda   #0
        sta   NCOL
NXCH:   ldx   NSRC
        cpx   NAVLO
        beq   NROWE
        lda   FNRPLY,x
        inc   NSRC
        cmp   #10
        beq   NROWE
        cmp   #13
        beq   NXCH
        ldy   NCOL
        cpy   #FNTCOL
        bcs   NXCH
        sta   FNRSEL+FH_TCHR
        inc   NCOL
        jmp   NXCH
NROWE:  jsr   NENDW
        inc   NROW
        lda   NSRC
        cmp   NAVLO
        bcc   NXROW
NSHOWN:

```

`NGOACK` is the transaction's tail in one place — commit, wait, check the ACK — and on any failure it drops its own return address and goes to the error screen, so that no call site needs a branch after it. `NENDW` renders a row and waits for `TEXTGEN` to change.

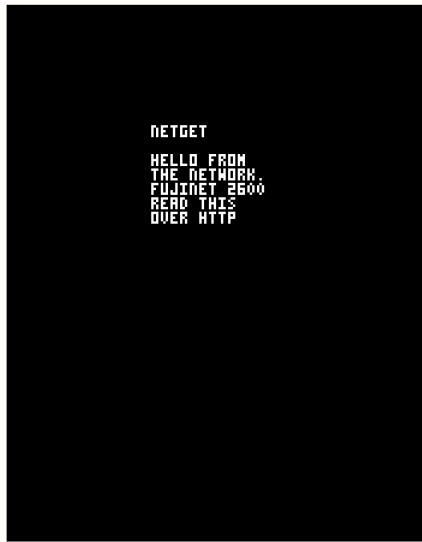


Figure 6. netget.asm, showing the fixture file.

netget in BASIC

The BASIC version is the same program with the helpers of Section 8. Its settle loop draws three frames between readings, and its READ caps the count in two compares because BASIC has no 16-bit compare:

NETGET.BAS: STATUS AND READ

```
status
  FNA_DEV = 0
  FNCMT = NETDEV
  FNA_CMD = 0
  FNCMT = NCSTAT
  FNA_NPAR = 0
  FNCMT = 2
  FNA_DRST = 0
  FNCMT = 0
  FNTX = 1
  FNTX = 0
  FNTX = 1
  FNTX = 0
  gosub fngo
  if err <> 0 then goto fail
  avlo = FNRPLY[0]
  avhi = FNRPLY[1]
  if avlo <> prlo then goto again
  if avhi <> prhi then goto again
  if avhi <> 0 then goto readit
  if avlo <> 0 then goto readit
again
  prlo = avlo
  prhi = avhi
  tries = tries + 1
  if tries = 30 then goto fail
  drawscreen
  drawscreen
  drawscreen
  goto status
readit
  rem READ: ONE two-byte parameter, the count, capped at 240 (twenty rows)
  if avhi <> 0 then avlo = 240
```

```

if avlo > 240 then avlo = 240
avhi = 0
FNA_DEV = 0
FNCMT = NETDEV
FNA_CMD = 0
FNCMT = NCREAD
FNA_NPAR = 0
FNCMT = 1
FNA_DRST = 0
FNCMT = 0
FNTX = 2
FNTX = avlo
FNTX = avhi
gosub fngo
if err <> 0 then goto fail

```

NETGET.BAS: A ROW PER LINE, A ROW PER FRAME

```

rem show it: a text row per line of the reply, a row per frame
src = 0
row = FT_ROW0
nextrow

```

It is the one program in this handbook that needed the trimmed module list of Section 8: with batari Basic's stock modules it was five bytes over.



Figure 7. `netget.bas`, with five rows under the playfield.

JSON

An open connection can be switched to the adapter's JSON parser with `CHANNEL_MODE ($FC, mode 1` in the second of two parameters), the body parsed with `PARSE ('P')`, and a single value fetched with `QUERY ('Q',` the path such as `/0/` name as the payload) followed by `STATUS` and `READ` for its text. That is the difference between parsing JSON on a 6507 and not. The games in this family skip it by asking their servers for binary with `?bin=1`, which is what Battleship does. The cards in Section 16 have the details.

The FujiNet's other reason to exist: a cartridge that can be any cartridge. The adapter mounts hosts — an SD card, a TNFS server across the room or across the world — lists their directories, and streams an image to the cartridge, which stages it and, on the program's signal, becomes it.

Hosts and directories

A host slot is one of eight configured hosts; `READ_HOST_SLOTS` answers all eight names, 32 bytes each, and `MOUNT_HOST` with a slot number connects one. Host slot 0 is the SD card on a standard `fujinet-pc`. Then:

Command	Params	Payload and reply
<code>OPEN_DIRECTORY</code> §F7	host slot	256 bytes: the path, a NUL, an optional filter such as <code>*.BIN</code> , the rest NUL
<code>READ_DIR_ENTRY</code> §F6	maxlen, flags	reply: one entry, NUL-terminated; a trailing / marks a directory; two §7F bytes mean the end
<code>SET_DIRECTORY_POSITION</code> §E4	one two-byte index	seek; <code>GET_DIRECTORY_POSITION</code> §E5 tells
<code>CLOSE_DIRECTORY</code> §F5	—	—

CAUTION: Ask for a `maxlen` of 30, not 31: at exactly 31 the firmware prepends icon bytes to the name. And stop at the two §7F bytes: past the end a `fujinet-pc` SD host answers `..` over and over, and every later seek is refused for the rest of the session.

`fujidir.asm` is the bring-up's browser: it mounts host 0, opens `/`, reads an entry per row into the screen, and moves a cursor with the joystick. No name is ever held in RAM. The cursor is an index; moving it re-reads the listing — instant in emulation, about a second a step on hardware, which is why `CONFIG` caches a page in the cartridge instead.

FUJIDIR.ASM: `OPEN_DIRECTORY`, AND THE 256-BYTE PAYLOAD

```

ODIR:  lda    #FNDEVF
        sta    FNDEV
        lda    #FNCODIR
        sta    FNCMD
        lda    #1
        sta    FNNPR
        jsr    FNBEG
        lda    #HOSTSL
        jsr    FNPB
        jsr    FNPBEG
        lda    #(TROOT)&$$FF
        sta    FNPTRL
        lda    #(TROOT)>>8
        sta    FNPTRH

```

```

        jsr     FNPSTR
        jsr     FNPEND
        jsr     FNGO
        sta     APERR
        cmp     #FNEOK
        bne     ODE
        jsr     FNACK
        sta     APERR
        cmp     #FNEOK
        beq     ODOK
ODE:    lda     #2
        sta     APSTEP
        lda     #1
        rts
ODOK:   lda     #0
        rts

```

FUJIDIR.ASM: AN ENTRY PER ROW

```

DRLOOP: stx     DIRWANT
        jsr     RDENT
        bne     DRBAD
        jsr     FNEOF
        beq     DREND          ; the $7F,$7F marker: stop, and do NOT read on
        ldx     DIRWANT

        ; Row: the cursor gutter, then the name straight from the reply.
        txa
        clc
        adc     #ROW0
        jsr     FNROWA
        lda     DIRWANT
        cmp     DIRSEL
        bne     DRNOCUR
        lda     #'>'
        jmp     DRCUR
DRNOCUR: lda     #' '
DRCUR:  sta     FNRSEL+FH_TCHR
        ldx     #0          ; reply offset: the name starts at 0
        ldy     #FNTCOL-1
        jsr     FNRRPX
        jsr     FNENDR

        ldx     DIRWANT
        inx
        stx     DIRCNT
        cpx     #NROWS
        bcc     DRLOOP

```



Figure 8. `fujidir.asm` listing the SD card. The `>` is the cursor; there is no inverse video.

`dir.bas` is the same browser in BASIC, and it keeps its working directory in the cartridge: path buffer 0 is set to `/` once, and the cartridge emits it, padded, into every `OPEN_DIRECTORY` with one store:

DIR.BAS: THE PATH BUFFER, AND THE LISTING

```
rem the working directory: path buffer 0 holds "/"
FNH_PATH0 = FP_SEL0
FNH_PATH0 = FP_RST
FNH_PATHC = 47
rem MOUNT_HOST(0)
dev = FNDEVF
cmd = FNCMHST
npar = 1
gosub fnbeg
FNTX = 1
FNTX = 0
gosub fngo
if err <> 0 then goto fail
```

DIR.BAS: OPEN_DIRECTORY AND READ_DIR_ENTRY

```
redraw
gosub draw
if err <> 0 then goto fail
main
drawscreen
if joy0down then goto down
if joy0up then goto up
goto main
down
ch = sel + 1
if ch = cnt then goto main
sel = ch
gosub release
goto redraw
up
if sel = 0 then goto main
sel = sel - 1
gosub release
```

```

goto redraw
release
drawscreen
if joy0down then goto release
if joy0up then goto release
return
fail
COLUBK = $34
drawscreen
goto fail
rem draw: OPEN_DIRECTORY(host 0, the path buffer, no filter) -- one
rem parameter, then the 256-byte path the cartridge emits and pads --
rem then READ_DIR_ENTRY(30, 0) once a row until the $7F,$7F end marker.
draw
dev = FNDEVF
cmd = FNCDIR
npar = 1
gosub fnbeg
FNTX = 1
FNTX = 0
FNH_PATHO = FP_TX
gosub fngo
if err <> 0 then return
cnt = 0
entry
dev = FNDEVF
cmd = FNCRDIR
npar = 2
gosub fnbeg
FNTX = 1
FNTX = 30
FNTX = 1
FNTX = 0
gosub fngo
if err <> 0 then return

```

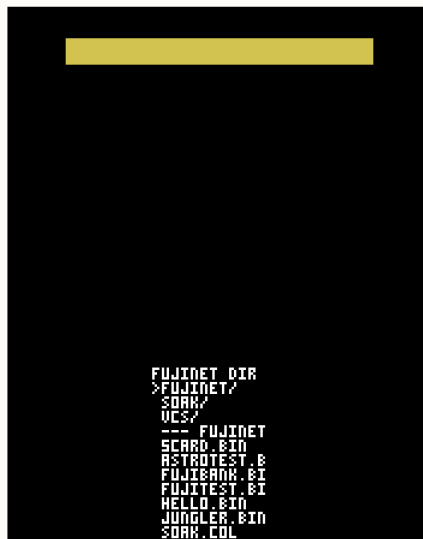


Figure 9. `dir.bas`. Twelve rows of listing under a one-line playfield, with the score kernel removed to pay for them.

Mounting and booting

Booting is three commands and a swap. `SET_DEVICE_FULLPATH` names the file for a device slot — three one-byte parameters, device slot, host slot and mode,

then the path as exactly 256 bytes. MOUNT_IMAGE mounts it — device slot and mode — and this is the command that takes real time: while its reply is outstanding, the adapter streams the image back to the cartridge over the same USB link, addressed to a device of its own (\$FF, the “DBC” device) in NET_OPEN, NET_WRITE and NET_CLOSE frames the cartridge acknowledges itself, and the cartridge stages it in a second buffer. If a .cfg file with the image’s name sits beside it, that arrives first and names the bank-switching scheme.

The program watches two cells. BOOT_STATE at \$1F06 goes 1 while the image is arriving, 2 when it is staged, \$80 if it failed, with a reason in BOOT_ERR; BOOT_PCT at \$1F07 counts 0 to 100 for a progress bar. Then:

- A. Commit \$B5 to the BOOTLOCK register (\$11). This arms the swap, and only once an image is staged.
- B. Copy the swap stub into zero-page RAM and jump to it. The swap replaces every byte of the window, including the code that triggered it.
- C. The stub silences the sprites, the sound and the playfield; sets the RIOT ports back to inputs; blanks the screen; stores to \$1DFE. The cartridge flips the window to the staged image between that store and the next fetch, which is safe because the next fetch is in RAM.
- D. The stub sets the stack pointer and jumps through (\$1FFC): the new image’s own reset vector, exactly what the 6507 fetches at power-on.

CAUTION: Three stores in the stub are easy to leave out and each breaks a different thing: AUDV0 and AUDV1, or a tone screams until the booted game’s cold start; and SWACNT and SWBCNT back to zero, because almost no game writes them and a client that left the RIOT ports as outputs would brick every game booted after it. The library’s FNSWAP has all three.

FUJIBOOT.ASM: MOUNT_IMAGE, THE WAIT, AND THE SWAP

```

MIMG:  lda    #FNDEVF
        sta    FNDEV
        lda    #FNCMIMG
        sta    FNCMD
        lda    #2
        sta    FNNPR
        jsr    FNBEG
        lda    #DEVSL0T
        jsr    FNPB
        lda    #FMREAD
        jsr    FNPB
        jsr    FNGO
        sta    APERR
        cmp    #FNEOK
        bne    ERR3
        jsr    FNACK
        sta    APERR
        cmp    #FNEOK
        beq    WAITB
ERR3:  lda    #3
        sta    APSTEP
        jmp    SHOW

; ---- wait for the staged image -----
WAITB:  ldy    #0
WAITB1: ldx    #0
WAITB2: lda    FNBST
        cmp    #FNBRDY
        beq    BOOT
        cmp    #FNBFAIL
        beq    BFAIL

```

```

        dex
        bne    WAITB2
        dey
        bne    WAITB1
        lda    #FNEWAIT        ; the image never arrived
        sta    APERR
        lda    #4
        sta    APSTEP
        jmp    SHOW
BFAIL:  lda    FNBER
        sta    APERR
        lda    #5
        sta    APSTEP
        jmp    SHOW

; ---- boot it -----
BOOT:   jsr    FNBLK        ; arm the swap
        jmp    FNSWAP       ; does not return

```

FUJILIB.INC: THE SWAP STUB

```

FNSWAP: ldx    #FNSTE-FNSTB-1
FNSW1:  lda    FNSTB,x
        sta    $80,x
        dex
        bpl    FNSW1
        jmp    $0080

FNSTB:  lda    #0
        sta    GRP0        ; silence the sprites
        sta    GRP1
        sta    AUDV0        ; *** and the sound, or it screams ***
        sta    AUDV1
        sta    PF0
        sta    PF1
        sta    PF2
        sta    WSYNC        ; land on a line boundary
        sta    SWACNT        ; *** RIOT port A back to inputs ***
        sta    SWBCNT        ; *** and port B ***
        lda    #2
        sta    VBLANK        ; blank across the swap
        sta    VSYNC        ; a clean discontinuity, not a rolling frame
        sta    FNRSEL+FH_SWAP ; THE SWAP. The next fetch is at $00xx, in
                                ; RAM with A12 low, so the window may flip
                                ; between this store and it.

        ldx    #$FF
        txs
        jmp    ($1FFC)        ; a clean stack for the new image
                                ; the NEW image's own reset vector, exactly
                                ; what the 6507 fetches at power-on

FNSTE:

```

In BASIC the stub is an `asm` block, and the wait is a loop that draws frames:

BOOT.BAS: THE WAIT AND THE SWAP

```

rem wait for the staged image
tries = 0
staged
drawscreen
state = FNBST
if state = FNBRDY then goto swap

```



Figure 10. boot.bas, a frame before the swap.

A booted image with no claim at $\$1F10$ is a game, and the mailbox goes dead for it; the harness that verified these boots waits for the claim to vanish, then compares every one of the 4096 served bytes with the file. The SD’s `hello.bin` is a 2K image, served mirrored, and the compare allows for that.

The mappers

A game that arrives is served by its own bank-switching scheme. The cartridge implements the nine that MAME implements — a scheme with no reference could not be verified against anything — and works out which from the image’s size, the Superchip signature, and the `.cfg` sibling if there is one.

Scheme	Image	How it switches
FLAT	2K or 4K	never; a 2K image is mirrored
F8	8K	reads of $\$1FF8$ – $\$1FF9$
F6	16K	$\$1FF6$ – $\$1FF9$
F4	32K	$\$1FF4$ – $\$1FFB$
FA	12K	$\$1FF8$ – $\$1FFA$, plus 256 bytes of RAM
E0	8K in 1K slices	$\$1FE0$ – $\$1FF7$, the top slice fixed
UA	8K	$\$0200$ – $\$027F$: below A12, where the cartridge is not selected and only watches
FE	8K	$\$01FE$ arms; the bank comes from D5 of the next access
CV	2K + 1K RAM	never; the RAM is read low and written high

An 8K F8, E0, UA and FE are all 8192 bytes and nothing inside the file tells them apart; the `.cfg` is how the adapter says which. Its content is the scheme’s name — F8SC, E0, UA — and it is spent on the image it came with, so that a hint left standing is never applied to the next one.

NOTE: A bank-switch hotspot returns the old bank's byte. The access that switches still reads what was there before, and the switch applies to the next fetch. MAME's own memory system settled it, the ColecoVision found the same thing, and it is invisible until a game reads its own hotspot for data as well as for the side effect — which several do.

Three more things the adapter does that a game wants, each a transaction or two.

Appkeys

An appkey is a small file on the adapter's SD card — `/FujiNet/CCCCAAKK.key`, named by a 16-bit creator id, an app id and a key number — and it is how a FujiNet program remembers something between sessions. Creator 1, app 1, key 0 is the slot every game in the family reads the player's name from.

`OPEN_APPKEY` selects the key and the direction, and its payload is one six-byte struct, not four parameters: the creator as two bytes low first, the app, the key, the mode (0 read, 1 write), and a reserved byte that is not optional — leave it off and the adapter waits for a byte that never comes, which reads back as a timeout. `READ_APPKEY` answers a two-byte length and then the value; `WRITE_APPKEY` takes one parameter, the length, and the value as its payload. A write resets the selection, so each write needs a fresh open.

APPKEY.ASM: OPEN, READ, WRITE

```
; AKOPEN -- OPEN_APPKEY for the key in AKCRL/AKCRH/AKKEYN, mode AKMODE.
AKOPEN: inc     APSTEP
        lda     #FNDEVF
        sta     FNDEV
        lda     #FNCAKOP
        sta     FNCMD
        lda     #0
        sta     FNNPR
        jsr     FNBEG
        lda     AKCRL                ; the six-byte struct, as the payload
        sta     FNTX
        lda     AKCRH
        sta     FNTX
        lda     #AKAPP
        sta     FNTX
        lda     AKKEYN
        sta     FNTX
        lda     AKMODE
        sta     FNTX
        lda     #0
        sta     FNTX                ; reserved, and not optional
        jmp     NGOACK

; AKREAD -- READ_APPKEY. The value is left in the reply window: two bytes
; of length, then the bytes.
AKREAD: inc     APSTEP
        lda     #FNDEVF
        sta     FNDEV
        lda     #FNCAKRD
        sta     FNCMD
        lda     #0
        sta     FNNPR
        jsr     FNBEG
        jmp     NGOACK

; AKWRITE -- WRITE_APPKEY: one parameter, the length, then the value.
AKWRITE: inc    APSTEP
        lda     #FNDEVF
        sta     FNDEV
        lda     #FNCAKWR
        sta     FNCMD
        lda     #1
        sta     FNNPR
        jsr     FNBEG
        lda     #TVALE-TVAL
```

```

      jsr      FNPB
      ldy      #0
AKW1: lda      TVAL,y
      sta      FNTX
      iny
      cpy      #TVALE-TVAL
      bne      AKW1
      jmp      NGOACK

```

APPKEY.BAS: THE SAME

```

rem akopen: OPEN_APPKEY -- the six-byte struct as the payload: creator
rem (two bytes, low first), app, key, mode, and a reserved byte
akopen
dev = FNDEVF
cmd = FNCAKOP
npar = 0
gosub fnbeg
FNTX = crlo
FNTX = crhi
FNTX = AKAPP
FNTX = keyn
FNTX = mode
FNTX = 0
gosub fngo
return
rem akread: READ_APPKEY -- the reply is a length word, then the value
akread
dev = FNDEVF
cmd = FNCAKRD
npar = 0
gosub fnbeg
gosub fngo
return

```

Both programs show the shared username, then write HELLO 2600 into creator \$2600, app 1, key 0 and read it back.



Figure 11. `appkey.bas`. The name is whatever the last FujiNet game stored.

NOTE: The open fails outright when no SD card is mounted on the adapter. That is not a client bug: Battleship treats it as “no name yet” and puts up its keyboard.

The clock

The clock is device \$45, and GET_ISO_LOCAL (\$49, I) answers the local time as text in the adapter's configured zone: 2026-09-15T20:45:31 followed by the offset and a NUL. GET_ISO_UTC (\$5A, Z) is the same in UTC. Twelve columns is not nineteen, so the date goes on one row and the time on the next, straight out of the reply window.

CLOCK.ASM: GET_ISO_LOCAL

```
GOTCART:
    lda    #CLKDEV
    sta    FNDEV
    lda    #CKISOL
    sta    FNCMD
    lda    #0
    sta    FNNPR
    jsr    FNBEG
    jsr    FNGO
    sta    APERR
    bne    SHOW
    jsr    FNACK
    sta    APERR
    bne    SHOW

    lda    #(TLOCAL)&$FF
    sta    FNPTRL
    lda    #(TLOCAL)>>8
    sta    FNPTRH
    lda    #2
    jsr    FNRSTR
    lda    #3                ; the date: ten characters from offset 0
    ldx    #0
    ldy    #10
    jsr    FNRRPL
    lda    #4                ; the time: eight from offset 11, past the T
    ldx    #11
    ldy    #8
    jsr    FNRRPL
```

CLOCK.BAS: THE SAME

```
rem showtime: ask the clock (cmd), then the date on row, the time on row+1
showtime
dev = CLKDEV
npar = 0
```

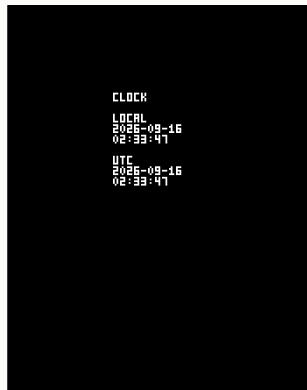


Figure 12. clock.asm. Local and UTC agree on this adapter because no zone is set.

The other formats answer too: GETTIME (\$93) is seven binary bytes, GET_PRODOS (\$50) four, GET_SIMPLE_HUNDREDTHS (\$4D) adds hundredths, GET_GENERAL (\$47) is the zone string and GETTZ_LEN (\$4C) its length. SETTZ (\$99) takes a POSIX zone string as its payload and sets the zone for the session. Section 17 has the cards.

Entropy, hashing, encoding, QR codes

GENERATE_GUID (\$BB) answers a fresh 36-character UUID and a NUL. The hash and base64 pipelines take four transactions each: INPUT with a two-byte count and the bytes as payload (repeatable, for long data), COMPUTE — for a hash, with the algorithm as a parameter: 0 MD5, 1 SHA-1, 3 SHA-256, 4 SHA-512 — then LENGTH and OUTPUT, each with a byte parameter that is 1 for hex text. A QR code is the same shape: INPUT, ENCODE with version, error-correction level and a shorten flag, LENGTH in the mode you want, OUTPUT. The adapter returns the module data ready to blit; a version-1 code is 21 by 21 modules, which the playfield can show.

6502 ASSEMBLY

```

; SHA-256 of a string, as hex text
        lda     #FNDEVF
        sta     FNDEV
        lda     #FNCHSHI      ; HASH_INPUT
        sta     FNCMD
        lda     #1
        sta     FNNPR
        jsr     FNBEG
        lda     #TXTE-TXT      ; the count, two bytes
        ldx     #0
        jsr     FNPW
        ldy     #0
HI1:    lda     TXT,y          ; the bytes
        sta     FNTX
        iny
        cpy     #TXTE-TXT
        bne     HI1
        jsr     FNGO
        lda     #FNCHSHC      ; HASH_COMPUTE, algorithm 3
        sta     FNCMD
        lda     #1
        sta     FNNPR
        jsr     FNBEG
        lda     #HASHA256
        jsr     FNPB
        jsr     FNGO
        lda     #FNCHSHO      ; HASH_OUTPUT, 1 = hex text
        sta     FNCMD
        jsr     FNBEG
        lda     #1
        jsr     FNPB
        jsr     FNGO          ; 64 hex digits in the reply window

```

BATARI BASIC

```

dev = FNDEVF
cmd = FNCHSHI
npar = 1
gosub fnbeg
FNTX = 2
FNTX = txt_length
FNTX = 0
for l = 0 to txt_length - 1
FNTX = txt[l]
next
gosub fngo
cmd = FNCHSHC
gosub fnbeg

```

```
FNTX = 1
FNTX = HASHA256
gosub fngo
cmd = FNCHSH0
gosub fnbeg
FNTX = 1
FNTX = 1
gosub fngo
rem 64 hex digits in the reply window
```

A 2K bank holds a screen and a few transactions. A program that wants more — CONFIG has seven screens, Battleship seven banks — switches banks, and on this cartridge that is a store: \$1D80 plus the bank number, and the switch is complete before the console's next fetch. What makes it usable is the fixed tail, the 220 bytes at \$1F20–\$1FFB that every bank sees at the same address.

A switch is a jump

A bank switch replaces every byte of \$1000–\$17FF. The instruction after the store is fetched from the new bank, so the store must be made from somewhere that does not change: the tail. Battleship's trampoline is the whole of it, and its stack reset is not decoration:

BSCORE.INC: THE TRAMPOLINE

```
; BSGOTO is not a label: bsdefs.inc gives it a fixed address and this ORG is
; what makes that true, so the two cannot drift apart. It must be FIRST in the
; tail, because it is the one address a bank has to know before build/tail.inc
; exists.
    ORG      BSGOTO

; -----
; BSGOTO -- select bank A and enter it at $1000.
;
; `sta FNRSEL,x' is the documented-safe indexed form: the base low byte is $00
; so the index cannot carry, which means the dummy read STA abs,X always
; performs lands on the same address as the write.
    clc
    adc     #FH_BANK
    tax
    sta     FNRSEL,x
; RESET THE STACK. A bank switch is a JUMP and nothing ever returns through
; one, so every switch abandons whatever return addresses were on the stack --
; and the switches here are taken from inside APPVBL, which DLOOP reaches with
; a JSR. Two bytes leaked per poll, a poll every ninety frames, and after
; thirty-five of them the stack had grown down out of $C0-$FF and was
; overwriting BSMOVE, BSENT and BSFRAME: a bank entry dispatching on an entry
; code that was really the low byte of a return address.
;
; This is the one place it can be got right once. The store above has already
; switched the bank, but these instructions are fetched from the FIXED tail,
; which is not banked, so they still execute.
    ldx     #$FF
    txs
    jmp     $1000
```

Nothing returns through a bank switch, and Battleship takes its switches from inside the per-frame hook, which the display loop reached with a JSR. Two bytes leaked per poll, a poll every ninety frames, and after thirty-five of them the stack had grown down out of its region and was overwriting the program's cells. The trampoline resets the stack on every switch, and because it is fetched from the fixed tail, the reset still executes after the bank has already changed.

"Where to carry on" is therefore data: Battleship keeps an entry code in a zero-page cell, and every bank's entry point dispatches on it.

The cold stub

The RESET vector points at the tail, never into a bank, because the RESET switch restarts the 6507 with whatever bank was last selected still mapped. The stub is deliberately tiny — the arming pair, because banking is a control-page

operation and the control page decodes nothing until the pair has arrived, then bank 0 — and it forces a cold entry:

BSCORE.INC: THE COLD STUB

```
; BSCOLD -- power-on and RESET.
;
; Deliberately TINY. Everything a cold start has to do -- clear the 128 bytes,
; blank the screen, empty the cartridge's four path buffers -- is done by bank
; 0 at BSCOLD2, because the tail is 220 bytes and the shared transport and
; text primitives want 160 of them. All that has to be here is what cannot be
; anywhere else: the arming pair, because banking is a control-page operation
; and that page decodes nothing until an ordered pair of stores carrying two
; specific values arrives, and the bank switch itself.
;
; A cold stub in bank 0 would not do. This console has NO RESET LINE to the
; cartridge: the RESET switch restarts the 6507 with whatever bank was last
; selected still mapped, so the stub has to be somewhere every bank can see.
BSCOLD: sei
        cld
        ldx    #$FF
        txs
; FORCE A COLD ENTRY. This console does not clear its RAM on a reset, so
; BSENT -- the byte that says what the bank being entered should do -- still
; holds whatever the last screen set it to. A reset taken at the keyboard came
; back into the LOBBY'S WARM PATH, which redraws a table list out of a reply
; window that is holding a Game, and reads table names out of the middle of
; lastResult.
        lda    #ENCOLD
        sta    BSENT
        lda    #FNAM1
        sta    FNRSEL+FH_ARM1
        lda    #FNAM2
        sta    FNRSEL+FH_ARM2
        lda    #BANKLOB
        jmp    BSGOTO
```

The console does not clear RAM on a reset. A reset taken at Battleship's keyboard once came back into the lobby's warm path, which redrew a table list out of a reply window that was holding a game.

Every bank carries its own library

There is no shared code region but the tail. The high 2K is the mailbox in its entirety, so each bank includes its own copy of the transport and the display kernel, about 600 bytes — which is what F8 games have always done. Battleship puts the routines every bank calls in the tail (the transport, 151 bytes; the four text primitives, 24) and generates the equates for them from the tail's own listing, so that no hand-kept list can go stale; the rest is per bank, behind feature flags so that a bank carries only what it calls. A byte added to a shared include costs a byte in every bank that includes it.

Building a banked image

The image is N banks of 2K then the fixed half, and N is 1, 3, 7 or 15 so that the size is one MAME will load; Battleship pads seven banks to eight. Each bank is assembled at \$1000–\$17FF and the tail at \$1800–\$1FFF, and the parts are concatenated in order. The bring-up's `build.sh` has a `build_banked` that does exactly this, and Battleship's has two checks it grew the hard way:

CAUTION: `p2bin` emits its whole address range whatever was assembled, so an over-long bank is truncated in silence: the image builds, the size is right, the static check passes, and the instructions past `$17FF` are simply gone. The assembler's listing is the only thing that knows, and Battleship's `checkbanks.py` asks it. Its `mktail.py` likewise fails a tail that runs past `$1FFB` into the vectors.

Then the claim is stamped and `checkrom.py` scans every bank, because every bank is entered at `$1000` and every bank is code. Section 19 has Battleship's bank map and budget.

batari Basic

A batari Basic program is one bank, and this section is why. Its own bank statements assume 4K banks with trampolines and vectors at the top of each, which is the mailbox; a program that outgrows 2K is written in assembly, or splits its work between the cartridge and the adapter. Most of what a 2K program cannot hold, it does not have to: the cartridge holds the strings and composes the screen, and the adapter does the network.

Text is twelve columns of white. A game board wants colour, and on this console colour comes from the playfield: forty bits across the screen, one colour per scanline, redrawn by the processor every line. Battleship draws up to four 10×10 boards at once in blue, red, white and gold on that hardware, and the trick is shared between the cartridge, which composes the tables, and a kernel that reads them on a schedule with no slack.

Two boards in forty bits

A cell is two playfield bits, eight pixels wide, so two boards side by side are the whole 40-bit asymmetric playfield: PF0, PF1 and PF2 rewritten twice a scanline, once for the left board and once for the right. The colour is decided by the line **within** the cell. Each line of an eight-line cell draws a different table in a different colour:

Cell line	Table	Colour	Shows
0	AUX	gold	your own hulls; the cursor on an enemy board
1–2	HIT	red	the top of a hit
3–4	MID	white	a hit's core, or the whole of a miss
5–6	HIT	red	the bottom of a hit
7	AUX	gold	

So a hit is a red block with a white core, a miss a white dash, and a hull or the cursor a pair of gold bars above and below the cell: a bracket that never hides what is under it, which is why the cursor does not blink.

The tables

Six registers — PF0, PF1, PF2 for the left half, then the right — times three kinds is eighteen tables, one entry per cell row, twenty rows for a top pair and a bottom pair. They live in the text-plane region, which nothing but a text render ever writes: plane r holds register r 's three tables at bytes 30 to 89, after text rows 0–4 and before rows 15–20. A program in board mode never renders text rows 5 to 14.

Cells	Bits
0–1	PF0 bits 4–5, 6–7
2–5	PF1 bits 7–6, 5–4, 3–2, 1–0
6–9	PF2 bits 0–1, 2–3, 4–5, 6–7

The TIA draws PF0 and PF2 low bit first and PF1 high bit first, which is why the map above zigzags. A slot is a board position — 0 top-left, 1 top-right, 2 bottom-left, 3 bottom-right — and every table byte is written whole and in place, so the kernel's `LDA table,Y` never sees a half-composed byte. The highest table byte is `$1AD9`, so every table sits inside one page and every read is four cycles.

Composing them

Four blit transforms fill the tables straight from the reply window, and the console never sees a cell:

Transform	Effect
PFCLR (10)	clear the kinds in a mask — <i>src</i> low byte: 1 HIT, 2 MID, 4 AUX — in slot <i>dst</i>
PFIELD (11)	a game field of 100 bytes (0 sea, 1 hit, 2 miss) at reply offset <i>src</i> into slot <i>dst</i> : 1 sets HIT and MID, 2 sets MID; AUX is untouched
PFHULL (12)	<i>cnt</i> ship placements (<i>pos</i> + 100 × <i>dir</i>) at reply offset <i>src</i> OR-ed into slot <i>dst</i> 's AUX; never clears
PFCELL (13)	one cell, <i>cnt</i> = 0–99, of slot <i>dst</i> : set the kinds in <i>src</i> , or clear them if <i>src</i> bit 7 is set — how the cursor moves

Doing that on the console would be four hundred reads through a 16-bit reply cursor in a bank that has not got the bytes, into RAM it has not got at all. The blit costs six stores.

BSCOMP.ASM: THE BOARDS, FROM THE REPLY

```
GBOARDS: lda    BSCLASS
          cmp    #CLPLAY
          bcs    GB0
          rts
GB0:     ldx     #0
GB1:     cpx     BSPCNT
          bcs    GB1X
          stx     BSIDX
          jsr     PLRECP
          lda     FNPTRL
          clc
          adc     #PLFIELD
          sta     FNRSEL+FH_BSL
          lda     FNPTRH
          adc     #0
          sec
          sbc     #(FNRPLY)>>8
          sta     FNRSEL+FH_BSH
          ldx     BSIDX
          jsr     PSLOT
          sta     FNRSEL+FH_BDL
          lda     #0
          sta     FNRSEL+FH_BDH
          lda     #FB_PFLD
          jsr     FNBLIT
          ldx     BSIDX
          inx
          jmp     GB1
GB1X:    rts

; GBAUX -- the AUX plane, in the vblank of the frame after: cleared
; everywhere, then your hulls, the winner's at game over, and the cursor on
; every live enemy. One frame behind the boards, which nobody can see, in
; exchange for a frame that is 262 lines.
```

The kernel

Each line of a board is six table reads and six playfield writes, with deadlines the TIA sets: the left *PF0* before colour clock 22, *PF1* before 28, *PF2* before 38; the right *PF0* between 28 and 49, *PF1* between 39 and 54, *PF2* between 50 and 65. The three line bodies are identical but for the table base and the colour:

DISPGAME.INC: ONE LINE OF A BOARD

```
LAUX:  sta     WSYNC
        lda     #CAUX          ; 2
        sta     COLUPF        ; 5
        lda     TAUX0,y        ; 9
        sta     PF0           ; 12
        lda     TAUX1,y        ; 16
        sta     PF1           ; 19
        lda     TAUX2,y        ; 23
        sta     PF2           ; 26
        lda     TAUX3,y        ; 30
        sta     PF0           ; 33
        lda     TAUX4,y        ; 37
        nop                     ; 39
        sta     PF1           ; 42
        lda     TAUX5,y        ; 46
        nop                     ; 48
        nop                     ; 50
        sta     PF2           ; 53
        rts
```

IMPORTANT: The `NOPS` are not padding. Without them the right half's `PF2` lands at cycle 49, and its first colour clock is inside pixel 79, the left board's last column. Every window here has at least ten colour clocks of margin, and `tools/pfcheck.py` in the Battleship tree reads every cell line of every board back from a snapshot and requires the colour the picture says.

Between the two boards of a pair runs a two-clock black missile, the divider, positioned once in `DINIT` and enabled only on board lines — because a missile replicates with its player's copy count, and the three close copies the text needs would put three dividers across the boards.

Four seats

Three or four players use the quadrant layout: two pairs of 8-line cells, 80 lines each, with the names over each pair and a three-line fleet strip under the names. Two players get one pair of 12-line cells, which is square on a 4:3 set, and the lower half of the screen becomes text. The 262 lines of the quadrant frame are counted below, and there is no slack in them: the kernel is 191 lines of a band of 192.

```
1  the unblank line
6  status row      (text row 0: five ink lines and a seam)
6  names over pair A (text row 1)
3  fleet strip     (text row 3)
1  the strip's seam, WHICH IS ALSO the board's lead-in
80 board pair A
1  lead-out
2  gap
6  names over pair B (text row 2)
3  fleet strip     (text row 4)
1  seam and lead-in
80 board pair B
1  lead-out
1  timed pad
```

The fleet strips are three ink lines rather than five because that is what the gap and the pad could pay for, and the font survives the clip: `#` keeps its hash and `=`

its bar. Digits would not — clipped to three rows, 8 and 9 are the same picture — which is why the strips show pips and not a count.

Trying it without a network

`layout.asm` in the Battleship tree is the board kernel with tables baked into the image from a picture in `tools/mklayout.py`: four boards, no two alike, every mark on every one.

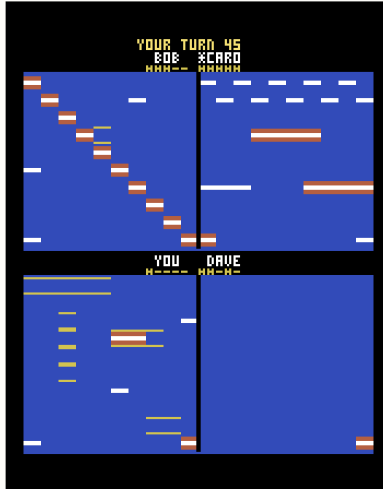


Figure 13. The layout ROM: every kind of mark on every board, so that a table read from the wrong slot or the wrong row is visible.

`make layout` && `make shot` builds it, snapshots it in MAME and reads every cell back. It is the place to start when changing anything about the kernel, because a wrong table read and a right one look the same until the picture is compared, cell by cell, against what was asked for.

Every command device \$70 answers on this cartridge, verified against the firmware's dispatch tables: the RS232 switch in `rs232Fuji.cpp` and the four mixins every FujiNet shares (appkeys, base64, hashing, QR codes). Conventions: *nparam* is the value written to the NPARAM register; a *byte* parameter is a size byte of 1 then the value, a *word* is 2 then two bytes little-endian. Replies land in the reply window at \$1B00. Each card carries an example in 6502 assembly, using the library of Section 7, and in batari Basic, using the helpers of Section 8: `dev`, `cmd` and `npar` `set`, `gosub` `fnbeg`, the parameters and payload as stores to `FNTX`, `gosub` `fngo`. Any command not on a card is answered with a NAK on this build; the last section names them.

WiFi and the adapter

GET_ADAPTERCONFIG_EXTENDED

\$C4

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	240 bytes: ssid[33], hostname[64], binary localIP, gateway, netmask, dnsIP (4 each), mac[6], bssid[6], fn_version[15] at 125, then the text fields sLocalIP[16] at 140, sGateway, sNetmask, sDnsIP, sMacAddress[18] at 204, sBssid — all NUL-terminated		

The first-contact command, and the one every status screen wants: no parameters, works before WiFi is up, and its text fields save a print-an-IP routine. The compact `GET_ADAPTERCONFIG` (§E8) answers the first 140 bytes only, addresses in binary.

6502 ASSEMBLY

```

lda    #FNDEVF
sta    FNDEV
lda    #FNCADPX
sta    FNCMD
lda    #0
sta    FNNPR
jsr    FNBEG
jsr    FNGO
; ssid at FNRPLY+0, version +125, ip text +140
    
```

BATARI BASIC

```

dev = FNDEVF
cmd = FNCADPX
npar = 0
gosub fnbeg
gosub fngo
ch = FNRPLY[140] : rem first digit of the IP
    
```

SCAN_NETWORKS

\$FD

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	1 byte: the number of networks found		

Starts a fresh scan and blocks until it finishes — seconds. Follow with GET_SCAN_RESULT once per index. CONFIG shows a “scanning” row first.

GET_SCAN_RESULT

\$FC

DEVICE	\$70	NPARAM	1
PARAMS	byte: result index, 0-based		
PAYLOAD	none		
REPLY	34 bytes: ssid[33] then rssi, a signed dBm		

One scanned network. The RSSI byte at offset 33 is a negative dBm — closer to zero is stronger — so bar-graph its magnitude.

6502 ASSEMBLY

```
lda    #FNCSCNR
sta    FNCMD
lda    #1
sta    FNNPR
jsr    FNBEG
lda    IDX          ; the index
jsr    FNPB
jsr    FNGO
; rssi at FNRPLY+33, signed
```

BATARI BASIC

```
cmd = FNCSCNR
npar = 1
gosub fnbeg
FNTX = 1
FNTX = idx
gosub fngo
rssi = FNRPLY[33]
```

SET_SSID

\$FB

DEVICE	\$70	NPARAM	1
PARAMS	byte: any value — required but ignored		
PAYLOAD	exactly 97 bytes: ssid[33] then password[64], NUL-padded		
REPLY	none		

Joins a network and saves it as the default. Two sharp edges: the parameter that must be there, and the payload that must be 97 bytes — a short one fails the transaction outright. CONFIG builds the two strings in the cartridge’s path buffers 0 and 1 and emits them raw.

6502 ASSEMBLY

```
lda    #FNCSSSD
sta    FNCMD
```

```

        lda    #1
        sta    FNNPR
        jsr    FNBEG
        lda    #0
        jsr    FNPB          ; the dummy parameter
        ldx    #0
SS1:    lda    SSIDPW,x      ; 97 bytes: ssid[33] + password[64]
        sta    FNTX
        inx
        cpx    #97
        bne    SS1
        jsr    FNGO

```

BATARI BASIC

```

cmd = FNCSSSD
npar = 1
gosub fnbeg
FNTX = 1
FNTX = 0
for l = 0 to 96
  FNTX = ssidpw[l]
next
gosub fngo

```

GET_SSID

\$FE

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	97 bytes: ssid[33] then password[64], NUL-padded		

The current network configuration, password in clear text; the adapter is a trusted friend.

GET_WIFISTATUS / GET_WIFI_ENABLED

\$FA / \$EA

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	\$FA: 1 byte, 3 = connected; \$EA: 1 byte, is the radio on		

The poll target while a join is in flight.

BATARI BASIC

```

cmd = FNCWIFI
npar = 0
gosub fnbeg
gosub fngo
if FNRPLY[0] = 3 then goto connected

```

RESET

\$FF

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	ACK, then the adapter reboots		

Reboots the ESP32-S3. The RP2040 and the mailbox stay up while the link bit in STATUS drops and returns; the cartridge waits up to three seconds for the link before its first transaction after that.

STATUS / DEVICE_READY

\$53 / \$00

DEVICE	\$70	NPARAM	1 / 0
PARAMS	STATUS: byte, request type — 1 for mount times		
PAYLOAD	none		
REPLY	STATUS type 1: a 4-byte time per device slot, 0 = unmounted; DEVICE_READY: the bus self-test response		

STATUS requires its parameter on this build. DEVICE_READY is the round trip that proves the mailbox, the RP2040, the USB link and the adapter are all telling the truth.

Hosts, slots and mounting

READ_HOST_SLOTS

\$F4

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	256 bytes: 8 slots x hostname[32]		

The host list, one transaction, rendered straight from the reply window. Slot 0 is the SD card on a standard fujinet-pc.

6502 ASSEMBLY

```
lda    #FNCRHST
sta    FNCMD
lda    #0
sta    FNNPR
jsr    FNBEG
jsr    FNGO
lda    #3                ; row 3: host 0's name
ldx    #0                ; FNRPLY + slot*32
ldy    #FNTCOL
jsr    FNRRPL
```

BATARI BASIC

```
cmd = FNCRHST
npar = 0
gosub fnbeg
gosub fngo
FNH_TROW = 3
for l = 0 to 11
  FNH_TCHR = FNRPLY[l]
```

```
next
gosub fnend
```

WRITE_HOST_SLOTS

\$F3

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	256 bytes: all 8 slots, full width		
REPLY	ACK		

All-or-nothing; there is no write-one-slot command. Read the table with READ_HOST_SLOTS, and stream it back out of the reply window into the TX page with the one edited slot substituted — the window is stable until the commit.

MOUNT_HOST

\$F9

DEVICE	\$70	NPARAM	1
PARAMS	byte: host slot, 0-based		
PAYLOAD	none		
REPLY	ACK, or NAK if the host is unreachable		

Connects the named host — SD, TNFS, SMB and the rest. Required before OPEN_DIRECTORY or a boot. UNMOUNT_HOST (\$E6) is not dispatched on this build.

6502 ASSEMBLY

```
lda    #FNCMHST
sta    FNCMD
lda    #1
sta    FNNPR
jsr    FNBEG
lda    #0                ; host slot 0
jsr    FNPB
jsr    FNGO
```

BATARI BASIC

```
cmd = FNCMHST
npar = 1
gosub fnbeg
FNTX = 1
FNTX = 0
gosub fngo
```

READ_DEVICE_SLOTS / WRITE_DEVICE_SLOTS

\$F2 / \$F1

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	\$F2: the device-slot table, hostSlot, mode, filename[] per slot; \$F1 takes the same table back as its payload		

What is configured in each virtual drive. This console boots images rather than serving drives, so CONFIG leaves these screenless — but they answer.

SET_DEVICE_FULLPATH

\$E2

DEVICE	\$70	NPARAM	3
PARAMS	bytes: device slot, host slot, mode (1 = read)		
PAYLOAD	256 bytes: the path, NUL-padded		
REPLY	ACK		

Names the file for a device slot: the first half of booting. The path is exactly 256 bytes; the library pads a ROM string, and the cartridge pads its own path buffer, which is where a browser has the name already.

6502 ASSEMBLY

```
lda    #FNCSDFP
sta    FNCMD
lda    #3
sta    FNNPR
jsr    FNBEG
lda    #0
jsr    FNPB          ; device slot 0
lda    #0
jsr    FNPB          ; host slot 0
lda    #FMREAD
jsr    FNPB
lda    #(PATH)&$FF
sta    FNPTRL
lda    #(PATH)>>8
sta    FNPTRH
jsr    FNPTRH        ; padded to 256
jsr    FNGO
```

BATARI BASIC

```
cmd = FNCSDFP
npar = 3
gosub fnbeg
FNTX = 1
FNTX = 0
FNTX = 1
FNTX = 0
FNTX = 1
FNTX = 0
FNTX = 1
FNTX = FMREAD
FNH_PATH0 = FP_TX : rem path buffer, padded
gosub fngo
```

MOUNT_IMAGE / UNMOUNT_IMAGE

\$F8 / \$E9

DEVICE	\$70	NPARAM	2 / 1
PARAMS	\$F8: device slot, access mode (1 read); \$E9: device slot		
PAYLOAD	none		
REPLY	ACK when the mount — and, on this console, the push — completes		

The second half of booting, and the cartridge's most theatrical command: while it is outstanding the adapter streams the image to the cartridge over the same link, and BOOT_STATE and BOOT_PCT narrate the progress. The cartridge allows it sixty seconds. Section 11 has the swap that follows.

6502 ASSEMBLY

```
lda    #FNCMIMG
sta    FNCMD
lda    #2
```

```

sta      FNNPR
jsr      FNBEG
lda      #0
jsr      FNPB
lda      #FMREAD
jsr      FNPB
jsr      FNGO
W1:      lda      FNBST      ; then watch BOOT_STATE
        cmp      #FNBRDY
        bne      W1

```

BATARI BASIC

```

cmd = FNCMIMG
npar = 2
gosub fnbeg
FNTX = 1
FNTX = 0
FNTX = 1
FNTX = FMREAD
gosub fngo
staged
drawscreen
if FNBST <> FNBRDY then goto staged

```

MOUNT_ALL / SET_BOOT_MODE / CONFIG_BOOT

\$D7 / \$D6 / \$D9

DEVICE	\$70	NPARAM	0 / 1 / 1
PARAMS	none		
PAYLOAD	none		
REPLY	ACK		

MOUNT_ALL mounts every configured slot; the other two select what the adapter offers at power-on on disk-serving platforms. On this console the claim, not a flag, decides who boots, and CONFIG never sends CONFIG_BOOT.

NEW_DISK

\$E7

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	numSectors (word), sectorSize (word), hostSlot, deviceSlot, filename[256]		
REPLY	ACK on creation		

Creates a blank image on a host: a full-struct payload with the 256-pad rule again.

Directories

OPEN_DIRECTORY

\$F7

DEVICE	\$70	NPARAM	1
PARAMS	byte: host slot		
PAYLOAD	256 bytes: the path, a NUL, then an optional *.BIN-style filter, the rest NUL		
REPLY	ACK		

Opens a directory on a mounted host; the filter is applied on the adapter. CONFIG keeps the filter in path buffer 1 and the directory in 0, because the payload wants both in one block.

6502 ASSEMBLY

```
lda    #FNCODIR
sta    FNCMD
lda    #1
sta    FNNPR
jsr    FNBEG
lda    #0
jsr    FNPB          ; host slot
jsr    FNWTX         ; the path buffer, padded to 256
jsr    FNGO
```

BATARI BASIC

```
cmd = FNCODIR
npar = 1
gosub fnbeg
FNTX = 1
FNTX = 0
FNH_PATH0 = FP_TX
gosub fngo
```

READ_DIR_ENTRY

\$F6

DEVICE	\$70	NPARAM	2
PARAMS	bytes: maxlen (the width to crunch the name to — 30, not 31), flags (0)		
PAYLOAD	none		
REPLY	one entry, NUL-terminated; a trailing / marks a subdirectory; the end is two \$7F bytes		

One entry per transaction; the entry advances the cursor itself. At a maxlen of exactly 31 the firmware prepends icon bytes. Never read past the end: a fujinet-pc SD host answers .. forever and every later seek is refused.

6502 ASSEMBLY

```
lda    #FNCRDIR
sta    FNCMD
lda    #2
sta    FNNPR
jsr    FNBEG
lda    #30
jsr    FNPB
lda    #0
jsr    FNPB
jsr    FNGO
jsr    FNEOF          ; Z: the end
```

BATARI BASIC

```
cmd = FNCRDIR
npar = 2
gosub fnbeg
FNTX = 1
FNTX = 30
FNTX = 1
FNTX = 0
gosub fngo
if FNRPLY[0] <> 127 then goto entry
if FNRPLY[1] = 127 then goto atend
entry
```

SET_DIRECTORY_POSITION /
GET_DIRECTORY_POSITION

\$E4 / \$E5

DEVICE	\$70	NPARAM	1 / 0
PARAMS	\$E4: one word , the absolute entry index		
PAYLOAD	none		
REPLY	\$E5: a word, the current index; \$E4: ACK		

Seek and tell for the open directory: paging in one command. The seek's parameter is one two-byte parameter, not two one-byte ones; the wrong shape is a NAK with nothing to point at.

6502 ASSEMBLY

```
lda    #FNCS DPS
sta    FN CMD
lda    #1
sta    FNNPR
jsr    FNBEG
lda    PAGE0          ; index, low
ldx    #0              ; high
jsr    FNPW            ; ONE param of TWO bytes
jsr    FNGO
```

BATARI BASIC

```
cmd = FNCS DPS
npar = 1
gosub fnbeg
FNTX = 2
FNTX = page0
FNTX = 0
gosub fngo
```

CLOSE_DIRECTORY

\$F5

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	ACK		

Closes the walk. Directories are a scarce resource on the adapter; close what you open.

Files, prefixes and paths

COPY_FILE

\$D8

DEVICE	\$70	NPARAM	2
PARAMS	bytes: source host, destination host — both 1-based		
PAYLOAD	exact length , no padding: sourcepath destdir/		
REPLY	ACK when the copy completes; the cartridge allows sixty seconds		

Host-to-host copy without a byte passing through the console. The payload is read as a string, so it goes at exact length — one padding NUL would land inside the destination — and its host parameters are 1-based, alone among slot parameters. When the destination ends in /, the adapter appends the source's basename itself, which is why CONFIG's working directory keeps its trailing slash.

SET_HOST_PREFIX / GET_HOST_PREFIX

\$E1 / \$E0

DEVICE	\$70	NPARAM	1
PARAMS	byte: host slot; SET carries the prefix as its payload		
PAYLOAD	none		
REPLY	GET: the prefix		

A per-host working directory prepended to relative paths.

GET_DEVICE_FULLPATH

\$DA

DEVICE	\$70	NPARAM	1
PARAMS	byte: device slot		
PAYLOAD	none		
REPLY	the stored path		

Reads back what SET_DEVICE_FULLPATH stored.

Appkeys

OPEN_APPKEY

\$DC

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	6 bytes: creator (word), app, key, mode (0 read, 1 write), reserved		
REPLY	ACK; NAK with no SD card, a creator of 0, or a bad mode		

Selects which key the next read or write touches. The payload is one packed struct; leave off the reserved byte and the adapter waits for it, which reads back as a timeout. The file is /FujiNet/CCCCAAKK.key; the value cap is 64 bytes.

6502 ASSEMBLY

```
lda    #FNCAKOP
sta    FN CMD
lda    #0
sta    FNNPR
jsr    FNBEG
lda    #AKCREAT      ; creator, low then high
```

```

sta      FNTX
lda      #0
sta      FNTX
lda      #AKAPP
sta      FNTX
lda      #AKKEY
sta      FNTX
lda      #AKMRD          ; mode
sta      FNTX
lda      #0              ; reserved: not optional
sta      FNTX
jsr      FNGO

```

BATARI BASIC

```

cmd = FNCAKOP
npar = 0
gosub fnbeg
FNTX = AKCREAT
FNTX = 0
FNTX = AKAPP
FNTX = AKKEY
FNTX = AKMRD
FNTX = 0
gosub fngo

```

READ_APPKEY / WRITE_APPKEY / CLOSE_APPKEY

\$DD / \$DE / \$DB

DEVICE	\$70	NPARAM	0 / 1 / 0
PARAMS	WRITE: byte, the length		
PAYLOAD	WRITE: the value, up to 64 bytes		
REPLY	READ: a word length, then the value; the others ACK		

READ_APPKEY's reply is length-prefixed on this bus. A successful write resets the selection, so each write needs a fresh open in write mode. Section 12 has both directions.

6502 ASSEMBLY

```

lda      #FNCAKRD
sta      FNCMD
lda      #0
sta      FNNPR
jsr      FNBEG
jsr      FNGO
lda      #3
ldx      #2          ; past the length word
ldy      #FNTCOL
jsr      FNRRLPL

```

BATARI BASIC

```

cmd = FNCAKRD
npar = 0
gosub fnbeg
gosub fngo
len = FNRPLY[0]
ch = FNRPLY[2] : rem the first byte of the value

```

Utilities

GENERATE_GUID

\$BB

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	a 36-character UUID and a NUL		

A fresh printable unique id. RANDOM_NUMBER (\$D3) is not dispatched on this build; the adapter's entropy reaches the console through a hash, or a GUID.

BATARI BASIC

```
cmd = FNCGUID
npar = 0
gosub fnbeg
gosub fngo
FNH_TROW = 5
for l = 0 to 11
  FNH_TCHR = FNRPLY[l]
next
gosub fnend
```

BASE64 / HASH: INPUT, COMPUTE, LENGTH, OUTPUT

\$D0--\$C9 / \$C8--\$C2

DEVICE	\$70	NPARAM	varies
PARAMS	INPUT: word, the byte count; HASH COMPUTE: byte, the algorithm (0 MD5, 1 SHA-1, 3 SHA-256, 4 SHA-512); LENGTH and OUTPUT: byte, 1 = hex text		
PAYLOAD	INPUT: the bytes		
REPLY	LENGTH: the size; OUTPUT: the result		

Four-step pipelines on the adapter: feed bytes with INPUT, as often as needed; COMPUTE; ask LENGTH; drain with OUTPUT. The hash set is HASH_INPUT \$C8, HASH_COMPUTE \$C7, HASH_LENGTH \$C6, HASH_OUTPUT \$C5; HASH_COMPUTE_NO_CLEAR (\$C3) keeps the input for another algorithm and HASH_CLEAR (\$C2) empties it. Base64 has an encode set — INPUT \$D0, COMPUTE \$CF, LENGTH \$CE, OUTPUT \$CD — and a decode set — \$CC, \$CB, \$CA, \$C9 — of the same four steps. Section 12 has the hash in both languages.

QR CODE: INPUT, ENCODE, LENGTH, OUTPUT, CLEAR

\$BC / \$BD / \$BE / \$BF / \$BA

DEVICE	\$70	NPARAM	varies
PARAMS	INPUT: word count; ENCODE: version (0 = auto), error-correction level (0–3), shorten; LENGTH: output mode (0 binary, 2 bitmap); OUTPUT: word count		
PAYLOAD	INPUT: the text		
REPLY	LENGTH: a 4-byte size; OUTPUT: the module data		

The adapter renders a QR matrix. A version-1 code is 21 × 21 modules, and the playfield is 40 bits wide, so it fits, at two playfield bits per module and a row per pair of lines.

Answered with a NAK on this build

These `fujiiCommandID.h` opcodes fall through every dispatch table in the RS232 build and earn a NAK: `ENABLE_UDPSTREAM ($F0)`, `SET_BAUDRATE ($EB)`, `UNMOUNT_HOST ($E6)`, `SET_HSIO_INDEX ($E3)`, `SET_SIO_EXTERNAL_CLOCK ($DF)`, `ENABLE_DEVICE` and `DISABLE_DEVICE ($D5, $D4)`, `RANDOM_NUMBER ($D3)`, `GET_TIME ($D2 — use the clock device)`, `DEVICE_ENABLE_STATUS ($D1)`, `GET_HEAP ($C1)`, `GET_DEVICE1..10_FULLPATH ($A0—$A9)`, `UPDATE_FIRMWARE ($90)` and `HSIO_INDEX ($3F)`. Most are other platforms' bus tuning.

Every command devices \$71–\$78 answer, verified against the RS232 network dispatcher in `network.cpp`. Most opcodes are the ASCII letter of their name — 'O'pen, 'R'ead, 'W'rite — a habit from the Atari. The connection, the devicespec grammar and each protocol's behaviour belong to *The FujiNet Network Protocol Handbook*; these are the cards. Cap a read at 512 bytes, one slice of the window, and a write's payload at what fits the 320-byte TX stream.

The lifecycle five

NET_OPEN

\$4F 'O'

DEVICE	\$71	NPARAM	2
PARAMS	bytes: access mode (4 read, 8 write, 12 read/write), translation (0 none, 1 CR, 2 LF, 3 CR LF)		
PAYLOAD	the devicespec, N: and the URL, at its own length		
REPLY	ACK when the connection is established		

On HTTP, mode 4 and mode 12 are both GET — 12 allows headers to be set through the channel modes — and the fetch is deferred to the first STATUS. Modes 13 POST, 14 PUT with headers, 5 DELETE. A URL that streams from a cartridge path buffer goes in with `FNWRAW` or `FNH_PATHO = FP_TXRAW`, unpadded: padding in the middle of a URL corrupts the request.

6502 ASSEMBLY

```

        lda    #NETDEV
        sta    FNDEV
        lda    #NCOPEN
        sta    FNCMD
        lda    #2
        sta    FNNPR
        jsr    FNBEG
        lda    #NMHTTGP      ; 12: GET, headers allowed
        jsr    FNPB
        lda    #NTRNONE
        jsr    FNPB
        ldy    #0
U1:     lda    TURL,y        ; "N:https://..."
        beq    U2
        sta    FNTX
        iny
        bne    U1
U2:     jsr    FNGO

```

BATARI BASIC

```

dev = NETDEV
cmd = NCOPEN
npar = 2
gosub fnbeg
FNTX = 1
FNTX = NMHTTGP
FNTX = 1
FNTX = NTRNONE
for l = 0 to url_length - 1
FNTX = url[l]
next
gosub fngo

```

NET_STATUS

\$53 'S'

DEVICE	\$71	NPARAM	2
PARAMS	two zero bytes — the build reads the request type from the second		
PAYLOAD	none		
REPLY	4 bytes: avail (word), connected, devstatus (1 success, 136 end of file)		

The heartbeat. A terminal gates on `connected`; a fetch-and-render program gates on `devstatus`; everyone reads `avail`, and reads it until two readings agree.

6502 ASSEMBLY

```
lda    #NCSTAT
sta    FNCMD
lda    #2
sta    FNNPR
jsr    FNBEG
lda    #0
jsr    FNPB
lda    #0
jsr    FNPB
jsr    FNGO
lda    FNRPLY+NSAVLO    ; avail, low
ldx    FNRPLY+NSAVHI    ; avail, high
ldy    FNRPLY+NSDEVST    ; 1 = success
```

BATARI BASIC

```
cmd = NCSTAT
npar = 2
gosub fnbeg
FNTX = 1
FNTX = 0
FNTX = 1
FNTX = 0
gosub fngo
avlo = FNRPLY[0]
avhi = FNRPLY[1]
if FNRPLY[3] <> 1 then goto fail
```

NET_READ

\$52 'R'

DEVICE	\$71	NPARAM	1
PARAMS	one word: the byte count — clamp to <code>avail</code> and to 512		
PAYLOAD	none		
REPLY	exactly that many bytes, in the reply window		

The adapter sends exactly `want` bytes on ACK; a short or failed read comes back as a NAK. Render or copy before any other transaction repaints the window.

6502 ASSEMBLY

```
lda    #NCREAD
sta    FNCMD
lda    #1
sta    FNNPR
jsr    FNBEG
lda    WANT            ; low
ldx    #0              ; high
jsr    FNPW
jsr    FNGO
; the bytes are at FNRPLY; RXLEN says how many
```

BATARI BASIC

```
cmd = NCREAD
npar = 1
gosub fnbeg
FNTX = 2
FNTX = want
FNTX = 0
gosub fngo
```

NET_WRITE

\$57 'W'

DEVICE	\$71	NPARAM	1
PARAMS	word: the byte count		
PAYLOAD	exactly that many bytes		
REPLY	ACK		

The count rides twice, as the parameter and as the payload the cartridge measures. Keep it well inside the 320-byte TX stream.

6502 ASSEMBLY

```
        lda     #NCWRITE
        sta     FNCMD
        lda     #1
        sta     FNNPR
        jsr     FNBEG
        lda     #MSGE-MSG
        ldx     #0
        jsr     FNPW
        ldy     #0
W1:     lda     MSG,y
        sta     FNTX
        iny
        cpy     #MSGE-MSG
        bne     W1
        jsr     FNGO
```

BATARI BASIC

```
cmd = NCWRITE
npar = 1
gosub fnbeg
FNTX = 2
FNTX = msg_length
FNTX = 0
for l = 0 to msg_length - 1
FNTX = msg[l]
next
gosub fngo
```

NET_CLOSE

\$43 'C'

DEVICE	\$71	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	ACK, and an empty reply window		

Hang up. A program that reopens per request closes at the **start** of the next request, never between a READ and the rendering that follows it. The first close of a session earns a NAK, which is harmless.

BATARI BASIC

```
cmd = NCCLOSE
npar = 0
gosub fnbeg
gosub fngo
```

Structured channels, seek and line discipline

NET_CHANNEL_MODE / NET_PARSE /
NET_QUERY \$FC / \$50 'P' / \$51 'Q'

DEVICE	\$71	NPARAM	2 / 0 / 0
PARAMS	CHANNEL_MODE: byte (ignored), then the mode (0 protocol, 1 JSON)		
PAYLOAD	QUERY: the path, such as /0/name		
REPLY	QUERY: the value as text — then STATUS and READ it		

Switch an open connection to the adapter’s JSON parser (the mode is the **second** parameter), parse the body, then query one element: the difference between parsing JSON on a 6507 and not. The games in this family ask their servers for binary with ?bin=1 instead.

6502 ASSEMBLY

```
lda    #NCJSON
sta    FNCMD
lda    #2
sta    FNNPR
jsr    FNBEG
lda    #0
jsr    FNPB
lda    #1                ; JSON
jsr    FNPB
jsr    FNGO
lda    #NCPARSE
sta    FNCMD
lda    #0
sta    FNNPR
jsr    FNBEG
jsr    FNGO
lda    #NCQUERY
sta    FNCMD
jsr    FNBEG
ldy    #0
Q1:    lda    PATH,y      ; "/0/name"
      beq    Q2
      sta    FNTX
      iny
      bne    Q1
Q2:    jsr    FNGO        ; then STATUS, READ
```

BATARI BASIC

```
cmd = NCJSON
npar = 2
gosub fnbeg
FNTX = 1
FNTX = 0
FNTX = 1
FNTX = 1
gosub fngo
cmd = NCPARSE
npar = 0
gosub fnbeg
gosub fngo
```

```

cmd = NCQUERY
gosub fnbeg
for l = 0 to path_length - 1
  FNTX = path[l]
next
gosub fngo

```

NET_SET_CHANNEL_MODE

\$4D 'M'

DEVICE	\$71	NPARAM	2
PARAMS	byte (ignored), then the HTTP channel mode: 0 body, 1 collect headers, 2 get headers, 3 set headers, 4 POST data		
PAYLOAD	none		
REPLY	ACK; NAK on a connection that is not HTTP		

The HTTP-only channel switch. In mode 3 a WRITE sets a request header; in mode 4 a WRITE supplies the POST body; in mode 2 a READ returns the collected response headers. Mode 0 is the page.

NET_SEEK / NET_TELL

\$25 '%' / \$26 '&'

DEVICE	\$71	NPARAM	1 / 0
PARAMS	SEEK: a 4-byte offset		
PAYLOAD	none		
REPLY	TELL: a 4-byte position		

Random access where the protocol supports it: TNFS files, HTTP ranges.

NET_TRANSLATION /

NET_SET_EOL /

NET_SET_INT_RATE

\$54 'T' / \$4C 'L' / \$5A 'Z'

DEVICE	\$71	NPARAM	2
PARAMS	TRANSLATION: byte (ignored) then the code; SET_EOL: the EOL pair, a first byte of 0 restores the default; SET_INT_RATE: byte (ignored) then the rate		
PAYLOAD	none		
REPLY	ACK		

Line-ending translation and custom terminators after OPEN. SET_INT_RATE paces a bus interrupt on platforms that have one; the cartridge has no line back to the console, so it is accepted and irrelevant. Polling ACKSEQ is the only completion mechanism here.

Filesystems, sockets and credentials

NET_RENAME / DELETE / LOCK / UNLOCK / MKDIR / RMDIR / CHDIR / GETCWD									
		\$20	\$21	\$23	\$24	\$2A	\$2B	\$2C	\$30
DEVICE	\$71								
PARAMS	none								
PAYLOAD	the target path; RENAME: old,new								
REPLY	ACK; GETCWD answers the path								

Filesystem verbs on the protocols that have files — TNFS, FTP, SMB. CHDIR and GETCWD move and report the working directory.

NET_CONTROL / NET_CLOSE_CLIENT									
						\$41	'A'	/	\$63
							'c'		
DEVICE	\$71								
PARAMS	none								
PAYLOAD	none								
REPLY	ACK								

TCP server duty: a listening N:TCP://:port/ accepts a waiting client with 'A' and hangs up on that client with 'c' while keeping the listener. The console can be the BBS.

NET_GET_REMOTE / NET_SET_DESTINATION / NET_USERNAME / NET_PASSWORD									
						\$72	'r'	/	\$44
							'D'	/	\$FD
								/	\$FE
DEVICE	\$71								
PARAMS	none								
PAYLOAD	SET_DESTINATION: host:port; USERNAME and PASSWORD: the credential								
REPLY	GET_REMOTE: the last datagram's sender								

UDP's who-said-that and talk-to-them-instead, plus stored credentials for the next OPEN on protocols that log in — FTP, SSH, SMB. Send them before the OPEN.

Answered with a NAK on this build

Enumerated but not dispatched, so a NAK and no example:

NET_GET_DSTATS_VALUE (\$FF), NET_SET_PARAMETERS (\$FB), NET_SET_CHANNEL (\$FA), NET_SET_HSIO_INDEX (\$E3), NET_QUERY_ALT and NET_PARSE_ALT (\$81, \$80), NET_GET_ERROR (\$45 — read the STATUS device byte instead), and NET_HSIO_INDEX (\$3F).

The mailbox is a transport, not a translator: whatever byte goes into the DEVICE register goes on the wire. Beyond the FUJI and network devices, the RS232 build answers four more — disks, a printer, a clock and a Hayes-style modem. Only the clock is exercised by a program in this handbook; the others are reachable capability, documented as such, their behaviour transcribed from the RS232 handlers.

Disk — devices \$31–\$38

Eight disk devices, \$31 plus the drive number. On this build every sector is 512 bytes and the offset is `sector × 512`. Read and write name the sector in the first parameter; a missing parameter throws on the adapter, so always supply it.

DISK_READ / DISK_WRITE / DISK_PUT \$52 / \$57 / \$50

DEVICE	\$31	NPARAM	1
PARAMS	the sector number, as one two-byte parameter		
PAYLOAD	WRITE and PUT: 512 bytes		
REPLY	READ: 512 bytes; WRITE and PUT: ACK		

\$53 is not a status call on this build: it falls through into the write path, so never send it to a disk expecting a status. With no image mounted, READ answers NAK.

6502 ASSEMBLY

```

lda    #DSKDEV          ; disk 1
sta    FNDEV
lda    #$52              ; READ
sta    FNCMD
lda    #1
sta    FNNPR
jsr    FNBEG
lda    SECL0
ldx    SECHI
jsr    FNPW
jsr    FNGO
; 512 bytes at FNRPLY, one slice

```

BATARI BASIC

```

dev = DSKDEV
cmd = $52
npar = 1
gosub fnbeg
FNTX = 2
FNTX = seclo
FNTX = sechi
gosub fngo

```

DISK_FORMAT / FORMAT_MEDIUM / PERCOM_READ / PERCOM_WRITE

\$21 / \$22 / \$4E / \$4F

DEVICE	\$31	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	FORMAT: a bad-sector map; PERCOM_READ: a 12-byte geometry block		

Vestigial on this build: the two FORMATS write nothing to the image, and the PERCOM block is a 12-byte scratch area with no effect on geometry.

Printer — device \$40

PRINTER_WRITE / PRINTER_PUT / PRINTER_STATUS

\$57 / \$50 / \$53

DEVICE	\$40	NPARAM	0 / 0 / 1
PARAMS	STATUS: byte, request type, required but ignored		
PAYLOAD	WRITE and PUT: the bytes to print, fewer than 255		
REPLY	WRITE and PUT: ACK; STATUS: 4 bytes		

Output goes to a paper file on the adapter, retrieved through its web interface, in whatever printer emulation is configured. A disabled printer does not reply at all, and the cartridge reports a timeout.

6502 ASSEMBLY

```
        lda    #PRNDEV
        sta    FNDEV
        lda    #$57          ; WRITE
        sta    FNCMD
        lda    #0
        sta    FNNPR
        jsr    FNBEG
        ldy    #0
P1:     lda    LINE,y
        beq    P2
        sta    FNTX
        iny
        bne    P1
P2:     jsr    FNGO
```

BATARI BASIC

```
dev = PRNDEV
cmd = $57
npar = 0
gosub fnbeg
for l = 0 to line_length - 1
  FNTX = line[l]
next
gosub fngo
```

Clock — device \$45

Present when APETime is enabled in the adapter's configuration, which it is by default. Reads come in many formats; a parameter of 1 on a read selects the alternate zone set by SETTZ.

GETTIME / GET_ISO_LOCAL / GET_ISO_UTC

\$93 / \$49 'I' / \$5A 'Z'

DEVICE	\$45	NPARAM	0
PARAMS	optional byte: 1 = the alternate zone		
PAYLOAD	none		
REPLY	GETTIME: 7 bytes, binary; the ISO forms: 2026-09-15T20:45:31+0000 and a NUL		

Section 12 shows both. The other formats: GET_PRODOS (\$50, 4 bytes), GET_SOS (\$53, text), GET_SIMPLE_HUNDREDTHS (\$4D, 8 bytes), GET_GENERAL (\$47, the zone string), GETTZ_LEN (\$4C), and GETTZTIME (\$9A).

6502 ASSEMBLY

```
lda    #CLKDEV
sta    FNDEV
lda    #CKISOL
sta    FNCMD
lda    #0
sta    FNNPR
jsr    FNBEG
jsr    FNGO
lda    #3          ; the date, ten characters
ldx    #0
ldy    #10
jsr    FNRRPL
```

BATARI BASIC

```
dev = CLKDEV
cmd = CKISOL
npar = 0
gosub fnbeg
gosub fngo
FNH_TROW = 3
for l = 0 to 9
  FNH_TCHR = FNRRPLY[l]
next
gosub fnend
```

SETTZ

\$99

DEVICE	\$45	NPARAM	0
PARAMS	none		
PAYLOAD	a POSIX zone string, such as CST6CDT		
REPLY	ACK		

Sets the zone the local-time reads use. SETTZ_ALT (\$74) and SETTZ_ALT2 (\$54) set the alternate zone the reads select with a parameter of 1.

Modem — device \$50

A Hayes-style virtual modem that dials TCP instead of phone lines: ATDT host:port opens a socket, ATNET1 runs the session through telnet negotiation. Ten packet commands configure and poll it; the character stream itself is unframed. Reachable but unexercised on this console.

**MODEM_STATUS / MODEM_WRITE /
MODEM_STREAM**

\$53 / \$57 / \$58

DEVICE	\$50	NPARAM	1 / 0 / 0
PARAMS	STATUS: byte, ignored but required		
PAYLOAD	WRITE: an AT command line, or data once connected		
REPLY	STATUS: 2 bytes, the second carrying the handshake bits (\$CD connected with data, \$CC connected idle, \$00 idle); WRITE: ACK; STREAM: a 9-byte baud table		

The other seven — CONTROL \$41, CONFIGURE \$42, SET_DUMP \$44, LISTEN \$4C, UNLISTEN \$4D, BAUDRATELOCK \$4E, AUTOANSWER \$4F — configure the virtual modem. Disabled in the configuration, it does not reply.

Below the FujiBus devices is the cartridge's own protocol: the register file and the hotspots the earlier sections used, the blit port, the path buffers, and the boot doorbells. This is all of it in one place. None of these reaches the adapter; each is decoded by the RP2040 on the console's bus, and takes effect before the next fetch.

Registers

Arm with a store to $\$1D00$ + the register number; commit with a store of the value to $\$1DFF$. The library's `FNRW` is the pair; in BASIC, `FNA_*` are the arm addresses and `FNCMT` the commit.

Register	Name	Value
$\$00$	DEVICE	the device id
$\$01$	CMD	the command
$\$02$	NPARAM	parameters at the head of the TX stream
$\$05$	DATA_RST	any: rewind the TX stream
$\$06$	RXSLICE	0 or 1: the reply slice shown at $\$1B00$; SLICE_ECHO is painted last
$\$10$	SEQ	ACKSEQ + 1, wrapping 255 to 1: launch
$\$11$	BOOTLOCK	$\$B5$: arm the swap of a staged image
$\$12, \13	BOOTSEL_1, BOOTSEL_2	$\$B5$ then $\$4A$ as consecutive writes: reboot the RP2040 into its USB bootloader for a firmware update. One stray access can never do it.

One-shot operations

The top half of the control page. The store's data byte is the operand, and none of these need a commit.

Store to	Name	Effect
$\$1D80 + b$	BANK	serve bank <i>b</i> at $\$1000$; <i>b</i> up to $\$6F$
$\$1DF0$	TROW	begin composing text row = data
$\$1DF1$	TCHR	append the character; saturates at twelve
$\$1DF2$	TEND	render the row into the planes; TEXTGEN changes when it has
$\$1DF3$	PATH_CH	append the character to the selected path buffer
$\$1DF4$	PATH_OP	a path-buffer operation, below
$\$1DF5 - \$1DF9$	BLIT_SL, SH, DL, DH, CNT	the blit's source offset, destination offset and count
$\$1DFA$	BLIT_GO	fire the blit with transform = data; BLITGEN changes when it has landed

\$IDFC, \$IDFD	ARM1, ARM2	\$B5 then \$4A: open the decode gate
\$IDFE	SWAP	serve the staged image; only after BOOTLOCK
\$IDFF	COMMIT	commit the armed register with the data

The path buffers

Four 256-byte buffers in the cartridge, one selected at a time. `PATH_CH` appends; `PATH_OP` with one of these values acts:

Value	Name	Effect
0	RST	empty the selected buffer
1	POP	drop the last path component, keeping the <code>/</code> : <code>/a/b/c/</code> becomes <code>/a/b/</code> , and <code>/</code> stays <code>/</code>
2	TX	emit the buffer into the TX stream, NUL-padded to exactly 256 bytes
3	TXRAW	emit just its bytes, unpadded — for a URL
4	POPCH	drop one character: the keyboard's backspace
5–8	SEL0–SEL3	subsequent operations act on that buffer; 3 is the edit scratch
9	COMMIT	selected = the scratch: accept an edit
10	SEED	scratch = selected: begin an edit; cancelling is simply neither

`PATHLEN` at `$1F17` is the selected buffer's length, republished on every change and every select, which is how a program pads the rest of a 256-byte payload after the cartridge has emitted the directory part raw.

6502 ASSEMBLY

```

lda    #FP_SEL0      ; buffer 0
sta    FNRSEL+FH_PATH0
lda    #FP_RST
sta    FNRSEL+FH_PATH0
lda    #'/'
sta    FNRSEL+FH_PATHC
; ... later, inside a transaction:
lda    #FP_TX        ; 256 bytes, one store
sta    FNRSEL+FH_PATH0

```

BATARI BASIC

```

FNH_PATH0 = FP_SEL0
FNH_PATH0 = FP_RST
FNH_PATHC = 47
rem ... later, inside a transaction:
FNH_PATH0 = FP_TX

```

The blit port

Six stores: source offset into the reply window (two bytes), destination offset into the text planes (two), a count, and the transform, which fires it. What each field means depends on the transform.

Transform	Name	What it does
-----------	------	--------------

0	RAW	copy <i>cnt</i> bytes from the reply to the planes unchanged
1	TEXT	compose text row <i>dst</i> from up to twelve ASCII bytes at reply offset <i>src</i>
2	FIELD	a 10 × 10 game field at reply offset <i>src</i> (0 sea, 1 hit, 2 miss) into ten text rows from <i>dst</i> , with a row digit in column 0; <i>cnt</i> is the cursor cell or \$FF for none
3	HULLS	<i>cnt</i> ship placements (<i>pos</i> + 100 × <i>dir</i>) at reply offset <i>src</i> as # over the composed board, then paint from row <i>dst</i>
4, 5, 6	SEA, CELL, PAINT	compose a board that is not in the reply: fill with sea; set <i>board[cnt]</i> to the low byte of <i>src</i> ; paint it into ten rows from <i>dst</i>
7	PATH	<i>cnt</i> characters of the selected path buffer from offset <i>src</i> into text row <i>dst</i> : how a program sees what it typed
8	TCELL	one character, the low byte of <i>src</i> , into one cell, <i>dst</i> = row × 12 + column: how a list moves its cursor
9	CARD	five playing cards from <i>hand[11]</i> at reply offset <i>src</i> — two lowercase bytes each, rank then suit — across the top two text rows from <i>dst</i> ; <i>cnt</i> bit 0 draws the first face down
10	PFCLR	clear kinds <i>src</i> (a mask) in playfield slot <i>dst</i>
11	PFIELD	a game field at reply offset <i>src</i> into playfield slot <i>dst</i> : hits and misses
12	PFHULL	<i>cnt</i> placements at reply offset <i>src</i> OR-ed into slot <i>dst</i> 's hull plane
13	PFCELL	one cell <i>cnt</i> of slot <i>dst</i> : set the kinds in the mask <i>src</i> , or clear them if bit 7 is set

The playfield transforms (10–13) compose the six tables the Battleship board kernel reads; Section 14 explains them. The cell glyphs are . sea, x hit, o miss, # hull, + cursor.

6502 ASSEMBLY

```
; FNBARG: A = src low, X = dst low, Y = cnt; FNBLIT: fire A and wait
    lda    #GFIELD      ; reply offset of gamefield[100]
    ldx    #0            ; slot 0
    ldy    #0
    jsr    FNBARG
    lda    #FB_PFLD
    jsr    FNBLIT
```

BATARI BASIC

```
FNH_BLSL = 110 : rem reply offset of the field
FNH_BLSH = 0
FNH_BLDL = 0 : rem slot 0
FNH_BLDH = 0
FNH_BLCNT = 0
```

```
gen = FNBGEN
FNH_BLG0 = FB_PFLD
for l = 0 to 100
  if FNBGEN <> gen then goto landed
next
landed
```

CAUTION: A blit is a single-slot request. On the RP2040 the bus core hands it to the other core and there is no queue, so the next blit's argument stores would overwrite this one's before it ran. Wait for BLITGEN at \$1F19 to change before the next — bounded, because in emulation it has already changed by the time you look.

The card transform

A suit is not in ASCII, and at three pixels wide a heart, a spade and a club are nearly the same picture, so a card is a transform and not a glyph. Five cards sit on a six-pixel pitch — five of art, one of gap — filling pixels 2 to 30, which is planes 0 to 3; planes 4 and 5 are untouched, so a name or a purse can share the row. A card is two text rows tall, the rank above and the pip below, with the blank sixth line of the cell keeping them apart. The bed starts at pixel 2 because pixel 7 cannot be drawn by any client (Section 5). `hand[11]` is a C string: rank in `23456789tjqka`, suit in `hdcs`, `??` for a hole card, and the first NUL rank ends the hand.

The boot doorbells

BOOTLOCK is honoured only once an image is staged (`BOOT_STATE = 2`), and SWAP only once BOOTLOCK has been honoured; an unarmed swap hotspot is inert. BOOTSEL wants its two registers written consecutively with `$B5` then `$4A`, and the cartridge reboots into the RP2040's USB bootloader, from which a new firmware is copied as a file. There is no acknowledgement to poll.

Battleship is a complete networked game for the Video Computer System: a 6502 client for the FujiNet Battleship server, in the binary wire format the Intellivision, Astrocade, Arcadia and Channel F clients share. Up to four 10 × 10 boards on screen at once, in colour, with the fleet strips saying which ships are afloat; a lobby, a placement screen that rolls you a fleet, a menu on the RESET switch, a keyboard for your name, and sound — on a console with 128 bytes of RAM, no framebuffer, and a playfield with one colour per line. It is 7,800 lines of source in seven banks and a tail, and this section walks through every part of it. Appendix D prints it.

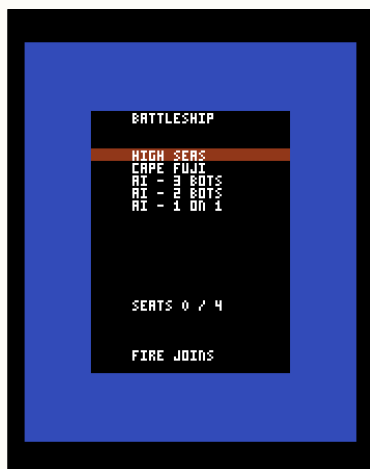


Figure 14. The lobby, against the live server: its tables, the seats, and FIRE to join.

The screen

```
YOUR TURN 45      the status row, synthesised
BOB >            names over the pair: left owner, a marker each, right
####            each seat's fleet: '#' afloat, '=' sunk; three lines tall

+-----+
| Q1    | Q2    |
+-----+
YOU      CAROL
####    ####
+-----+
| Q0    | Q3    |
+-----+
```

board pair A, 80 lines: two 10x10 boards of 8x8-pixel cells, abutting at pixel 80 with a two-clock black missile down the seam

pair B

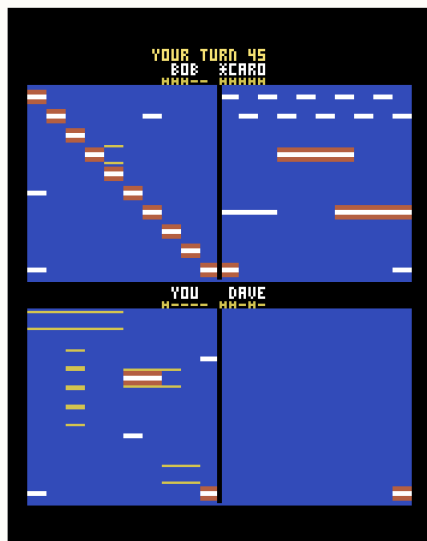


Figure 15. The four-seat screen, from the layout ROM: status row, names with the turn marker, fleet strips, and two pairs of boards — blue water, red-and-white hits, white misses, gold hulls and cursor, and the black divider down each seam.

Three or four seats use the family's quadrant convention: you bottom-left, the others clockwise from top-left. Two seats — the common case, a table against the server's AI — get one pair of 8×12 cells, square on a 4:3 set, and the lower half becomes text: the server's prompt, each fleet as pips, and a hint.

The status row is synthesised, because the server's prompt is empty for the whole of play: MISS YOU 45, HIT ENEMY, SUNK YOU 12 — the last result, whose turn, and your clock, which counts down locally and polls when it runs out.

Controls

	Lobby	Placement	In play
stick	—	move the ship	move the cursor
FIRE	ready up, a toggle	keep the ship where it is, or where you moved it	attack that cell
SELECT	poll now	rotate	poll now
RESET		the menu	the menu: resume, how to play, leave

RESET is a switch on this console, a bit in a RIOT register the program reads; it restarts nothing, and what it means is the client's to choose. A shot lands on every live enemy at once — the one rule this game does not share with the board game, and the reason the help page exists.

Seven banks and a tail

A bank switch replaces every byte of \$1000–\$17FF, so each bank carries its own copy of every module it calls. Only zero page crosses; the text planes, the playfield tables and the reply window are cartridge state, and survive.

Bank	File	What it does	Bytes
0	bslobby	the cold start and the table list	1016
1	bsgame	the board kernel, the cursor, the cues, a shot	1668
2	bsnet	one request, with the picture up, and back	1993
3	bsmenu	the RESET menu, the help, leaving	1092
4	bsname	the keyboard and the shared username	1201
5	bsplace	the roll, the five ships	2013
6	bscomp	compose the game screen, in passes	2025
tail	bstail	the trampoline, the transport, the cold stub	220

Seven banks pad to eight, 16384 bytes. The composer is a bank of its own because the game bank came in 961 bytes over with it inside; the network bank exists because the URL and transport code is 550 bytes; and three things every bank used to carry whether it wanted them or not were split behind feature flags before the fleet strips and the roll would fit anywhere: reading the stick (104 bytes) from reading a player record (118), streaming the reply into a row from the string helpers, and the sound cues, numbered so that each bank's are a prefix of the list and a constant says where its copy stops.

The transport and the four text primitives live in the tail, the one region every bank sees at the same address, and `tools/mkttail.py` reads their addresses out of the tail's own listing into a generated include, so there is no hand-kept list to go stale. The trampoline resets the stack on every switch, for the reason Section 13 gave.

Where to carry on

A switch is a jump, so “what the bank being entered should do” is data: an entry code in `BSSENT`. Sixteen of them, and every bank's entry point dispatches on the one it finds:

BSDEFS.INC: THE ENTRY CODES

```
ENCOLD EQU 0 ; bank 0: power-on
ENLOBBY EQU 1 ; bank 0: the table list is in the window
ENGOLD EQU 2 ; bank 1, then 6: a table was just joined
ENGAME EQU 3 ; bank 6: back from a poll: recompose
ENFETCH EQU 4 ; bank 2: /state, or the staged request
ENTABLE EQU 5 ; bank 2: /tables, then the lobby
ENMENU EQU 6 ; bank 3: the RESET menu
ENLEAVE EQU 7 ; bank 2: /leave, /tables, then the lobby
ENNAME EQU 8 ; bank 4: the keyboard
ENGRUN EQU 9 ; bank 1: composed: finish the frame, run
ENPLACE EQU 10 ; bank 5: the server wants our ships
ENPLFAIL EQU 11 ; bank 5: the /place was refused: again
ENGNEXT EQU 12 ; bank 1: composed, between frames: run
ENRESUME EQU 13 ; bank 6: the menu closed, mid-frame
ENTEXT EQU 14 ; bank 6: the text pass, mid-frame
ENGRUND EQU 15 ; bank 1: as ENGRUN, after a text bank: DINIT
```

The zero page

\$80—\$BF is the program's; the stack must stay above \$C0. \$80—\$8C and \$A8—\$A9 are the transport's; \$8D—\$A7 is the swap stub's landing ground in the library's clients, and since this program never swaps, it is the program's here.

BSDEFS.INC: THE CLIENT'S CELLS

```
INRPT EQU $8D ; INREPT's auto-repeat countdown
BSDGT EQU $8E ; BSDEC's digit and leading-zero latch
BSREQ EQU $8F ; the request in flight (RQ*)
BSTRIES EQU $90 ; the settle loop's bound
BSDLY EQU $91 ; its inter-poll delay, in memory because
; NFRAME clobbers X and Y
BSINP EQU $92 ; THIS FRAME'S input mask, in a cell of its own
BSAVL0 EQU $92 ; NET_STATUS bytes waiting -- only ever live
BSAVL1 EQU $93 ; inside a transaction, when no input runs
BSPRV0 EQU $94 ; the previous reading, for the settle loop
BSPRV1 EQU $95
BSNUM0 EQU $94 ; BSDEC's 16-bit number: never live with the
BSNUM1 EQU $95 ; settle loop, which composes nothing
BSRXL0 EQU $96 ; the READ's reply length, captured at once
BSRXL1 EQU $97
BSMIN0 EQU $98 ; the length this request will settle for
BSMIN1 EQU $99
BSERR EQU $9A ; the last failure
BSSTEP EQU $9B ; where it happened: 1 open 2 status 3 read
; 4 validate
BSROW EQU $9C ; text kernel: the row being drawn
ROWEND EQU $9C ; board kernel: one past the last cell row
BSLINE EQU $9D ; text kernel: ink lines left in the row
BSBAR EQU $9E ; text kernel: the red bar's row, or $FF
BSTMP2 EQU $9E ; board banks: a second scratch cell
SNDPTR EQU $9F ; the sound script step, 0 = silent
SNDCNT EQU $A0 ; frames left on that step
BSPOLL EQU $A1 ; frames until the next /state
BSFRAME EQU $A2 ; free-running frame counter
BSSENT EQU $A3 ; EN*: what the bank being entered should do
```

Three regions of it have rules. \$AE—\$B0 is the game bank's edge memory — the previous active player, status and last attack — and it must survive placement, or the first poll after placing hears a phantom shot. Cells with two names are aliases that are never live at the same time, and each carries the reason next to it: BSINP and BSAVL0 share a byte because the network's byte count is only live inside a transaction, when no input runs. And a bank's scratch must never alias a cell a loop in another bank is counting in: a "done" flag in the network bank once aliased the settle loop's delay counter, its hook ran inside every frame that loop drew, and the symptom was a cold start after the first poll.

The place bank needed four more cells for the roll and took BSSEL, BSERR2, BSBLINK and BSTMP2, each dead in that bank for a reason written beside it.

SHIPS[4] is the roll's flag: the slots fill in order, so \$FF in the last one means the roll is still running, and no cell was spent on a state byte.

The transport

The tail's transport is the library's, and the network bank builds on it. Five rules from the siblings are written into net.inc: CLOSE at the start of the next request, never after the READ, because every screen renders straight out of the window between polls; capture RXLEN at once, because it belongs to the most recent transaction; STATUS until two readings agree and the count reaches a minimum, because the adapter reports early; the fourth STATUS byte is where an HTTP error shows; and unsigned compares go through the carry.

NET.INC: THE WHOLE ROUND TRIP

```
APICALL: sta    BSREQ
         jsr    NSMIN
         jsr    NCLOSE           ; the PREVIOUS request's connection
         jsr    NOPEN
         beq    APOK
         rts
APOK:    lda    #0
         sta    BSPRV0
         sta    BSPRV1
         lda    #20             ; the settle loop's bound
         sta    BSTRIES
APSET:   jsr    NSTATUS
         beq    APS1
         jmp    APFAIL
; Enough yet? Two tests, and the first is what the Channel F port added after
; watching a reply settle a third of the way in.
APS1:    lda    BSAVL1           ; avail >= BSMIN, unsigned, through the carry
         cmp    BSMIN1
         bcc    APAGN
         bne    APS2
         lda    BSAVL0
         cmp    BSMIN0
         bcc    APAGN
APS2:    lda    BSAVL0           ; and the same reading twice running
         cmp    BSPRV0
         bne    APAGN
         lda    BSAVL1
         cmp    BSPRV1
         beq    APRD
APAGN:   lda    BSAVL0
         sta    BSPRV0
         lda    BSAVL1
         sta    BSPRV1
         lda    #3              ; ~3 frames between polls
         sta    BSDLY           ; NOT in X or Y: NFRAME is register-counted
APDLY:   jsr    NFRAME           ; either way and comes back with both gone
         dec    BSDLY
         bne    APDLY
         dec    BSTRIES
         bne    APSET
APFAIL:  jsr    NCLOSE           ; failed before the READ: nothing to keep
         lda    #$FF
         rts
APRD:    jsr    NREAD
         beq    APRD1
         jmp    APFAIL
; A validation failure AFTER the read returns nonzero WITHOUT closing: the
; next APICALL's leading NCLOSE retires the connection, and closing here would
; wipe a window the caller may still be rendering the last good reply from.
APRD1:   jmp    VALID8
```

NPG0 is FNG0 with a frame drawn per poll, and one frame drawn **before** the first look at the acknowledgement:

NET.INC: A FRAME BEFORE THE FIRST LOOK

```
NPG02:  lda    #NPQUANT
         sta    FNTMO
; ONE FRAME BEFORE THE FIRST LOOK. In emulation the cartridge answers inside
; the commit itself, so without this every step of a transaction -- and the
; setup of the next, a URL streamed a character at a time -- piled into the
; overscan of a single frame and ran it fifty lines long. A frame per step
; is what real hardware costs anyway, and a poll is ninety frames apart.
         jsr    NFRAME
; ACKSEQ is published LAST, after the reply, its length and the error code are
; all in place, so a match means everything behind it is already readable.
NPG03:  lda    FNACKS
         cmp    FNSEQ
```

A validation failure after the read returns without closing: the next request's leading CLOSE retires the connection, and closing here would wipe a window the caller may still be rendering the last good reply from.

The URL

Nothing is assembled in RAM. The cartridge counts the payload itself, so nothing tracks a length, which is the only reason a console with 128 bytes can build N:https://battleship.carr-designs.com/attack/47?table=ai1&player=VCS&bin=1&v=2 at all. The endpoint comes from ROM, the request path from one of six strings, a cell number through a decimal converter that emits digits straight into the TX page, and the table id and player name from two cartridge path buffers — pushed there once from the server's reply, and emitted raw into every later URL:

URL.INC: THE QUERY STRING FROM THE PATH BUFFERS

```
BUQ:    ldx    # (UQTAB)&$FF
        ldy    # (UQTAB)>>8
        jsr    UPUT
        lda    #PBTABLE
        jsr    FNWSEL
        jsr    FNWRAW
        ldx    # (UQPLR)&$FF
        ldy    # (UQPLR)>>8
        jsr    UPUT
        lda    #PBNAME
        jsr    FNWSEL
        jsr    FNWRAW
        ldx    # (UQBIN)&$FF
        ldy    # (UQBIN)>>8
        jmp    UPUT

        INCLUDE "../build/endpoint.inc"

UTABLES: DB    "tables?bin=1",0
USTATE:  DB    "state",0
UREADY:  DB    "ready",0
UPLACE:  DB    "place/",0
UATTCK:  DB    "attack/",0
ULEAVE:  DB    "leave",0
UQTAB:   DB    "?table=",0
UQPLR:   DB    "&player=",0
; v=2: the winner and the ship reveal at game over. No &be=1: the Go server's
; big-endian path is the road much less travelled.
UQBIN:   DB    "&bin=1&v=2",0
```

Request	URL
tables	tables?bin=1 — no table, no player
state	state?table=T&player=P&bin=1&v=2
ready	ready?... — a bare /ready toggles
place	place/P,P,P,P,P?... — each P is pos + 100 × dir
attack	attack/P?... — P is y × 10 + x; its reply is the next state
leave	leave?... , then a fresh tables

A staged request rides the poll: /attack, /ready and /place all return the next state, so there is no separate submit anywhere in this client. v=2 brings the winner and the ship reveal at game over.

The state record

The server's ?bin=1&v=2 reply is one record, parsed in place. Nothing copies. A 38-byte header is always present; what follows depends on the phase.

Offset	Size	Field
--------	------	-------

0	1	player count, at most 4
1	33	the prompt
34	1	status: 0 lobby, 1 placing, 10 started, 11 miss, 12 hit, 13 sunk, 99 over
35	1	your own status: 0 playing, 1 defeated, 2 viewing, 3 ready, 10 placing
36	1	the active player; \$FF nobody; 0 is you; at game over, the winner
37	1	seconds left on the move clock
38	1	in play: the last attack position
39	10	in play: your five ships, then the winner's five
49	115 each	in play: a record per player — name (9), status (1), the 100-cell field, five ships-left bytes

In the lobby the body is a server name and ten-byte records of name and status; during placement the records are ten bytes too, and **a program that indexes a game field in the placement phase reads garbage**. In play the second player's record starts past 256, out of an 8-bit index's reach, which is why `PLRECP` is a pointer. `VALID8` checks the reply's length against what the header implies — a minimum, since a longer reply is harmless and a shorter one means stale bytes — then caches the four bytes every bank keys on.

STATE.INC: A POINTER TO A PLAYER'S RECORD

```

PLRECP: lda    #(FNRPLY)>>8
        sta    FNPTRH
        lda    BSSTAT
        cmp    #STSTART
        bcs    PLR2
        cmp    #STPLACE
        beq    PLR1
        lda    #GLPLYRS        ; the lobby
        jmp    PLR1A
PLR1:   lda    #GIPLYRS        ; placement: ten-byte records
PLR1A:  cpx    #0
        beq    PLR9
PLR1B:  clc
        adc    #LBSTRID
        dex
        bne    PLR1B
        jmp    PLR9
PLR2:   lda    #GIPLYRS        ; in play: 115-byte records, 16 bits
        cpx    #0
        beq    PLR9
PLR2A:  clc
        adc    #PLSTRID
        bcc    PLR2B
        inc    FNPTRH
PLR2B:  dex
        bne    PLR2A
PLR9:   sta    FNPTL
        rts

```

The largest reply, four players in play, is 509 bytes: it fits slice 0 whole, which is why the reply window is 512.

The game bank

bsgame is the board kernel and the per-frame hook: tick the sound, take the text pass the composer owes, read the stick with auto-repeat, RESET to the menu, SELECT to poll now, the stick to move the cursor and FIRE to shoot, then the clock, and when the poll clock reaches zero, the switch to the network bank — taken here, inside the vblank, so that the network bank finishes this frame.

BSGAME.ASM: THE HOOK

```
APPVBL: jsr      SNDTICK
; The composer's text pass, owed from the last poll: taken here, at the top
; of a vblank, and the composer finishes this frame.
        bit      BSREDRW
        bpl      GA0
        lda      #ENTEXT
        sta      BSENT
        lda      #BANKCMP
        jmp      BSGOTO
GA0:     jsr      INREPT
        sta      BSINP
        and      #IN_RST
        beq      GA1
        lda      #ENMENU
        sta      BSENT
        lda      #BANKMNU
        jmp      BSGOTO
GA1:     lda      BSINP
        and      #IN_SEL
        beq      GA2
        lda      #0
        sta      BSPOLL
        lda      BSCLASS
        cmp      #CLPLAY
        bne      GA5
        lda      BSMYST
        lda      GA6
        lda      BSINP
        and      #INDIRS
        beq      GA3
        jsr      GMOVE
GA3:     lda      BSINP
        and      #IN_FIRE
        beq      GA6
        jsr      GFIRE
        jmp      GA6
GA5:     lda      BSCLASS
        lda      GA6
        bne      BSINP
        and      #IN_FIRE
        beq      GA6
        lda      #RQREADY
        sta      BSREQ2
        lda      #0
        sta      BSPOLL
        lda      #SNDSEL
        jsr      SNDFIRE
GA6:     jsr      GCLOCK
; The poll clock stops AT zero and the switch is taken here, inside the
; vblank: the network bank finishes this frame. The test comes before the
; decrement, or a due poll would wrap the clock to 255.
        lda      BSPOLL
        beq      GA7
        dec      BSPOLL
        rts
GA7:     lda      #ENFETCH
        sta      BSENT
        lda      #BANKNET
        jmp      BSGOTO
; -----
```

A shot is refused with a tone if the cell is already resolved on every live enemy, because the server refuses it silently and the turn never passes. Moving the cursor is a PFCELL blit that clears the gold bracket on every live board, a step, a click, and a blit that puts it back.

Every frame is 262 lines

The blanked bands are timed with the RIOT timer, and so is the visible one: the kernel's own lines add up to less than 192 and a timed pad at the bottom absorbs the difference, so an overrun moves nothing. Four things keep a network poll from blanking the screen or making it shudder:

- The network bank carries the board kernel and spends every wait for the cartridge drawing a frame out of the tables it is still holding.
- The overscan is waited out at the **start** of the next frame, not the end of the one before, so whatever a bank does between frames — stream a URL, recompose the boards, switch banks — is spent inside the overscan, as long as it fits in thirty lines. With the wait at the end, a played game measured 96 frames in 2,400 that were 273 to 310 lines.
- A switch into a frame is taken inside the vblank hook, and the bank entered finishes the frame. A switch that started a fresh frame from the hook would throw a 40-line frame at the set.
- A poll's recompose is two passes in a bank of its own: the boards between frames, in the overscan; the text a frame later, in the vblank, and only the rows a poll can change.

`TIMINT`, not `INTIM`, is what the waits read. Past zero the timer free-runs at a tick a cycle, and a seven-cycle `LDA INTIM / BNE` walks over the single cycle it reads zero on; `TIMINT`'s bit 7 latches on the underflow and stays latched, so an overrun costs exactly the overrun. Polling `INTIM` cost an overrunning frame 290 lines where the work was 151.

The seam line of a text row is a budget, not a glitch. A seam that runs past cycle 76 does not glitch; it silently costs a second scanline. The first cut of the text kernel reached a `LDA` at cycle 77 on one path, every seat row was seven lines instead of six, and the frame was 278 lines — a picture that rolls on anything that cares, and nothing about it looked wrong on screen.

The composer

`bscomp` composes the game screen in passes, because a poll's recompose is about 3,900 cycles and the overscan between frames has about 2,200. The boards go between frames from the reply window with four blits a seat; the AUX plane — hulls and cursor — in the next frame's vblank; the text a frame later, and only the rows a poll can change: the status row, the fleets, and the two turn-marker cells of each name row through a TCELL blit. A screen change is three passes more.

BSCOMP.ASM: THE PASS SCHEDULE

```
CENTRY: lda    BSENT
        cmp    #ENGSCOLD
        beq    CCOLD
        cmp    #ENRESUME
        beq    CRESUM
        cmp    #ENEXT
        beq    CTEXT
; ENGAGE: back from a poll. A failed poll leaves the reply window alone, so
; the boards are still the last good ones; only the status row says.
        lda    BSERR2
        beq    CW1
        jsr    GFALL
        jmp    CNEXT
CW1:    jsr    GCOMPB
CNEXT:  lda    #ENGNEXT
        sta    BSENT
```

```

        lda    #BANKGAM
        jmp    BSGOTO

CTEXT:  jsr    GCOMPT
CRUN:   lda    #ENGRUN
        sta    BSENT
        lda    #BANKGAM
        jmp    BSGOTO

CRESUM: lda    #1
        sta    BSREDRW      ; everything: the menu drew over it all
        jsr    GCOMPB
        lda    #ENGRUND     ; the menu's kernel was a text one
        sta    BSENT
        lda    #BANKGAM
        jmp    BSGOTO

CCOLD:  jsr    GCLEAR
        jmp    CNEXT

; APPVBL -- never called: this bank draws no frame.
APPVBL: rts

; -----

```

The result cue rides the pair of status and last-attack position, because the status repeats across polls when nobody has fired since; the turn cue rides the active player's edge and fires last, so it is what is heard when both happen at once.

The lobby, the name, the menu, the placement

The lobby lists the server's tables from a `/tables` reply — 36-byte records starting at offset 1, after a count byte — and joining is not a call: fetching the state for a table is the join. The row count comes from the listing's count byte, not from dividing the length by the stride, which says 5 only by accident of rounding.

The name bank reads the shared username from the appkey and puts it straight into a cartridge path buffer; if the slot is empty or there is no SD card, a three-row keyboard types one, shown through a `PATH` blit because the console cannot read the buffer back, and writes it back best-effort.

The menu bank exists for `RESET`: resume, how to play, leave. `LEAVE` sends `/leave` and then a fresh `/tables`, so the seat is given up rather than abandoned.

The placement screen opens on a fleet, not an empty board: five random legal placements are rolled — one candidate per frame in the `vblank` hook, because a whole fleet does not fit in one overscan and the first version, which looped, dropped every button pressed at it — the whole fleet is drawn, and the stick moves whichever ship's turn it is from where the roll put it. Overlaps are checked here, by walking each staged hull, because a hundred-byte occupancy map is most of the console's RAM.

BSPLACE.ASM: ONE CANDIDATE A FRAME

```

PROLLF: jsr    PROLL
        jsr    FITS
        bne    PRFX
        jsr    PCAND
        sta    BSCND
        jsr    POVL
        bne    PRFX
        lda    BSCND
        ldx    BSSHIP
        sta    SHIPS,x
        inx

```

```

        cpx      #NSHIPS
        bcs      PRF4
        stx      BSSHIP
PRFX:   rts
PRF4:   lda      #0
        sta      BSSHIP
        jsr      PLOAD
        jsr      PTEXT
        jmp      PHULLS

; PLOAD -- the cursor onto the seeded placement of ship BSSHIP, so the stick
; starts where the roll put it rather than at the origin.
PLOAD:  ldx      BSSHIP
        lda      SHIPS,x
        jsr      PDEC
        lda      BSPDIR
        sta      BSDIR
        lda      BSP0S
        ldx      #$FF
PLD1:   inx
        sec
        sbc      #BRDDIM
        bcs      PLD1
        adc      #BRDDIM      ; C is clear: this adds ten back
        sta      BSCURX
        stx      BSCURY
        rts

; PSL0T0 -- A = your slot: top-right with two players, bottom-left with more.
PSLOT0: lda      #SLTR
        ldx      BSMODE
        beq      PSL1
        lda      #SLBL
PSL1:   rts

```

Sound

One TIA channel and a script engine that steps once a frame. A cue is a list of steps — frames, control, frequency, volume — ending in a zero. Twelve cues, in bank order, so that each bank's are a prefix of the list and the constant SNDLAST truncates its copy of the scripts there: the place bank fires four of the twelve and used to carry all of them, which was 148 bytes it did not have.

SOUND.INC: THE CUES, IN BANK ORDER

```

SNDMOVE EQU      0      ; the cursor stepped one cell: a click
SNDSEL  EQU      1      ; a choice made, a table joined, readied
SNDPLC  EQU      2      ; a ship committed
SNDERR  EQU      3      ; refused: low, short, deliberately ugly
SNDSHOT EQU      4      ; the shot went out: a falling swoosh
SNDCLK  EQU      5      ; the clock's last seconds
SNDMISS EQU      6      ; a splash, low and short
SNDHIT  EQU      7      ; an explosion, brighter and longer
SNDUNK  EQU      8      ; the explosion, then three falling tones
SNDTURN EQU      9      ; your turn: three rising, carries a room
SNDOVER EQU     10      ; game over: three long falling tones
SNDJOIN EQU     11      ; the start countdown began

; SNDFIRE -- start cue A, replacing whatever is playing.
SNDFIRE: tax
        lda      SNDTAB,x
        sta      SNDPTR
        jmp      SNDSTEP

        IF      SNDFULL
; SNDISOFT -- start cue A only if nothing is playing. For the cursor click,
; which must never cut a hit short.
SNDISOFT: ldx      SNDPTR
        bne      SNDX
        jmp      SNDFIRE

; SNDTICK -- once a frame.
SNDTICK: lda      SNDPTR

```

How it is checked

Target	What it proves
make hosttest	the four playfield transforms against a picture of the tables built from the bit map in prose, every slot, poisoned outside
make layout && make shot	the kernel with no network: tables baked in, snapshotted, and every cell line of every board read back and required to be the colour the picture says, plus the divider
make drive	a game against the live server at two seats, reading the table off the screen through the font; after every poll, the reply's game fields and the cartridge's tables must agree byte for byte
make drive4	the same at four seats, reading the fleet strips back and requiring the pips to match each record
make frames, make frames4	a VSYNC tap over a whole game: nothing but 262
make resettest, make resetleave	a 6507 restart, and the RESET switch, which are two different things
build gates	the equates against the firmware header, the image's size and claim and control-page opcodes, each bank's extent from the assembler's own listing, and the tail's

Traps paid for here

- A blit is single-slot on the RP2040: a burst of thirteen would lose most of them on hardware and none in MAME. BLITGEN exists because of this program.
- Missiles replicate with their player's copy count: with the text's three close copies there were three dividers.
- A text row leaks: the six-copy kernel goes on displaying its last two bytes until something blanks them, and a JSR right after the row routine's RTS lands at cycle 79 and costs a scanline without anything looking wrong.
- The seam line's two deadlines are not the same deadline: COLUBK before pixel 0 at cycle 22, the sprite blanking before the text block at cycle 39. Blanking first, where it is free, put the water in four clocks late.
- NTSC hue 4 is pink; red is hue 3. The text block is at clocks 52–99, not 46–93: a player positioned by RESP_N completing at cycle N lands at clock 3N – 63, measured from the raster.
- The poll's recompose was three lines too long at four seats, and had been all along: make frames was two seats. emu/banktime.lua put the switch into the composer at +235.9 lines and the switch out at +263.5, so the pass wanted 27.6 lines of the 26.1 it had. The AUX plane became a pass of its own.
- A cold start's RAM clear writes every zero-page cell, so a write tap sees a spurious request in bank 0: that is the clear, not a fetch.

- Throttled is not a performance choice: the server's countdown and move clock are wall clock, and an unthrottled MAME sits at "starting in 2" forever.

CONFIG is the cartridge's own program: baked into the RP2040 firmware, served the instant the console starts fetching, because at power-up there is no network and nothing to load one from. It is the other real client, and it lives in `fujinet-config/atari-2600`, standalone: the shared CONFIG core assumes a keyboard, four device slots and kilobytes of buffers, and this console has a joystick, one device slot and 128 bytes. Feature parity with the Intellivision's — WiFi setup, host slots, browsing, copy, the game lobby and boot — on a twelve-column display with no text mode at all.

Seven screens, seven banks

Bank	Screen	Keys
0	host slots	up and down move, FIRE mounts and browses, SELECT the menu
1	browser	up and down move, left and right page, left at page 0 goes up a directory, FIRE descends or boots, SELECT the menu
2	adapter info	FIRE or SELECT back
3	keyboard	the stick moves the grid, FIRE types, SELECT the menu
4	WiFi	up and down move, FIRE picks a network, SELECT skips
5	boot	a progress bar; FIRE dismisses a failure
6	copy	FIRE returns early

Seven 2K banks pad to eight, a 16K image; each bank carries its own copy of the library, and the fixed tail holds the trampoline and the cold stub, as Section 13 describes.



Figure 16. CONFIG's host-slot screen, from the cartridge's own build. Selection is a > in column 0.

Almost nothing lives in console RAM

The working directory, the filter, a pending copy's source path and the text being typed are all 256-byte strings, and all four of the cartridge's path buffers are in use:

Buffer	Holds
0	the working directory — and the SSID, during WiFi setup
1	the directory filter — and the passphrase, during WiFi setup
2	a pending copy's source path, which has to survive browsing to a destination
3	the edit scratch: what the keyboard is typing into, so that cancelling an edit is possible at all

Buffers 0 and 1 double up because WiFi setup happens before anything is mounted. `SEED` copies the selected buffer into the scratch to begin an edit, `COMMIT` copies it back on accept, and cancel is neither. The console holds a cursor and a handful of flags.

The cartridge's path buffer 0 already holds the full path of the thing to boot: the browser appends the filename before it leaves, so the boot screen needs no argument, and on failure `POP` drops the filename and puts the browser back where it was.

Two blits it could not do without

There is no inverse video: the row renderer takes no attribute, so a selected row cannot be highlighted. Every list marks its selection with a > in column 0, which is why content gets eleven columns, and moving the cursor changes exactly two characters — two TCELL blits, not two recomposed rows, because

a scan list's text arrives one `GET_SCAN_RESULT` at a time and recomposing would cost two round trips per cursor step.

The keyboard is twelve by eight cells mapped directly onto ASCII 32 to 127, so the cursor position **is** the character: `ch = 32 + row × 12 + col`. What has been typed is shown through a PATH blit, the only way to see a buffer the console cannot read back.

Building it

```
cd ~/Workspace/fujinet-config/atari-2600
make                # build/config.bin, 16384 bytes
make rom.h          # bake it into the cartridge firmware
```

Its build adds one check the bring-up's lacked: `p2bin` silently truncates a bank that grows past `$17FF`, so the script re-runs it over `$1800–$1FFF` and requires zero bytes there.

The bring-up was seven milestones, and each one found something no earlier console in the family had. This is what they were, and what they cost, kept here because every one of them is a thing you will otherwise pay for yourself.

The milestones

	Milestone	The exit test
M0	the cartridge-composed display	the raster decoded back into plane bytes: 756 of 756 match the renderer
M-novel	the bus decode and the write-sampling model	a host test, because MAME structurally cannot test the sampling — its cartridge device is handed a clean byte
M1	the mailbox round trip, and reset survival	live SSID, IP and version on screen; ACKSEQ 01 and then 02 across a reset
M2	network boot, byte-identical	4096 of 4096 served bytes match the file, and the booted image runs
M3	the directory browser	navigates to a file by name, boots it, 4096 of 4096
M4	the RP2040 firmware, the ESP32 board, CI	the firmware builds, core 1 is SRAM-resident, three CI jobs
M5	the real cartridge mappers	nine schemes against MAME's own handlers, 200,000 fuzzed accesses each
M6	soak	thirteen synthetic cartridges across all nine schemes, every bank byte-compared
M7	CONFIG and Battleship	CONFIG browses and boots through a subfolder; Battleship plays a real game against the live server

M0 was pulled forward deliberately: every sibling console had a character generator or a framebuffer, so the cartridge-composed display was the one component with no template anywhere in the family. M-novel came with it, because MAME cannot test the write sampling, so it had to be proven before anything depended on it.

Designing the mailbox

- **Sixteen pages for everything.** Three decisions bought pages back. The REGDATA and DATA hotspots are not console addresses: the firmware's dispatcher only needs three distinct page numbers, so two of the three live outside the window and the bus layer synthesises the events. The reply is 512 bytes, not 256 or 1024, because 512 is the size at which the flagship

client never pages a slice. And the claim and the vectors live in the fixed half, so a RESET is survivable whatever bank is mapped.

- **A read of a write port is a write.** The Channel F's cartridge can tell a store from a fetch and its register pages are inert to reads; this one cannot, so the arm-then-commit pair stayed, and here it is the defence. The disarm-after-one-use in the firmware does real work on this console.
- **The 6507 has no interrupts**, which is the one structural gift: it is what lets the static check be a proof.
- **A bank-switch hotspot returns the old bank's byte.** MAME installs a read bank and a read tap over the same range and runs the tap after the read, so the access that switches still returns what was there before. The port had it inverted until the MAME source settled it, and it is invisible until a game reads its own hotspot for data as well as for the side effect.
- **Banking is a pointer swap, never a copy.** The first version copied 2K into the window, which is microseconds against an 838-nanosecond bus, and it passed every test, because MAME's device has all the time in the world. Every real 2600 mapper swaps a pointer, and so does this one now.
- **UA and FE switch on addresses below A12**, where the cartridge is not selected at all and is only watching the bus. Neither the RP2040 loop nor the MAME device was looking there.
- **Size detection needs the .cfg.** An 8K F8, E0, UA and FE are all 8192 bytes and nothing inside the file tells them apart. And the hint has to be spent on the image it came with, or a hint left standing is applied to the next image — the ColecoVision learned this the same way.
- **A blit is single-slot.** Battleship fired thirteen in a burst; on hardware most would be lost and in MAME none, so BLITGEN was published and the client waits for it. TEXTGEN is the same thing for a row.
- **The equates drift.** The spec put the control page at \$1D00 and the client's header said \$1C00; every register write went to a page that decodes nothing, reads of it fell through to the served window, and the client came up, drew a screen, and never armed the mailbox. `checkdefs.py` cross-checks every equate against the header and the build runs it first.
- **Equates and code go in different include files.** Including the code half before the `ORG` assembles the whole transport at \$0000, and every `JSR` to it becomes `20 00 00`.

The display

Three real bugs came out of decoding the raster instead of squinting at it: a missing seventh `GRP` write, an end-of-frame drain that toggled `VDELP` off and on and so restored the stale registers, and a `LDX`-versus-`TSX` cycle — the copies are 2.67 cycles apart and a zero-page store is 3, so the four late writes drift a full cycle and that one cycle decides whether the schedule closes at all. Three glyph pairs were bit-identical at 3×5 and had to be redrawn, because `SOAK.BIN` and `50AK.BIN` were the same picture. And harnesses that look for text on screen compare rendered forms, not decoded text, because decoding is lossy at this size and always will be.

The first programs

- The `READ_DIR_ENTRY` width is 30, not 31; at 31 the firmware prepends icon bytes.
- `SET_DIRECTORY_POSITION` is one parameter of two bytes, not two of one.

- Reading past the end of a directory poisons the session on an SD host.
- `OPEN_DIRECTORY` and `SET_DEVICE_FULLPATH` read exactly 256 bytes; every port in this family has paid for a short payload once.
- `OPEN_APPKEY`'s struct has a sixth, reserved byte; without it the adapter waits for a byte that never comes.
- The swap stub must silence the sound and return the RIOT ports to inputs.
- In Battleship: the stack leaked two bytes per poll through the trampoline until it reset the stack; a bank's scratch aliased the settle loop's counter; a whole fleet did not fit in one overscan; a seam line that ran past cycle 76 silently cost a second scanline and a 278-line frame; the four-seat recompose was three lines too long and had been all along, because the frame test ran at two seats.

The MAME bench

- `SDL_VIDEO_DRIVER=dummy` is required wherever there is no display, even under `-video none`, because SDL comes up before the video backend is chosen.
- MAME must run from its own tree or `-autoboot_script` is silently ignored.
- `fujinet-pc`'s bus-over-IP listener takes one client; a stray MAME starves the next run, and the symptom is a hang.
- MAME's Lua binds neither the beam position nor the cycle counter, and a write tap does not fire on the TIA range, so raster instrumentation comes from decoding snapshots.
- The grafted device must be re-applied after every edit to a shared source; MAME builds the copies.
- Without symbol prefixing, a MAME tree that already carries the Arcadia and Astrocade FujiNet devices links one `fujimail_paint()` for all three, and the rest paint the wrong offsets.

This handbook's own

- A batari Basic program built for 4K puts its score font and vectors on the status page; `set romsize 2k` is the whole answer.
- A name batari Basic has not seen as a `const` compiles to a load from memory when used as a value: `FNCMT = FNDEVF` went out as `LDA $0070`, and the transaction, addressed to whatever the TIA read as, timed out. Every value is a `const` now, generated and prepended.
- A playfield under 120 lines wraps batari Basic's overscan timer and the frame comes out short — 163 lines — with nothing looking wrong in the source. `pfrowheight` of 6 is the knob; 5 is not.
- Composing three text rows between two frames makes one 268-line frame. A row per frame does not.
- `pkill -f "mame a2600"` in a run script matches any shell whose command line contains the pattern — including the one running the script. Anchor it.
- The harness must resolve the image's path before it changes directory into MAME's tree.

Building

SYMPTOM	PROBABLE CAUSE AND REMEDY
checkdefs fails before anything assembles	An equate in <code>fujinet.inc</code> no longer matches <code>fuji_mailbox.h</code> . Fix the equate; never the check.
checkrom: an RMW or indirect store on the control pages	Find the <code>INC</code> , <code>DEC</code> , <code>shift</code> or <code>STA (zp),Y</code> and replace it with a plain store. In BASIC, an arithmetic assignment to a mailbox name is the usual cause.
checkrom: image size	The image must be $(N+1) \times 2048$ and a size MAME loads: 4K, 8K, 16K or 32K. Pad the bank count.
Every JSR into the library lands at \$0000-\$01FF	<code>fujilib.inc</code> was included before the <code>ORG</code> . Equates first, code inside the <code>ORG</code> .
A bank assembles, the image is the right size, and code past \$17FF is simply gone	<code>p2bin</code> truncates in silence. Check the bank's extent in the assembler's listing, as Battleship's <code>checkbanks.py</code> does.
batari Basic: "Origin Reverse-indexed"	The program is over 2K. Trim with <code>default.inc</code> , <code>const noscore = 1</code> , or less BASIC.
batari Basic: "EQU: Value mismatch"	A <code>dim</code> or <code>const</code> redefines a name the kernel or the headers already own — a single-letter variable, or an equate with a different value.
batari Basic: an unresolved symbol at link time	A name used in BASIC exists in no header. Check the spelling against <code>fujinet.h</code> and <code>fujibas.h</code> .

The mailbox

SYMPTOM	PROBABLE CAUSE AND REMEDY
Magic F, N not at \$1F09	No cartridge, or MAME started with the wrong slot: - <code>cartslot fujinet</code> . The bring-up's plain slot serves the image but nothing paints.
FLAGS bit 1 clear: the claim was not honoured	FUJI is not at file offset $(\text{size} - 2048) + \$710$. The build stamps it; a hand-built image must too.
The program runs, draws, and no transaction ever completes	The gate is closed: \$B5 to \$1DFC then \$4A to \$1DFD before anything else. Or the control page is one page off — see <code>checkdefs</code> .
ERR = 2, timeout, on every transaction	In BASIC: a value used without a <code>const</code> , so the device or command went out as a load from memory. In assembly: a parameter count that does not match the parameters written. In MAME: a stray MAME holding the bus-over-IP listener.

SYMPTOM	PROBABLE CAUSE AND REMEDY
ERR = 1, no link	The USB link to the adapter is down, or <code>fujinet-pc</code> is not running. The cartridge waits three seconds for the link before its first transaction after a reset.
REPLY_CMD = \$15, a NAK, with ERR = 0	The adapter refused the command: wrong parameter shape, a short 256-byte payload, a command this build does not dispatch, an unmounted host. The transport is fine.
The first transaction after a RESET is ignored	The program derived SEQ from a counter in RAM and collided with a sequence already answered. Use <code>ACKSEQ + 1</code> .
The reply changes under the program	Something launched a transaction or selected a slice. Read what you need before the next commit.

The display

SYMPTOM	PROBABLE CAUSE AND REMEDY
Text, but the wrong characters in the wrong columns	The positioning constants were changed. Restore <code>RNOP 16, R3 1, HM0 15, HM1 0</code> and decode the raster to be sure.
The last text line repeats down the blank lines	The blank at the end of the kernel toggled <code>VDELP</code> instead of writing <code>GRP0, GRP1, GRP0</code> .
A row shows part of the previous row's text	The next row was composed before the render landed. Wait for <code>TEXTGEN</code> to change.
One character of a row is stale after a blit	Two blits in a row without waiting for <code>BLITGEN</code> .
The picture rolls, in BASIC	The frame is not 262 lines. Run <code>frames.lua</code> ; fewer text rows, <code>pfrowheight</code> of 6, <code>noscore</code> , or a row per frame.
The picture rolls once on a screen change	More than a frame's worth of composing between two frames. Spread it, or accept it.
<code>s</code> and <code>5</code> , <code>o</code> and <code>0</code> look alike in a test	They are alike at 3×5 in most fonts; this one draws them apart. Compare rendered forms in harnesses, not decoded text.

The network

SYMPTOM	PROBABLE CAUSE AND REMEDY
OPEN succeeds, STATUS says nothing is waiting	The fetch is deferred to the first STATUS; take another reading. If the device status is not 1, the URL failed — that is where an HTTP error shows.
The first half of the page, then garbage	STATUS was read once. Read until two readings agree.

SYMPTOM	PROBABLE CAUSE AND REMEDY
A URL from a path buffer fails	It was emitted padded. Use the raw emit for a URL; padding lands inside the query.
The screen goes blank after a READ	CLOSE was sent after the READ. Close at the start of the next request.
A count compare goes wrong past 127	A signed branch. Compare through the carry.

Booting

SYMPTOM	PROBABLE CAUSE AND REMEDY
BOOT_STATE goes to \$80, BOOT_ERR = 1 or 3	The image is larger than 32K, or its size is not a multiple of 2K.
BOOT_STATE stays at 1	The push is in progress; MOUNT_IMAGE has sixty seconds. A very slow host is a slow host.
The swap does nothing	BOOTLOCK was not committed, or was committed before BOOT_STATE was 2. It is honoured only once an image is staged.
The booted game screams	The stub did not silence AUDV0 and AUDV1.
The booted game cannot read its joystick	The stub left SWACNT and SWBCNT as outputs.
An 8K game runs the wrong banks	It is E0, UA or FE and was served as F8. Put a .cfg with the scheme's name beside it.
The mailbox is dead after the boot	As it should be: the booted image has no claim.

The MAME bench

SYMPTOM	PROBABLE CAUSE AND REMEDY
Could not initialize SDL	No display: SDL_VIDEO_DRIVER=dummy, which emu/run.sh sets.
The harness never runs	MAME was not started from its own tree; - autoboot_script is silently ignored.
A run hangs with no error	A stray MAME holds the one-client bus-over-IP listener. emu/run.sh kills it first.
A Lua write tap reports a request in bank 0 at frame 0	That is the cold start clearing every zero-page cell, not a fetch.
The server says "starting in 2" forever	The run is unthrottled. The server's clocks are wall clock; throttled is not a performance choice.

Everything in this handbook is transcribed from these, at these revisions. When ordering repair parts, always include the part number.

The cartridge

The bring-up: mailbox spec, RP2040 firmware, MAME device, testroms, tools. . .	fn-2600/pico/atari-2600, 2026-09-15
The single source of truth.	firmware/include/fuji_mailbox.h
The client library.	testrom/fujinet.inc, fujilib.inc, fujidisp.inc, vcs.inc
The static check and the equate check. . .	tools/checkrom.py, tools/checkdefs.py
The font.	tools/vcsfont.py
The MAME cartridge device.	emu/fujinet.cpp, emu/apply.sh

The adapter

FujiNet firmware, RS232 build.	fujinet-firmware d38e32543
The command ids and device ids.	include/fujiCommandID.h, fujiDeviceID.h
The RS232 devices.	lib/device/rs232/
The shared mixins: appkeys, base64, hash, QR.	lib/device/fujiDevice/
fujinet-pc, the adapter on a PC.	fujinet-pc-rs232 2a9e2c23f, bus-over-IP 127.0.0.1:9995

The programs

Battleship for the Atari 2600.	fujinet-battleship b59ec45
The Battleship server.	battleship.carr-designs.com
CONFIG for the Atari 2600.	fujinet-config b4e3805
This handbook's examples.	fujinet-manuals/atari-2600/programmers-handbook/listings/

The tools

Macroassembler AS.	asl, p2bin, Bld 305
batari Basic.	v1.7 native binaries, source efd6dcd
dasm.	2.20.16, c361b82
MAME.	ea1dad92a34, with the FujiNet cartridge device grafted
Typst.	0.15.1
fontforge, for the screen font.	20251009

Further reading

The FujiNet Network Protocol Handbook. . .	every scheme the N: device speaks
The FujiNet Platform Bring-Up Guide.	how a cartridge like this one is made

The FujiNet Programming Guides for the Intellivision, Bally Astrocade and ColecoVision.	the same mailbox on the siblings
Stella Programmer's Guide, Steve Wright, 1979.	the TIA, from the people who built it

The window

Address	Region
\$1000—\$17FF	the bank; \$1D80 + b selects
\$1800, \$1880, \$1900, \$1980, \$1A00, \$1A80	text planes 0—5, 128 bytes each; Y = row × 6 + line
\$1B00—\$1CFF	the reply window, 512 bytes; slice 0 or 1
\$1D00—\$1D7F	arm register n (write-only)
\$1D80—\$1DFF	one-shot operations (write-only)
\$1E00—\$1EFF	the TX stream: any store appends (write-only)
\$1F00—\$1F1F	status cells
\$1F10	FUJI
\$1F20—\$1FFB	the fixed tail
\$1FFC, \$1FFE	RESET, BRK

Status cells

Cell	Name	Cell	Name
\$1F00	ACKSEQ	\$1F09—\$1F0A	F, N
\$1F01	STATUS: bit 0 link, bit 1 busy	\$1F0B	PROTO_VER = 2
\$1F02	ERR	\$1F0C	SLICE_ECHO
\$1F03	REPLY_CMD \$06 / \$15	\$1F0D	TEXTGEN
\$1F04—\$1F05	RXLEN	\$1F0E	BANK
\$1F06	BOOT_STATE	\$1F0F	FLAGS: bit 1 claim
\$1F07	BOOT_PCT	\$1F17—\$1F18	PATHLEN
\$1F08	BOOT_ERR	\$1F19	BLITGEN

Registers and one-shots

Reg		Store	
\$00	DEVICE	\$1D80+b	bank b
\$01	CMD	\$1DF0—\$1DF2	text row, char, render
\$02	NPARAM	\$1DF3—\$1DF4	path char, path op
\$05	DATA_RST	\$1DF5—\$1DFA	blit SL SH DL DH CNT GO
\$06	RXSLICE	\$1DFC—\$1DFD	arm \$B5, \$4A
\$10	SEQ	\$1DFE	swap
\$11	BOOTLOCK \$B5	\$1DFF	commit
\$12—\$13	BOOTSEL \$B5, \$4A		

Path operations and blits

Path: 0 reset, 1 pop, 2 emit padded, 3 emit raw, 4 pop char, 5–8 select 0–3, 9 commit, 10 seed. Blits: 0 raw, 1 text, 2 field, 3 hulls, 4 sea, 5 cell, 6 paint, 7 path, 8 tcell, 9 card, 10 pfclr, 11 pfield, 12 pfhull, 13 pfccl.

The transaction

DEVICE, CMD, NPARAM, DATA_RST; then NPARAM × (size, value) and the payload into \$1E00; then SEQ = ACKSEQ + 1 (255 wraps to 1); wait for ACKSEQ; read ERR, REPLY_CMD, RXLEN, the window.

The library's zero page

\$80 FNSEQ, \$81 PAD3, \$82 SAVSP, \$83 FNCNT, \$86 FNDEV, \$87 FNCMD, \$88 FNNPR, \$89 FNTMO, \$8A–\$8B FNPTRL/H, \$8E FNPCNT, \$A8–\$A9 INCUR/INPREV. The swap stub lands at \$80–\$9D.

Where	Value	Meaning
ERR \$1F02	0	ok
	1	no link to the adapter
	2	timeout: five seconds, or sixty for a mount or copy
	3	bad frame: SLIP, length or checksum
	4	too big for the link
REPLY_CMD \$1F03	\$06	ACK
	\$15	NAK: the adapter refused the command
the library, FNGO and FNACK	0	FNEOK
	\$FF	FNEWAIT: the cartridge never answered
	\$EE	FNENAK: the adapter said NAK
	\$FD	FNENOC: no cartridge answered at all
BOOT_STATE \$1F06	0	idle
	1	transferring; BOOT_PCT counts
	2	staged and ready
	\$80	failed; see BOOT_ERR
BOOT_ERR \$1F08	1	too big (over 32K)
	2	truncated: the push was aborted
	3	no mapper: the size is not a multiple of 2K
	4	the store was busy
N: STATUS byte 3	1	success
	136	end of file
	other	the protocol's error; the HTTP status shows here
WiFi status	3	connected
READ_DIR_ENTRY	\$7F, \$7F	the end of the directory

The library, this handbook's additions, and the example programs, as built and run. The Battleship sources are in Appendix D.

LISTINGS/COMMON/FUJINET.INC

```

1 : fujinet.inc -- equates for the FujiNet
2 : cartridge mailbox.
3 :
4 : Hand-mirrored from firmware/include/
5 : fujinet_mailbox.h; keep the two in step.
6 :
7 : Split from fujilib.inc because that file
8 : contains CODE, and code has to be
9 : assembled inside the ORG'd program. The
10 : equates have to come first, so they
11 : live here. (Including the code half early
12 : instead assembles the whole
13 : transport at address $0000 and every JSR to it
14 : becomes '20 00 00' -- which
15 : assembles cleanly and fails at run time.)
16 :
17 : ----- cart -> console, painted
18 :
19 : FNTEXT EQU $1800 ; six 128-byte
20 : text planes
21 : FNRPY EQU $1B00 ; the reply
22 : window, 512 bytes
23 : FNACKS EQU $1F00 ; echoes SEQ
24 : when the reply is ready
25 : FNSTAT EQU $1F01 ; bit0 link up,
26 : bit1 busy
27 : FNERR EQU $1F02 ; FNE+ below
28 : FNRCMD EQU $1F03 ; $06 ACK / $15
29 : NAK
30 : FNRLX EQU $1F04 ; reply length,
31 : little-endian
32 : FNRXHI EQU $1F05
33 : FNBSST EQU $1F06 ; boot state
34 : FNBPCC EQU $1F07 ; boot percent
35 : FNBER EQU $1F08 ; boot error
36 : FNMAAG EQU $1F09 ; 'F'
37 : FNMAI EQU $1F0A ; 'N'
38 : FNMPV EQU $1F0B
39 : FNSECH EQU $1F0C ; slice echo,
40 : published last after a repaint
41 : FNBTXG EQU $1F0D ; last text row
42 : rendered, plus a toggle
43 : FNBNBK EQU $1F0E ; the live bank
44 : FNBFGL EQU $1F0F ; bit0 armed,
45 : bit1 claim honoured
46 : FNPLNL EQU $1F17 ; path buffer
47 : length, little-endian
48 : FNPLNH EQU $1F18
49 :
50 : ----- console -> cart, write-only
51 :
52 : NEVER READ THESE PAGES. The cart does not
53 : drive them -- not driving them is
54 : exactly what lets it sample the data bus -- so
55 : a read returns the floating
56 : bus, and worse, is decoded as an access.
57 : FNRSLE EQU $1D00 ; + n (n < $80):
58 : arm register n
59 : FNCRMT EQU $1DFF ; commit the
60 : armed register with this data
61 : FNCTX EQU $1E00 ; a write
62 : anywhere in the page appends
63 :
64 : Registers, unchanged from the sibling ports so
65 : fujimail.c compiles verbatim.
66 : FR_DEV EQU $00
67 : FR_CMD EQU $01
68 : FR_NPAR EQU $02
69 : FR_DRST EQU $05 ; any value:
70 : rewind the TX write pointer
71 : FR_RXSL EQU $06
72 : FR_SEQ EQU $10 ; nonzero and !=
73 : ACKSEQ: launch
74 : FR_BLK EQU $11 ; FNBLKM: arm
75 : the ROM swap
76 :
77 : One-shot operations, in the bit7-set half of
78 : the control page.
79 : FH_BANK EQU $80
80 : FH_TROW EQU $F0
81 : FH_TCHR EQU $F1
82 : FH_TEND EQU $F2
83 : FH_PATHC EQU $F3 ; append A to
84 : the cartridge path buffer
85 : FH_PATHO EQU $F4 ; act on it:
86 : FP+ below
87 : FP_RST EQU 0
88 : FP_POP EQU 1
89 : FP_TX EQU 2
90 : FP_TXRAW EQU 3
91 : FH_ARM1 EQU $FC
92 : FH_ARM2 EQU $FD
93 : FH_SWAP EQU $FE
94 :
95 : FNAM1 EQU $85 ; the arming
96 : pair
97 : FNAM2 EQU $4A
98 : FNBLKM EQU $85 ; bootlock
99 :
100 : FNMTOL EQU 12 ; text columns
101 :
102 : $1F20-$1FFB is the client's fixed tail: the
103 : cold stub, the bank trampolines,
104 : and anything else that must be at the same
105 : address from every bank.
106 : FNNTAL EQU $1F20
107 :
108 : Devices and commands, from include/
109 : fujiCommandID.h and fujiDeviceID.h.
110 : FNDEVF EQU $70 ; the FUJI
111 : device
112 : FNDEVN EQU $71 ; N1:
113 : FNCADEX EQU $C4 ;
114 : GET_ADAPTERCONFIG EXTENDED
115 : FNCHST EQU $F9 ; MOUNT_HOST
116 : FNCSDFP EQU $F8 ; MOUNT_IMAGE
117 : FNCSDFP EQU $E2
118 : SET_DEVICE FULLPATH
119 : FNCRST EQU $F4 ;
120 : READ_HOST_SLOTS
121 : FNCDIR EQU $F7 ; OPEN_DIRECTORY
122 : FNCRDIR EQU $F6 ; READ_DIR_ENTRY
123 : FNCCDIR EQU $F5 ;
124 : CLOSE_DIRECTORY
125 : FNCSDFS EQU $E4 ;
126 : SET_DIRECTORY_POSITION
127 :
128 : FMREAD EQU 1 ; disk access
129 : mode: read
130 :
131 : FN_R_BOOT_STATE values.
132 : FNBDL EQU 0
133 : FNBFKR EQU 1
134 : FNBRDY EQU 2
135 : FNBFALL EQU $80
136 :
137 : FNEOK EQU 0
138 : FNEWAIT EQU $FF ; the cart never
139 : answered at all
140 : FNEAK EQU $EE ; the server
141 : said NAK
142 : FNEOC EQU $FD ; no cartridge
143 : answered at all -- deliberately ; not 1, which
144 : is FN_ERR_NOLINK
145 :
146 : ----- zero page -----
147 :
148 : FNSEQ EQU $80 ; the sequence
149 : we are waiting for
150 : FNCRMT EQU $83 ; timeout quanta
151 : remaining
152 : FNDEV EQU $86 ; transaction
153 : setup, read by FNBEQ
154 : FNCRMT EQU $87
155 : FNPNPR EQU $88
156 : FNTRMO EQU $89 ; timeout in
157 : byte
158 : FNPRTR EQU $8A
159 : FNPRTH EQU $8B
160 : FNPCNT EQU $8E ; bytes written
161 : into the current path payload
162 : INCUR EQU $A8 ; input, above
163 : the swap stub's landing ground
164 : INPREV EQU $A9
165 :
166 : ----- input -----
167 :
168 : Joystick 0 in SWCHA bits 7-4 and the console
169 : switches in SWCHB, all active
170 : low, folded by INSCAN into one active-high
171 : byte.
172 : IN_UP EQU $01
173 : IN_DOWN EQU $02
174 : IN_LEFT EQU $04
175 : IN_RIGHT EQU $08
176 : IN_FIRE EQU $10
177 : IN_SEL EQU $20
178 : IN_RST EQU $40
179 :

```

LISTINGS/COMMON/FUJILIB.INC

```

1 ; fujilib.inc -- the 6502 side of the FujiNet
  cartridge mailbox: the code.
2 ;
3 ; Equates are in fujinet.inc, which must be
  included BEFORE the ORG. This file
4 ; is code and must be included inside it.
5 ;
6 ; A register write is TWO stores on this
  console, not one:
7 ;
8 ;       lda #value
9 ;       sta FNRSEL+reg      ; arm; this
  store's data is ignored
10 ;       sta FNCMT          ; commit:
  register 'reg' = value
11 ;
12 ; That is not ceremony. The cartridge port has
  no R/W line, so the cart cannot
13 ; tell a read of a write port from a write to it
  -- a stray LDA into page $2C
14 ; is decoded as an access carrying whatever the
  floating bus held. Arming and
15 ; committing separately means ONE stray mutates
  nothing: an arm carries no
16 ; value, and a commit with nothing armed is
  discarded. The Channel F keeps the
17 ; same pair as protocol shape; here it is the
  defence.
18 ;
19 ; The other rule that will bite: NEVER use a
  read-modify-write instruction on
20 ; the control or TX pages. INC/DEC/ASL/LSR/ROL/
  ROR absolute are three bus
21 ; cycles at one address -- read, write old,
  write new -- and the cart's "last
22 ; value before the address changed" sampling has
  no defined answer in that
23 ; burst. Only STA/STX/STY. tools/checkrom.py
  fails the build on the rest.
24 ;
25 -----
26 ; FNRW -- register X = A. The whole transport is
  built on this.
27 ;
28 ; 'sta FNRSEL,x' is safe for any X: the base low
  byte is $00 so the index can
29 ; never carry, which means the dummy read STA
  abs,X always performs lands on
30 ; the SAME address as the write -- one parked
  access, not two, and an arm
31 ; ignores its data anyway.
32 FNRW:  sta FNRSEL,x
33       sta FNCMT
34       rts
35 ;
36 ; FNARM -- open the decode gate.
37 ; Two stores, in order, carrying two specific
  values. A 7800's BIOS probes
38 ; cartridge space on startup and cannot produce
  that, which is the point.
39 FNARM:  lda #FNAM1
40       sta FNRSEL+FH_ARM1
41       lda #FNAM2
42       sta FNRSEL+FH_ARM2
43       rts
44 ;
45 ; FNCHK -- Z set if a FujiNet cartridge is
  answering.
46 FNCHK:  lda FNMAG0
47       cmp #'F'
48       bne FNCHK1
49       lda FNMAG1
50       cmp #'N'
51 FNCHK1: rts
52 ;
53 ; FNBEG -- begin a transaction described by
  FNDEV / FNCMD / FNNPR.
54 ; Rewinds the TX stream last, so parameters may
  be appended immediately.
55 FNBEG:  lda #16
56       ; ~9s; longer
  than the cart's own 5s budget so
57       sta FNTMO
58       ; a real
  timeout is reported as ITS error
59       lda FNDEV
60       ldx #FR_DEV
61       jsr FNRW
62       lda FNCMD
63       ldx #FR_CMD
64       jsr FNRW
65       lda FNNPR
66       ldx #FR_NPAR
67       jsr FNRW
68       lda #0
69       ldx #FR_DRST
70       jsr FNRW
71       rts
72 ;
73 ; FNTXB -- append A to the TX stream.
74 FNTXB:  sta FNTX
75       rts
76 ;
77 ; FNPB -- append a one-byte parameter with value
  A.
78 ; The stream is NPARAM x { size byte, then that
  many value bytes } then the
79 ; payload, identical to every sibling port.
80 FNPB:  pha
81       lda #1
82       sta FNTX
83       sta FNTX
84       rts
85 ;
86 ; FNPW -- append a two-byte little-endian
  parameter. A = low, X = high.
87 ; SET_DIRECTORY_POSITION takes ONE parameter of
  TWO bytes -- fujiDevice reads
88 ; it as packet.param(0) declared uint16_t -- not
  two one-byte parameters, and
89 ; getting that wrong produces a NAK with nothing
  to point at.
90 FNPW:  pha
91       lda #2
92       sta FNTX
93       pla
94       sta FNTX
95       txa
96       sta FNTX
97       rts
98 ;
99 ; FNGO -- commit and wait. Returns A = FNE* (0 =
  the transaction completed).
100 ;
101 ; THE RULE THAT EVERY PORT IN THIS FAMILY
  LEARNED THE HARD WAY: the next
102 ; sequence number comes from the CART'S OWN
  persisted ACKSEQ + 1, never from a
103 ; counter in RAM. A console RESET restarts the
  client and re-zeroes its
104 ; variables but does NOT reset the cartridge --
  there is no reset line on the
105 ; connector and the RESET switch is a bit in a
  RIOT register the cart never
106 ; sees. A program-local counter would restart at
  1, collide with a sequence
107 ; already answered, and the transaction would be
  dropped in silence.
108 FNGO:  lda FNAK
109       clc
110       adc #1
111       bne FNGO1
112       lda #1
113       ; as "never used"
114       sta FNSQ
115       ldx #FR_SEQ
116       jsr FNRW
117       ; is this single commit
118       sta FNCNT
119       ldx #0
120       ; ACKSEQ is published LAST, after the
  whole reply is in place, so a
121 ; match means everything behind it is
  already readable.
122 FNGO4:  lda FNAK
123       cmp FNSQ
124       beq FNGO5
125       dex
126       bne FNGO4
127       dey
128       bne FNGO3
129       dec FNCNT
130       bne FNGO2
131       lda #FNEWAIT
132       rts
133 FNGO5:  lda FNERR
134       rts
135 ;
136 ; FNAK -- 0 on success, FNE* on transport
  failure, FENAK if the server said
137 ; no. Call after FNGO returned 0.
138 FNAK:  lda FNRCD
139       cmp #0
140       beq FNAK1
141       lda #FENAK
142       rts
143 FNAK1:  lda #FNEOK
144       rts
145 ;
146 ;
147 ; Text, composed by the cartridge.
148 ;
149 ; FNROWA -- begin text row A.
150 FNROWA: sta FNRSEL+FH_TROW
151       rts
152 ;
153 ; FNEHDR -- render the composed row.
154 FNEHDR: lda #0
155       sta FNRSEL+FH_TEND
156       rts
157 ;
158 ; FNRRLPL -- render up to Y bytes of the reply
  window at offset X into text
159 ; row A, stopping at NUL.
160 ;
161 ; This is the zero-copy path and on this console
  it is not an optimisation.
162 ; The reply window is repainted only by a SEQ
  commit or a slice select, so it

```

```

163 ; is stable while the row is streamed -- and the
164 bytes go from the cartridge
165 ; back to the cartridge without ever occupying
166 any of the console's 128 bytes
167 ; of RAM, of which the stack already owns the
168 top.
169 ;
170 ; X is a single byte, so this reaches the first
171 256 bytes of the reply. Every
172 ; field of GET_ADAPTERCONFIG EXTENDED that a
173 client wants lives below that.
174 FNRR1: sta FNRSEL+FH_TROW
175 FNRR1: lda FNRPLY,x
176 beq FNR2
177 sta FNRSEL+FH_TCHR
178 inx
179 dey
180 bne FNRR1
181 FNRR2: lda #0
182 sta FNRSEL+FH_TEND
183 rts
184 ;
185 ; FNRSTR -- render the NUL-terminated string at
186 FNPTRL/FNPTRH into text row A.
187 FNRSTR: sta FNRSEL+FH_TROW
188 ldy #0
189 (FNPTRL),y
190 beq FNR2
191 sta FNRSEL+FH_TCHR
192 iny
193 cpy #FNTCOL
194 bne FNR1
195 FNRS2: lda #0
196 sta FNRSEL+FH_TEND
197 rts
198 ;
199 ; FNHEX -- render A as two hex digits into the
200 row being composed.
201 FNHEX: pha
202 lsr a
203 lsr a
204 lsr a
205 lsr a
206 jsr FNHEX1
207 pla
208 and #$0F
209 cmp #10
210 bcc FNHEX2
211 adc #6
212 here, so this adds 7 in total ; carry is set
213 FNHEX2: adc #'0'
214 sta FNRSEL+FH_TCHR
215 rts
216 ;
217 ; FNPATH -- append the NUL-terminated string at
218 FNPTRL/H to the TX stream,
219 ; padded with NULs to EXACTLY 256 bytes.
220 ;
221 ; Not optional and not a rounding:
222 OPEN_DIRECTORY and SET_DEVICE_FULPATH both
223 ; read exactly 256 bytes and a short payload
224 ; fails the server's read. Every
225 ; port in this family has paid for this one.
226 FNPATH: ldy #0
227 FNP1: ldy (FNPTRL),y
228 beq FNP2
229 sta FNP2
230 FNTX
231 iny
232 bne FNP1
233 rts
234 ; exactly 256
235 path bytes; no padding needed
236 FNP2: lda #0
237 FNP3: sta FNTX ; Y wraps to 0
238 after exactly 256 - Y stores
239 iny
240 bne FNP3
241 rts
242 ;
243 ; FNBK -- arm the ROM swap (FN_REG_BOOTLOCK).
244 FNBK: lda #FNBKLM
245 ldx #FR_BLKCK
246 jmp FNRW
247 ;
248 ; FNSWAP -- boot the staged image. Does not
249 return.
250 ;
251 ; The swap replaces every byte of the 4K window
252 including the code that
253 ; triggered it, so the trigger has to run from
254 somewhere else -- and on this
255 ; console the only RAM is $80-$FF, which is ALSO
256 the stack ($0100-$01FF
257 ; mirrors into it). The stub lives at $80 and
258 the caller must keep SP above
259 ; it; there is no JSR inside the stub and the
260 6507 has no interrupts, so
261 ; nothing else can touch the stack while it
262 runs.
263 ;
264 ; Three of the stub's stores are easy to leave
265 out and each breaks a different
266 ; thing: AUDV0/AUDV1, or a tone screams until
267 the boot game's cold start
268 ; runs; and SWACNT/SWBCNT back to inputs,
269 because almost no game writes them
270 ; and a client that left the RIOT ports as
271 outputs would brick every game
272 ; booted after it.
273 FNSWAP: ldx #FNSTE-FNSTB-1
274 FNSW1: lda FNSTB,x
275 sta $80,x
276 dex
277 bpl FNSW1
278 jmp $0080
279 ;
280 FNSTB: lda #0
281 sta GRP0 ; silence the
282 sprites
283 sta GRP1
284 AUDV0 ; *** and the
285 sound, or it screams ***
286 sta AUDV1
287 PF0
288 sta PF1
289 sta PF2
290 WSYNC ; land on a line
291 boundary
292 sta SWACNT ; *** RIOT port
293 A back to inputs ***
294 sta SWBCNT ; *** and port B
295 ***
296 lda #2
297 sta VBLANK ; blank across
298 the swap
299 sta VSYNC ; a clean
300 discontinuity, not a rolling frame
301 sta FNRSEL+FH_SWAP ; THE SWAP. The
302 next fetch is at $00xx, in ; RAM with A12
303 low, so the window may flip ; between this
304 store and it.
305 ldx #$FF
306 txs ; a clean stack
307 for the new image
308 jmp ($1FFC) ; the NEW
309 image's own reset vector, exactly ; what the
310 6507 fetches at power-on
311 FNSTE:
312 ;
313 ; Building a 256-byte path payload a piece at a
314 time.
315 ; SET_DEVICE_FULPATH and OPEN_DIRECTORY both
316 read EXACTLY 256 bytes, so the
317 ; payload has to be padded whatever it is built
318 from. FNPATH does that for a
319 ; string already in ROM; these do it for a path
320 assembled from a prefix plus a
321 ; filename that only exists in the cartridge's
322 reply window.
323 FNPBEG: lda #0
324 sta FNPcnt
325 rts
326 ;
327 ; FNPCH -- append A, Preserves X and Y.
328 FNPCH: sta FNTX
329 inc FNPcnt
330 rts
331 ;
332 ; FNPSTR -- append the NUL-terminated string at
333 FNPTRL/H.
334 FNPSTR: ldy #0
335 FNP1: ldy (FNPTRL),y
336 beq FNP2
337 sta FNP2
338 FNTX
339 iny
340 bne FNP1
341 FNP2: rts
342 ;
343 ; FNPRL -- append up to Y bytes of the reply
344 window at offset X, stopping at
345 ; NUL.
346 ; This is the zero-copy path, and on this
347 console it is not an optimisation.
348 ; The name being appended is sitting in the
349 cartridge's reply window; it goes
350 ; straight back to the cartridge through the TX
351 page without ever occupying
352 ; any of the console's 128 bytes of RAM, of
353 which the stack already owns the
354 ; top and the swap stub the bottom.
355 FNPRL: lda FNRPLY,x
356 beq FNP2
357 FNTX
358 FNPCH
359 inx
360 dey
361 bne FNPRL
362 FNP2: rts
363 ;
364 ; FNPEND -- pad out to exactly 256 bytes.
365 FNPEND: lda #0
366 FNP1: ldx FNPcnt
367 beq FNP2
368 bytes written
369 sta FNTX
370 inc FNPcnt
371 bne FNP1
372 FNP2: rts
373 ;
374 ; The working directory, held in the CARTRIDGE.
375 ; A browser has to remember which directory it
376 is in. This console has 128
377 ; bytes of RAM, the stack owns the top of them
378 and the swap stub the bottom,

```

```

332 ; and a path is 256 bytes. So the path lives in
333 the cartridge: built a
334 ; character at a time, popped a component at a
335 time, and emitted straight into
336 ; the TX stream when a transaction needs it. It
337 never occupies console RAM.
338 ;
339 ; A directory is held WITH its trailing "/", so
340 appending a filename to it is
341 ; just FNWRPL and needs no separator logic here.
342 ;
343 ; IT SURVIVES A CONSOLE RESET, because there is
344 no reset line on this
345 ; connector and the cartridge never sees the
346 switch. That is the behaviour you
347 ; want from a browser -- but it means a client
348 must not assume the buffer is
349 ; empty at startup. Call FNWRST.
350 ;
351 ; FNWRST -- empty it.
352 FNWRST: lda #FP_RST
353 sta FNRSEL+FH_PATHO
354 rts
355 ;
356 ; FNWCH -- append A. Preserves X and Y.
357 FNWCH: sta FNRSEL+FH_PATHC
358 rts
359 ;
360 ; FNWPOP -- drop the last component. "/a/b/c/"
361 becomes "/a/b/", and "/"
362 ; stays "/", so walking up from the root is
363 harmless.
364 FNWPOP: lda #FP_POP
365 sta FNRSEL+FH_PATHO
366 rts
367 ;
368 ; FNWTX -- emit the path into the TX stream as
369 EXACTLY 256 bytes, padded.
370 ; One store; the cartridge does the other 255.
371 FNWTX: lda #FP_TX
372 sta FNRSEL+FH_PATHO
373 rts
374 ;
375 ; FNPWD -- append the working directory to the
376 payload being built, WITHOUT
377 ; padding, and advance the payload counter past
378 it.
379 ;
380 ; This is the half of the path buffer that
381 SET_DEVICE_FULLPATH needs: the
382 ; directory and the filename go into ONE 256-
383 byte payload, so the cartridge
384 ; emits its buffer raw and FNPEND pads what is
385 left. The console cannot know
386 ; how many bytes that was, so the cartridge
387 publishes it at FNPLNL.
388 FNPWD: lda #FP_TXRAW
389 sta FNRSEL+FH_PATHO
390 lda FNP CNT
391 clc
392 adc FNPLNL
393 sta FNP CNT
394 rts
395 ;
396 ; FNWSTR -- append the NUL-terminated string at
397 FNPTL/H.
398 FNWSTR: ldy #0
399 FNWS1: lda (FNPTL),y
400 beq FNWS2
401 sta FNRSEL+FH_PATHC
402 iny
403 bne FNWS1
404 FNWS2: rts
405 ;
406 ; FNWRPL -- append up to Y bytes of the reply
407 window at offset X, stopping at
408 ; NUL. Cartridge to cartridge: the name never
409 lands in console RAM.
410 FNWRPL: lda FNRPLY,x
411 beq FNWR2
412 sta FNRSEL+FH_PATHC
413 inx
414 dey
415 bne FNWRPL
416 FNWR2: rts
417 ;
418 ; FNEOF -- Z set if the reply is the $7F,$7F
419 end-of-directory marker.
420 ;
421 ; Checking this is NOT optional. Past the end,
422 fujiDevice returns $7F,$7F
423 ; but a fujiNet-pc SD host returns "." over and
424 over, and once a client has
425 ; read past the end every later
426 SET_DIRECTORY_POSITION NAKs for the rest of
427 ; the session.
428 FNEOF: lda FNRPLY
429 cmp #$7F
430 bne FNEO1
431 lda FNRPLY+1
432 cmp #$7F
433 FNEO1: rts
434 ;
435 ;
436 ; INSCAN -- fold the joystick and console
437 switches into one active-high byte
438 ; and return only what is NEWLY pressed, so a
439 held stick steps once.
440 INSCAN: lda SWCHA

```

```

416 eor #$FF ; the port is
417 active low and #$F0 ; right, left,
418 down, up in bits 7-4
419 lsr a
420 lsr a
421 lsr a
422 sta INCUR
423 lda INPT4
424 bmt INSC1 ; bit 7 high =
425 fire not pressed
426 lda INCUR
427 ora #IN_FIRE
428 INCUR: lda SWCHB
429 and #$01
430 bne INSC2
431 lda INCUR
432 ora #IN_RST
433 INCUR: sta SWCHB
434 lda SWCHB
435 and #$02
436 bne INSC3
437 lda INCUR
438 ora #IN_SEL
439 INCUR: sta INPREV
440 INSC3: lda INPREV ; newly pressed
441 = current AND NOT previous
442 eor #$FF
443 and INCUR
444 pha
445 lda INCUR
446 sta INPREV
447 pla
448 rts

```

LISTINGS/COMMON/FUJIDISP.INC

```

1 ; fujidisp.inc -- the 48-pixel text display,
2 ; shared by every 2600 client.
3 ; The cartridge composes the glyphs (firmware/
4 ; src/vcs_render.c) and publishes
5 ; them as six 128-byte planes at $1800-$1AFF;
6 ; this is the console half, which
7 ; does nothing but stream them into GRP0/GRP1 on
8 ; a cycle-exact schedule.
9 ;
10 ; DINIT sets the TIA up and positions the
11 ; players. DLOOP runs frames forever.
12 ; Everything about WHY the constants are what
13 ; they are is in the comments
14 ; below -- they are the coordinates of a one-
15 ; pixel-wide window, found by
16 ; byte-comparing the raster against the renderer
17 ; with emu/dispcheck.py.
18 ;
19 ; ----- kernel timing constants
20 ;
21 ; These four are not taste. The 48-pixel block
22 ; is six player copies 2.67 cycles
23 ; apart and the kernel's stores are 3 cycles, so
24 ; where the block sits decides
25 ; whether each graphics update lands between
26 ; copies or inside one. The window
27 ; that satisfies all six groups is ONE PIXEL
28 ; wide, and these are its
29 ; coordinates, found by byte-comparing the
30 ; raster against the renderer
31 ; (emu/dispcheck.py) rather than by eye -- every
32 ; wrong setting still looks like
33 ; text.
34 ;
35 ; RNP0 EQU 16 ; NOPs before
36 ; RESP0 (2 cycles each)...
37 ; R3 EQU 1 ; ...plus this,
38 ; so RESP0 lands on cycle 38
39 ; HMP0 EQU 15 ; HMP0 = $F0;
40 ; move P0 right one pixel...
41 ; HM1 EQU 0 ; ...and leave
42 ; P1, closing the gap to eight
43 ; VDEL EQU 1 ; vertical delay
44 ; on both players
45 ;
46 ; ----- text plane bases
47 ;
48 ; FN_T_PLANE(0..5) from firmware/include/
49 ; fujit_mailbox.h.
50 ;
51 ; TP0 EQU $1800
52 ; TP1 EQU $1880
53 ; TP2 EQU $1900
54 ; TP3 EQU $1980
55 ; TP4 EQU $1A00
56 ; TP5 EQU $1A80
57 ;
58 ; TROWS EQU 21 ; FN_T_ROWS
59 ; TCELLH EQU 6 ; FN_T_CELL_H
60 ; TLINE EQU TROWS*TCELLH ; 126 scanlines
61 ; of text
62 ;
63 ; NTSC: 3 vsync + 37 vblank + 192 visible + 30
64 ; overscan = 262.
65 ;
66 ; VBLINES EQU 37
67 ; OSLINES EQU 30
68 ; PADLINE EQU (192-TLINE)/2 ; 33 above and
69 ; below
70 ;
71 ; ----- DINIT: TIA setup and player
72 ; positioning
73 ;
74 ; Two players, three copies each, eight pixels
75 ; apart: P0 draws groups 0,2,4
76 ; and P1 groups 1,3,5, interleaving to 48
77 ; contiguous pixels. VDELP is what
78 ; makes the six-write sequence land in the right
79 ; copies.
80 ;
81 ; ldx #03 ; NUSIZ: three
82 ; copies, close
83 ; sta NUSIZ0
84 ; sta NUSIZ1
85 ; ldx #VDEL
86 ; sta VDELP0
87 ; sta VDELP1
88 ; ldx #0E ; white
89 ; sta COLUP0
90 ; sta COLUP1
91 ; ldx #00
92 ; sta COLUBK
93 ;
94 ; Player positioning, transcribed from batari
95 ; Basic's std_kernel.asm rather
96 ; than re-derived. That kernel is proven, and
97 ; its setup is exact: RESP0 strobed
98 ; at cycle 43, RESP1 three cycles later at 46
99 ; (nine pixels, since two stores
100 ; cannot be closer), HMP0 = $E0 and HMP1 = $F0
101 ; to pull them to eight apart, and
102 ; HMOVE at cycle 73 -- late in the same line,
103 ; NOT immediately after a WSYNC.
104 ;
105 ; RDLY is the delay before the RESP0 store, in
106 ; cycles, built from NOPs plus an
107 ; optional 3-cycle zero-page store so that every
108 ; value is reachable: the copies
109 ; are 2.67 cycles apart, so this has to tune to
110 ; the cycle, and a NOP alone only
111 ; gets even numbers. RESP0 therefore completes
112 ; at RDLY+3; batari's 43 is
113 ; RDLY = 40,
114 ;
115 ; sta WSYNC
116 ; REPT RNOP
117 ; nop
118 ; ENDM
119 ; IF R3
120 ; sta PAD3 ; 3 cycles, so
121 ; odd delays are reachable
122 ; ENDF
123 ; sta RESP0
124 ; sta RESP1
125 ; ldx #HM0*16
126 ; sta HMP0
127 ; ldx #(HM1*16)&$FF
128 ; sta HMP1
129 ;
130 ; An EARLY HMOVE -- WSYNC, then strobe -- rather
131 ; than batari's late one at
132 ; cycle 73. A late HMOVE does not apply the
133 ; motion nibbles cleanly: nudging
134 ; HMP0 by one here widened the player gap from 8
135 ; pixels to 9 (the block grew
136 ; from 43 to 44 pixels of span) instead of
137 ; sliding the block. With the early
138 ; strobe the nibbles behave, and one-pixel steps
139 ; are what this kernel needs --
140 ; the wind between group 0 and group 5 failing
141 ; is a single pixel wide.
142 ;
143 ; sta WSYNC
144 ; HMOVE
145 ;
146 ; rts
147 ;
148 ; ----- DLOOP: draw frames, forever
149 ;
150 ; ----- frame -----
151 ;
152 ; ldx #2
153 ; sta VSYNC
154 ; sta WSYNC
155 ; sta WSYNC
156 ; sta WSYNC
157 ; ldx #0
158 ; sta VSYNC
159 ;
160 ; ldx #2
161 ; sta VBLANK
162 ; ldx #VBLINES
163 ; VBLP: sta WSYNC
164 ; dex
165 ; bne VBLP
166 ;
167 ; The client's per-frame hook, called while the
168 ; screen is still blanked and
169 ; the raster is nowhere near the text. Input
170 ; scanning belongs here; so does a
171 ; transaction, with the caveat that a network
172 ; round trip takes far longer than
173 ; a frame and the picture will stall for it.
174 ; Clients that need nothing define
175 ; APPVBL as an RTS.
176 ;
177 ; jsr APPVBL
178 ;
179 ; ldx #0
180 ; sta VBLANK
181 ; ldx #PADLINE
182 ; PADA: sta WSYNC
183 ; dex
184 ; bne PADA
185 ;
186 ; ----- the kernel -----
187 ;
188 ; 53 cycles of the 76 available. Y is never
189 ; clobbered: the textbook score
190 ; kernel reloads it from scratch because there
191 ; it holds the font row, but here
192 ; Y is the screen-wide scanline index and must
193 ; survive the whole frame.
194 ;
195 ; The stack pointer is a spare register here,
196 ; and that is not a flourish: the
197 ; six player copies are 2.67 cycles apart while
198 ; a zero-page store is 3, so the
199 ; four late writes drift a full cycle across the
200 ; block. Recovering a byte
201 ; from zero page costs 3 cycles; 'tsx' costs 2.
202 ; That one cycle is the whole difference between a
203 ; schedule that closes and one that does not --
204 ; it is why batari Basic's kernel
205 ; parks a byte in S with 'tsx' and recovers it
206 ; with 'tsx', and why the seventh
207 ; write is 'sty GRP0': its value is a don't-
208 ; care, so Y (the scanline counter)
209 ; serves and no register need be spared for it.
210 ;
211 ; Nothing here may use the stack between the txs
212 ; and the restore below -- no
213 ; JSR; and the 6507 has no interrupts to worry
214 ; about.
215 ;
216 ; tsx
217 ; stx SAVSP
218 ; ldx #0
219 ; KERN: sta WSYNC
220 ; ldx TP0,y ; 4
221 ; sta GRP0 ; 3 -> 7
222 ;
223 ; new GRP0 = group 0

```

```

150      lda      TP1,y      ; 4 -> 11
151      sta      GRP1      ; 3 -> 14
      displays_group 0
152      lda      TP2,y      ; 4 -> 18
153      sta      GRP0      ; 3 -> 21
      displays_group 1
154      lda      TP4,y      ; 4 -> 25
155      tax      ; 2 -> 27
156      txs      ; 2 -> 29
      park_group 4 in 5
157      lda      TP3,y      ; 4 -> 33
158      tax      ; 2 -> 35
159      lda      TP5,y      ; 4 -> 39
160      stx      GRP1      ; 3 -> 42
      displays_group 2
161      tsx      ; 2 -> 44
      recover_group 4
162      stx      GRP0      ; 3 -> 47
      displays_group 3
163      sta      GRP1      ; 3 -> 50
      displays_group 4
164      sty      GRP0      ; 3 -> 53
      displays_group 5; value unused
165      iny
166      cpy      #TLINES
167      bne      KERN
168      ldx      SAVSP
169      txs
170
171      ; Blank both players. Three writes, not two:
172      ; with VDELP set, a write to GRP0
173      ; only reloads the DELAYED GRP1 and vice versa,
174      ; so clearing both displayed
175      ; registers takes GRP0, GRP1, GRP0. Toggling
176      ; VDELP off and back on instead --
177      ; the obvious-looking version -- RESTORES the
178      ; stale delayed registers, and the
179      ; padding lines above the text then redraw the
180      ; last kernel line.
181      lda      #0
182      sta      GRP0
183      sta      GRP1
184      sta      GRP0
185
186      ldx      #PADLINE
187      sta      WSYNC
188      dex
189      bne      PADB
190
191      lda      #2
192      sta      VBLANK
193      ldx      #OSLINES
194      sta      WSYNC
195      dex
196      bne      OSLP
197      jmp      DLOOP

```

LISTINGS/COMMON/VCS.INC

```

1 ; vcs.inc -- TIA and RIOT equates for
2 Macroassembler AS.
3 ; The 2600 world's standard vcs.h is a dasm
4 companion file and is explicitly
5 ; "please do not redistribute", so this is
6 written from the hardware register
7 ; map rather than copied. AS is the assembler
8 the rest of this family already
9 ; uses -- the 02 (8048), arcadia (2650) and
10 channelf (F8) ports all build with
11 ; it and CI already caches its build -- so the
12 2600 client joins them rather
13 ; than adding dasm as a seventh toolchain.
14
15 ; The 6507 has thirteen address lines, so
16 everything mirrors. These are the
17 ; canonical low addresses; TIA writes are $00-
18 $2C, TIA reads $00-$DD, and the
19 ; RIOT sits at $0280 with its RAM at $0800.
20
21 ; ----- TIA, write -----
22 VSYNC EQU $00 ; bit 1:
23 vertical_sync
24 VBLANK EQU $01 ; bit 1: blank,
25 bit 6/7: input latch/dump
26 WSYNC EQU $02 ; halt the CPU
27 until the next scanline
28 RSYNC EQU $03
29 NUSIZ0 EQU $04 ; player 0
30 copies / missile size
31 NUSIZ1 EQU $05
32 COLUP0 EQU $06
33 COLUP1 EQU $07
34 COLUPF EQU $08
35 COLUBK EQU $09
36 CTRLPF EQU $0A
37 REFP0 EQU $0B
38 REFP1 EQU $0C
39 PF0 EQU $0D
40 PF1 EQU $0E
41 PF2 EQU $0F
42 RESR0 EQU $10 ; strobe: reset
43 player 0 to the beam
44 RESP1 EQU $11
45 RESM0 EQU $12
46 RESM1 EQU $13
47 RESBL EQU $14
48
49 AUDC0 EQU $15
50 AUDC1 EQU $16
51 AUDF0 EQU $17
52 AUDF1 EQU $18
53 AUDV0 EQU $19
54 AUDV1 EQU $1A
55 GRP0 EQU $1B ; player 0
56 graphics
57 GRP1 EQU $1C
58 ENAM0 EQU $1D
59 ENAM1 EQU $1E
60 ENABL EQU $1F
61 HMP0 EQU $20 ; fine motion,
62 high nibble, +7..-8
63 HMP1 EQU $21
64 HMM0 EQU $22
65 HMM1 EQU $23
66 HMBL EQU $24 ; bit 0: delay
67 VDLP0 EQU $25
68 GRP0 by one write
69 VDLP1 EQU $26
70 VDELBL EQU $27
71 RESMP0 EQU $28
72 RESMP1 EQU $29
73 HMOVE EQU $2A ; strobe: apply
74 the HM registers
75 HMCCLR EQU $2B
76 CXCLR EQU $2C
77
78 ; ----- TIA, read -----
79 CXM0P EQU $00
80 CXM1P EQU $01
81 CXP0FB EQU $02
82 CXP1FB EQU $03
83 CXM0FB EQU $04
84 CXM1FB EQU $05
85 CXBLPF EQU $06
86 CXPPPM EQU $07
87 INPT0 EQU $08
88 INPT1 EQU $09
89 INPT2 EQU $0A
90 INPT3 EQU $0B
91 INPT4 EQU $0C ; bit 7 low:
92 joystick 0 fire
93 INPT5 EQU $0D
94
95 ; ----- RIOT -----

```

```

78 SWCHA EQU $0280 ; joysticks: P0
   in bits 7-4 (R,L,D,U low)
79 SWACNT EQU $0281
80 SWCHB EQU $0282 ; bit 0 RESET,
   bit 1 SELECT (low = pressed)
81 SWBCNT EQU $0283
82 INTIM EQU $0284
83 TIM1T EQU $0294
84 TIM8T EQU $0295
85 TIM64T EQU $0296
86 TI024T EQU $0297
87
88 ; Joystick 0 direction bits in SWCHA, active
   low.
89 J0RIGHT EQU $80
90 J0LEFT EQU $40
91 J0DOWN EQU $20
92 J0UP EQU $10
93

```

LISTINGS/COMMON/NETDEFS.INC

```

1 ; netdefs.inc -- the N: (network) device, for
   the client library.
2 ;
3 ; Device ids from include/fujiDeviceID.h,
   command bytes from
4 ; include/fujiCommandID.h, open modes from lib/
   network-protocol/HTTP.cpp and
5 ; the STATUS reply layout from lib/device/rs232/
   network.cpp. The testrom's
6 ; fujinet.inc covers the FUJI device only; this
   is the rest of what a client
7 ; that talks to the network needs, kept separate
   so that file stays verbatim.
8 NETDEV EQU $71 ; N1: (N2:-N8:
   are $72-$78)
9 NCOPEX EQU $4F ; 'O' open:
   params mode, translation; payload = devicespec
10 NCCLSE EQU $43 ; 'C' close
11 NCREAD EQU $52 ; 'R' read: ONE
   two-byte param, the count
12 NCWRITE EQU $57 ; 'W' write: ONE
   two-byte param, then the payload
13 NCSTAT EQU $53 ; 'S' status:
   two one-byte params, both 0
14 NCPARSE EQU $50 ; 'P' parse the
   reply as JSON
15 NCQUERY EQU $51 ; 'Q' query a
   JSON path; payload = the path
16 NCJSON EQU $FC ; channel mode:
   second param 0 = protocol, 1 = JSON
17 NCHTTPM EQU $4D ; 'M' HTTP
   channel mode: second param 0 body, 1 collect
   headers, 2
18 get headers, 3 set headers, 4 POST data
19 NMREAD EQU 4 ; open mode:
   read (HTTP GET)
20 NMWRITE EQU 8 ; write (HTTP
   PUT)
21 NMHTTTPG EQU 12 ; read/write
   (HTTP GET, headers allowed)
22 NMPOST EQU 13 ; HTTP POST
23 NTRNONE EQU 0 ; translation:
   none
24 NTRCR EQU 1 ; CR to/from the
   host's EOL
25 NTRLF EQU 2 ; LF
26 NTRCRLF EQU 3 ; CR LF
27 NSAVLO EQU 0 ; STATUS reply:
   bytes waiting, low
28 NSAVHI EQU 1 ; high
29 NSCONN EQU 2 ; connected
30 NSDEVST EQU 3 ;
   nDevStatus_t: 1 = success
31

```

LISTINGS/COMMON/DEVDEFS.INC

```

1 ; devdefs.inc -- the other devices, and the FUJI
   commands the testrom's
2 ; fujinet.inc does not name. Values from
   include/fujiDeviceID.h and
3 ; include/fujiCommandID.h; the appkey struct
   from lib/device/fujiDevice/
4 ; AppKeyMixin.h. Kept apart from fujinet.inc so
   that file stays verbatim.
5 ;
6 ; ----- device ids -----
7 DSKDEV EQU $31 ; D1: (D2:-D8:
   are $32-$38)
8 PRNDEV EQU $40 ; P1:
9 CLKDEV EQU $45 ; the clock
   (APETIME)
10 MDMDEV EQU $50 ; R: the modem
11 CPMDEV EQU $5A ; CP/M
12 ; ----- FUJI ($70) commands
13 ; -----
14 FNCIRST EQU $FF ; RESET
15 FNCSSSD EQU $FE ; GET_SSID
16 FNCSKAN EQU $FD ; SCAN_NETWORKS
17 FNCSCLR EQU $FC ;
   GET_SCAN_RESULT
18 FNCSSSD EQU $FB ; SET_SSID
19 FNCWIFI EQU $FA ; GET_WIFISTATUS
20 FNCWHST EQU $F3 ;
   WRITE_HOST_SLOTS
21 FNCRDEV EQU $F2 ;
   READ_DEVICE_SLOTS
22 FNCWDEV EQU $F1 ;
   WRITE_DEVICE_SLOTS
23 FNCWIEN EQU $EA ;
   GET_WIFI_ENABLED
24 FNCUIG EQU $E9 ; UNMOUNT_IMAGE
25 FNCADPC EQU $E8 ;
   GET_ADAPTERCONFIG
26 FNCNEWD EQU $E7 ; NEW_DISK
27 FNCUHSST EQU $E6 ; UNMOUNT_HOST
28 FNCGDPS EQU $E5 ;
   GET_DIRECTORY_POSITION
29 FNCSPHX EQU $E1 ;
   SET_HOST_PREFIX
30 FNCGPX EQU $E0 ;
   GET_HOST_PREFIX
31 FNCANWR EQU $DE ; WRITE_APPKEY
32 FNCANRD EQU $DD ; READ_APPKEY
33 FNCAPKO EQU $DC ; OPEN_APPKEY
34 FNCACKL EQU $DB ; CLOSE_APPKEY
35 FNCGCP EQU $DA ;
   GET_DEVICE_FULPATH
36 FNCBCT EQU $D9 ; CONFIG_BOOT
37 FNCOCOPY EQU $D8 ; COPY_FILE
38 FNCMALL EQU $D7 ; MOUNT_ALL

```

```

39 FNC5BM EQU $D6 ; SET_BOOT_MODE
40 FNC6RAND EQU $D3 ; RANDOM_NUMBER
41 FNCB64EI EQU $D0 ;
   BASE64_ENCODE_INPUT
42 FNCB64EC EQU $CF ;
   BASE64_ENCODE_COMPUTE
43 FNCB64EL EQU $CE ;
   BASE64_ENCODE_LENGTH
44 FNCB64EO EQU $CD ;
   BASE64_ENCODE_OUTPUT
45 FNCB64DI EQU $CC ;
   BASE64_DECODE_COMPUTE
46 FNCB64DC EQU $CB ;
   BASE64_DECODE_COMPUTE
47 FNCB64DL EQU $CA ;
   BASE64_DECODE_LENGTH
48 FNCB64DO EQU $C9 ;
   BASE64_DECODE_OUTPUT
49 FNCBSHI EQU $C8 ; HASH_INPUT
50 FNCBSHC EQU $C7 ; HASH_COMPUTE
51 FNCBSHL EQU $C6 ; HASH_LENGTH
52 FNCBSHO EQU $C5 ; HASH_OUTPUT
53 FNCBSHN EQU $C3 ;
   HASH_COMPUTE_NO_CLEAR
54 FNCBSHX EQU $C4 ; HASH_CLEAR
55 FNCQRO EQU $BF ; ORCODE_OUTPUT
56 FNCQRL EQU $BE ; ORCODE_LENGTH
57 FNCQRE EQU $BD ; ORCODE_ENCODE
58 FNCQRI EQU $BC ; ORCODE_INPUT
59 FNCGUID EQU $BB ; GENERATE_GUID
60 FNCSTAT EQU $B3 ; STATUS
61 FNCRDY EQU $B0 ; DEVICE_READY
62 ;
63 ; ----- appkeys -----
64 ; OPEN_APPKEY's payload is ONE six-byte packed
   struct: creator (u16, little-
65 ; endian), app, key, mode, and a reserved byte
   that is not optional.
66 AKMRD EQU 0 ; mode: read
67 AKMRW EQU 1 ; mode: write
68 AKCREAT EQU 1 ; the shared
   FujiNet username slot...
69 AKAPP EQU 1 ; ...creator 1,
   app 1, key 0, which every
70 AKKEY EQU 0 ; game in the
   family reads
71 ;
72 ; ----- the clock ($45)
   -----
73 CKGTIME EQU $93 ; GETTIME: 7
   bytes, binary
74 CKISOL EQU $49 ; 'I'
   GET_ISO_LOCAL: "YYYY-MM-DDTHH:MM:SS"...
75 CKISOU EQU $5A ; 'Z'
   GET_ISO_UTC
76 CKGEN EQU $47 ; 'G'
   GET_GENERAL
77 CKTZLEN EQU $4C ; 'L' GETTZ_LEN
78 CKSETTZ EQU $59 ; SETTZ: payload
   = the POSIX TZ string
79 CKTZTIM EQU $9A ; GETTZTIME
80 ;
81 ; ----- hash algorithms
   (HASH_COMPUTE's parameter) -----
82 HAMD5 EQU 0
83 HASHA1 EQU 1
84 HASHA256 EQU 3
85 HASHA512 EQU 4
86 ;
87 ; ----- the path buffers, beyond the
   testrom's four ops -----
88 FP_POPCH EQU 4 ; drop ONE
   character (the keyboard's backspace)
89 FP_SEL0 EQU 5 ; subsequent ops
   act on buffer 0...
90 FP_SEL1 EQU 6
91 FP_SEL2 EQU 7
92 FP_SEL3 EQU 8 ; ...buffer 3,
   the edit scratch
93 FP_CMIT EQU 9 ; selected <-
   the edit scratch
94 FP_SEED EQU 10 ; the edit
   scratch <- selected
95 ;
96 ; ----- the blit port
   -----
97 FH_BSL EQU $F5 ; source offset,
   low (into the reply window)
98 FH_BSH EQU $F6 ; source offset,
   high
99 FH_BDL EQU $F7 ; destination
   offset, low (into the planes)
100 FH_BDH EQU $F8 ; destination
   offset, high
101 FH_BCNT EQU $F9 ; count
102 FH_BGO EQU $FA ; data = the
   transform: fire
103 FNBGEN EQU $1F19 ; FN_B_BLITGEN:
   bumped when a blit has landed
104 FB_RAW EQU 0 ; copy bytes
   unchanged
105 FB_TEXT EQU 1 ; ASCII -> a
   text row
106 FB_FIELD EQU 2 ; a 10x10 game
   field -> ten text rows
107 FB_HULLS EQU 3 ; five
   placements -> a hull overlay
108 FB_SEA EQU 4 ; fill the
   composed board with sea
109 FB_CELL EQU 5 ; board[cnt] =
   the low byte of src

```

LISTINGS/ASM/HELLO.ASM

```
1 ; hello.asm -- M0: the cart-rendered 48-pixel
2 ; text display, proven standalone.
3 ;
4 ; This is the rung worth failing early. Every
5 ; sibling console in this family
6 ; had a character generator or a framebuffer and
7 ; rendered console-side; the
8 ; 2600 has neither, so the cartridge composes
9 ; GRP bytes and the console runs a
10 ; cycle-exact kernel over them. Nothing else in
11 ; the port has a template for
12 ; that, and if it does not come up cleanly the
13 ; CONFIG rung is dead and the
14 ; memory map changes shape.
15 ;
16 ; So this image contains NO mailbox and NO
17 ; network: the text planes are baked
18 ; at $1800-$1AFF by tools/vcsfont.py, which is
19 ; the same renderer the cartridge
20 ; runs in C (firmware/src/vcs_render.c, byte-
21 ; compared against it by
22 ; host_test/test_render.c). It is a plain 4K
23 ; cartridge and stock MAME boots it
24 ; with no patching whatsoever -- which is the
25 ; point. The display is proven
26 ; before a single line of emulator work exists.
27 ;
28 ; Exit test: emu/dispcheck.py decodes the raster
29 ; back into plane bytes and
30 ; requires all 756 to be byte-identical to the
31 ; renderer's.
32 ;
33 CPU 6502
34 INCLUDE "vcs.inc"
35 ;
36 ; ----- zero page -----
37 PAD3 EQU $81 ; scratch: a 3-
38 cycle store used only as a delay
39 SAVSP EQU $82 ; stack pointer,
40 parked across the kernel
41
42 ORG $1000
43
44 START: sei
45 cld
46 ldx #$FF
47 txs
48 lda #0
49 CLRLP: sta $00,x ; $00-$7F is the
50 TIA, $80-$FF is RAM
51 dex
52 bne CLRLP
53 sta $00
54
55 jsr DINIT
56 jmp DLOOP
57
58 APPVBL: rts ; this client
59 needs no per-frame work
60
61 INCLUDE "fujidisp.inc"
62 ;
63 ; ----- the text planes
64 ORG $1800
65 INCLUDE "screen.inc"
66 ;
67 ; ----- the fixed tail
68
69 ORG $1FFC
70 DW START ; RESET
71 DW START ; BRK -- a
72 runaway reboots rather than hangs
73
74 END
```

LISTINGS/ASM/FUJITEST.ASM

```
1 ; fujitest.asm -- M1: one real transaction, and
2 ; proof the cart survives a reset.
3 ;
4 ; Asks the FujiNet for
5 ; GET_ADAPTERCONFIG_EXTENDED and puts the live
6 ; SSID, IP
7 ; address and firmware version on screen.
8 ; Nothing is faked: the bytes come off
9 ; a socket to fujinet-pc, through the
10 ; cartridge's own fujimail.c, and are
11 ; rendered by the cartridge's own compositor.
12 ;
13 ; The reply is 240 bytes and the window is 512,
14 ; so it lands in slice 0 whole
15 ; and no slice paging is needed -- which is why
16 ; the reply window is 512 and not
17 ; 256 (see fuji_mailbox.h).
18 ;
19 ; THE TRANSACTION RUNS BEFORE THE DISPLAY
20 ; STARTS, with the screen blanked.
21 ; Under emulation the socket round trip is
22 ; synchronous so this is invisible;
23 ; on real hardware it is a brief blank frame
24 ; before the picture appears. A
25 ; client that wants to stay in sync while it
26 ; talks has to do its transactions
27 ; from inside the frame loop, which fujicfg
28 ; will.
29 ;
30 ; Exit test: emu/drive.lua reads the rendered
31 ; planes back and requires the SSID
32 ; to be on screen; emu/resettest.lua then
33 ; toggles the RESET switch and requires
34 ; the NEXT sequence number to be ACKSEQ+1, not 1
35 ; -- the cart has no reset line,
36 ; so it keeps its state across a console reset
37 ; and a client that restarted its
38 ; own counter would collide with a sequence
39 ; already answered.
40 ;
41 CPU 6502
42 INCLUDE "vcs.inc"
43 ;
44 ; ----- zero page -----
45 PAD3 EQU $81 ; the display
46 kernel's 3-cycle pad target
47 SAVSP EQU $82 ; stack pointer,
48 parked across the kernel
49 APERR EQU $8C ; what went
50 wrong, if anything
51
52 INCLUDE "fujinet.inc"
```

```

34 ; Field offsets inside
   GET_ADAPTERCONFIG_EXTENDED, from the packed
35 ; AdapterConfigExtended in lib/device/
   fujDevice/fujDevice.h. Every field this
36 ; client wants is below 256, so a single-byte
   index reaches all of them.
37 AC_SSID EQU 0 ; char[33]
38 AC_VER EQU 125 ; char[15]
39 AC_SIP EQU 140 ; char[16], the
   IP already formatted as text
40
41 ORG $1000
42
43 ; ----- cold start -----
44 START: sei
45 cld
46 ldx # $FF
47 txs
48 lda #0
49 CLRLP: sta $00,x ; $00-$7F is the
   TIA, $80-$FF is RAM
50 dex
51 bne CLRLP
52 sta $00
53
54 lda #2 ; keep the
   screen blanked while we talk
55 sta VBLANK
56 lda #0
57 sta APERR
58
59 ; ----- the transaction
60 jsr FNARM ; open the
   decode gate
61 jsr FNCHK
62 beq GOTCART
63 lda #FNECART ; no cartridge
   is answering
64 sta APERR
65 jmp SHOW
66
67 GOTCART:
68 lda #FNDEVF
69 sta FNDEV
70 lda #FNCDPX
71 sta FNCD
72 lda #0
73 sta FNNPR
74 jsr FNBEG
75 jsr FNGO
76 sta APERR
77 cmp #FNEOK
78 bne SHOW
79 jsr FNACK
80 sta APERR
81
82 ; ----- put it on screen
83 SHOW: lda #(TTITLE)&$FF
84 sta FNPTL
85 lda #(TTITLE)>>8
86 sta FNPTRH
87 lda #0
88 jsr FNRSTR
89
90 lda APERR
91 beq SHOWOK
92
93 ; The failure screen is as useful as the success
   one: it is what a burned
94 ; EPROM shows when there is no cartridge behind
   it.
95 lda #(TFAIL)&$FF
96 sta FNPTL
97 lda #(TFAIL)>>8
98 sta FNPTRH
99 lda #2
100 jsr FNRSTR
101 lda #3
102 jsr FNROWA
103 lda APERR
104 jsr FNHEX
105 jsr FNEHDR
106 jmp RUN
107
108 SHOWOK: lda #(TSSID)&$FF
109 sta FNPTL
110 lda #(TSSID)>>8
111 sta FNPTRH
112 lda #2
113 jsr FNRSTR
114 lda #3 ; row
115 ldx #AC_SSID ; reply offset
116 ldw #FNTCOL ; at most a
   rowful
117 jsr FNRRL
118
119 lda #(TIP)&$FF
120 sta FNPTL
121 lda #(TIP)>>8
122 sta FNPTRH
123 lda #5
124 jsr FNRSTR
125 lda #6
126 ldx #AC_SIP
127 ldw #FNTCOL
128 jsr FNRRL
129
130 lda #(TVER)&$FF
131 sta FNPTL
132 lda #(TVER)>>8
133
134 sta FNPTRH
135 lda #8
136 jsr FNRSTR
137 lda #9
138 ldx #AC_VER
139 ldw #FNTCOL
140 jsr FNRRL
141
142 ; The sequence number, in hex. This is the
   rung's real evidence: after a
   console RESET it must read 02, not 01.
143 lda #(TSEQ)&$FF
144 sta FNPTL
145 lda #(TSEQ)>>8
146 sta FNPTRH
147 lda #11
148 jsr FNRSTR
149 lda #12
150 jsr FNROWA
151 lda FNACKS
152 jsr FNHEX
153 jsr FNEHDR
154
155 RUN: lda #0
156 sta VBLANK
157 jsr DINIT
158 jmp DLOOP
159
160 ; ----- strings -----
161 TTITLE: DB "FUJINET 2600",0
162 TSSID: DB "SSID",0
163 TIP: DB "IP",0
164 TVER: DB "VERSION",0
165 TSEQ: DB "ACKSEQ",0
166 TFAIL: DB "FAILED ERR",0
167
168 INCLUDE "fujilib.inc" ; this client
   APPVBL: rts ; needs no per-frame work
169
170 INCLUDE "fujidisp.inc"
171
172 ; ----- the fixed tail
173
174 ; $1FFC must live in the fixed half so a console
   RESET is survivable whatever
175 ; bank happens to be mapped low.
176 ORG $1FFC
177 DW START ; RESET
178 DW START ; BRK -- a
   runaway reboots rather than hangs
179
180 END

```

LISTINGS/ASM/NETGET.ASM

```

1 ; netget.asm -- fetch a URL through the N:
2 ; device and show the text.
3 ; OPEN the devicespec as an HTTP GET, STATUS
4 ; until the byte count settles,
5 ; READ up to 228 bytes -- nineteen rows of
6 ; twelve -- into the reply window,
7 ; then stream the reply into text rows a line at
8 ; a time: LF ends a row, CR is
9 ; skipped, a row is at most twelve characters.
10 ; Then CLOSE. Nothing of the
11 ; reply ever lands in console RAM: the cartridge
12 ; holds it and the cartridge
13 ; composes it, and the console's part is a
14 ; cursor and a row number.
15 ;
16 ; The transaction runs before the display
17 ; starts, with the screen blanked, as
18 ; fujitest.asm does; a client that wants to keep
19 ; a picture up while it talks
20 ; does its transactions from inside the frame
21 ; loop, which Battleship shows.
22
23 CPU 6502
24 INCLUDE "vcs.inc"
25
26 ; ----- zero page -----
27 PAD3 EQU $81 ; the display
28 kernel's 3-cycle pad target
29 SAVSP EQU $82 ; stack pointer,
30 parked across the kernel
31 APERR EQU $8F ; what went
32 wrong, if anything
33 NAVLO EQU $90 ; bytes waiting,
34 from STATUS
35 NAVHI EQU $91
36 NPRLO EQU $92 ; the previous
37 STATUS reading
38 NPRHI EQU $93
39 NSRC EQU $94 ; the reply
40 cursor
41 NROW EQU $95 ; the text row
42 being composed
43 NCOL EQU $96 ; characters in
44 it so far
45 NTRY EQU $97 ; STATUS
46 readings taken
47
48 INCLUDE "fujinet.inc"
49 INCLUDE "netdefs.inc"
50
51 NMAX EQU 228 ; nineteen rows
52 of twelve rows 2-20
53 NROW EQU 2 ; the first text
54 row of the reply
55 NTRIES EQU 30 ; STATUS
56 readings before giving up
57
58 ORG $1000
59
60 ; ----- cold start -----
61 START: sei
62 cld
63 ldx #0
64 txs
65 lda #0
66 CLRLP: sta $00,x
67 TIA, $80-$FF is RAM ; $00-$7F is the
68 dex
69 bne CLRLP
70 sta $00
71
72 lda #2 ; keep the
73 screen blanked while we talk
74 sta VBLANK
75 lda #0
76 sta APERR
77
78 jsr FNARM ; open the
79 decode gate
80 jsr FNCHK
81 beq GOTCART
82 lda #FNENOC ; no cartridge
83 is answering
84 sta APERR
85 jmp SHOW
86
87 ; ----- OPEN -----
88 ; Two one-byte parameters -- the mode and the
89 ; translation -- then the
90 ; devicespec as the payload, streamed a byte at
91 ; a time from ROM.
92 GOTCART: lda #NETDEV
93 sta #FNDEV
94 lda #NCOPEN
95 sta #FNCMD
96 lda #2
97 sta #FNNPR
98 jsr #FNBEQ
99 lda #NMHTTPG
100 jsr #FNPB
101 lda #NTRNONE
102 jsr #FNPB
103 ldy
104 NURL: ldy
105 beq NURL
106 sta FNTX
107
108 iny
109 bne NURL
110 NURL: jsr NGOACK
111
112 ; ----- STATUS, until it settles
113 ; The adapter reports STATUS as soon as SOME of
114 ; the response has arrived, so
115 ; one reading can be a fraction of the page.
116 ; Take readings until two agree
117 ; and the count is not zero.
118 lda #0
119 sta #NPRLO
120 lda #NPRHI
121 lda #0
122 sta #NTRY
123 NSTAT: lda #NETDEV
124 sta #FNDEV
125 lda #FNCSTAT
126 lda #FNCMD
127 lda #2
128 sta #FNNPR
129 jsr #FNBEQ
130 lda #0
131 jsr #FNPB
132 lda #0
133 jsr #FNPB
134 lda #0
135 jsr #FNPB
136 lda #0
137 jsr #FNPB
138 lda #0
139 jsr #FNPB
140 lda #0
141 jsr #FNPB
142 lda #0
143 jsr #FNPB
144 lda #0
145 jsr #FNPB
146 lda #0
147 jsr #FNPB
148 lda #0
149 jsr #FNPB
150 lda #0
151 jsr #FNPB
152 lda #0
153 jsr #FNPB
154 lda #0
155 jsr #FNPB
156 lda #0
157 jsr #FNPB
158 lda #0
159 jsr #FNPB
160 lda #0
161 jsr #FNPB
162 lda #0
163 jsr #FNPB
164 lda #0
165 jsr #FNPB
166 lda #0
167 jsr #FNPB
168 lda #0
169 jsr #FNPB
170 lda #0
171 jsr #FNPB
172 lda #0
173 jsr #FNPB
174 lda #0
175 jsr #FNPB
176 lda #0
177 jsr #FNPB
178 lda #0
179 jsr #FNPB
180 lda #0
181 jsr #FNPB
182 lda #0
183 jsr #FNPB
184 lda #0
185 jsr #FNPB
186 lda #0
187 jsr #FNPB
188 lda #0
189 jsr #FNPB
190 lda #0
191 jsr #FNPB
192 lda #0
193 jsr #FNPB
194 lda #0
195 jsr #FNPB
196 lda #0
197 jsr #FNPB
198 lda #0
199 jsr #FNPB
200 lda #0
201 jsr #FNPB
202 lda #0
203 jsr #FNPB
204 lda #0
205 jsr #FNPB
206 lda #0
207 jsr #FNPB
208 lda #0
209 jsr #FNPB
210 lda #0
211 jsr #FNPB
212 lda #0
213 jsr #FNPB
214 lda #0
215 jsr #FNPB
216 lda #0
217 jsr #FNPB
218 lda #0
219 jsr #FNPB
220 lda #0
221 jsr #FNPB
222 lda #0
223 jsr #FNPB
224 lda #0
225 jsr #FNPB
226 lda #0
227 jsr #FNPB
228 lda #0
229 jsr #FNPB
230 lda #0
231 jsr #FNPB
232 lda #0
233 jsr #FNPB
234 lda #0
235 jsr #FNPB
236 lda #0
237 jsr #FNPB
238 lda #0
239 jsr #FNPB
240 lda #0
241 jsr #FNPB
242 lda #0
243 jsr #FNPB
244 lda #0
245 jsr #FNPB
246 lda #0
247 jsr #FNPB
248 lda #0
249 jsr #FNPB
250 lda #0
251 jsr #FNPB
252 lda #0
253 jsr #FNPB
254 lda #0
255 jsr #FNPB
256 lda #0
257 jsr #FNPB
258 lda #0
259 jsr #FNPB
260 lda #0
261 jsr #FNPB
262 lda #0
263 jsr #FNPB
264 lda #0
265 jsr #FNPB
266 lda #0
267 jsr #FNPB
268 lda #0
269 jsr #FNPB
270 lda #0
271 jsr #FNPB
272 lda #0
273 jsr #FNPB
274 lda #0
275 jsr #FNPB
276 lda #0
277 jsr #FNPB
278 lda #0
279 jsr #FNPB
280 lda #0
281 jsr #FNPB
282 lda #0
283 jsr #FNPB
284 lda #0
285 jsr #FNPB
286 lda #0
287 jsr #FNPB
288 lda #0
289 jsr #FNPB
290 lda #0
291 jsr #FNPB
292 lda #0
293 jsr #FNPB
294 lda #0
295 jsr #FNPB
296 lda #0
297 jsr #FNPB
298 lda #0
299 jsr #FNPB
300 lda #0
301 jsr #FNPB
302 lda #0
303 jsr #FNPB
304 lda #0
305 jsr #FNPB
306 lda #0
307 jsr #FNPB
308 lda #0
309 jsr #FNPB
310 lda #0
311 jsr #FNPB
312 lda #0
313 jsr #FNPB
314 lda #0
315 jsr #FNPB
316 lda #0
317 jsr #FNPB
318 lda #0
319 jsr #FNPB
320 lda #0
321 jsr #FNPB
322 lda #0
323 jsr #FNPB
324 lda #0
325 jsr #FNPB
326 lda #0
327 jsr #FNPB
328 lda #0
329 jsr #FNPB
330 lda #0
331 jsr #FNPB
332 lda #0
333 jsr #FNPB
334 lda #0
335 jsr #FNPB
336 lda #0
337 jsr #FNPB
338 lda #0
339 jsr #FNPB
340 lda #0
341 jsr #FNPB
342 lda #0
343 jsr #FNPB
344 lda #0
345 jsr #FNPB
346 lda #0
347 jsr #FNPB
348 lda #0
349 jsr #FNPB
350 lda #0
351 jsr #FNPB
352 lda #0
353 jsr #FNPB
354 lda #0
355 jsr #FNPB
356 lda #0
357 jsr #FNPB
358 lda #0
359 jsr #FNPB
360 lda #0
361 jsr #FNPB
362 lda #0
363 jsr #FNPB
364 lda #0
365 jsr #FNPB
366 lda #0
367 jsr #FNPB
368 lda #0
369 jsr #FNPB
370 lda #0
371 jsr #FNPB
372 lda #0
373 jsr #FNPB
374 lda #0
375 jsr #FNPB
376 lda #0
377 jsr #FNPB
378 lda #0
379 jsr #FNPB
380 lda #0
381 jsr #FNPB
382 lda #0
383 jsr #FNPB
384 lda #0
385 jsr #FNPB
386 lda #0
387 jsr #FNPB
388 lda #0
389 jsr #FNPB
390 lda #0
391 jsr #FNPB
392 lda #0
393 jsr #FNPB
394 lda #0
395 jsr #FNPB
396 lda #0
397 jsr #FNPB
398 lda #0
399 jsr #FNPB
400 lda #0
401 jsr #FNPB
402 lda #0
403 jsr #FNPB
404 lda #0
405 jsr #FNPB
406 lda #0
407 jsr #FNPB
408 lda #0
409 jsr #FNPB
410 lda #0
411 jsr #FNPB
412 lda #0
413 jsr #FNPB
414 lda #0
415 jsr #FNPB
416 lda #0
417 jsr #FNPB
418 lda #0
419 jsr #FNPB
420 lda #0
421 jsr #FNPB
422 lda #0
423 jsr #FNPB
424 lda #0
425 jsr #FNPB
426 lda #0
427 jsr #FNPB
428 lda #0
429 jsr #FNPB
430 lda #0
431 jsr #FNPB
432 lda #0
433 jsr #FNPB
434 lda #0
435 jsr #FNPB
436 lda #0
437 jsr #FNPB
438 lda #0
439 jsr #FNPB
440 lda #0
441 jsr #FNPB
442 lda #0
443 jsr #FNPB
444 lda #0
445 jsr #FNPB
446 lda #0
447 jsr #FNPB
448 lda #0
449 jsr #FNPB
450 lda #0
451 jsr #FNPB
452 lda #0
453 jsr #FNPB
454 lda #0
455 jsr #FNPB
456 lda #0
457 jsr #FNPB
458 lda #0
459 jsr #FNPB
460 lda #0
461 jsr #FNPB
462 lda #0
463 jsr #FNPB
464 lda #0
465 jsr #FNPB
466 lda #0
467 jsr #FNPB
468 lda #0
469 jsr #FNPB
470 lda #0
471 jsr #FNPB
472 lda #0
473 jsr #FNPB
474 lda #0
475 jsr #FNPB
476 lda #0
477 jsr #FNPB
478 lda #0
479 jsr #FNPB
480 lda #0
481 jsr #FNPB
482 lda #0
483 jsr #FNPB
484 lda #0
485 jsr #FNPB
486 lda #0
487 jsr #FNPB
488 lda #0
489 jsr #FNPB
490 lda #0
491 jsr #FNPB
492 lda #0
493 jsr #FNPB
494 lda #0
495 jsr #FNPB
496 lda #0
497 jsr #FNPB
498 lda #0
499 jsr #FNPB
500 lda #0
501 jsr #FNPB
502 lda #0
503 jsr #FNPB
504 lda #0
505 jsr #FNPB
506 lda #0
507 jsr #FNPB
508 lda #0
509 jsr #FNPB
510 lda #0
511 jsr #FNPB
512 lda #0
513 jsr #FNPB
514 lda #0
515 jsr #FNPB
516 lda #0
517 jsr #FNPB
518 lda #0
519 jsr #FNPB
520 lda #0
521 jsr #FNPB
522 lda #0
523 jsr #FNPB
524 lda #0
525 jsr #FNPB
526 lda #0
527 jsr #FNPB
528 lda #0
529 jsr #FNPB
530 lda #0
531 jsr #FNPB
532 lda #0
533 jsr #FNPB
534 lda #0
535 jsr #FNPB
536 lda #0
537 jsr #FNPB
538 lda #0
539 jsr #FNPB
540 lda #0
541 jsr #FNPB
542 lda #0
543 jsr #FNPB
544 lda #0
545 jsr #FNPB
546 lda #0
547 jsr #FNPB
548 lda #0
549 jsr #FNPB
550 lda #0
551 jsr #FNPB
552 lda #0
553 jsr #FNPB
554 lda #0
555 jsr #FNPB
556 lda #0
557 jsr #FNPB
558 lda #0
559 jsr #FNPB
560 lda #0
561 jsr #FNPB
562 lda #0
563 jsr #FNPB
564 lda #0
565 jsr #FNPB
566 lda #0
567 jsr #FNPB
568 lda #0
569 jsr #FNPB
570 lda #0
571 jsr #FNPB
572 lda #0
573 jsr #FNPB
574 lda #0
575 jsr #FNPB
576 lda #0
577 jsr #FNPB
578 lda #0
579 jsr #FNPB
580 lda #0
581 jsr #FNPB
582 lda #0
583 jsr #FNPB
584 lda #0
585 jsr #FNPB
586 lda #0
587 jsr #FNPB
588 lda #0
589 jsr #FNPB
590 lda #0
591 jsr #FNPB
592 lda #0
593 jsr #FNPB
594 lda #0
595 jsr #FNPB
596 lda #0
597 jsr #FNPB
598 lda #0
599 jsr #FNPB
600 lda #0
601 jsr #FNPB
602 lda #0
603 jsr #FNPB
604 lda #0
605 jsr #FNPB
606 lda #0
607 jsr #FNPB
608 lda #0
609 jsr #FNPB
610 lda #0
611 jsr #FNPB
612 lda #0
613 jsr #FNPB
614 lda #0
615 jsr #FNPB
616 lda #0
617 jsr #FNPB
618 lda #0
619 jsr #FNPB
620 lda #0
621 jsr #FNPB
622 lda #0
623 jsr #FNPB
624 lda #0
625 jsr #FNPB
626 lda #0
627 jsr #FNPB
628 lda #0
629 jsr #FNPB
630 lda #0
631 jsr #FNPB
632 lda #0
633 jsr #FNPB
634 lda #0
635 jsr #FNPB
636 lda #0
637 jsr #FNPB
638 lda #0
639 jsr #FNPB
640 lda #0
641 jsr #FNPB
642 lda #0
643 jsr #FNPB
644 lda #0
645 jsr #FNPB
646 lda #0
647 jsr #FNPB
648 lda #0
649 jsr #FNPB
650 lda #0
651 jsr #FNPB
652 lda #0
653 jsr #FNPB
654 lda #0
655 jsr #FNPB
656 lda #0
657 jsr #FNPB
658 lda #0
659 jsr #FNPB
660 lda #0
661 jsr #FNPB
662 lda #0
663 jsr #FNPB
664 lda #0
665 jsr #FNPB
666 lda #0
667 jsr #FNPB
668 lda #0
669 jsr #FNPB
670 lda #0
671 jsr #FNPB
672 lda #0
673 jsr #FNPB
674 lda #0
675 jsr #FNPB
676 lda #0
677 jsr #FNPB
678 lda #0
679 jsr #FNPB
680 lda #0
681 jsr #FNPB
682 lda #0
683 jsr #FNPB
684 lda #0
685 jsr #FNPB
686 lda #0
687 jsr #FNPB
688 lda #0
689 jsr #FNPB
690 lda #0
691 jsr #FNPB
692 lda #0
693 jsr #FNPB
694 lda #0
695 jsr #FNPB
696 lda #0
697 jsr #FNPB
698 lda #0
699 jsr #FNPB
700 lda #0
701 jsr #FNPB
702 lda #0
703 jsr #FNPB
704 lda #0
705 jsr #FNPB
706 lda #0
707 jsr #FNPB
708 lda #0
709 jsr #FNPB
710 lda #0
711 jsr #FNPB
712 lda #0
713 jsr #FNPB
714 lda #0
715 jsr #FNPB
716 lda #0
717 jsr #FNPB
718 lda #0
719 jsr #FNPB
720 lda #0
721 jsr #FNPB
722 lda #0
723 jsr #FNPB
724 lda #0
725 jsr #FNPB
726 lda #0
727 jsr #FNPB
728 lda #0
729 jsr #FNPB
730 lda #0
731 jsr #FNPB
732 lda #0
733 jsr #FNPB
734 lda #0
735 jsr #FNPB
736 lda #0
737 jsr #FNPB
738 lda #0
739 jsr #FNPB
740 lda #0
741 jsr #FNPB
742 lda #0
743 jsr #FNPB
744 lda #0
745 jsr #FNPB
746 lda #0
747 jsr #FNPB
748 lda #0
749 jsr #FNPB
750 lda #0
751 jsr #FNPB
752 lda #0
753 jsr #FNPB
754 lda #0
755 jsr #FNPB
756 lda #0
757 jsr #FNPB
758 lda #0
759 jsr #FNPB
760 lda #0
761 jsr #FNPB
762 lda #0
763 jsr #FNPB
764 lda #0
765 jsr #FNPB
766 lda #0
767 jsr #FNPB
768 lda #0
769 jsr #FNPB
770 lda #0
771 jsr #FNPB
772 lda #0
773 jsr #FNPB
774 lda #0
775 jsr #FNPB
776 lda #0
777 jsr #FNPB
778 lda #0
779 jsr #FNPB
780 lda #0
781 jsr #FNPB
782 lda #0
783 jsr #FNPB
784 lda #0
785 jsr #FNPB
786 lda #0
787 jsr #FNPB
788 lda #0
789 jsr #FNPB
790 lda #0
791 jsr #FNPB
792 lda #0
793 jsr #FNPB
794 lda #0
795 jsr #FNPB
796 lda #0
797 jsr #FNPB
798 lda #0
799 jsr #FNPB
800 lda #0
801 jsr #FNPB
802 lda #0
803 jsr #FNPB
804 lda #0
805 jsr #FNPB
806 lda #0
807 jsr #FNPB
808 lda #0
809 jsr #FNPB
810 lda #0
811 jsr #FNPB
812 lda #0
813 jsr #FNPB
814 lda #0
815 jsr #FNPB
816 lda #0
817 jsr #FNPB
818 lda #0
819 jsr #FNPB
820 lda #0
821 jsr #FNPB
822 lda #0
823 jsr #FNPB
824 lda #0
825 jsr #FNPB
826 lda #0
827 jsr #FNPB
828 lda #0
829 jsr #FNPB
830 lda #0
831 jsr #FNPB
832 lda #0
833 jsr #FNPB
834 lda #0
835 jsr #FNPB
836 lda #0
837 jsr #FNPB
838 lda #0
839 jsr #FNPB
840 lda #0
841 jsr #FNPB
842 lda #0
843 jsr #FNPB
844 lda #0
845 jsr #FNPB
846 lda #0
847 jsr #FNPB
848 lda #0
849 jsr #FNPB
850 lda #0
851 jsr #FNPB
852 lda #0
853 jsr #FNPB
854 lda #0
855 jsr #FNPB
856 lda #0
857 jsr #FNPB
858 lda #0
859 jsr #FNPB
860 lda #0
861 jsr #FNPB
862 lda #0
863 jsr #FNPB
864 lda #0
865 jsr #FNPB
866 lda #0
867 jsr #FNPB
868 lda #0
869 jsr #FNPB
870 lda #0
871 jsr #FNPB
872 lda #0
873 jsr #FNPB
874 lda #0
875 jsr #FNPB
876 lda #0
877 jsr #FNPB
878 lda #0
879 jsr #FNPB
880 lda #0
881 jsr #FNPB
882 lda #0
883 jsr #FNPB
884 lda #0
885 jsr #FNPB
886 lda #0
887 jsr #FNPB
888 lda #0
889 jsr #FNPB
890 lda #0
891 jsr #FNPB
892 lda #0
893 jsr #FNPB
894 lda #0
895 jsr #FNPB
896 lda #0
897 jsr #FNPB
898 lda #0
899 jsr #FNPB
900 lda #0
901 jsr #FNPB
902 lda #0
903 jsr #FNPB
904 lda #0
905 jsr #FNPB
906 lda #0
907 jsr #FNPB
908 lda #0
909 jsr #FNPB
910 lda #0
911 jsr #FNPB
912 lda #0
913 jsr #FNPB
914 lda #0
915 jsr #FNPB
916 lda #0
917 jsr #FNPB
918 lda #0
919 jsr #FNPB
920 lda #0
921 jsr #FNPB
922 lda #0
923 jsr #FNPB
924 lda #0
925 jsr #FNPB
926 lda #0
927 jsr #FNPB
928 lda #0
929 jsr #FNPB
930 lda #0
931 jsr #FNPB
932 lda #0
933 jsr #FNPB
934 lda #0
935 jsr #FNPB
936 lda #0
937 jsr #FNPB
938 lda #0
939 jsr #FNPB
940 lda #0
941 jsr #FNPB
942 lda #0
943 jsr #FNPB
944 lda #0
945 jsr #FNPB
946 lda #0
947 jsr #FNPB
948 lda #0
949 jsr #FNPB
950 lda #0
951 jsr #FNPB
952 lda #0
953 jsr #FNPB
954 lda #0
955 jsr #FNPB
956 lda #0
957 jsr #FNPB
958 lda #0
959 jsr #FNPB
960 lda #0
961 jsr #FNPB
962 lda #0
963 jsr #FNPB
964 lda #0
965 jsr #FNPB
966 lda #0
967 jsr #FNPB
968 lda #0
969 jsr #FNPB
970 lda #0
971 jsr #FNPB
972 lda #0
973 jsr #FNPB
974 lda #0
975 jsr #FNPB
976 lda #0
977 jsr #FNPB
978 lda #0
979 jsr #FNPB
980 lda #0
981 jsr #FNPB
982 lda #0
983 jsr #FNPB
984 lda #0
985 jsr #FNPB
986 lda #0
987 jsr #FNPB
988 lda #0
989 jsr #FNPB
990 lda #0
991 jsr #FNPB
992 lda #0
993 jsr #FNPB
994 lda #0
995 jsr #FNPB
996 lda #0
997 jsr #FNPB
998 lda #0
999 jsr #FNPB
1000 lda #0
1001 jsr #FNPB
1002 lda #0
1003 jsr #FNPB
1004 lda #0
1005 jsr #FNPB
1006 lda #0
1007 jsr #FNPB
1008 lda #0
1009 jsr #FNPB
1009 lda #0
1010 jsr #FNPB
1011 lda #0
1012 jsr #FNPB
1013 lda #0
1014 jsr #FNPB
1014 lda #0
1015 jsr #FNPB
1015 lda #0
1016 jsr #FNPB
1016 lda #0
1017 jsr #FNPB
1017 lda #0
1018 jsr #FNPB
1018 lda #0
1019 jsr #FNPB
1019 lda #0
1020 jsr #FNPB
1020 lda #0
1021 jsr #FNPB
1021 lda #0
1022 jsr #FNPB
1022 lda #0
1023 jsr #FNPB
1023 lda #0
1024 jsr #FNPB
1024 lda #0
1025 jsr #FNPB
1025 lda #0
1026 jsr #FNPB
1026 lda #0
1027 jsr #FNPB
1027 lda #0
1028 jsr #FNPB
1028 lda #0
1029 jsr #FNPB
1029 lda #0
1030 jsr #FNPB
1030 lda #0
1031 jsr #FNPB
1031 lda #0
1032 jsr #FNPB
1032 lda #0
1033 jsr #FNPB
1033 lda #0
1034 jsr #FNPB
1034 lda #0
1035 jsr #FNPB
1035 lda #0
1036 jsr #FNPB
1036 lda #0
1037 jsr #FNPB
1037 lda #0
1038 jsr #FNPB
1038 lda #0
1039 jsr #FNPB
1039 lda #0
1040 jsr #FNPB
1040 lda #0
1041 jsr #FNPB
1041 lda #0
1042 jsr #FNPB
1042 lda #0
1043 jsr #FNPB
1043 lda #0
1044 jsr #FNPB
1044 lda #0
1045 jsr #FNPB
1045 lda #0
1046 jsr #FNPB
1046 lda #0
1047 jsr #FNPB
1047 lda #0
1048 jsr #FNPB
1048 lda #0
1049 jsr #FNPB
1049 lda #0
1050 jsr #FNPB
1050 lda #0
1051 jsr #FNPB
1051 lda #0
1052 jsr #FNPB
1052 lda #0
1053 jsr #FNPB
1053 lda #0
1054 jsr #FNPB
1054 lda #0
1055 jsr #FNPB
1055 lda #0
1056 jsr #FNPB
1056 lda #0
1057 jsr #FNPB
1057 lda #0
1058 jsr #FNPB
1058 lda #0
1059 jsr #FNPB
1059 lda #0
1060 jsr #FNPB
1060 lda #0
1061 jsr #FNPB
1061 lda #0
1062 jsr #FNPB
1062 lda #0
1063 jsr #FNPB
1063 lda #0
1064 jsr #FNPB
1064 lda #0
1065 jsr #FNPB
1065 lda #0
1066 jsr #FNPB
1066 lda #0
1067 jsr #FNPB
1067 lda #0
1068 jsr #FNPB
1068 lda #0
1069 jsr #FNPB
1069 lda #0
1070 jsr #FNPB
1070 lda #0
1071 jsr #FNPB
1071 lda #0
1072 jsr #FNPB
1072 lda #0
1073 jsr #FNPB
1073 lda #0
1074 jsr #FNPB
1074 lda #0
1075 jsr #FNPB
1075 lda #0
1076 jsr #FNPB
1076 lda #0
1077 jsr #FNPB
1077 lda #0
1078 jsr #FNPB
1078 lda #0
1079 jsr #FNPB
1079 lda #0
1080 jsr #FNPB
1080 lda #0
1081 jsr #FNPB
1081 lda #0
1082 jsr #FNPB
1082 lda #0
1083 jsr #FNPB
1083 lda #0
1084 jsr #FNPB
1084 lda #0
1085 jsr #FNPB
1085 lda #0
1086 jsr #FNPB
1086 lda #0
1087 jsr #FNPB
1087 lda #0
1088 jsr #FNPB
1088 lda #0
1089 jsr #FNPB
1089 lda #0
1090 jsr #FNPB
1090 lda #0
1091 jsr #FNPB
1091 lda #0
1092 jsr #FNPB
1092 lda #0
1093 jsr #FNPB
1093 lda #0
1094 jsr #FNPB
1094 lda #0
1095 jsr #FNPB
1095 lda #0
1096 jsr #FNPB
1096 lda #0
1097 jsr #FNPB
1097 lda #0
1098 jsr #FNPB
1098 lda #0
1099 jsr #FNPB
1099 lda #0
1100 jsr #FNPB
1100 lda #0
1101 jsr #FNPB
1101 lda #0
1102 jsr #FNPB
1102 lda #0
1103 jsr #FNPB
1103 lda #0
1104 jsr #FNPB
1104 lda #0
1105 jsr #FNPB
1105 lda #0
1106 jsr #FNPB
1106 lda #0
1107 jsr #FNPB
1107 lda #0
1108 jsr #FNPB
1108 lda #0
1109 jsr #FNPB
1109 lda #0
1110 jsr #FNPB
1110 lda #0
1111 jsr #FNPB
1111 lda #0
1112 jsr #FNPB
1112 lda #0
1113 jsr #FNPB
1113 lda #0
1114 jsr #FNPB
1114 lda #0
1115 jsr #FNPB
1115 lda #0
1116 jsr #FNPB
1116 lda #0
1117 jsr #FNPB
1117 lda #0
1118 jsr #FNPB
1118 lda #0
1119 jsr #FNPB
1119 lda #0
1120 jsr #FNPB
1120 lda #0
1121 jsr #FNPB
1121 lda #0
1122 jsr #FNPB
1122 lda #0
1123 jsr #FNPB
1123 lda #0
1124 jsr #FNPB
1124 lda #0
1125 jsr #FNPB
1125 lda #0
1126 jsr #FNPB
1126 lda #0
1127 jsr #FNPB
1127 lda #0
1128 jsr #FNPB
1128 lda #0
1129 jsr #FNPB
1129 lda #0
1130 jsr #FNPB
1130 lda #0
1131 jsr #FNPB
1131 lda #0
1132 jsr #FNPB
1132 lda #0
1133 jsr #FNPB
1133 lda #0
1134 jsr #FNPB
1134 lda #0
1135 jsr #FNPB
1135 lda #0
1136 jsr #FNPB
1136 lda #0
1137 jsr #FNPB
1137 lda #0
1138 jsr #FNPB
1138 lda #0
1139 jsr #FNPB
1139 lda #0
1140 jsr #FNPB
1140 lda #0
1141 jsr #FNPB
1141 lda #0
1142 jsr #FNPB
1142 lda #0
1143 jsr #FNPB
1143 lda #0
1144 jsr #FNPB
1144 lda #0
1145 jsr #FNPB
1145 lda #0
1146 jsr #FNPB
1146 lda #0
1147 jsr #FNPB
1147 lda #0
1148 jsr #FNPB
1148 lda #0
1149 jsr #FNPB
1149 lda #0
1150 jsr #FNPB
1150 lda #0
1151 jsr #FNPB
1151 lda #0
1152 jsr #FNPB
1152 lda #0
1153 jsr #FNPB
1153 lda #0
1154 jsr #FNPB
1154 lda #0
1155 jsr #FNPB
1155 lda #0
1156 jsr #FNPB
1156 lda #0
1157 jsr #FNPB
1157 lda #0
1158 jsr #FNPB
1158 lda #0
1159 jsr #FNPB
1159 lda #0
1160 jsr #FNPB
1160 lda #0
1161 jsr #FNPB
1161 lda #0
1162 jsr #FNPB
1162 lda #0
1163 jsr #FNPB
1163 lda #0
1164 jsr #FNPB
1164 lda #0
1165 jsr #FNPB
1165 lda #0
1166 jsr #FNPB
1166 lda #0
1167 jsr #FNPB
1167 lda #0
1168 jsr #FNPB
1168 lda #0
1169 jsr #FNPB
1169 lda #0
1170 jsr #FNPB
1170 lda #0
1171 jsr #FNPB
1171 lda #0
1172 jsr #FNPB
1172 lda #
```

```

179      beq     NXCH          ; CR: nothing to
180      ldy     NCOL
181      cpy     #FNTCOL
182      bcs     NXCH          ; past twelve:
the rest of the line is lost
183      sta     FNRSEL+FH_TCHR
184      inc     NCOL
185      jmp     NXCH
186 NROW:      jsr     NENDW
187      inc     NROW
188      lda     NSRC
189      cmp     NAVLO
190      bcc     NXROW
191 NSHOWN:
192
193 ; ----- CLOSE -----
194      lda     #NETDEV
195      sta     FNDEV
196      lda     #NCLOSE
197      sta     FNCMD
198      lda     #0
199      sta     FNNPR
200      jsr     FNBEG
201      jsr     FNGO
202      lda     #0
203      sta     APERR
204
205 ; ----- the title, and the verdict
206 SHOW:   lda     #(TTITLE)&$FF
207          sta     FNPTRL
208          lda     #(TTITLE)>>8
209          sta     FNPTRH
210          lda     #0
211          jsr     FNRSTR
212          lda     APERR
213          beq     RUN
214          lda     #(TFAIL)&$FF
215          sta     FNPTRL
216          lda     #(TFAIL)>>8
217          sta     FNPTRH
218          lda     #1
219          jsr     FNRSTR
220          lda     #1
221          jsr     FNROWA
222          lda     #(TFAIL)&$FF
223          sta     FNPTRL
224          lda     #(TFAIL)>>8
225          sta     FNPTRH
226          ldy     #0
227 NFL:    lda     (FNPTRL),y
228          beq     NFLE
229          sta     FNRSEL+FH_TCHR
230          tny
231          bne     NFL
232 NFLE:   lda     #' '
233          sta     FNRSEL+FH_TCHR
234          lda     APERR
235          jsr     FNHEX
236          jsr     FNENDR
237
238 RUN:    lda     #0
239          sta     VBLANK
240          jsr     DINIT
241          jmp     DLOOP
242
243 ; NGOACK -- commit and wait, then check the ACK.
244 ; on success it returns; on
245 ; any failure it leaves the code in APERR, drops
246 ; its own return address and
247 ; goes straight to SHOW, so a caller needs no
248 ; branch after it (and none of
249 ; them would reach SHOW in one branch anyway).
250 NGOACK: jsr     FNGO
251          sta     APERR
252          bne     NGOA1
253          jsr     FNACK
254          sta     APERR
255          beq     NGOA2
256 NGOA1:  pla
257          pla
258          jmp     SHOW
259 NGOA2:  rts
260
261 ; NENDW -- render the composed row and wait
262 ; (bounded) for the cartridge to
263 ; say it has landed on the planes. Instant in
264 ; emulation; on the RP2040 the
265 ; row is composed on the other core, and a
266 ; client that starts the next row's
267 ; characters before then hands it a row already
268 ; being overwritten.
269 NENDW:  ldx     FNBTXG
270          jsr     FNENDR
271          ldy     #0
272 NENDW1: cpx     FNBTXG
273          bne     NENDW2
274          dey
275          bne     NENDW1
276 NENDW2: rts
277
278 ; ----- strings -----
279 TURL:   DB      "N:http://127.0.0.1:8765/
280          hello.txt",0
281 TTITLE: DB      "NETGET",0
282 TFAIL:  DB      "FAILED",0
283
284 INCLUDE "fujilib.inc"
285 APPVBL: rts          ; this client
286          needs no per-frame work

```

LISTINGS/ASM/FUJIDIR.ASM

```

1 ; fujidir.asm -- M3: browse a host's directory
  and boot what you pick.
2 ;
3 ; MOUNT_HOST --> OPEN_DIRECTORY -> READ_DIR_ENTRY
  x N: one entry rendered
4 ; straight from the reply window into a text
  row. Joystick up/down moves the
5 ; cursor, FIRE mounts the selection and boots
  it.
6 ;
7 ; NO FILENAME IS EVER HELD IN CONSOLE RAM. There
  is nowhere to put one: 128
8 ; bytes total, the stack owns the top and the
  swap stub the bottom. So a name
9 ; goes cartridge -> text plane to be drawn, and
  cartridge -> TX page to be
10 ; mounted, and the console only ever holds an
  index. That is the ColecoVision
11 ; port's lesson, and this console needs it more.
12 ;
13 ; The cost of that is what moving the cursor
  does: with no names in RAM there
14 ; is nothing to redraw from, so the list is re-
  read from the host. Under
15 ; emulation that is instant; on real hardware it
  is about a second per step.
16 ; The fix, when CONFIG needs it, is a cartridge-
  side page cache -- the blit
17 ; port already exists for exactly this shape of
  problem.
18 ;
19 ; Exit test: emu/dirttest.lua walks the cursor to
  a NAMED file by reading the
20 ; rendered planes back, presses FIRE, and
  requires the served window to be
21 ; byte-identical to that file.
22 ;
23 CPU 6502
24 INCLUDE "vcs.inc"
25 ;
26 ; ----- zero page -----
27 ; $80-$A6 is the swap stub's landing ground and
  $A8+ is ours; the stack runs
28 ; down from $FF and must stay above $C0.
29 PAD3 EQU $81
30 SAVSP EQU $82
31 DIRSEL EQU $AA ; cursor row, 0-
  based
32 DIRCNT EQU $AB ; entries
33 actually on screen
34 APERR EQU $AC
35 APSTEP EQU $AD ; entry index
36 DIRWANT EQU $AE
  wanted while re-reading
37 ;
38 INCLUDE "fujinet.inc"
39 ;
40 HOSTSL EQU 0 ; host slot 0 =
  SD on the standard fujinet-pc
41 DEVSLT EQU 0
42 NAMELN EQU 30 ; READ_DIR_ENTRY
  maxlen. NOT 31: at exactly 31
43 ; the firmware
44 prepends icon bytes.
45 NROWS EQU 14 ; list rows,
  leaving a title and a footer
46 ROW0 EQU 3 ; first list row
47 ;
48 ORG $1000
49 ;
50 START: sei
  cld
51 ldx $FFF
52 txs
53 ldx $0
54 CLRLP: sta $00,x
  dex
55 bne CLRLP
56 sta $00
57 ;
58 ldx #2
59 sta VBLANK
60 ldx #0
61 sta APERR
62 sta APSTEP
63 sta DIRSEL
64 ;
65 jsr FNARM
66 jsr FNCHK
67 beq HAVE
68 ldx #FNEOC
69 sta APERR
70 jmp FAIL
71 ;
72 HAVE: jsr MHOST
  bne FAIL
73 jsr DRAW
  bne FAIL ; first listing
74 ;
75 ldx #0
76 sta VBLANK
77 jsr DINIT
78 jmp DLOOP
79 ;
80 FAIL: jsr SHOWERR
  ldx #0
81 sta VBLANK
82 jsr DINIT
83 ;
84 ;
85 jmp DLOOP
86 ;
87 ;
88 ;
89 ; APPVBL -- one frame's worth of client work,
  called from the display loop
90 ; while the screen is blanked.
91 APPVBL: jsr INSCAN
  sta APERR+0 ; reuse as
92 scratch; APERR is only live on FAIL
93 and #IN_DOWN
94 beq AV1
95 ldx DIRSEL
96 clc
97 adc #1
98 cmp DIRCNT
99 bcs AVDONE ; already on the
  last entry
100 sta DIRSEL
101 jmp AVREDRAW
102 AV1: ldx APERR+0
103 and #IN_UP
104 beq AV2
105 ldx DIRSEL
106 beq AVDONE ; already at the
  top
107 sec
108 sbc #1
109 sta DIRSEL
110 jmp AVREDRAW
111 AV2: ldx APERR+0
112 and #IN_FIRE
113 beq AVDONE
114 jmp BOOTSEL
115 AVDONE: rts
116 ;
117 AVREDRAW:
118 ldx #2 ; blank while we
  talk; a listing is many
119 sta VBLANK ; round trips
  and the frame will not finish
120 jsr DRAW
121 ldx #0
122 sta VBLANK
123 rts
124 ;
125 ;
126 ; MHOST -- MOUNT_HOST(HOSTSL). Z set on success.
127 MHOST: ldx #FNDEVF
128 sta FNDEV
129 ldx #FNCMHST
130 sta FNCMD
131 ldx #1
132 sta FNNPR
133 jsr FNBEG
134 ldx #HOSTSL
135 jsr FNPB
136 jsr FNGO
137 sta APERR
138 cmp #FNEOK
139 bne MHE
140 jsr FNACK
141 sta APERR
142 cmp #FNEOK
143 beq MHOK
144 MHE: ldx #1
145 sta APSTEP
146 ldx #1 ; Z clear
147 rts
148 MHOK: ldx #0
149 rts
150 ;
151 ;
152 ; ODIR -- OPEN_DIRECTORY(host, "/"). Z set on
  success.
153 ;
154 ; One parameter and then a FULL 256-byte
  payload: the path, its NUL, and then
155 ; padding -- which the server reads as "no
  filter".
156 ODIR: ldx #FNDEVF
157 sta FNDEV
158 ldx #FNCODIR
159 sta FNCMD
160 ldx #1
161 sta FNNPR
162 jsr FNBEG
163 ldx #HOSTSL
164 jsr FNPB
165 jsr FNPBEG
166 ldx #((TROOT)&$FF
  #((TROOT)>>8
167 sta FNPTRL
168 ldx #((TROOT)>>8
169 sta FNPTRH
170 jsr FNPSTR
171 jsr FNPEND
172 jsr FNGO
173 sta APERR
174 cmp #FNEOK
175 bne ODE
176 jsr FNACK
177 sta APERR
178 cmp #FNEOK
179 beq ODOK
180 ODE: ldx #2

```

```

181      sta      APSTEP
182      lda      #1
183      rts
184 ODOK:  lda      #0
185      rts
186
187 ;
188 ; RDENT -- READ_DIR_ENTRY(NAMELN, 0). Z set on
189 ; success; the entry is left in
190 ; the reply window and is NOT copied anywhere.
191 RDENT:  lda      #FNDEVF
192      sta      FNDEV
193      lda      #FNCRDIR
194      fncmddir
195      lda      #2
196      sta      FNNPR
197      jsr      FNBEG
198      lda      #NAMELN
199      jsr      FNPB
200      lda      #0
201      jsr      FNPB
202      jsr      FNGO
203      cmp      #FNEOK
204      bne      RDE
205      jsr      FNACK
206      cmp      #FNEOK
207      bne      RDE
208      lda      #0
209 RDE:    sta      APERR
210      jsr      #3
211      sta      APSTEP
212      lda      #1
213      rts
214
215 ;
216 ; DRAW -- re-open the directory and paint the
217 ; visible list. Z set on success.
218 DRAW:  jsr      ODIR
219      bne      DRBAD
220      lda      #(TTITLE)&$FF
221      sta      FNPTL
222      lda      #(TTITLE)>>8
223      sta      FNPTRH
224      lda      #0
225      jsr      FNRSTR
226
227      lda      #0
228      sta      DIRCNT
229      ldx      #0 ; X = row index
230
231 within the list
232 DRLOOP: stx      DIRWANT
233      jsr      RDENT
234      bne      DRBAD
235      jsr      FNEOF
236      beq      DREND ; the $7F,$7F
237 marker: stop, and do NOT read on
238      ldx      DIRWANT
239
240 ; Row: the cursor gutter, then the name
241 straight from the reply.
242      txa
243      clc
244      adc      #ROW0
245      jsr      FNROWA
246      lda      DIRWANT
247      jsr      DIRSEL
248      bne      DRNOCUR
249      lda      #' '
250      jsr      DRCUR
251      lda      #' '
252      jsr      DRCUR
253      lda      #FNRSEL+FH_TCHR
254      sta      DRCUR
255      ldx      #0 ; reply offset:
256      the name starts at 0
257      ldy      #FNTCOL-1
258      jsr      FNRRPX
259      jsr      FNENDR
260
261      ldx      DIRCNT
262      cpx      #NROWS
263      bcs      DROK
264      txa
265      clc
266      adc      #ROW0
267      jsr      FNROWA
268      jsr      FNENDR
269      jsr      FNENDR
270      jmp      DRBL
271 DROK:  lda      #0
272      rts
273 DRBAD: lda      #1
274      rts
275
276 ; FNRRPX -- append up to Y characters of the
277 ; reply window at offset X to the
278 ; row being composed, stopping at NUL. (FNRRPL
279 ; selects the row itself; this
280 ; one does not, so a cursor character can go
281 ; first.)
282 FNRRPX: lda      FNRRPLY,x
283
284      beq      FNRRX2
285      sta      FNRRSEL+FH_TCHR
286      inx
287      dey
288      bne      FNRRPX
289      rts
290
291 ; BOOTSEL -- mount the selected entry and boot
292 ; it.
293 ; The name is not in RAM, so the directory is
294 ; re-opened and re-read up to the
295 ; selection, and the name is then streamed from
296 ; the reply window straight into
297 ; the path payload.
298 BOOTSEL:
299      lda      #2
300      sta      VBLANK
301      jsr      ODIR
302      beq      BSOPEN ; branch range:
303      invert and JMP
304      jmp      BSBAD
305 BSOPEN:  lda      #0
306      sta      DIRWANT
307      BSSKIP: jsr      RDENT
308      beq      BSSK1
309      jmp      BSBAD
310 BSSK1:  jsr      FNEOF
311      bne      BSSK2
312      jmp      BSBAD ; ran off the
313      end: the selection is gone
314 BSSK2:
315      lda      DIRWANT
316      cmp      DIRSEL
317      beq      BSGOT
318      inc      DIRWANT
319      jmp      BSSKIP
320
321 ; --- SET_DEVICE_FULLPATH(dev, host, mode, "/"
322 ; + name)
323 BSGOT:  lda      #FNDEVF
324      sta      FNDEV
325      lda      #FNCSDFP
326      sta      FNCMD
327      lda      #3
328      sta      FNNPR
329      jsr      FNBEG
330      lda      #DEVSLT
331      jsr      FNPB
332      lda      #HOSTSL
333      jsr      FNPB
334      lda      #FMREAD
335      jsr      FNPB
336      lda      #FNPTL
337      jsr      FNPTRH
338      jsr      FNPSTR
339      ldx      #0
340      ldy      #NAMELN
341      jsr      FNRPPL
342      cartridge to cartridge
343      jsr      FNPEND
344      jsr      FNGO
345      sta      APERR
346      cmp      #FNEOK
347      bne      BSBAD2
348      jsr      FNACK
349      sta      APERR
350      cmp      #FNEOK
351      bne      BSBAD2
352
353 ; --- MOUNT_IMAGE(dev, mode)
354      lda      #FNDEVF
355      sta      FNDEV
356      lda      #FNCMIMG
357      sta      FNCMD
358      lda      #2
359      sta      FNNPR
360      jsr      FNBEG
361      lda      #DEVSLT
362      jsr      FNPB
363      lda      #FMREAD
364      jsr      FNPB
365      jsr      FNGO
366      sta      APERR
367      cmp      #FNEOK
368      bne      BSBAD2
369      jsr      FNACK
370      sta      APERR
371      cmp      #FNEOK
372      bne      BSBAD2
373
374 BSWAIT: ldy      #0
375      ldx      #0
376      BSW1:  lda      FNBST
377      cmp      #FNBRDY
378      beq      BSSOOT
379      cmp      #FNBFALL
380      beq      BSBAD2
381      dex
382      bne      BSW2
383      dey
384      bne      BSW1
385      lda      #FNEWAIT
386      sta      APERR

```

```

382 BSBAD2: lda    #5
383         sta    APSTEP
384 BSBAD:  jsr     SHOWERR
385         lda    #0
386         sta    VBLANK
387         rts
        display loop with the error up ; back to the
388
389 BSBOOT: jsr     FNBLK
390         jmp     FNSWAP ; does not
391
392 ;
-----
393 SHOWERR:
394         lda    #(TFAIL)&$FF
395         sta    FNPTL
396         lda    #(TFAIL)>>8
397         sta    FNPTRH
398         lda    #1
399         jsr     FNRSTR
400         lda    #2
401         jsr     FNROWA
402         lda    #'S'
403         sta    FNRSEL+FH_TCHR
404         lda    APSTEP
405         jsr     FNHEX
406         lda    #' '
407         sta    FNRSEL+FH_TCHR
408         lda    #'E'
409         sta    FNRSEL+FH_TCHR
410         lda    APERR
411         jsr     FNHEX
412         jsr     FNEHDR
413         rts
414
415 TTITLE: DB      "FUJINET DIR",0
416 TFAIL:  DB      "FAILED",0
417 TROOT:  DB      "/",0
418
419         INCLUDE "fujilib.inc"
420         INCLUDE "fujidisp.inc"
421
422         ORG     $1FFC
423         DW      START
424         DW      START
425
426         END
427

```

LISTINGS/ASM/FUJIBOOT.ASM

```

1 ; fujiboot.asm -- M2: mount a cartridge image
2 over the network and boot it.
3
4 ; MOUNT_HOST -> SET_DEVICE_FULLPATH ->
5 MOUNT_IMAGE. The last one makes the
6 ; ESP32 side stream the image back to the
7 cartridge on the same link, addressed
8 ; to the DBC device, while MOUNT_IMAGE's own
9 reply is still outstanding; the
10 ; cart stages it and reports BOOT_STATE.
11
12 ; The boot itself: arm the swap (BOOTLOCK), then
13 FNSWAP copies a stub into
14 ; zero-page RAM and jumps to it. The stub stores
15 to the swap hotspot -- the
16 ; cart flips the window to the staged image
17 between that store and the next
18 ; fetch, which is safe because the next fetch is
19 in RAM with A12 low -- and
20 ; then JMPs through ($1FFC), the new image's own
21 reset vector, exactly what the
22 ; 6507 fetches at power-on.
23
24 ; Target host slot and path come from
25 bootcfg.inc, written by build.sh:
26 ; BOOT_HOST=0 BOOT_PATH=/vcsgame.bin ./
27 build.sh fujiboot
28
29 ; Exit test: emu/boottest.lua dumps the whole
30 served 4K window after the swap
31 ; and requires it to be byte-identical to the
32 file on disk.
33
34 CPU      6502
35 INCLUDE "vcs.inc"
36
37 ; ----- zero page -----
38 ; $80-$A6 is the swap stub's landing ground (see
39 FNSWAP), so nothing the
40 ; client keeps may live there. The stack starts
41 at $FF and must not descend
42 ; past $A8.
43 PAD3 EQU    $81 ; the display
44 kernel's 3-cycle pad target
45 SAVSP EQU    $82 ; stack pointer,
46 parked across the kernel
47 APERR EQU    $8C ; the failing
48 code
49 APSTEP EQU    $8D ; which step
50 failed
51
52 INCLUDE "fujinet.inc"
53
54 DEVSLOT EQU    0
55
56 ORG     $1000
57
58 START: sei
59
60 cld
61 ldx
62 txs
63 lda    #0
64 CLRLP: sta    $00,x
65         bne    CLRLP
66         sta    $00
67         lda    #2 ; blanked while
68         sta    VBLANK
69         lda    #0
70         sta    APERR
71         sta    APSTEP
72
73         jsr     FNARM
74         jsr     FNCHK
75         beq     HAVE
76         lda    #FNEOC
77         sta    APERR
78         jmp     SHOW
79
80 ; ---- MOUNT_HOST(host)
81 HAVE:  lda    #FNDEVF
82         sta    FNDEV
83         lda    #FNCMHST
84         sta    FNCMD
85         lda    #1
86         sta    FNNPR
87         jsr     FNBEQ
88         lda    #BOOTHST
89         jsr     FNPB
90         jsr     FNGO
91         sta    APERR
92         cmp     #FNEOK
93         bne    ERR1
94         jsr     FNACK
95         sta    APERR
96         cmp     #FNEOK
97         beq     SDFP
98 ERR1:  lda    #1
99         sta    APSTEP
100        jmp     SHOW
101
102 ; ---- SET_DEVICE_FULLPATH(dev, host, mode,
103 path) -----
104 ; Three one-byte parameters and then a FULL 256-
105 byte NUL-padded path. The
106 ; server reads exactly 256 and fails a short
107 one.
108 SDFP:  lda    #FNDEVF
109         sta    FNDEV
110         lda    #FNCSDFP
111         sta    FNCMD
112         lda    #3
113         sta    FNNPR

```

```

93      jsr     FNBEG
94      lda     #DEVSLOT
95      jsr     FNPB
96      lda     #BOOTHST
97      jsr     FNPB
98      lda     #FMREAD
99      jsr     FNPB
100     lda     #(BOOTPTH)&$FF
101     sta     FNPTRL
102     lda     #(BOOTPTH)>>8
103     sta     FNPTRH
104     jsr     FNPTRH
105     jsr     FNGO
106     sta     APERR
107     cmp     #FNEOK
108     bne     ERR2
109     jsr     FNACK
110     sta     APERR
111     cmp     #FNEOK
112     beq     MIMG
113     ERR2:   lda     #2
114             sta     APSTEP
115             jmp     SHOW
116
117 ; ---- MOUNT_IMAGE(dev, mode)
118 ; This is the one that takes real time: the
119 ; ESP32 fetches the file and streams
120 ; it back to the cartridge as DBC push frames
121 ; while this reply is still
122 ; outstanding. FNBEG's default timeout is
123 ; deliberately generous.
124 MIMG:      lda     #FNDEVF
125             sta     FNDEV
126             lda     #FNCIMG
127             sta     FNCMD
128             lda     #2
129             sta     FNNPR
130             jsr     FNBEG
131             lda     #DEVSLOT
132             jsr     FNPB
133             lda     #FMREAD
134             jsr     FNPB
135             jsr     FNGO
136             sta     APERR
137             cmp     #FNEOK
138             bne     ERR3
139             jsr     FNACK
140             sta     APERR
141             cmp     #FNEOK
142             beq     WAITB
143             ERR3:   lda     #3
144                     sta     APSTEP
145                     jmp     SHOW
146
147 ; ---- wait for the staged image
148
149 WAITB:     ldy     #0
150 WAITB1:    ldx     #0
151 WAITB2:    lda     FNBST
152             cmp     #FNBBDY
153             beq     BOOT
154             cmp     #FNBFAIL
155             beq     BFAIL
156             dex
157             bne     WAITB2
158             dey
159             bne     WAITB1
160             lda     #FNEWAIT
161             never arrived ; the image
162
163             sta     APERR
164             lda     #4
165             sta     APSTEP
166             jmp     SHOW
167             BFAIL:   lda     FNBST
168                     sta     APERR
169                     lda     #5
170                     sta     APSTEP
171                     jmp     SHOW
172
173 ; ---- boot it
174
175 BOOT:      jsr     FNBK
176             jmp     FNSWAP ; arm the swap
177
178 return
179
180 ; ---- the failure screen
181
182 ; As useful as the success one: it says WHICH
183 ; step failed and with what, which
184 ; is the difference between a bug report and a
185 ; shrug.
186 SHOW:      lda     #(TTITLE)&$FF
187             sta     FNPTRL
188             lda     #(TTITLE)>>8
189             sta     FNPTRH
190             lda     #0
191             jsr     FNRSTR
192
193             lda     #(TFAIL)&$FF
194             sta     FNPTRL
195             lda     #(TFAIL)>>8
196             sta     FNPTRH
197             lda     #2
198             jsr     FNRSTR
199
200             lda     #4
201             jsr     FNROWA
202             lda     #'S'
203             sta     FNRSEL+FH_TCHR
204             lda     APSTEP

```

LISTINGS/ASM/APPKEY.ASM

```

1 ; appkey.asm -- read the shared FujiNet
2 username, then write a key of our
3 ; own and read it back.
4 ;
5 ; An appkey is a small file on the adapter's SD
6 card, named by a creator id,
7 ; an application id and a key number. Creator 1,
8 app 1, key 0 is the slot
9 ; every FujiNet game reads the player's name
10 from. This client shows it, then
11 ; writes "HELLO 2600" into creator $2600 / app
12 1 / key 0 and reads that back.
13 ;
14 ; OPEN_APPKEY's payload is ONE six-byte packed
15 struct -- creator (u16, little-
16 endian), app, key, mode, and a reserved byte
17 that is not optional -- not
18 ; four parameters. READ_APPKEY's reply is a two-
19 byte length and then the
20 ; value. WRITE_APPKEY takes one parameter, the
21 length, then the bytes.
22
23 CPU 6502
24 INCLUDE "vcs.inc"
25
26 PAD3 EQU $81
27 SAVSP EQU $82
28 APERR EQU $8F ; what went
29 ; wrong, if anything
30 APSTEP EQU $90 ; which step
31 AKCRH EQU $91 ; the key being
32 opened: creator lo, hi...
33 AKCRH EQU $92
34 AKKEYN EQU $93 ; ...key
35 AKMODE EQU $94 ; ...and mode
36 (AKMRD / AKMWR)
37
38 INCLUDE "fujinet.inc"
39 INCLUDE "devdefs.inc"
40
41 OURCRE EQU $2600 ; our creator
42 id: the
43 OURKEY EQU 0
44
45 ORG $1000
46
47 START: sei
48 cld
49 ldx #FFF
50 txs
51 lda #0
52 CLRLP: sta $00,x
53 dex
54 bne CLRLP
55 sta $00
56 lda #2
57 sta VBLANK
58 lda #0
59 sta APERR
60 sta APSTEP
61
62 jsr FNARM
63 jsr FNCHK
64 beq GOTCART
65 lda #FNENOC
66 sta APERR
67 jmp SHOW
68
69 ; ----- the shared username: creator
70 1, app 1, key 0 -----
71 GOTCART:
72 lda #(TNAME)&$FF
73 sta FNPTL
74 lda #(TNAME)>>8
75 sta FNPTRH
76 lda #2
77 jsr FNRSTR
78 lda #AKCREAT
79 sta AKCRH
80 lda #0
81 sta AKCRH
82 lda #AKKEY
83 sta AKKEYN
84 lda #AKMRD
85 sta AKMODE
86 jsr AKOPEN
87 jsr AKREAD
88 lda #3
89 jsr AKSHOW
90
91 ; ----- write our own key, then read
92 it back -----
93 lda #(TOURS)&$FF
94 sta FNPTL
95 lda #(TOURS)>>8
96 sta FNPTRH
97 lda #6
98 jsr FNRSTR
99 lda #(OURCRE)&$FF
100 sta AKCRH
101 lda #(OURCRE)>>8
102 sta AKCRH
103 lda #AKKEY
104 sta AKKEYN
105 lda #AKMWR
106 sta AKMODE
107 jsr AKOPEN
108
109 jsr AKWRITE
110 lda #AKMRD
111 sta AKMODE
112 jsr AKOPEN
113 AKREAD: lda #7
114 jsr AKSHOW
115 lda #0
116 sta AKSHOW
117
118 ; (TTITLE)&$FF
119 FNPTL
120 ; (TTITLE)>>8
121 FNPTRH
122 #0
123 FNRSTR
124 APERR
125 RUN
126 #10
127 FNROWA
128 #'S'
129 FNRSEL+FH_TCHR
130 APSTEP
131 FNHEX
132 #' '
133 FNRSEL+FH_TCHR
134 #'E'
135 FNRSEL+FH_TCHR
136 APERR
137 FNHEX
138 FNENDR
139 #0
140 VBLANK
141 DINIT
142 DLOOP
143
144 ; AKOPEN -- OPEN_APPKEY for the key in AKCRH /
145 AKCRH/AKKEYN, mode AKMODE.
146 AKOPEN: inc APSTEP
147 lda #FNDEVF
148 sta FNDEV
149 lda #FNCAKOP
150 sta FNCMD
151 lda #0
152 sta FNNPR
153 jsr FNBEG
154 lda AKCRH ; the six-byte
155 struct, as the payload
156 sta FNTX
157 lda AKCRH
158 sta FNTX
159 lda #AKAPP
160 sta FNTX
161 lda AKKEYN
162 sta FNTX
163 lda AKMODE
164 sta FNTX
165 lda #0
166 sta FNTX ; reserved, and
167 not optional
168 jmp NGOACK
169
170 ; AKREAD -- READ_APPKEY. The value is left in
171 the reply window: two bytes
172 ; of length, then the bytes.
173 AKREAD: inc APSTEP
174 lda #FNDEVF
175 sta FNDEV
176 lda #FNCAKRD
177 sta FNCMD
178 lda #0
179 sta FNNPR
180 jsr FNBEG
181 jmp NGOACK
182
183 ; AKWRITE -- WRITE_APPKEY: one parameter, the
184 length, then the value.
185 AKWRITE: inc APSTEP
186 lda #FNDEVF
187 sta FNDEV
188 lda #FNCAKWR
189 sta FNCMD
190 lda #1
191 sta FNNPR
192 jsr FNBEG
193 lda #TVALE-TVAL
194 jsr FNPB
195 lda #0
196 TVAL: lda TVAL,y
197 sta FNTX
198 iny
199 cpy #TVALE-TVAL
200 bne AKW1
201 jmp NGOACK
202
203 ; AKSHOW -- the value just read, into text row
204 A; "(EMPTY)" if none.
205 AKSHOW: pha
206 lda FNRPLY+0
207 ora FNRPLY+1
208 bne AKSH1
209 pla
210 pha
211 lda #(EMPTY)&$FF
212 sta FNPTL
213 lda #(EMPTY)>>8
214 sta FNPTRH

```

```

194      pla
195      jmp      FNRSTR
196 AKSH1: pla
197      ldx      #2          ; the value
198      starts after the length word
199      ldy      #FNICOL
200      jmp      FNRRPL
201 ; NGOACK -- commit and wait, then check the ACK;
202 on failure, straight to SHOW.
203 NGOACK: jsr      FNGO
204      sta      APERR
205      bne      NGOA1
206      jsr      FNACK
207      sta      APERR
208      beq      NGOA2
209 NGOA1: pla
210      pla
211      jmp      SHOW
212 NGOA2: rts
213 TTITLE: DB      "APPEYS",0
214 TNAME:  DB      "USERNAME",0
215 TOURS:  DB      "KEY 2600:1:0",0
216 TEMPTY: DB      "(EMPTY)",0
217 TVAL:   DB      "HELLO 2600"
218 TVALUE:
219
220 INCLUDE "fujilib.inc"
221 APPVBL: rts
222 INCLUDE "fujidisp.inc"
223
224      ORG      $1FFC
225      DW      START
226      DW      START
227      END
228

```

LISTINGS/ASM/CLOCK.ASM

```

1 ; clock.asm -- the time of day, from the
2 ; adapter's clock.
3 ; The clock is a device of its own ($45).
4 GET_ISO_LOCAL answers with the
5 ; local time as text -- "2026-09-15T20:45:31"
6 and a NUL -- in the adapter's
7 ; configured time zone, and GET_ISO_UTC the same
8 in UTC. Twelve columns is
9 ; not nineteen, so the date goes on one row and
10 the time on the next, both
11 ; straight out of the reply window.
12
13 CPU      6502
14 INCLUDE  "vcs.inc"
15
16 PAD3     EQU      $81
17 SAVSP    EQU      $82
18 APERR    EQU      $8F
19
20 INCLUDE  "fujinet.inc"
21 INCLUDE  "devdefs.inc"
22
23 ORG      $1000
24
25 START:   sei
26          cld
27          ldx      #$FF
28          txs
29          lda      #0
30          sta      CLRLP: $00,x
31          dex
32          bne      CLRLP
33          sta      $00
34          lda      #2
35          sta      VBLANK
36          lda      #0
37          sta      APERR
38
39          jsr      FNARM
40          jsr      FNCHK
41          beq      GOTCART
42          lda      #FNENOC
43          sta      APERR
44          jmp      SHOW
45
46 ; ----- GET_ISO_LOCAL: no parameters
47 GOTCART:
48          lda      #CLKDEV
49          lda      #FNDEV
50          lda      #CKISOL
51          sta      FNCMD
52          lda      #0
53          sta      FNNPR
54          jsr      FNBEQ
55          jsr      FNGO
56          sta      APERR
57          bne      SHOW
58          jsr      FNACK
59          sta      APERR
60          bne      SHOW
61          lda      #(TLOCAL)&$FF
62          sta      FNPTL
63          lda      #(TLOCAL)>>8
64          sta      FNPTRH
65          lda      #2
66          jsr      FNRSTR
67
68          lda      #3
69          characters from offset 0
70          ldx      #0
71          ldy      #10
72          jsr      FNRRPL
73          lda      #4
74          eight from offset 11, past the T
75          ldx      #11
76          ldy      #8
77          jsr      FNRRPL
78
79 ; ----- GET_ISO_UTC -----
80          lda      #CLKDEV
81          lda      #FNDEV
82          lda      #CKISOU
83          sta      FNCMD
84          lda      #0
85          sta      FNNPR
86          jsr      FNBEQ
87          jsr      FNGO
88          sta      APERR
89          bne      SHOW
90          jsr      FNACK
91          sta      APERR
92          bne      SHOW
93          lda      #(TUTC)&$FF
94          sta      FNPTL
95          lda      #(TUTC)>>8
96          sta      FNPTRH
97          lda      #6
98          jsr      FNRSTR
99          lda      #7
100          ldx      #0
101          ldy      #10
102          jsr      FNRRPL
103          lda      #8
104          ldx      #11
105          ldy      #8
106          jsr      FNRRPL
107
108 SHOW:     lda      #(TTITLE)&$FF
109          sta      FNPTL
110          lda      #(TTITLE)>>8
111          sta      FNPTRH
112          lda      #0
113          jsr      FNRSTR
114          lda      #APERR
115          beq      RUN
116          lda      #10
117          jsr      FNROWA
118          lda      #'E'
119          sta      FNRSSEL+FH_TCHR
120          lda      #APERR
121          jsr      FNHEX
122          jsr      FNENDR
123          lda      #0
124          sta      VBLANK
125          jsr      DINIT
126          jmp      DLOOP
127
128 TTITLE:   DB      "CLOCK",0
129 TLOCAL:   DB      "LOCAL",0
130 TUTC:     DB      "UTC",0
131
132 INCLUDE  "fujilib.inc"
133 APPVBL:  rts
134 INCLUDE  "fujidisp.inc"
135
136      ORG      $1FFC
137      DW      START

```

```

132         DW      START
133     END
134

```

LISTINGS/ASM/BUILD.SH

```

1 #!/usr/bin/env bash
2 # build.sh -- assemble the handbook's assembly
3 # clients with Macroassembler AS.
4 #
5 # ./build.sh [client ...]      default:
6 # every client here
7 #
8 # Output: build/<client>.bin, exactly 4096 bytes
9 # -- one 2K bank plus the fixed
10 # half -- with the "FUJI" claim stamped in and
11 # tools/checkrom.py run over it.
12 # The library (../common) is the cartridge
13 # bring-up's testrom, verbatim; its
14 # equates are checked against the firmware
15 # header before anything assembles,
16 # because a drifted address assembles cleanly,
17 # runs, draws a screen, and
18 # silently talks to a page that decodes nothing.
19 set -euo pipefail
20 cd "$(dirname "$0")"
21 FN2600="$FN2600:~$HOME/Workspace/fn-2600/pico/
22 atari-2600)"
23 if command -v asl >/dev/null 2>&1; then AS=asl
24 P2BIN=p2bin
25 elif [ -x "$HOME/asl/asl" ]; then AS="$HOME/asl/
26 asl" P2BIN="$HOME/asl/p2bin"
27 else echo "build.sh: no Macroassembler AS found
28 (tried PATH and ~/asl)" >&2; exit 1; fi
29 mkdir -p build
30 python3 "$FN2600/tools/checkdefs.py" "$FN2600/
31 firmware/include/fuji_mailbox.h" ../common/
32 fuji.net.inc
33
34 claim_off() { echo "${1} - 0x800 + 0x0710 ); }
35 build_one() {
36     local name=$1
37     if [ "$name" = hello ]; then
38         python3 "$FN2600/tools/vcsfont.py" --
39         screen-file screen.txt > screen.inc
40     fi
41     if [ "$name" = fujiboot ]; then
42         { echo "; generated by build.sh -- do
43         not edit"
44         echo "BOOTHST EQU      ${BOOT_HOST:-0}"
45         printf 'BOOTPTH:DB      "%s"\n'
46         "${BOOT_PATH:-/vcsgame.bin}"
47         echo "          DB      0"; } >
48         bootcfg.inc
49     fi
50     $AS -g -L -i ../common "$name.asm"
51     mv "$name.p" "build/$name.p"
52     mv -f "$name.lst" "build/$name.lst" 2>/dev/
53     null || true
54     $P2BIN "build/$name.p" "build/$name.bin" -r
55     '$1000-$1FFF' -l 0 0
56     rm -f "build/$name.p"
57     local size; size=$(stat -c%s "build/
58     $name.bin")
59     printf 'FUJI' | dd of="build/$name.bin" bs=1
60     seek=$((claim_off "$size") conv=notrunc
61     status=none
62     python3 "$FN2600/tools/checkrom.py" "build/
63     $name.bin"
64     echo "build/$name.bin: $size bytes"
65 }
66
67 clients=("$@")
68 [ ${#clients[@]} -eq 0 ] && clients=(hello
69 fuji.test fujiboot fujidir netget)
70 for c in "${clients[@]}; do build_one "$c";
71 done
72

```

LISTINGS/DASM/FUJIBAS.H

```

1 ; fuji.bas.h -- the mailbox as batari Basic sees
2 ; it.
3 ;
4 ; batari Basic emits any name it does not know
5 ; verbatim, so a dasm equate is
6 ; a BASIC variable: 'FNH_TROW = 3' assembles to
7 ; LDA #3 / STA $1Df0, and
8 ; 'x = FNRPly[i]' to LDX i / LDA $1B00,x. BASIC
9 ; has no way to write
10 ; FNHSEL+FH_TROW, so every register arm address
11 ; and every one-shot hotspot
12 ; gets a name of its own here -- each DEFINED
13 ; FROM the fuji.net.h equates, so
14 ; nothing in this file can drift from the
15 ; cartridge header on its own.
16 ;
17 ;
18 ; Include AFTER fuji.net.h and devdefs.h. Never
19 ; write 'FNH_T = FNH_T + 1' or any other
20 ; in-place arithmetic on one of these: batari
21 ; Basic compiles that to an
22 ; INC on the control page, which is the one
23 ; instruction the cartridge
24 ; cannot sample. Compute in a variable, then
25 ; assign.
26 ;
27 ; ---- register arms: store anything here, then
28 ; the value to FNH_T ----
29
30 FNA_DEV = FNHSEL+FR_DEV ; $1D00 device
31 FNA_CMD = FNHSEL+FR_CMD ; $1D01 command
32 id
33 FNA_NPAR = FNHSEL+FR_NPAR ; $1D02
34 parameter count
35 FNA_DRST = FNHSEL+FR_DRST ; $1D05 rewind
36 the TX stream
37 FNA_RXSL = FNHSEL+FR_RXSL ; $1D06 select a
38 reply slice
39 FNA_SEQ = FNHSEL+FR_SEQ ; $1D10 launch:
40 commit ACKSEQ+1 here
41 FNA_BLK = FNHSEL+FR_BLK ; $1D11 arm the
42 ROM swap ($B5)
43
44 ; ---- one-shot operations: the store's data is
45 ; the operand ----
46 FNH_BANK = FNHSEL+FH_BANK ; $1D80 + b:
47 select bank b (store to FNH_BANK+b)
48 FNH_TROW = FNHSEL+FH_TROW ; $1DF0 begin
49 text row (data = row)
50 FNH_TCHR = FNHSEL+FH_TCHR ; $1DF1 append a
51 character
52 FNH_TEND = FNHSEL+FH_TEND ; $1DF2 render
53 the row (data = 0)
54 FNH_PATHC = FNHSEL+FH_PATHC ; $1DF3 append
55 to the path buffer
56

```

```

30 FNH_PATHO = FNRSSEL+FH_PATHO      ; $1DF4 path
   buffer operation (FP_*)
31 FNH_BLSL = FNRSSEL+$F5            ; $1DF5 blit
   source, low
32 FNH_BLSH = FNRSSEL+$F6            ; $1DF6 blit
   source, high
33 FNH_BLDL = FNRSSEL+$F7            ; $1DF7 blit
   destination, low
34 FNH_BLDH = FNRSSEL+$F8            ; $1DF8 blit
   destination, high
35 FNH_BLCNT = FNRSSEL+$F9            ; $1DF9 blit
   count
36 FNH_BLGO = FNRSSEL+$FA            ; $1DFA fire the
   blit (data = transform)
37 FNH_ARM1 = FNRSSEL+FH_ARM1         ; $1DFC the
   arming pair: $B5...
38 FNH_ARM2 = FNRSSEL+FH_ARM2         ; $1DFD ...then
   $A4
39 FNH_SWAP = FNRSSEL+FH_SWAP         ; $1DFE serve
   the staged image (armed only)
40
41 ; ---- the second reply slice page, for a 16-bit
   reply offset ----
42 FNRPY2 = FNRPY+$100                ; $1C00: bytes
   256-511 of the reply slice
43
44 ; The blit transforms (FB_*), the rest of the
   path operations (FP_*) and
45 ; the playfield masks (PFM_*) come from
   devdefs.h, as they do for assembly.
46

```

LISTINGS/BAS/FUJITAIL.ASM

```

1 : fujitail.asm -- the fixed half of a batari
   Basic FujiNet client.
2 ;
3 ; A batari Basic program built with `set romsize
   2k` is exactly the 2K the
4 ; cartridge serves at $1000-$17FF, with its
   `start` at $1000. The cartridge
5 ; needs one more 2K behind it: the claim that
   says "this image is a FujiNet
6 ; client, keep the mailbox alive", and the
   RESET/BRK vectors, which the 6507
7 ; fetches from $1FFC-$1FFF -- the fixed half --
   whatever bank happens to be
8 ; mapped low. Everything between $1800 and $1F0F
   is served by the cartridge
9 ; itself (text planes, reply window, control and
   TX pages, status cells), so
10 ; those bytes of this image are never seen by
   the console.
11 ;
12 ; Assemble once with dasm: dasm fujitail.asm -f3
   -obuild/fujitail.bin
13 ; (2048 bytes, $1800-$1FFF). tools/bbfix.py
   appends it to the program.
14 ; processor 6502
15
16 FNRSSEL = $1D00                    ; + n: arm
   register n / one-shot ops
17 FH_BANK = $80                      ; + b: select
   bank b
18 FH_ARM1 = $FC                      ; the arming
   pair...
19 FH_ARM2 = $FD
20 FNAM1 = $B5
21 FNAM2 = $A4
22
23 ORG $1800                          ; nothing here
   is ours to drive; one byte so
24 .byte 0                            ; the raw
   image starts at $1800
25 ORG $1F10                          ;
   .byte "FUJI"
26 FN_R_CLAIM                         ; the claim --
   ORG $1F14
27 .byte 1, 6, 1                      ; FN_R_HDR: bank
   count, cell height, layout rev
28
29 ; The cold stub. The RESET switch restarts the
   6507 with whatever bank was
30 ; last selected still mapped, and this console's
   cartridge port has no reset
31 ; line -- so the entry point lives in the fixed
   half and reselects bank 0
32 ; before jumping to the program's own `start`.
   Banking is a control-page
33 ; operation and the control page decodes nothing
   until the arming pair has
34 ; arrived, so that comes first.
35
36 COLD ORG $1F20
37 sei
38 cld
39 ldx #FFF
40 txs
41 lda #FNAM1
42 sta FNRSSEL+FH_ARM1
43 lda #FNAM2
44 sta FNRSSEL+FH_ARM2
45 lda #0
46 sta FNRSSEL+FH_BANK ; bank 0: the
   batari Basic program
47 jmp $1000 ; batari Basic's
   `start`
48
49 ORG $1FFC
50 .word COLD ; RESET

```

LISTINGS/BAS/FUJITEXT.ASM

```

1 ; fujitext.asm -- a batari Basic minikernel that
  shows the cartridge's text.
2 ;
3 ; The FujiNet cartridge composes 12-column text
  into six 128-byte planes at
4 ; $1800-$1AFF (fuj1_mailbox.h); this draws
  FT_ROWS of those rows, starting at
5 ; text row FT_ROW0, between batari Basic's
  playfield and its score, in the
6 ; slot the standard kernel offers a
  'minikernel'. The ink body is the
7 ; bring-up's fujidisp.inc kernel, transcribed
  and NOT retimed: its four
8 ; positioning constants are the coordinates of a
  one-pixel-wide window.
9 ;
10 ; In the program:
11 ;   include fujitext.asm
12 ;   const FT_ROW0 = 18      ; first text row
  shown (0-20)
13 ;   const FT_ROWS = 3      ; how many
  (1-21)
14 ;   const pfres = 8         ; and shrink the
  playfield to pay for the
15 ;   const pfrowheight = 8   ; 3 +
  6*FT_ROWS scanlines this costs
16 ;
17 ; batari Basic's frame is timer-locked, so the
  lines this draws come out of
18 ; the time the BASIC program has between frames
  unless the playfield gives
19 ; them back; emu/frames.lua is what says whether
  the frame is still 262.
20 FT_ROW0 = 18
21 ifnconst FT_ROWS
22   endif
23 ifnconst FT_ROWS
24 FT_ROWS = 3
25   endif
26 ifnconst FT_COLOR
27 FT_COLOR = $0E      ; white
28   endif
29 FT_Y0 = FT_ROW0*6
30 FT_Y1 = FT_Y0+FT_ROWS*6
31 FT_PAD = temp1      ; a 3-cycle
  store target
32 FT_SP = temp2      ; the stack
  pointer, parked across the rows
33 ;
34 ; The plane bases, FN_T_PLANE(0..5).
35 FT_P0 = $1800
36 FT_P1 = $1880
37 FT_P2 = $1900
38 FT_P3 = $1980
39 FT_P4 = $1A00
40 FT_P5 = $1A80
41 ;
42 minikernel
43   sta WSYNC
44 ; Two players, three copies each, eight pixels
  apart: P0 draws groups 0,2,4
45 ; and P1 groups 1,3,5, interleaving to 48
  contiguous pixels. VDELP is what
46 ; makes the six-write sequence land in the right
  copies.
47   lda #03
48   sta NUSIZ0
49   sta NUSIZ1
50   lda #1
51   sta VDELP0
52   sta VDELP1
53   lda #FT_COLOR
54   sta COLUP0
55   sta COLUP1
56   lda #0
57   sta GRP0
58   sta GRP1
59   sta GRP0
60 ; Positioning: 16 NOPs plus one 3-cycle store,
  RESP0 at cycle 38, RESP1 three
61 ; cycles later, HMP0 = $F0 to pull P0 right one
  pixel, and an EARLY HMOVE.
62   sta WSYNC
63   REPEAT 16
64     nop
65   REPEND
66   sta FT_PAD
67   sta RESP0
68   sta RESP1
69   lda #$F0
70   sta HMP0
71   lda #0
72   sta HMP1
73   sta WSYNC
74   sta HMOVE
75 ; The rows. 53 cycles of the 76; Y is the
  absolute scanline into the planes,
76 ; so 'lda plane,y' is always four cycles and no
  per-row setup exists. The
77 ; stack pointer parks group 4, because a zero-
  page store is 3 cycles and the
78 ; six copies are 2.67 apart. Nothing here may
  touch the stack.
79   tsx
80   stx FT_SP
81   ldy #FT_Y0
82 ftkern sta WSYNC
83   lda FT_P0,y      ; 4
84   sta GRP0         ; 3 -> 7
  new GRP0 = group 0
85   lda FT_P1,y      ; 4 -> 11
86   sta GRP1         ; 3 -> 14
  displays group 0
87   lda FT_P2,y      ; 4 -> 18
88   sta GRP0         ; 3 -> 21
  displays group 1
89   lda FT_P4,y      ; 4 -> 25
90   sta GRP0         ; 2 -> 27
91   txs             ; 2 -> 29
  park group 4 in S
92   lda FT_P3,y      ; 4 -> 33
93   tax             ; 2 -> 35
94   lda FT_P5,y      ; 4 -> 39
95   stx GRP1        ; 3 -> 42
  displays group 2
96   tsx             ; 2 -> 44
  recover group 4
97   stx GRP0        ; 3 -> 47
  displays group 3
98   sta GRP1        ; 3 -> 50
  displays group 4
99   sty GRP0        ; 3 -> 53
  displays group 5; value unused
100  iny
101  cpy #FT_Y1
102  ftkern
103  ldx FT_SP
104  txs
105 ; Blank both players: three writes, not two --
  with VDELP set a write to
106 ; GRP0 only reloads the DELAYED GRP1 and vice
  versa.
107   lda #0
108   sta GRP0
109   sta GRP1
110   sta GRP0
111   rts
112

```

LISTINGS/BAS/FUJICONST.BAS (GENERATED)

```

1 rem fujiconst.bas -- GENERATED by tools/
2 mkfujiconst.py from the dasm
3 rem headers; prepended to every program by
4 build.sh. Do not edit.
5 const FR_DEV = $00
6 const FR_CMD = $01
7 const FR_NPAR = $02
8 const FR_DRST = $05
9 const FR_RXSL = $06
10 const FR_SEQ = $10
11 const FR_BLKX = $11
12 const FH_BANK = $80
13 const FH_TROW = $F0
14 const FH_TCHR = $F1
15 const FH_TEND = $F2
16 const FH_PATHC = $F3
17 const FH_PATHO = $F4
18 const FP_RST = 0
19 const FP_POP = 1
20 const FP_TX = 2
21 const FP_TXRAW = 3
22 const FH_ARM1 = $FC
23 const FH_ARM2 = $FD
24 const FH_SWAP = $FE
25 const FNAM1 = $B5
26 const FNAM2 = $4A
27 const FNBLKM = $B5
28 const FNTCOL = 12
29 const FNDEVF = $70
30 const FNCADPX = $C4
31 const FNCMHST = $F9
32 const FNCIMMG = $F8
33 const FNCSDFP = $E2
34 const FNCRHRST = $F4
35 const FNCODIR = $F7
36 const FNCRRDIR = $F6
37 const FNCCDIR = $F5
38 const FNCSDPS = $E4
39 const FNREAD = 1
40 const FNBDL = 0
41 const FNBRDY = 2
42 const FNBFATL = $80
43 const FNROK = 0
44 const FNEWAIT = $FF
45 const FENAK = $EE
46 const FENOC = $FD
47 const IN_UP = $01
48 const IN_DOWN = $02
49 const IN_LEFT = $04
50 const IN_RIGHT = $08
51 const IN_FIRE = $10
52 const IN_SEL = $20
53 const IN_RST = $40
54 const NETDEV = $71
55 const NCOPEN = $4F
56 const NCCLOSE = $43
57 const NCREAD = $52
58 const NCWRITE = $57
59 const NCSTAT = $53
60 const NCPARSE = $50
61 const NCQUERY = $51
62 const NCJSON = $FC
63 const NCHITPM = $4D
64 const NMREAD = 4
65 const NMWRITE = 8
66 const NMHTTPG = 12
67 const NMPOST = 13
68 const NTRIOME = 0
69 const NTRCR = 1
70 const NTRLF = 2
71 const NTRCRLF = 3
72 const NSAVLO = 0
73 const NSVLO = 1
74 const NSCONN = 2
75 const NSDEVST = 3
76 const DSKDEV = $31
77 const PRNDEV = $40
78 const CLKDEV = $45
79 const MDDEV = $50
80 const CPMDEV = $5A
81 const FNCRST = $FF
82 const FNCSSD = $FE
83 const FNCSAN = $FD
84 const FNCSNR = $FC
85 const FNCSSD = $FB
86 const FNCWIFI = $FA
87 const FNCWHST = $F3
88 const FNCRDEV = $F2
89 const FNCWDEV = $F1
90 const FNCWIEN = $EA
91 const FNCUIMG = $E9
92 const FNCADPC = $E8
93 const FNCENMD = $E7
94 const FNCUHST = $E6
95 const FNCGDPX = $E5
96 const FNCSDPX = $E1
97 const FNCSDPX = $E0
98 const FNCAKWR = $DE
99 const FNCAKRD = $DD
100 const FNCAKOP = $DC
101 const FNCAKCL = $DB
102 const FNCSDFP = $DA
103 const FNCBOP = $D9
104 const FNCOPY = $D8
105 const FNCMALL = $D7
106 const FNCBSM = $D6
107 const FNCRAND = $D3
108 const FNCB64E1 = $D0
109 const FNCB64EC = $CF
110 const FNCB64EL = $CE
111 const FNCB64EO = $CD
112 const FNCB64DI = $CC
113 const FNCB64DC = $CB
114 const FNCB64DL = $CA
115 const FNCB64DO = $C9
116 const FNCCHSHI = $C8
117 const FNCCHSHC = $C7
118 const FNCCHSHL = $C6
119 const FNCCHSHO = $C5
120 const FNCCHSHN = $C3
121 const FNCCHSHX = $C2
122 const FNCQRO = $BF
123 const FNCQRL = $BE
124 const FNCQRE = $BD
125 const FNCQRI = $BC
126 const FNCGUID = $BB
127 const FNCSTAT = $53
128 const FNCRDY = $00
129 const AKMRD = 0
130 const AKMWR = 1
131 const AKCREAT = 1
132 const AKAPP = 1
133 const AKKEY = 0
134 const CKTIME = $93
135 const CKISOL = $49
136 const CKISOU = $5A
137 const CKGEN = $47
138 const CKTZLEN = $4C
139 const CKSETTZ = $9A
140 const CKTZTIM = $9A
141 const HAMD5 = 0
142 const HASHA1 = 1
143 const HASHA256 = 3
144 const HASHA512 = 4
145 const FP_POPCH = 4
146 const FP_SELO = 5
147 const FP_SEL1 = 6
148 const FP_SEL2 = 7
149 const FP_SEL3 = 8
150 const FP_CMIT = 9
151 const FP_SEED = 10
152 const FH_BSL = $F5
153 const FH_BSH = $F6
154 const FH_BDL = $F7
155 const FH_BDH = $F8
156 const FH_BCNT = $F9
157 const FH_BGO = $FA
158 const FB_RAW = 0
159 const FB_TEXT = 1
160 const FB_FIELD = 2
161 const FB_HULLS = 3
162 const FB_SEA = 4
163 const FB_CELL = 5
164 const FB_PAINT = 6
165 const FB_PATH = 7
166 const FB_TCELL = 8
167 const FB_CARD = 9
168 const FB_PFCCL = 10
169 const FB_PFLD = 11
170 const FB_PFHUL = 12
171 const FB_PFCCL = 13
172 const PFWROW = 30
173 const PFKLEN = 20
174 const PFM_HIT = $01
175 const PFM_MID = $02
176 const PFM_AUX = $04
177 const PFM_ALL = $07
178 const PFM_CLR = $08
179 const PFSLOTS = 4
180

```

LISTINGS/BAS/HELLO.BAS

```
1 rem hello.bas -- HELLO, WORLD: three rows of
2 text, composed by the cartridge
3 rem and shown under the playfield by the
4 fujitext minikernel. No network,
5 rem no transaction: the text rows are a one-
6 shot operation on the control
7 rem page, so all this needs is the gate open
8 and three loops of stores.
9 include fujinet.h
10 include fujibas.h
11 include devdefs.h
12 include fujitext.asm
13 set romsize 2k
14 const FT_ROW0 = 18
15 const FT_ROWS = 3
16 const pfrowheight = 6
17 dim gen = d
18 data msg1
19 "FUJINET 2600"
20 end
21 data msg2
22 "HELLO, WORLD"
23 end
24 data msg3
25 "BATARI BASIC"
26 end
27 COLUBK = 0
28 COLUPF = $84
29 playfield:
30 .....
31 ...XXXXX.X...X...X.XXXXX...
32 ...X...X...X...X...X...X...
33 ...XXXX..X..X...X...X...
34 ...X...X...X...X...X...X...
35 ...X...XXXXX.XXX..XXXXX...
36 .....
37 end
38 FNH_ARM1 = FNAM1
39 FNH_ARM2 = FNAM2
40 FNH_TROW = 18
41 for l = 0 to 11
42   FNH_TCHR = msg1[l]
43 next
44 gosub fnend
45 FNH_TROW = 19
46 for l = 0 to 11
47   FNH_TCHR = msg2[l]
48 next
49 gosub fnend
50 FNH_TROW = 20
51 for l = 0 to 11
52   FNH_TCHR = msg3[l]
53 next
54 gosub fnend
55 main
56 drawscreen
57 goto main
58 rem fnend: render the row, then wait (bounded)
59 for the cartridge to say it
60 rem has landed on the planes -- instant in
61 emulation, a few bus cycles later
62 rem on the real RP2040, which composes a row on
63 its other core.
64 fnend
65 gen = FNBTXG
66 FNH_TEND = 0
67 for l = 0 to 100
68   if FNBTXG <> gen then return
69 next
70 return
```

LISTINGS/BAS/ARM.BAS

```
1 rem arm.bas -- spike S2: open the decode gate
2 and run one transaction,
3 rem GET_ADAPTERCONFIG_EXTENDED, from BASIC
4 alone. The mailbox's names come
5 rem from the assembler headers: an assignment
6 to one is a store, a read of
7 rem one is a load. No dim is needed.
8 include fujinet.h
9 include fujibas.h
10 include devdefs.h
11 set romsize 2k
12 dim fnseq = a
13 dim ch = b
14 COLUBK = 0
15 FNH_ARM1 = FNAM1
16 FNH_ARM2 = FNAM2
17 rem device $70 (FUJI), command $C4, no
18 parameters, rewind TX
19 FNA_DEV = 0
20 FNCMT = FNDEVF
21 FNA_CMD = 0
22 FNCMT = FNCADPX
23 FNA_NPAR = 0
24 FNCMT = 0
25 FNA_DRST = 0
26 FNCMT = 0
27 rem launch: SEQ = the cartridge's ACKSEQ + 1,
28 wrapping 255 to 1
29 fnseq = FNACKS + 1
30 if fnseq = 0 then fnseq = 1
31 FNA_SEQ = 0
32 FNCMT = fnseq
33 wait
34 drawscreen
35 if FNACKS <> fnseq then goto wait
36 rem done; green screen if the transport
37 reported OK, red otherwise
38 COLUBK = $C4
39 if FNERR <> 0 then COLUBK = $34
40 ch = FNRPLY[0]
41 main
42 drawscreen
43 goto main
```

LISTINGS/BAS/TEXT.BAS

```
1 rem text.bas -- spike S3: the transaction, and
2 the reply on screen through
3 rem the cartridge's text planes and the
4 fujitext minikernel.
5 include fujinet.h
6 include fujibas.h
7 include devdefs.h
8 include fujitext.asm
9 set romsize 2k
10 const FT_ROW0 = 18
11 const FT_ROWS = 3
12 dim fnseq = a
13 dim ch = b
14 dim src = c
15 dim gen = d
16 COLUBK = 0
17 COLUPF = $84
18 playfield:
19 .....
20 .....XXXXX.X...X...X.XXXXXX....
21 .....X...X...X...X...X...X...
22 .....XXXXX.X...X...X...X...X...
23 .....X...X...X...X...X...X...
24 .....XXXXXX.XXX.XXXXXX....
25 .....
26 end
27 FNH_ARM1 = FNAM1
28 FNH_ARM2 = FNAM2
29 FNA_DEV = 0
30 FNCMT = FNDEVF
31 FNA_CMD = 0
32 FNCMT = FNCADPX
33 FNA_NPAR = 0
34 FNCMT = 0
35 FNA_DRST = 0
36 FNCMT = 0
37 fnseq = FNACKS + 1
38 if fnseq = 0 then fnseq = 1
39 FNA_SEQ = 0
40 FNCMT = fnseq
41 wait
42 drawscreen
43 if FNERR <> 0 then COLUBK = $34 : goto main
44 rem the reply: SSID at 0, version at 125, IP at
45 140 -- one row each
46 src = 0 : FNH_TROW = 18 : gosub putrow
47 src = 140 : FNH_TROW = 19 : gosub putrow
48 src = 125 : FNH_TROW = 20 : gosub putrow
49 main
50 drawscreen
51 goto main
52 rem putrow: stream up to 12 bytes of the reply
53 at src into the row begun by
54 rem the caller, stopping at NUL, then render
55 and wait for the render.
56 putrow
57 for i = 0 to 11
58 ch = FNRPLY[src]
59 if ch = 0 then goto putend
60 FNH_TCHR = ch
61 src = src + 1
62 next
63 putend
64 gen = FNBTXG
65 FNH_TEND = 0
66 for i = 0 to 100
67 if FNBTXG <> gen then goto putok
68 next
69 putok
70 return
71
```

LISTINGS/BAS/NETGET.BAS

```
1 rem netget.bas -- fetch a URL through the N:
2 device and show the text.
3 rem OPEN it as an HTTP GET, STATUS until the
4 byte count settles, READ up to
5 rem 240 bytes into the reply window, stream it
6 into text row a line at a
7 rem time, CLOSE. Nothing of the reply ever
8 lands in console RAM.
9 include fujinet.h
10 include fujibas.h
11 include devdefs.h
12 include netdefs.h
13 include fujitext.asm
14 set romsize 2k
15 const FT_ROW0 = 16
16 const FT_ROWS = 5
17 const FT_END = 21
18 const pfrowheight = 6
19 dim fnseq = a
20 dim ch = b
21 dim src = c
22 dim gen = d
23 dim avlo = e
24 dim avhi = f
25 dim prlo = g
26 dim prhi = h
27 dim row = j
28 dim tries = k
29 dim col = m
30 dim err = n
31 data url
32 "N:http://127.0.0.1:8765/hello.txt"
33 end
34 COLUBK = 0
35 COLUPF = $84
36 playfield:
37 .....
38 .....XXXXX.X...X...X.XXXXXX....
39 .....X...X...X...X...X...X...
40 .....XXXXX.X...X...X...X...X...
41 .....XXXXXX.XXX.XXXXXX....
42 .....
43 end
44 FNH_ARM1 = FNAM1
45 FNH_ARM2 = FNAM2
46 FNA_DEV = 0
47 FNCMT = NETDEV
48 FNA_CMD = 0
49 FNCMT = NCOPEN
50 FNA_NPAR = 0
51 FNCMT = 2
52 FNA_DRST = 0
53 FNCMT = 0
54 FNTX = 1
```

```

54 FNTX = NMHTTGP
55 FNTX = 1
56 FNTX = NTRNONE
57 for l = 0 to url_length - 1
58 FNTX = url[l]
59 next
60 gosub fngo
61 if err <> 0 then goto fail
62 rem STATUS until two readings agree and
something is waiting
63 prlo = 255
64 prhi = 255
65 tries = 0
66 status = 0
67 FNA_DEV = 0
68 FNCMT = NETDEV
69 FNA_CMD = 0
70 FNCMT = NCSTAT
71 FNA_NPAR = 0
72 FNCMT = 2
73 FNA_DRST = 0
74 FNCMT = 0
75 FNTX = 1
76 FNTX = 0
77 FNTX = 1
78 FNTX = 0
79 gosub fngo
80 if err <> 0 then goto fail
81 avlo = FNRPLY[0]
82 avhi = FNRPLY[1]
83 if avlo <> prlo then goto again
84 if avhi <> prhi then goto again
85 if avhi <> 0 then goto readit
86 if avlo <> 0 then goto readit
87 again
88 prlo = avlo
89 prhi = avhi
90 tries = tries + 1
91 if tries = 30 then goto fail
92 drawscreen
93 drawscreen
94 drawscreen
95 goto status
96 readit
97 rem READ: ONE two-byte parameter, the count,
capped at 240 (twenty rows)
98 if avhi <> 0 then avlo = 240
99 if avlo > 240 then avlo = 240
100 avhi = 0
101 FNA_DEV = 0
102 FNCMT = NETDEV
103 FNA_CMD = 0
104 FNCMT = NCREAD
105 FNA_NPAR = 0
106 FNCMT = 1
107 FNA_DRST = 0
108 FNCMT = 0
109 FNTX = 2
110 FNTX = avlo
111 FNTX = avhi
112 gosub fngo
113 if err <> 0 then goto fail
114 rem show it: a text row per line of the reply,
a row per frame
115 src = 0
116 row = FT_ROW0
117 nextrow
118 if row = FT_END then goto shown
119 FNH_TROW = row
120 col = 0
121 nextch
122 if src = avlo then goto rowend
123 ch = FNRPLY[src]
124 src = src + 1
125 if ch = 10 then goto rowend
126 if ch = 13 then goto nextch
127 if col = 12 then goto nextch
128 FNH_TCHR = ch
129 col = col + 1
130 goto nextch
131 rowend
132 gosub fnend
133 drawscreen
134 row = row + 1
135 if src < avlo then goto nextrow
136 shown
137 rem CLOSE, and the picture stays: the rows are
on the planes now
138 FNA_DEV = 0
139 FNCMT = NETDEV
140 FNA_CMD = 0
141 FNCMT = NCCLOSE
142 FNA_NPAR = 0
143 FNCMT = 0
144 FNA_DRST = 0
145 FNCMT = 0
146 gosub fngo
147 main
148 drawscreen
149 goto main
150 fail
151 COLUBK = $34
152 goto main
153 rem fngo: launch (SEQ = ACKSEQ + 1), draw
frames until the cartridge answers,
rem err = 0 on ACK, else the transport's code
or 255 for no answer / NAK
154 fngo
155 fnseq = FNACKS + 1
156 if fnseq = 0 then fnseq = 1

```

```

158 FNA_SEQ = 0
159 FNCMT = fnseq
160 tries = 0
161 fngowait
162 drawscreen
163 if FNACKS = fnseq then goto fngodone
164 tries = tries + 1
165 if tries <> 0 then goto fngowait
166 err = 255
167 return
168 fngodone
169 err = FNERR
170 if err <> 0 then return
171 if FNRCMD <> 6 then err = 255
172 return
173 fnend
174 gen = FNBTXG
175 FNH_TEND = 0
176 for l = 0 to 100
177 if FNBTXG <> gen then return
178 next
179 return
180

```

LISTINGS/BAS/DIR.BAS

```

1 rem dir.bas -- list a directory of host slot 0,
  with a cursor.
2 rem The working directory lives in the
  CARTRIDGE (path buffer 0): the
3 rem console never holds a path or a filename,
  only a cursor index. Each
4 rem entry is rendered straight from the reply
  window into a text row, so
5 rem moving the cursor re-reads the listing --
  instant in emulation, about a
6 rem second on real hardware, and CONFIG's page
  cache is the cure.
7 include fujinet.h
8 include fujibas.h
9 include devdefs.h
10 include fujitext.asm
11 set romsize 2k
12 const FT_ROW0 = 9
13 const FT_ROWS = 12
14 const pfrowheight = 6
15 const noscore = 1
16 const ROW0 = 10
17 const NROWS = 11
18 dim fnsseq = a
19 dim err = b
20 dim dev = c
21 dim cmd = d
22 dim npar = e
23 dim gen = f
24 dim ch = g
25 dim src = h
26 dim sel = j
27 dim cnt = k
28 dim row = m
29 dim tries = n
30 dim col = o
31 data title
32 "FUJINET DIR"
33 end
34 COLUBK = 0
35 COLUPF = $1A
36 playfield:
37 XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
38 end
39 FNH_ARM1 = FNAM1
40 FNH_ARM2 = FNAM2
41 rem the working directory: path buffer 0 holds
  "/"
42 FNH_PATH0 = FP_SELO
43 FNH_PATH0 = FP_RST
44 FNH_PATHC = 47
45 rem MOUNT_HOST(0)
46 dev = FNDEVF
47 cmd = FNCMHST
48 npar = 1
49 gosub fnbeg
50 FNTX = 1
51 FNTX = 0
52 gosub fngo
53 if err <> 0 then goto fail
54 FNH_TROW = 9
55 for l = 0 to 10
56 FNH_TCHR = title[l]
57 next
58 gosub fnend
59 sel = 0
60 redraw
61 gosub draw
62 if err <> 0 then goto fail
63 main
64 drawscreen
65 if joy0down then goto down
66 if joy0up then goto up
67 goto main
68 down
69 ch = sel + 1
70 if ch = cnt then goto main
71 sel = ch
72 gosub release
73 goto redraw
74 up
75 if sel = 0 then goto main
76 sel = sel - 1
77 gosub release
78 goto redraw
79 release
80 drawscreen
81 if joy0down then goto release
82 if joy0up then goto release
83 return
84 fail
85 COLUBK = $34
86 drawscreen
87 goto fail
88 rem draw: OPEN_DIRECTORY(host 0, the path
  buffer, no filter) -- one
89 rem parameter, then the 256-byte path the
  cartridge emits and pads --
90 rem then READ_DIR_ENTRY(30, 0) once a row until
  the $7F,$7F end marker.
91 draw
92 dev = FNDEVF
93 cmd = FNCODIR
94 npar = 1
95 gosub fnbeg
96 FNTX = 1
97 FNTX = 0
98 FNH_PATH0 = FP_TX
99 gosub fngo
100 if err <> 0 then return
101 cnt = 0
102 entry
103 dev = FNDEVF
104 cmd = FNCODIR
105 npar = 2
106 gosub fnbeg
107 FNTX = 1
108 FNTX = 30
109 FNTX = 1
110 FNTX = 0
111 gosub fngo
112 if err <> 0 then return
113 if FNRPLY[0] <> 127 then goto show
114 if FNRPLY[1] = 127 then goto blank
115 show
116 row = cnt + ROW0
117 FNH_TROW = row
118 ch = 32
119 if cnt = sel then ch = 62
120 FNH_TCHR = ch
121 src = 0
122 col = 1
123 nextch
124 ch = FNRPLY[src]
125 if ch = 0 then goto rowend
126 FNH_TCHR = ch
127 src = src + 1
128 col = col + 1
129 if col < 12 then goto nextch
130 rowend
131 gosub fnend
132 cnt = cnt + 1
133 if cnt < NROWS then goto entry
134 blank
135 row = cnt + ROW0
136 blankrow
137 if row = 21 then goto drawn
138 FNH_TROW = row
139 gosub fnend
140 row = row + 1
141 goto blankrow
142 drawn
143 err = 0
144 return
145 rem fnbeg: begin a transaction from dev / cmd /
  fnbeg
146 fnbeg
147 FNA_DEV = 0
148 FNCMT = dev
149 FNA_CMD = 0
150 FNCMT = cmd
151 FNA_NPAR = 0
152 FNCMT = npar
153 FNA_DRST = 0
154 FNCMT = 0
155 return
156 rem fngo: launch (SEQ = ACKSEQ + 1), draw
  frames until the cartridge
  rem answers; err = 0 on ACK, the transport's
  code, or 255
157 fngo
158 fnsseq = FNACKS + 1
159 if fnsseq = 0 then fnsseq = 1
160 FNA_SEQ = 0
161 FNCMT = fnsseq
162 tries = 0
163 fngowait
164 drawscreen
165 if FNACKS = fnsseq then goto fngodone
166 tries = tries + 1
167 if tries <> 0 then goto fngowait
168 err = 255
169 return
170 fngodone
171 err = FNERR
172 if err <> 0 then return
173 if FNCRCMD <> 6 then err = 255
174 return
175 fnend
176 gen = FNBTXG
177 FNH_TEND = 0
178 for l = 0 to 100
179 if FNBTXG <> gen then return
180 next
181 return
182

```

LISTINGS/BAS/BOOT.BAS

```

1 rem boot.bas -- mount a cartridge image over
  the network and boot it.
2 rem MOUNT_HOST, SET_DEVICE_FULLPATH,
  MOUNT_IMAGE -- and while that last
3 rem reply is outstanding the adapter streams
  the image to the cartridge,
4 rem which stages it and reports BOOT_STATE.
  Then the swap: arm it, copy the
5 rem stub into zero page and jump to it, because
  the swap replaces every byte
6 rem of the window including this program.
7 include fujinet.h
8 include fujibas.h
9 include devdefs.h
10 include fujitext.asm
11 set rmsize 2k
12 const FT_ROW0 = 18
13 const FT_ROWS = 3
14 const pfrowheight = 6
15 dim fnseq = a
16 dim err = b
17 dim dev = c
18 dim cmd = d
19 dim npar = e
20 dim gen = f
21 dim tries = g
22 dim state = h
23 data title
24 "FUJIBOOT"
25 end
26 data path
27 "/hello.bin"
28 end
29 data msg1
30 "MOUNTING"
31 end
32 data msg2
33 "FAILED"
34 end
35 COLUBK = 0
36 COLUPF = $84
37 playfield:
38 .....
39 .....
40 .....XXXXX.X...X...X.XXXXXX.....
41 .....X...X...X...X...X...X.....
42 .....XXXXX.X...X...X...X...X.....
43 .....X...X...X...X...X...X.....
44 .....X...XXXXX.XXX.XXXXXX.....
45 .....
46 end
47 FNH_ARM1 = FNAM1
48 FNH_ARM2 = FNAM2
49 FNH_TROW = 18
50 for l = 0 to 7
51 FNH_TCHR = title[l]
52 next
53 gosub fnend
54 FNH_TROW = 19
55 for l = 0 to 7
56 FNH_TCHR = msg1[l]
57 next
58 gosub fnend
59 rem MOUNT_HOST(0)
60 dev = FNDEVF
61 cmd = FNCMHST
62 npar = 1
63 gosub fnbeg
64 FNTX = 1
65 FNTX = 0
66 gosub fngo
67 if err <> 0 then goto fail
68 rem SET_DEVICE_FULLPATH(device 0, host 0, read,
  path): three parameters,
69 rem then the path as EXACTLY 256 bytes -- built
  in cartridge path buffer 1
70 rem and emitted, NUL-padded, by the cartridge
71 FNH_PATH0 = FP_SEL1
72 FNH_PATH0 = FP_RST
73 for l = 0 to 9
74 FNH_PATHC = path[l]
75 next
76 dev = FNDEVF
77 cmd = FNCSDFP
78 npar = 3
79 gosub fnbeg
80 FNTX = 1
81 FNTX = 0
82 FNTX = 1
83 FNTX = 0
84 FNTX = 1
85 FNTX = FMREAD
86 FNH_PATH0 = FP_TX
87 gosub fngo
88 if err <> 0 then goto fail
89 rem MOUNT_IMAGE(device 0, read): the one that
  takes real time
90 dev = FNDEVF
91 cmd = FNCIMMG
92 npar = 2
93 gosub fnbeg
94 FNTX = 1
95 FNTX = 0
96 FNTX = 1
97 FNTX = FMREAD
98 gosub fngo
99 if err <> 0 then goto fail
100 rem wait for the staged image
101 tries = 0
102 staged
103 drawscreen
104 state = FNBST
105 if state = FNBDRY then goto swap
106 if state = FNBFAIL then goto fail
107 tries = tries + 1
108 if tries <> 0 then goto staged
109 goto fail
110 swap
111 asm
112 lda #FNBCLKM ; arm the swap:
113 BOOTLOCK = the magic
114 sta FNRSEL+FR_BLK
115 sta FNCMT
116 ldx #bootstubend-bootstub-1
117 bootcopy
118 lda bootstub,x ; the stub must
119 run from RAM
120 sta $80,x
121 dex
122 bpl bootcopy
123 jmp $0080
124 bootstub
125 lda #0
126 sta GRP0 ; silence the
127 sprites
128 sta GRP1
129 sta AUDV0 ; and the sound,
130 or a tone screams
131 sta AUDV1
132 sta PF0
133 sta PF1
134 sta PF2
135 sta WSYNC
136 sta SWACNT ; RIOT ports
137 back to inputs, or the next
138 sta SWBCNT ; game cannot
139 read its joystick
140 lda #2
141 sta VBLANK ; blank across
142 the swap
143 sta VSYNC
144 sta FNRSEL+FH_SWAP ; THE SWAP: the
145 window flips before the
146 ldx #$FF ; next fetch,
147 which is here, in RAM
148 txx
149 jmp ($1FFC) ; the new
150 image's own reset vector
151 bootstubend
152 end
153 fail
154 FNH_TROW = 20
155 for l = 0 to 5
156 FNH_TCHR = msg2[l]
157 next
158 gosub fnend
159 COLUBK = $34
160 failed
161 drawscreen
162 goto failed
163 fnbeg
164 FNA_DEV = 0
165 FNCMT = dev
166 FNA_CMD = 0
167 FNCMT = cmd
168 FNA_NPAR = 0
169 FNCMT = npar
170 FNA_DRST = 0
171 FNCMT = 0
172 return
173 fngo
174 fnseq = FNACKS + 1
175 if fnseq = 0 then fnseq = 1
176 FNA_SEQ = 0
177 FNCMT = fnseq
178 tries = 0
179 fngowait
180 drawscreen
181 if FNACKS = fnseq then goto fngodone
182 tries = tries + 1
183 if tries <> 0 then goto fngowait
184 err = 255
185 return
186 fngodone
187 err = FNERR
188 if err <> 0 then return
189 if FNRCMD <> 6 then err = 255
190 return
191 fnend
192 gen = FNBTXG
193 FNH_TEND = 0
194 for l = 0 to 100
195 if FNBTXG <> gen then return
196 next
197 return
198

```

LISTINGS/BAS/APPKEY.BAS

```

1 rem appkey.bas -- read the shared FujiNet
  username, then write a key of
2 rem our own and read it back. An appkey is a
  small file on the adapter's
3 rem SD card named by creator, app and key;
  creator 1 / app 1 / key 0 is the
4 rem slot every FujiNet game reads the player's
  name from.
5 include fujinet.h
6 include fujibas.h
7 include devdefs.h
8 include fujitext.asm
9 set romsize 2k
10 const FT_ROW0 = 15
11 const FT_ROWS = 6
12 const pfrowheight = 6
13 dim fseq = a
14 dim err = b
15 dim dev = c
16 dim cmd = d
17 dim npar = e
18 dim gen = f
19 dim tries = g
20 dim crlo = h
21 dim crhi = j
22 dim keyn = k
23 dim mode = m
24 dim src = n
25 dim len = o
26 data tname
27 "USERNAME"
28 end
29 data tours
30 "KEY 2600:1:0"
31 end
32 data tval
33 "HELLO 2600"
34 end
35 COLUBK = 0
36 COLUPF = $84
37 playfield:
38 .....
39 .....
40 ...XXXX.X...X...X.XXXXX....
41 ...X...X...X...X...X...X...
42 ...XXX..X..X...X...X...X...
43 ...X...X...X.X.X...X...X...
44 ...X...XXXXX..XXX..XXXXX....
45 .....
46 end
47 FNH_ARM1 = FNAM1
48 FNH_ARM2 = FNAM2
49 rem the shared username: creator 1, app 1, key
  0
50 FNH_TROW = 15
51 for l = 0 to 7
52 FNH_TCHR = tname[l]
53 next
54 gosub fnend
55 crlo = AKCREAT
56 crhi = 0
57 keyn = AKKEY
58 mode = AKMRD
59 gosub akopen
60 if err <> 0 then goto fail
61 gosub akread
62 if err <> 0 then goto fail
63 FNH_TROW = 16
64 gosub akshow
65 rem our own key: creator $2600, app 1, key 0 --
  write it, read it back
66 FNH_TROW = 18
67 for l = 0 to 11
68 FNH_TCHR = tours[l]
69 next
70 gosub fnend
71 crlo = $00
72 crhi = $26
73 keyn = 0
74 mode = AKMWR
75 gosub akopen
76 if err <> 0 then goto fail
77 dev = FNDEVF
78 cmd = FNCAKWR
79 npar = 1
80 gosub fnbeg
81 FNTX = 1
82 FNTX = 10
83 for l = 0 to 9
84 FNTX = tval[l]
85 next
86 gosub fngo
87 if err <> 0 then goto fail
88 mode = AKMRD
89 gosub akopen
90 if err <> 0 then goto fail
91 gosub akread
92 if err <> 0 then goto fail
93 FNH_TROW = 19
94 gosub akshow
95 main
96 drawscreen
97 goto main
98 fail
99 COLUBK = $34
100 goto main

101 rem akopen: OPEN_APPKEY -- the six-byte struct
  as the payload: creator
102 rem (two bytes, low first), app, key, mode, and
  a reserved byte
103 akopen
104 dev = FNDEVF
105 cmd = FNCAKOP
106 npar = 0
107 gosub fnbeg
108 FNTX = crlo
109 FNTX = crhi
110 FNTX = AKAPP
111 FNTX = keyn
112 FNTX = mode
113 FNTX = 0
114 gosub fngo
115 return
116 rem akread: READ_APPKEY -- the reply is a
  length word, then the value
117 akread
118 dev = FNDEVF
119 cmd = FNCAKRD
120 npar = 0
121 gosub fnbeg
122 gosub fngo
123 return
124 rem akshow: the value just read into the row
  begun by the caller
125 akshow
126 len = FNRPPLY[0]
127 if len > 12 then len = 12
128 src = 2
129 for l = 1 to len
130 FNH_TCHR = FNRPPLY[src]
131 src = src + 1
132 next
133 gosub fnend
134 return
135 fnbeg
136 FNA_DEV = 0
137 FNCTM = dev
138 FNA_CMD = 0
139 FNCTM = cmd
140 FNA_NPAR = 0
141 FNCTM = npar
142 FNA_DRST = 0
143 FNCTM = 0
144 return
145 fngo
146 fseq = FNACKS + 1
147 if fseq < 0 then fseq = 1
148 FNA_SEQ = 0
149 FNCTM = fseq
150 tries = 0
151 fngowait
152 drawscreen
153 if FNACKS = fseq then goto fngodone
154 tries = tries + 1
155 if tries <> 0 then goto fngowait
156 err = 255
157 return
158 fngodone
159 err = FNERR
160 if err <> 0 then return
161 if FNRCMD <> 6 then err = 255
162 return
163 fnend
164 gen = FNBTXG
165 FNH_TEND = 0
166 for l = 0 to 100
167 if FNBTXG <> gen then return
168 next
169 return
170

```

LISTINGS/BAS/CLOCK.BAS

```

1 rem clock.bas -- the time of day, from the
  adapter's clock (device $45).
2 rem GET_ISO_LOCAL answers "2026-09-15T20:45:31"
  and a NUL; the date goes on
3 rem one row and the time on the next, straight
  out of the reply window.
4 include fujinet.h
5 include fujibas.h
6 include devdefs.h
7 include fujitext.asm
8 set romsize 2k
9 const FT_ROW0 = 15
10 const FT_ROWS = 6
11 const pfrowheight = 6
12 dim fnseq = a
13 dim err = b
14 dim dev = c
15 dim cmd = d
16 dim npar = e
17 dim gen = f
18 dim tries = g
19 dim src = h
20 dim row = j
21 data tlocal
22 "LOCAL"
23 end
24 data tutc
25 "UTC"
26 end
27 COLUBK = 0
28 COLUPF = $84
29 playfield:
30 .....
31 .....XXXXX.X...X.....X.XXXXX.....
32 .....X...X...X...X...X.....
33 .....XXXXX.X...X...X...X.....
34 .....X...X...X...X...X.....
35 .....X...X...X...X...X.....
36 .....X.....XXXXXX.XXX.XXXXX.....
37 .....
38 end
39 FNH_ARM1 = FNAM1
40 FNH_ARM2 = FNAM2
41 FNH_TROW = 15
42 for l = 0 to 4
43   FNH_TCHR = tlocal[l]
44 next
45 gosub fnend
46 cmd = CKISOL
47 row = 15
48 gosub showtime
49 if err <> 0 then goto fail
50 FNH_TROW = 18
51 for l = 0 to 2
52   FNH_TCHR = tutc[l]
53 next
54 gosub fnend
55 cmd = CKISOU
56 row = 19
57 gosub showtime
58 if err <> 0 then goto fail
59 main
60 drawscreen
61 goto main
62 fail
63 COLUBK = $34
64 goto main
65 rem showtime: ask the clock (cmd), then the
  date on row, the time on row+1
66 showtime
67 dev = CLKDEV
68 npar = 0
69 gosub fnbeg
70 gosub fngo
71 if err <> 0 then return
72 FNH_TROW = row
73 for src = 0 to 9
74   FNH_TCHR = FNRPLY[src]
75 next
76 gosub fnend
77 row = row + 1
78 FNH_TROW = row
79 for src = 11 to 18
80   FNH_TCHR = FNRPLY[src]
81 next
82 gosub fnend
83 return
84 fnbeg
85 FNA_DEV = 0
86 FNCMT = dev
87 FNA_CMD = 0
88 FNCMT = cmd
89 FNA_NPAR = 0
90 FNCMT = npar
91 FNA_DRST = 0
92 FNCMT = 0
93 return
94 fngo
95 fnseq = FNACKS + 1
96 if fnseq = 0 then fnseq = 1
97 FNA_SEQ = 0
98 FNCMT = fnseq
99 tries = 0
100 fngowait
101 drawscreen
102 if FNACKS = fnseq then goto fngodone
103 tries = tries + 1
104 if tries <> 0 then goto fngowait
105 err = 255
106 return
107 fngodone
108 err = FNERR
109 if err <> 0 then return
110 if FNRCMD <> 6 then err = 255
111 return
112 fnend
113 gen = FNBTXG
114 FNH_TEND = 0
115 for l = 0 to 100
116   if FNBTXG <> gen then return
117 next
118 return
119

```

LISTINGS/BAS/BUILD.SH

```
1#!/usr/bin/env bash
2# build.sh -- build a batari Basic FujiNet
3# client.
4#
5# ./build.sh prog listings/bas/
6# prog.bas -> build/prog.bin (4096 bytes)
7#
8# batari Basic's native toolchain (bB=...) and
9# dasm on PATH. The program is
10# compiled in build/ so bB's intermediates stay
11# out of the source tree; the
12# library files are reached through symlinks so
13# 'include fujilib.asm' and a
14# # project-local default.inc resolve from the
15# build directory,
16# set -euo pipefail
17# cd "$(dirname "$0")"
18# HERE=$(pwd)
19# export bB="${bB:-$HOME/Workspace/batari-Basic}"
20# export PATH="$HOME/.local/bin:$PATH"
21# FN2600="${FN2600:-$HOME/Workspace/fn-2600/pico/
22# atari-2600}"
23# export FN2600
24# PROG=${1:?usage: build.sh prog}
25
26# mkdir -p build
27# for f in fujinet.h fujibas.h netdefs.h devdefs.h
28# fujilib.asm fujidisp.asm vcs2600.h; do
29# ln -sf "$f" "$HERE/build/$f"
30# done
31# for f in fujitext.asm default.inc; do
32# [ -f "$f" ] && ln -sf "$f" "$HERE/build/$f"
33# done
34# [ -f "build/fujitail.bin" ] || dasm fujitail.asm -
35# f3 -obuild/fujitail.bin -lbuild/fujitail.lst
36# # The BASIC half of the header goes in front of
37# the program: batari Basic
38# # has no BASIC-level include, and a value it has
39# not seen as a 'const' is
40# # compiled as a memory reference. (Error line
41# numbers are offset by its length.)
42# cat fujiconst.bas "$PROG.bas" > "build/
43# $PROG.bas"
44# cd build
45# rm -f "$PROG.bas.bin"
46# "$bB/2600basic.native.sh" "$PROG.bas" 2>&1 |
47# grep -v "A basic version|^A dasm version|^A$|
48# relocateBB"
49# [ -f "$PROG.bas.bin" ] || { echo "build.sh: no
50# $PROG.bas.bin produced" >&2; exit 1; }
51# python3 "$HERE/../../tools/bbfix.py"
52# "$PROG.bas.bin" fujitail.bin "$PROG.bin"
53# grep -o "[0-9]* bytes of ROM space left"
54# "$PROG.bas.lst" | tail -1 || true
55
```

TOOLS/BBFIX.PY

```
1#!/usr/bin/env python3
2"""bbfix.py -- turn a batari Basic 2K build into
3# a FujiNet cartridge image.
4#
5# bbfix.py prog.bas.bin fujitail.bin out.bin
6#
7# The program must be exactly 2048 bytes ('set
8# romsize 2k'); the fixed half
9# (fujitail.asm, assembled) is appended, and the
10# result is a 4096-byte image the
11# tools/checkrom.py from the
12# cartridge tree is then run on it, because a
13# batari Basic statement can emit an
14# INC/DEC on a write-only page as easily as an
15# assembler can.
16#
17# import os
18# import re
19# import subprocess
20# import sys
21#
22# prog, tail, out = sys.argv[1:4]
23# p = open(prog, "rb").read()
24# t = open(tail, "rb").read()
25# if len(p) != 2048:
26#     sys.exit("bbfix: %s is %d bytes, not 2048 --
27# is 'set romsize 2k' missing?"
28#             % (prog, len(p)))
29# if len(t) != 2048 or t[0x718:0x714] != b"FUJI":
30#     sys.exit("bbfix: %s is not a 2048-byte fixed
31# half with the claim at $F100" % tail)
32#
33# open(out, "wb").write(p + t)
34# print("bbfix: %s = %s + fixed half, %d bytes" %
35#       (out, prog, 4096))
36# fn = os.environ.get("FN2600",
37#                     os.path.expanduser("~/Workspace/fn-2600/pico/
38# atari-2600"))
39# r = subprocess.run([sys.executable,
40#                     os.path.join(fn, "tools", "checkrom.py"), out],
41#                     capture_output=True,
42#                     text=True)
43# report = (r.stdout + r.stderr).splitlines()
44#
45# # checkrom bans STA (zp),Y everywhere, because a
46# static scan cannot know what
47#
48# # a zero-page pointer holds. batari Basic's
49# runtime (everything below the
50# # 'game' label) uses indirect stores that point
51# at RAM and the TIA, never at
52# # the mailbox; those are vouched for here, by
53# address. An indirect store in
54# # YOUR code -- at or above 'game' -- still fails
55# the build, as does any RMW.
56# game = None
57#
58# try:
59#     for line in open(prog.replace(".bin",
60#                                     ".sym")):
61#         m = re.match(r"^game\s+([0-9a-fA-F]{4})", line)
62#         if m:
63#             game = int(m.group(1), 16)
64# except OSError:
65#     pass
66# bad = 0
67#
68# for line in report:
69#     m = re.search(r"^\[[0-9a-fA-f]{4}\]: STA
70# \((zp),Y", line)
71#     if m and game is not None and
72#         int(m.group(1), 16) < game:
73#         print("bbfix: (runtime) " + line.split("
74# -- ")[0] + " -- batari Basic's own, vouched
75# for")
76#         continue
77#     if line.strip():
78#         print(line)
79#         bad += 1
80#
81# if bad:
82#     sys.exit("bbfix: checkrom found %d
83# problem(s) in %s" % (bad, out))
84# print("bbfix: checkrom clean above 'game' (%s
85# %04X)" % (game or 0))
86#
87#
```

TOOLS/AS2DASM.PY

```

1 #!/usr/bin/env python3
2 """as2dasm.py -- translate the Macroassembler AS
3 client sources to dasm.
4
5 The cartridge bring-up's client library
6 (testrom/fujinet.inc, fujilib.inc,
7 fujidisp.inc, vcs.inc) is written for
8 Macroassembler AS. batari Basic
9 programs go through dasm, so the library has to
10 exist in dasm syntax too -
11 and two hand-maintained copies of a transport
12 drift. This translates the AS
13 sources mechanically; the dasm twin in listings/
14 dasm/ is GENERATED and the
15 build diffs it against the committed copy.
16
17 as2dasm.py <in.inc|in.asm> <out>
18
19 Rules (all the AS constructs the client sources
20 use):
21 LABEL:                -> LABEL                (dasm
22 labels take no colon)
23 DB / DW               -> .byte / .word
24 REPT n .. ENDM        -> REPEAT n .. REPEND
25 lsr a (asl/rol/ror) -> lsr
26 (implied accumulator
27 (SYM)&$FF / (SYM)>8 -> <SYM / >SYM          (low /
28 high byte)
29 (expr)&$FF            -> [expr]&$FF          (dasm
30 groups with brackets)
31 'c'                  -> 'c                  (dasm
32 character constant)
33 CPU 6502              -> processor 6502
34 INCLUDE "x.inc"       -> include "x.h" / "x.asm"
35 (the dasm twins' names)
36 END                   -> dropped
37
38 The zero-page equates of fujinet.inc are wrapped
39 in IFMCONST so a batari
40 Basic program can place them on cells of its own
41 choosing before including
42 the header (bB's kernel owns $80-$A3, where the
43 AS clients keep theirs).
44 """
45 import re
46 import sys
47
48 ZP_CELLS = {"FENSEQ", "FNCNT", "FNDEV", "FNCMD",
49             "FNNPR", "FNTMO",
50             "FNPTL", "FNPTRH", "FNP CNT",
51             "INCUR", "INPREV",
52             "PAD3", "SAVSP"}
53 INC_MAP = {"vcs.inc": "vcs2600.h",
54            "fujinet.inc": "fujinet.h",
55            "fujilib.inc": "fujilib.asm",
56            "fujidisp.inc": "fujidisp.asm",
57            "screen.inc": "screen.asm",
58            "bootcfg.inc": "bootcfg.asm"}
59
60 def split_comment(line):
61     """split at the first ';' that is not inside
62     a double-quoted string."""
63     inq = False
64     for i, ch in enumerate(line):
65         if ch == '"':
66             inq = not inq
67         elif ch == ';' and not inq:
68             return line[:i], line[i:]
69     return line, ""
70
71 def xlate(line):
72     code, comment =
73     split_comment(line.rstrip("\n"))
74     if not code.strip():
75         return code + comment
76     m = re.match(r"^(CPU)\s+(CPU)\s+S+$", code, re.I)
77     if m:
78         return "processor 6502" +
79         comment
80     if re.match(r"^(END)\s+$", code, re.I):
81         return "." + code + comment
82     m = re.match(r"^(INCLUDE)\s+([^\s]+)$", code, re.I)
83     if m:
84         name = m.group(2)
85         return '%sinclude "%s"' % (m.group(1),
86         INC_MAP.get(name, name)) + comment
87     # label: -> label
88     code = re.sub(r"^[A-Za-z_][A-Za-z0-9_]*\s+",
89     r"l", code)
90     # directives
91     code = re.sub(r"^(DB|DW)\s+", r"l.byte|l.word",
92     code)
93     code = re.sub(r"^(DW)\s+", r"l.word",
94     code)
95     code = re.sub(r"^(REPT)\s+",
96     r"l1REPEAT", code)
97     code = re.sub(r"^(ENDM)\s+",
98     r"l1REPEND", code)
99     # implied accumulator
100     code = re.sub(r"^(b|l|s|a|b)\s+a\b",
101     r"l", code, flags=re.I)
102     # low / high byte of a symbol
103     code = re.sub(r"^\s+([A-Za-z_][A-Za-z0-9_]*)\s+&$FF",
104     r"l1", code)
105     code = re.sub(r"^\s+([A-Za-z_][A-Za-z0-9_]*)\s+>8",
106     r">l1", code)
107     # (expr)&$FF -> [expr]&$FF
108     code = re.sub(r"^\s+([^\s]+)\s+&$FF",
109     r"l1[" + [expr] + "]&$FF",
110     code)
111     # character constants
112     code = re.sub(r"^\s+'.'\s+", r"l1", code)
113     return code + comment
114
115 def main():
116     src, dst = sys.argv[1], sys.argv[2]
117     out = [""]; GENERATED by tools/as2dasm.py from
118     %s -- do not edit; edit the AS"
119     t = xlate(line)
120     % src.split("/")[-1],
121     "; source and regenerate. dasm
122     syntax; see the translator for the rules.]"
123     for line in open(src):
124         m = re.match(r"^[A-Za-z_][A-Za-z0-9_]*\s+(t)",
125         if m and m.group(1) in ZP_CELLS:
126             out.append("IFMCONST %s" %
127             m.group(1))
128         out.append(t)
129         out.append("ENDIF")
130     else:
131         out.append(t)
132     open(dst, "w").write("\n".join(out) + "\n")
133     print("wrote", dst)
134
135 if __name__ == "__main__":
136     main()

```

TOOLS/MKFUJICONST.PY

```
1#!/usr/bin/env python3
2"""mkfujiconst.py -- the BASIC half of the
3mailbox header.
4
5b Atari Basic emits a name it does not know as a
6MEMORY reference: 'x = FNDEVF'
7becomes 'LDA FNDEVF', a load from address $70,
8not the constant. A value has
9to be declared to the compiler with 'const'.
10This generates that block from
11the dasm headers (fujinet.h, fujibas.h): every
12plain numeric equate that is a
13VALUE -- device ids, command ids, register
14numbers, hotspot offsets, blit
15kinds, path ops, error codes -- and none that is
16an ADDRESS (the window at
17$1xxx, the zero-page cells), which BASIC must
18keep treating as memory.
19listings/bas/build.sh prepends the result to
20every program it compiles.
21
22mkfujiconst.py fujinet.h fujibas.h >
23fujiconst.bas
24"""
25import re
26import sys
27
28ZP = {"FNSEQ", "FNCNT", "FNDEV", "FNCMD",
29      "FNNPR", "FNTMO", "FNPTL",
30      "FNPTRH", "FNCPCNT", "INCUR", "INPREV",
31      "PAD3", "SAVSP"}
32
33out = ["rem fujiconst.bas -- GENERATED by
34tools/mkfujiconst.py from the dasm",
35      "rem headers; prepended to every program
36by build.sh. Do not edit."]
37
38seen = set()
39for path in sys.argv[1:]:
40    for line in open(path):
41        m = re.match(r"([A-Za-z_][A-Za-
42z0-9_]*)\s+(?:EQU|=)\s+(\[0-9A-Fa-f\]+\d+)\b",
43                    line)
44        if not m:
45            continue
46        name, val = m.group(1), m.group(2)
47        n = int(val[1:], 16) if
48            val.startswith("$") else int(val)
49        if name in ZP or name in seen or n >=
50            0x1000:
51            continue
52        seen.add(name)
53        out.append("const %s = %s" % (name,
54                                     val))
55print("\n".join(out))
```

EMU/RUN.SH

```
1#!/usr/bin/env bash
2# run.sh -- run a client image in the FujiNet-
3patched MAME.
4#
5# emu/run.sh <image.bin> [lua-script]
6# (script = basename in emu/)
7#
8# With a script it runs headless and exits;
9without one it opens a window.
10# The MAME tree must have had fn-2600/pico/
11atari-2600/emu/apply.sh run against
12# it, and for anything that touches the network
13a fujinet-pc must be listening
14# on BoIP 127.0.0.1:9995 -- which takes ONE
15client, so a stray MAME starves
16# the next run and the symptom is a hang. Kill
17strays first.
18# - MAME must run FROM ITS OWN TREE or -
19autoboot_script is silently ignored.
20# - SDL_VIDEODRIVER=dummy is required wherever
21there is no DISPLAY.
22# - THROTTLED unless FAST=1: servers with
23wall-clock timers need real time.
24set -euo pipefail
25ROM=$(realpath "${1:?usage: run.sh image.bin
26[script]}")
27cd "$(dirname "$0")"
28HERE=$(pwd)
29SCRIPT=${2:-}
30MAME=${MAME:-$HOME/Workspace/mame}
31SNAP=${SNAP:-$HERE/./build/snap}
32mkdir -p "$SNAP"
33kill -f "$[.]mame a2600" 2>/dev/null && sleep
341 || true # anchored: never our own shell
35sleep "${SETTLE:-1}" # let fujinet-pc's one-
36client BoIP listener come back
37args=(a2600 -cartslot "${SLOT:-fujinet}" -cart
38      "$ROM" -snapshot_directory "$SNAP")
39export A2600_EMU="$HERE"
40export FN2600_EMU="$[FN2600:-$HOME/Workspace/
41fn-2600/pico/atari-2600]/emu"
42export DRIVE_EXPECT="$[DRIVE_EXPECT:-$HERE/./
43build/expect.txt]"
44if [ -n "$SCRIPT" ]; then
45    args+=(-autoboot_script "$HERE/$SCRIPT.lua"
46          -video none -sound none
47          -seconds-to_run "${SECS:-10}")
48    [ -n "${FAST:-}" ] && args+=(-nothrottled)
49fi
50[ -n "${DISPLAY:-}" ] || export
51SDL_VIDEODRIVER=dummy
52export
53FUJINET_TCP="${FUJINET_TCP:-127.0.0.1:9995}"
54cd "$MAME"
55exec ./mame "$args[@]"
```

EMU/TXN.LUA

```
1 -- txn.lua -- did the client complete one
  transaction, and what came back?
2 -- Waits on the cartridge's own state (the 'FN'
  magic, then ACKSEQ != 0),
3 -- never on a frame number; prints the status
  cells and the reply's first
4 -- bytes as text; PASS when the transport
  reported no error and the reply is
5 -- the 240-byte AdapterConfigExtended (SSID at
  0, version at 125, IP at 140).
6 local sp =
  manager.machine.devices[":maincpu"].spaces["program"]
7 local waited, done = 0, false
8 local want = tonumber(os.getenv("TXN_RXLEN") or
  "240")
9 local function str(off, max)
10   local s = ""
11   for i = 0, max - 1 do
12     local c = sp:readv_u8(0x1B00 + off + i)
13     if c == 0 then break end
14     if c < 0x20 or c > 0x7E then c = 0x3F
15   end
16   s = s .. string.char(c)
17   return s
18 end
19 _G.txn =
  emu.add_machine_frame_notifier(function()
20   if done then return end
21   if sp:readv_u8(0x1F09) ~= 0x46 or
  sp:readv_u8(0x1F0A) ~= 0x4E then
22     waited = waited + 1
23     if waited > 600 then print("FAIL: no
  cartridge answered"); done = true;
  manager.machine:exit() end
24     return
25   end
26   local ack = sp:readv_u8(0x1F00)
27   if ack == 0 then
28     waited = waited + 1
29     if waited > 900 then print("FAIL: no
  transaction completed"); done = true;
  manager.machine:exit() end
30     return
31   end
32   done = true
33   local err, rcmd = sp:readv_u8(0x1F02),
  sp:readv_u8(0x1F03)
34   local rxlen = sp:readv_u8(0x1F04) + 256 *
  sp:readv_u8(0x1F05)
35   print(string.format("TXN ackseq=%02X err=%d
  reply=%02X rxlen=%d status=%02X bank=%02X
  zp=%02X %02X %02X",
36     ack, err, rcmd, rxlen,
  sp:readv_u8(0x1F01), sp:readv_u8(0x1F0E),
  sp:readv_u8(0xF0), sp:readv_u8(0xF1),
  sp:readv_u8(0xF2)))
37   print("TXN ssid=" .. str(0, 33) .. "
  ver=" .. str(125, 15) .. " ip=" .. str(140, 16))
38   if err == 0 and rcmd == 6 and rxlen == want
  then print("PASS") else print("FAIL") end
  manager.machine:exit()
40 end)
41 end)
42
```

EMU/TEXT.LUA

```
1 -- text.lua -- what is on the cartridge's text
  planes, decoded through the
2 -- font, and does it match what the reply said?
  Compares RENDERED FORMS: the
3 -- expected string is rendered to glyph keys
  with the same font table, so a
4 -- '5' and an 'S' (identical at 3x5) can never
  make a false failure.
5 -- TEXT_ROWS=18:0 19:140 20:125" row:reply-
  offset pairs to check
6 package.path = (os.getenv("A2600_EMU") or
  ".") .. "/?.lua;" .. package.path
7 local font = require("vcfont")
8 local sp =
  manager.machine.devices[":maincpu"].spaces["program"]
9 local waited, done = 0, false
10 local spec = os.getenv("TEXT_ROWS") or "18:0
  19:140 20:125"
11 -- TEXT_WANT="18:HELLO WORLD|19:SECOND ROW"
  compares against literal text instead
12 local want_lit = os.getenv("TEXT_WANT")
13 local function rowkeys(row)
14   local keys = {}
15   for col = 0, 11 do
16     local plane = 0x1B00 + (col // 2) * 0x80
17     local key = 0
18     for line = 0, 4 do
19       local b = sp:readv_u8(plane + row *
20         6 + line)
21       local ink = (col % 2 == 0) and ((b
22         >> 5) & 7) or ((b >> 1) & 7)
23       key = key * 8 + ink
24     end
25     keys[#keys + 1] = key
26   end
27   return keys
28 end
29 local function wantkeys(off)
30   local keys = {}
31   for i = 0, 11 do
32     local c = sp:readv_u8(0x1B00 + off + i)
33     if c == 0 then break end
34     local ch = string.char(c):upper()
35     keys[#keys + 1] = font.key[ch] or
36     font.key["?"]
37   end
38   while #keys < 12 do keys[#keys + 1] =
39     font.key[" "] end
40   return keys
41 end
42 local function litkeys(txt)
43   local keys = {}
44   for i = 1, #txt do
45     local ch = txt:sub(i, i):upper()
46     keys[#keys + 1] = font.key[ch] or
47     font.key["?"]
48   end
49   while #keys < 12 do keys[#keys + 1] =
50     font.key[" "] end
51   return keys
52 end
53 local function decode(keys)
54   local s = ""
55   for _, k in ipairs(keys) do s = s ..
56     (font.glyph[k] or "#") end
57   return s
58 end
59 _G.text =
  emu.add_machine_frame_notifier(function()
60   if done then return end
61   local ack = sp:readv_u8(0x1F00)
62   if sp:readv_u8(0x1F09) ~= 0x46 or (ack == 0
  and not want_lit) then
63     waited = waited + 1
64   end
65 end)
66
```

```

57         if waited > 900 then print("FAIL: no
transaction"); done = true;
manager.machine:exit() end
58     return
59 end
60 waited = waited + 1
61 if waited < (want_lit and 240 or 60) then
return end -- let the rows land
62 done = true
63 local ok = true
64 local checks = {}
65 if want_lit then
66     for pair in want_lit:gmatch("[^"]+") do
67         local row, txt = pair:match("(%d+):
(.*?)$")
68         checks[#checks + 1] =
{ tonumber(row), litkeys(txt) }
69     end
70 else
71     for pair in spec:gmatch("%S+") do
72         local row, off = pair:match("(%d+):
(%d+)" )
73         checks[#checks + 1] =
{ tonumber(row), wantkeys(tonumber(off)) }
74     end
75 for _, c in ipairs(checks) do
76     local row, want = c[1], c[2]
77     local got = rowkeys(row)
78     local same = true
79     for i = 1, 12 do if got[i] ~= want[i]
80 then same = false end end
81     print(string.format("TEXT row %2d %-12s
%s", row, decode(got),
82 ("MISMATCH want " .. same and "ok" or
decode(want))))
83     ok = ok and same
84 end
85 print(string.format("TEXT err=%d rxlen=%d",
sp:readv_u8(0x1F02),
86 sp:readv_u8(0x1F04) +
256 * sp:readv_u8(0x1F05)))
87 print(ok and "PASS" or "FAIL")
88 manager.machine:exit()
89 end)
90

```

EMU/Frames.LUA

```

1 -- frames.lua -- every frame must be 262 lines.
2 --
3 -- Not by counting frames spent in a bank: that
measures where the program is,
4 -- not whether anything is on screen. Not by
sampling the screen either:
5 -- screen:pixel() reads a bitmap MAME updates on
its own schedule. What is
6 -- unambiguous is the program's own VSYNC. Tap
the write, measure the gap to
7 -- the previous one, and anything but 262 lines
is a frame that moved the
8 -- picture -- a poll that blanked, a recompose
that overran, a kernel line
9 -- that silently took two.
10 --
11 -- DRIVE_LUA=emu/drive.lua ./run.sh battleship
frames
12 --
13 -- Without DRIVE_LUA it boots and sits wherever
it lands.
14 if os.getenv("DRIVE_LUA") then
dofile(os.getenv("DRIVE_LUA")) end
15
16 local FRAME = 1 / 59.92
17 local LINE = FRAME / 262
18 local hist = {}
19 local last, lastbank, n, bad = nil, 0, 0, 0
20
21 local function report()
22     local keys = {}
23     for k in pairs(hist) do keys[#keys+1] = k
24 end
25     table.sort(keys)
26     local out = {}
27     for _, k in ipairs(keys) do out[#out+1] =
string.format("%d:%d", k, hist[k]) end
28     print("LINES " .. table.concat(out, " "))
29     print(string.format("FRAMES: %d of %d were
not 262 lines", bad, n))
30 end
31
32 local sp =
manager.machine.devices["maincpu"].spaces["program"]
33 _G._frames = sp:install_write_tap(0x00, 0x00,
"vsync", function(off, data, mask)
34     if (data & 0x02) == 0 then return end
35 -- VSYNC going ON only
36     local t = manager.machine.time:as_double()
37     local bank = sp:readv_u8(0x1F0E)
38     if last then
39         local gap = t - last
40         n = n + 1
41         local lines = math.floor(gap / LINE +
0.5)
42         hist[lines] = (hist[lines] or 0) + 1
43         if lines ~= 262 then
44             bad = bad + 1
45             if bad <= 40 then
46                 print(string.format("BAD FRAME
47 #d: %d lines at %.2fs, bank %d -> %d",
48 t, lastbank, bank))
49                 bad, lines,
50 last, lastbank = t, bank
51 end)
52 _G._frames_stop =
emu.add_machine_stop_notifier(function()
53     report() end)
54

```

fujinet-battleship/atari2600 at b59ec45, in the order the banks include the files.

SRC/VCS.INC

```

1 ; vcs.inc -- TIA and RIOT equates for Macroassembler
2 AS.
3 ; The 2600 world's standard vcs.h is a dasm
4 companion file and is explicitly
5 ; "please do not redistribute", so this is written
6 from the hardware register
7 ; map rather than copied. AS is the assembler the
8 rest of this family already
9 ; uses -- the o2 (8048), arcadia (2650) and channelf
10 (F8) ports all build with
11 ; it and CI already caches its build -- so the 2600
12 client joins them rather
13 ; than adding dasm as a seventh toolchain.
14 ;
15 ; The 6507 has thirteen address lines, so everything
16 mirrors. These are the
17 ; canonical low addresses; TIA writes are $00-$2C,
18 TIA reads $00-$0D, and the
19 ; RIOT sits at $0280 with its RAM at $0080.
20
21 ; ----- TIA, write -----
22 VSYNC EQU $00 ; bit 1: vertical
23 sync
24 VBLANK EQU $01 ; bit 1: blank, bit
25 6/7: input latch/dump
26 WSYNC EQU $02 ; halt the CPU until
27 the next scanline
28 RSYNC EQU $03 ;
29 NUSIZ0 EQU $04 ; player 0 copies /
30 missile size
31 NUSIZ1 EQU $05
32 COLUP0 EQU $06
33 COLUP1 EQU $07
34 COLUPF EQU $08
35 COLUBK EQU $09
36 CTRLPF EQU $0A
37 REFP0 EQU $0B
38 REFP1 EQU $0C
39 PF0 EQU $0D
40 PF1 EQU $0E
41 PF2 EQU $0F
42 RESP0 EQU $10 ; strobe: reset
43 player 0 to the beam
44 RESP1 EQU $11
45 RESM0 EQU $12
46 RESM1 EQU $13
47 RESBL EQU $14
48 AUDC0 EQU $15
49 AUDC1 EQU $16
50 AUDF0 EQU $17
51 AUDF1 EQU $18
52 AUDV0 EQU $19
53 AUDV1 EQU $1A
54 GRP0 EQU $1B ; player 0 graphics
55 GRP1 EQU $1C
56 ENAM0 EQU $1D
57 ENAM1 EQU $1E
58 ENABL EQU $1F
59 HMP0 EQU $20 ; fine motion, high
60 nibble, *7, -8
61 HMP1 EQU $21
62 HMM0 EQU $22
63 HMM1 EQU $23
64 HMBL EQU $24
65 VDELFP EQU $25 ; bit 0: delay GRP0
66 by one write
67 VDELP1 EQU $26
68 VDELBL EQU $27
69 RESMP0 EQU $28
70 RESMP1 EQU $29
71 HMOVE EQU $2A ; strobe: apply the
72 HW registers
73 HMCLE EQU $2B
74 CXCLR EQU $2C
75
76 ; ----- TIA, read -----
77 CXM0P EQU $00
78 CXM1P EQU $01
79 CXP0FB EQU $02
80 CXP1FB EQU $03
81 CXM0FB EQU $04
82 CXM1FB EQU $05
83 CXBLPF EQU $06
84 CXPPMW EQU $07
85 INPT0 EQU $08
86 INPT1 EQU $09
87 INPT2 EQU $0A
88 INPT3 EQU $0B
89 INPT4 EQU $0C ; bit 7 low:
90 joystick 0 fire
91 INPT5 EQU $0D
92
93 ; ----- RIOT -----
94 SWCHA EQU $0280 ; joysticks: P0 in
95 bits 7-4 (R,L,D,U low)
96 SWACNT EQU $0281
97 SWCHB EQU $0282 ; bit 0 RESET, bit 1
98 SELECT (low = pressed)
99 SWBCNT EQU $0283
100 INTIM EQU $0284 ; bit 7 latches on
101 TIMINT EQU $0285 ; stays latched
102 the timer's underflow and
103 until INTIM is read or the

```

SRC/FUJINET.INC

```
1 ; fujinet.inc -- equates for the FujiNet cartridge mailbox.
2 ;
3 ; Hand-mirrored from fujinet-firmware/pico/
4 ; atari-2600/firmware/include/
5 ; fujinet_mailbox.h. tools/checkdefs.py in that tree
6 ; compares this file against
7 ; that header on every build, so the copy cannot
8 ; drift -- and drift here is
9 ; invisible: a client whose control page is one page
10 ; off comes up, draws a
11 ; screen, and simply never arms the mailbox.
12 ;
13 ; TRIMMED: the FUJI device's file, directory and
14 ; boot commands are gone. This
15 ; program mounts nothing and boots nothing. What is
16 ; left is the transport, the
17 ; text ports, the blit port -- including the
18 ; PLAYFIELD transforms the cartridge
19 ; grew for this client -- the path buffers (where
20 ; the player name and the
21 ; table id live, because console RAM cannot hold
22 ; them), the appkey commands
23 ; and the input map.
24 ;
25 ; EQUATES ONLY. This file is included BEFORE the
26 ; ORG; the code half is
27 ; bslib.inc and goes inside it. Including the code
28 ; half early instead
29 ; assembles the whole transport at $0000 and every
30 ; JSR to it becomes
31 ; ; '20 00 00' -- which assembles cleanly and fails at
32 ; run time.
33 ;
34 ; ----- cart -> console, painted
35 ;
36 FNTEXT EQU $1800 ; six 128-byte text
37 planes
38 FNRPPLY EQU $1B00 ; the reply window,
39 512 bytes
40 FNACKS EQU $1F00 ; echoes SEQ when
41 the reply is ready
42 FNSTAT EQU $1F01 ; bit0 link up, bit1
43 busy
44 FNERR EQU $1F02 ; FME+ below
45 FNRCMD EQU $1F03 ; $06 ACK / $15 NAK
46 FNRXLO EQU $1F04 ; reply length,
47 little-endian
48 FNRXHI EQU $1F05
49 ; The three boot-progress bytes and the two bootlock
50 ; equates below are not
51 ; used by this program. They are kept because
52 ; checkdefs.py's shared table is
53 ; what EVERY client has in common, and forking that
54 ; tool to drop five lines
55 ; would create a second thing to drift.
56 FNST EQU $1F06 ; boot state
57 FNBPCC EQU $1F07 ; boot progress,
58 0-100
59 FNBER EQU $1F08 ; boot error
60 FNMGAG EQU $1F09 ; 'F'
61 FNMGAI EQU $1F0A ; 'N'
62 FNPMVER EQU $1F0B
63 FNSECH EQU $1F0C ; slice echo,
64 published last after a repaint ; last text row
65 FNBTXG EQU $1F0D ; last text row
66 rendered, plus a toggle
67 FNBBNK EQU $1F0E ; the live bank
68 FNBLGL EQU $1F0F ; bit0 armed, bit1
69 claim honoured
70 FNPLWL EQU $1F17 ; path buffer
71 length, little-endian
72 FNPLNH EQU $1F18
73 ; Bumped after every blit has actually landed on the
74 ; planes. A blit is a
75 ; single-slot request the bus core hands to the
76 ; cartridge's other core --
77 ; there is no queue -- so a client that fires the
78 ; next one before this has
79 ; run loses this one on hardware and notices nothing
80 ; in emulation, where the
81 ; blit is done inside the store. FNBLIT in bslib.inc
82 polls it.
83 FNBNGB EQU $1F19
84 ; ----- console -> cart, write-only
85 ;
86 ; NEVER READ THESE PAGES. The cart does not drive
87 ; them -- not driving them is
88 ; exactly what lets it sample the data bus -- so a
89 ; read returns the floating
90 ; bus and, worse, is decoded as an access. Only STA/
91 ; STX/STY may target them:
92 ; no INC/DEC/shl/ror (three bus cycles at one
93 ; address, and the "last
94 ; value before the address changed" sampling has no
95 ; defined answer in that
96 ; burst) and no indirect store anywhere (it performs
97 ; a read at the address it
98 ; writes). tools/checkrom.py fails the build on all
99 ; of these.
100 FNRSLE EQU $1D00 ; + n (n < $80): arm
101 register n
102 FNCMT EQU $1DFF ; commit the armed
103 register with this data
104 FNTX EQU $1E00 ; a write anywhere
105 in the page appends
106 ;
107 ; Registers.
108 FR_DEV EQU $00
109 FR_CMD EQU $01
110 FR_NPAR EQU $02
111 FR_DST EQU $03 ; any value: rewind
112 the TX write pointer
113 FR_RXSL EQU $06
114
115 FR_SEQ EQU $10 ; nonzero and !=
116 ACKSEQ: launch
117 FR_BLACK EQU $11 ; arm the ROM swap
118 -- unused here, see above
119 ;
120 ; One-shot operations, in the bit7-set half of the
121 ; control page.
122 FH_BANK EQU $00
123 FH_THROW EQU $F0 ; data = row: begin
124 composing it
125 FH_TCHR EQU $F1 ; data = char:
126 append (saturates at 12)
127 FH_TEND EQU $F2 ; data = attribute
128 (0): render the row
129 FH_PATHC EQU $F3 ; data = char:
130 append to the path buffer
131 FH_PATHO EQU $F4 ; data = FP_*: act
132 on the path buffer
133 FH_ARM1 EQU $F5 ;
134 FH_ARM2 EQU $F6 ;
135 FH_SWAP EQU $FE ;
136
137 FNAM1 EQU $85 ; the arming pair
138 FNAM2 EQU $8A ;
139 FNBLKM EQU $85 ; the bootlock magic
140 -- unused here
141
142 FNTCOL EQU $12 ; text columns
143 FNTRW EQU $21 ; text rows
144 FNCLH EQU $6 ; scanlines per
145 cell: 5 of ink, 1 of leading
146
147 ; Path buffer ops. FOUR buffers, and they SURVIVE A
148 ; CONSOLE RESET -- there is
149 ; no reset line on this connector -- so a cold start
150 ; must empty all four and
151 ; leave a known one selected.
152 FP_RST EQU $0
153 FP_POP EQU $1 ;
154 FP_TX EQU $2 ; emit, NUL-padded
155 to exactly 256
156 FP_TXRAW EQU $3 ; emit exactly what
157 is in it
158 FP_PCHC EQU $4 ; the text editor's
159 backspace
160 FP_SEL0 EQU $5
161 FP_SEL1 EQU $6
162 FP_SEL2 EQU $7
163 FP_SEL3 EQU $8
164 FP_COMMIT EQU $9
165 FP_SEED EQU $10
166
167 ; $1F20-$1F2B is the client's fixed tail: the cold
168 ; stub and the bank
169 ; trampoline, which must be at the same address seen
170 ; from every bank.
171 FNTAIL EQU $1F20
172
173 ; ----- the blit port -----
174 ; Six stores move a screenful of bytes the console
175 ; has neither the RAM nor the
176 ; raster time to move itself.
177 FH_BSL EQU $F5 ; source offset into
178 the reply, low
179 FH_BSH EQU $F6 ; ..high
180 FH_BDL EQU $F7 ; destination, low
181 (a text row, or a SLOT)
182 FH_BDH EQU $F8 ; ..high
183 FH_BCNT EQU $F9 ; count, or a cell
184 FH_BGO EQU $FA ; data = the
185 transform; fire
186 FB_RAW EQU $0
187 FB_TEXT EQU $1 ; cnt ASCII bytes of
188 the reply -> text row dst
189 FB_PATH EQU $7 ; cnt chars of the
190 path buffer -> text row dst
191 FB_TCELL EQU $8 ; ONE character (low
192 byte of src) into ONE
193 cell, dst = row
194 * FNTCOL + col
195
196 ; The PLAYFIELD board -- the four-quadrant display,
197 ; composed by the cartridge
198 ; into six tables the kernel streams into PF0/PF1/
199 ; PF2 twice a scanline.
200 ; r holds register r's tables (0-2 the left half's
201 ; PF0/PF1/PF2, 3-5 the
202 ; right's) at bytes PFRW0.. : three KINDS of PFKLEN
203 ; entries each, one entry
204 ; per cell row, 0-9 the top pair of boards and 10-19
205 ; the bottom pair. A SLOT
206 ; is a pairrow*2 + half: 0 top-left, 1 top-right, 2
207 ; bottom-left, 3 bottom-right.
208 ; The tables sit between text rows 4 and 15, so a
209 ; bank in board mode never
210 ; renders rows 5-14.
211 FB_PFLC EQU $10 ; dst = slot, src
212 low = a kind mask: clear
213 FB_PFLD EQU $11 ; src = reply offset
214 of a gamelfld[100],
215 ; dst = slot: HIT
216 and MID from the cells
217 FB_PRLU EQU $12 ; src = reply offset
218 of cnt placements,
219 ; dst = slot: OR
220 their cells into AUX
221 FB_PRLC EQU $13 ; dst = slot, cnt =
222 cell, src low = a kind
223 ; mask, PFM_CLR
224 set to clear instead
225 PFRW0 EQU $30 ; the first table
226 byte of every plane
227 PFKLEN EQU $20 ; entries per table
228 PFM_HIT EQU $01 ; the kind masks
229 PFM_MID EQU $02
230 PFM_AUX EQU $04
231 PFM_ALL EQU $07
```

```

148 PFM CLR EQU $80
149 PFSLOTS EQU 4
150
151 ; ----- error codes -----
152 FNEOK EQU 0
153 FNEWAIT EQU $FF ; the cart never
154 answered at all
155 FNEWAK EQU $EE ; the server said
156 NAK
157 FNEBADL EQU $FC ; the reply is
158 shorter than its header implies
159 FNEWOC EQU $FD ; no cartridge
160 answered -- deliberately not 1, which is
161 FN_ERR_NOLINK
162
163 ; ----- zero page, the transport's
164 FNSAQ EQU $80 ; the sequence we
165 are waiting for
166 FNCNT EQU $83 ; timeout quanta
167 remaining
168 FNDV EQU $86 ; transaction setup,
169 read by FNBEG
170 FNCMD EQU $87
171 FNNPR EQU $88
172 FNTMO EQU $89 ; timeout in ~0.6s
173 quanta
174 FNPTRL EQU $8A
175 FNPTRH EQU $8B
176 FNPCT EQU $8C ; bytes written into
177 the current path payload
178 INCUR EQU $A8 ; input, both halves
179 INPREV EQU $A9
180
181 ; ----- input -----
182 Jystick 0 in SWCHA bits 7-4 and the console
183 switches in SWCHB, all active
184 ; low, folded by INSCAN into one active-high byte.
185 IN_UP EQU $01
186 IN_DOWN EQU $02
187 IN_LEFT EQU $04
188 IN_RIGHT EQU $08
189 IN_FIRE EQU $10
190 IN_SEL EQU $20
191 IN_RST EQU $40
192 INDIRS EQU IN_UP|IN_DOWN|IN_LEFT|IN_RIGHT
193
194 ; ----- N: device -----
195 NETDEV EQU $71 ; N1:
196 NCOPEN EQU $4F ; 'O'
197 NCCLSE EQU $43 ; 'C'
198 NCREAD EQU $52 ; 'R'
199 NCSTAT EQU $53 ; 'S'
200 NMHTPG EQU $0C ; open mode: HTTP
201 GET
202 NTRNONE EQU 0 ; translation: none
203
204 ; NET STATUS reply: 4 bytes in slice 0.
205 NSAVLO EQU 0 ; bytes waiting, low
206 NSAVHI EQU 1 ; ..high
207 NSCONN EQU 2 ; connected flag
208 NSDEVST EQU 3 ; ndevStatus_t; 1 =
209 SUCCESS
210
211 ; ----- FUJI device, appkeys only
212
213 FNDEVF EQU $70
214 FCAKOPN EQU $DC
215 FCAKCLS EQU $DB
216 FCAKRD EQU $DD
217 FCAKWR EQU $DE
218
219 ; The shared Fujinet username: creator 1, app 1, key
220 0. Every game in the
221 ; family reads the same slot, which is the point of
222 it.
223 AKCREAT EQU 1
224 AKAPP EQU 1
225 AKKEY EQU 0
226 AKMRD EQU 0
227 AKMWR EQU 1

```

SRC/BSDEFS.INC

```

1 ; bsdefs.inc -- what every bank of the Battleship
2 client agrees on.
3 ;
4 ; EQUATES ONLY, included before the ORG next to
5 fujinet.inc.
6 ;
7 ; This console has 128 bytes of RAM and the stack
8 mirrors into the top of
9 ; them. What that buys, once you accept it, is a
10 design where the CARTRIDGE
11 ; holds the state: the server's reply sits in the
12 cartridge's 512-byte window
13 ; and is read from there in place; the four boards
14 are composed by the
15 ; cartridge into playfield tables the kernel streams
16 straight into the TIA;
17 ; the player name and the table id live in the
18 cartridge's path buffers; the
19 ; URL is streamed a character at a time into the
20 cartridge's TX page. What
21 ; the console holds is a cursor, a phase, and a
22 handful of counters.
23
24 ; ----- the banks -----
25
26 ; An image is (N+1) x 2048 bytes -- N banks then the
27 fixed half -- and MAME's
28 ; cart slot accepts only N in {1,3,7,15}. Seven
29 banks, and all seven used.
30 BANKLOB EQU 0 ; the cold start and
31 the table list
32 BANKGAM EQU 1 ; the game: the
33 kernel, the cursor, a shot
34 BANKNET EQU 2 ; one request, with
35 the picture up, and back
36
37 BANKMNU EQU 3 ; the RESET menu:
38 resume, help, leave
39 BANKNAM EQU 4 ; the keyboard and
40 the shared username
41 BANKPLC EQU 5 ; placing the five
42 ships
43 BANKCMP EQU 6 ; compose the game
44 screen, in one vblank
45 NBANKS EQU 7
46
47 ; The trampoline, at a fixed address in the fixed
48 tail because the banks are
49 ; assembled separately. bscore.inc ORGs it there.
50 BSGOTO EQU FNTAIL
51
52 ; A bank switch is a JUMP, never a call: the store
53 that switches is the last
54 ; instruction fetched from the old bank, so there is
55 no return address to
56 ; come back to. "Where to carry on" is therefore
57 DATA, in BSENT.
58 ENCOLD EQU 0 ; bank 0: power-on
59 ENLOBBY EQU 1 ; bank 0: the table
60 list is in the window
61 ENGOLD EQU 2 ; bank 1, then 6: a
62 table was just joined
63 ENGAME EQU 3 ; bank 6: back from
64 a poll: recompose
65 ENFETCH EQU 4 ; bank 2: /state, or
66 the staged request
67 ENTABLE EQU 5 ; bank 2: /tables,
68 then the lobby
69 ENMENU EQU 6 ; bank 3: the RESET
70 menu

```

```

40 ENLEAVE EQU 7 ; bank 2: /leave, /
   $FFB, then the lobby
41 ENNAME EQU 8 ; bank 4: the
   keyboard
42 ENGRUN EQU 9 ; bank 1: composed:
   finish the frame, run
43 ENPLACE EQU 10 ; bank 5: the server
   wants our ships
44 ENPFAIL EQU 11 ; bank 5: the /place
   was refused: again
45 ENQNEXT EQU 12 ; bank 1: composed,
   between frames: run
46 ENRESUME EQU 13 ; bank 6: the menu
   closed, mid-frame
47 ENTEXT EQU 14 ; bank 6: the text
   pass, mid-frame
48 ENGRUND EQU 15 ; bank 1: as ENGRUN,
   after a text bank: DINIT
49 ; ----- requests -----
50 ; Every one of them returns the same state blob, so
   a staged move REPLACES
51 ; the /state poll: its reply is the next state.
   Request 0 is /tables, so a
52 ; cleared "pending request" cell can never mean /
   state.
53 RQTABLE EQU 0
54 RQSTATE EQU 1
55 RQREADY EQU 2 ; a bare /ready
   TOGGLES
56 RQPLACE EQU 3 ; /place/P,P,P,P,P
57 RQATTCK EQU 4 ; /attack/P
58 RQLEAVE EQU 5
59
60 ; The cartridge's path buffers: where the two
   strings this client needs live.
61
62 PBTABLE EQU FP_SEL0 ; the joined table's
   id
63 PBNAME EQU FP_SEL1 ; the player name
64 PBEDIT EQU FP_SEL3 ; what the keyboard
   is typing into
65
66 ; ----- zero page -----
67 ; $80-$BF, and the stack must stay above $C0. $80-
   $8C is the transport's, as
68 ; is $A8/$A9 (fujinet.inc); the rest is ours. $8D-
   $A7 is the ROM swap stub's
69 ; landing ground in the firmware's own clients --
   this program never swaps,
70 ; so it is ours.
71 PA03 EQU $81 ; the display
   kernels' 3-cycle pad target
72 SAVSP EQU $82 ; the stack pointer,
   parked across a text row
73 BSDX EQU $84 ; a render loop's
   index. Its own cell: BSTMP
74
75 across a call
76 BSTMP EQU $85 ; scratch; any
   routine you call may clobber it
77 INRPT EQU $8D ; INRPT's auto-
   repeat countdown
78 BSDGT EQU $8E ; BSDEC's digit and
   leading-zero latch
79 BSREQ EQU $8F ; the request in
   flight (RQ*)
80 BSTRIES EQU $90 ; the settle loop's
   bound
81 BSDLY EQU $91 ; its inter-poll
   delay, in memory because
82
83 X and Y
84 BSINP EQU $92 ; THIS FRAME's input
   mask, in a cell of its own
85 BSAVLO EQU $92 ; NET_STATUS bytes
   waiting -- only ever live
86 BSAVL1 EQU $93 ; inside a
   transaction, when no input runs
87 BSPRV0 EQU $94 ; the previous
   reading, for the settle loop
88 BSPRV1 EQU $95
89 BSNUM0 EQU $94 ; BSDEC's 16-bit
   number: never live with the
90 BSNUM1 EQU $95 ; settle loop,
   which composes nothing
91 BSRXL0 EQU $96 ; the READ's reply
   length, captured at once
92 BSRXL1 EQU $97
93 BSMIN0 EQU $98 ; the length this
   request will settle for
94 BSMIN1 EQU $99
95 BSERR EQU $9A ; the last failure
96 BSSTEP EQU $9B ; where it happened:
   1 open 2 status 3 read
97
98 BSROW EQU $9C ; 4 validate
   row being drawn
99 ROWEND EQU $9C ; board kernel: one
   past the last cell row
100 BSLINE EQU $9D ; text kernel: ink
   lines left in the row
101 BSBAR EQU $9E ; text kernel: the
   red bar's row, or $FF
102 BSTMP2 EQU $9E ; board banks: a
   second scratch cell
103 SNDPTR EQU $9F ; the sound script
   step, 0 = silent
104 SNDCNT EQU $A0 ; frames left on
   that step
105 BSPOLL EQU $A1 ; frames until the
   next /state
106 BSFRAME EQU $A2 ; free-running frame
   counter
107 BSENT EQU $A3 ; EN*: what the bank
   being entered should do
108 BSPCIT EQU $A4 ; playerCount from
   the last reply
109 BSSTAT EQU $A5 ; game status from
   the last reply
110 BSMYST EQU $A6 ; YOUR player status

```

```

187 STSTART EQU 10
188 STMISSE EQU 11
189 STHIT EQU 12
190 STSUNK EQU 13
191 STOVER EQU 99
192
193 ; Player status.
194 PSPLAY EQU 0
195 PSDEFT EQU 1 ; defeated
196 PSVIEW EQU 2 ; spectator
197 PSREADY EQU 3
198 PSPLACE EQU 10
199
200 ; The screen class every sibling collapses the
201 ; status to. A change of class
202 ; -- or of seat count -- forces a full clear,
203 ; because opaque overdraw cannot
204 ; repair content that shrank.
205 CLOBBY EQU 0
206 CLPLACE EQU 1
207 CLPLAY EQU 2
208 CLOVER EQU 3
209
210 BRDDIM EQU 10
211 BRDCCELL EQU 100
212 NSHIPS EQU 5
213 MAXPLR EQU 4
214 NAMEMAX EQU 8 ; the wire field is
215 ; nine with the NUL
216
217 ; ----- colours (NTSC) -----
218 CBLACK EQU $00
219 CWHITE EQU $0E
220 CGREY EQU $06
221 CDIM EQU $04
222 CRED EQU $34 ; the text kernel's
223 ; bar
224 CGREEN EQU $C4
225 CWATER EQU $84 ; the sea
226 CHIT EQU $36 ; a hit: red (hue 4
227 ; is pink on an NTSC set)
228 CMISS EQU $0E ; a miss: white
229 CAUX EQU $1A ; hulls and the
230 ; cursor: gold
231 CLABEL EQU $0E ; text over the
232 ; boards
233 CSTATUS EQU $1C ; the status row
234
235 ; ----- the game screen -----
236 ; Text rows the board banks own. Rows 5-14 are the
237 ; playfield tables and are
238 ; NEVER rendered in board mode; rows 15-20 are the
239 ; lower text area of the
240 ; two-player layout.
241 RSTAT EQU 0 ; the status line
242 RLABA EQU 1 ; names over board
243 ; pair A
244 RLABB EQU 2 ; names over board
245 ; pair B
246 RFLTA EQU 3 ; the fleet strip
247 ; under each name row, three
248 RFLTB EQU 4 ; ink lines,
249 ; quadrant layout only
250 RLOW0 EQU 15 ; the two-player
251 ; layout's lower rows: 15-20
252 NLOWROW EQU 6
253
254 ; Slots: pairrow*2 + half. Player i's slot is a
255 ; mode-dependent table in the
256 ; game bank; these are the two layouts it holds.
257 SLTL EQU 0
258 SLTR EQU 1
259 SLBL EQU 2
260 SLBR EQU 3
261
262 ; Timing: in frames.
263 POLLFRM EQU 90 ; between /state
264 ; polls in play, ~1.5s
265 LOBFRM EQU 120 ; in the lobby, ~2s
266 OVERFRM EQU 180 ; at game over, ~3s:
267 ; the server resets
268 FAILFRM EQU 150 ; after a failed
269 ; poll
270 TURNFRM EQU 4 ; right after our
271 ; own shot: the reply is it
272 BLINKFR EQU 16 ; the pending ship's
273 ; blink
274
275 ; ----- the text screens -----
276 ; The 21-row text kernel (disptext.inc) is the 5
277 ; Card Stud one: a per-row
278 ; background table BSGT of 22 entries and a red bar
279 ; on row BSBAR. Every text
280 ; screen here is white ink, so the seat-colour pick
281 ; is gone from its seam
282 ; line.
283 RSEAT0 EQU 1
284 RBAR0 EQU 17
285 PFEDGE EQU $F0 ; the playfield
286 ; border on the text screens
287
288

```

SRC/BSCORE.INC

```

1 ; bscore.inc -- the contents of the fixed half.
2 ;
3 ; Included by btail.asm, which is the copy that
4 ; becomes the real client's
5 ; last 2K, AND by layout.asm, whose flat 4K image
6 ; contains its own fixed half.
7 ; Sharing the source is what makes the addresses in
8 ; build/tail.inc true for
9 ; both: the layout ROW used to include only the
10 ; generated equates, so every
11 ; ;jsr into the shared transport landed on p2bin's
12 ; filler.
13 ;
14 ; BSGOTO is not a label: bsdefs.inc gives it a fixed
15 ; address and this ORG is
16 ;
17 ; what makes that true, so the two cannot drift
18 ; apart. It must be FIRST in the
19 ; tail, because it is the one address a bank has to
20 ; know before build/tail.inc
21 ; exists
22 ;
23 ; ORG BSGOTO
24 ;
25 ; -----
26 ; BSGOTO -- select bank A and enter it at $1000.
27 ;
28 ; 'sta FNRSEL,x' is the documented-safe indexed
29 ; form: the base low byte is $00
30 ; so the index cannot carry, which means the dummy
31 ; read STA abs,X always

```

```

20 ; performs lands on the same address as the write.
21     cld
22     adc     #FH_BANK
23     tax
24     sta     FNRSSEL,x
25 ; RESET THE STACK. A bank switch is a JUMP and
    nothing ever returns through
26 ; one, so every switch abandons whatever return
    addresses were on the stack --
27 ; and the switches here are taken from inside
    APPVBL, which DLOOP reaches with
28 ; a 3SR. Two bytes leaked per poll, a poll every
    ninety frames, and after
29 ; thirty-five of them the stack had grown down out
    of $C0-$FF and was
30 ; overwriting BSMOVE, BSENT and BSFRAME: a bank
    entry dispatching on an entry
31 ; code that was really the low byte of a return
    address.
32 ;
33 ; This is the one place it can be got right once.
    The store above has already
34 ; switched the bank, but these instructions are
    fetched from the FIXED tail,
35 ; which is not banked, so they still execute.
36     ldx     #SFF
37     txs
38     jmp     $1000
39
40 -----
41 ; The shared transport. tools/mktail.py turns the
    addresses these assemble to
42 ; into build/tail.inc, which is what the banks
    include.
43     INCLUDE "bstrans.inc"
44
45 -----
46 ; BSCOLD -- power-on and RESET.
47 ;
48 ; Deliberately TINY. Everything a cold start has to
    do -- clear the 128 bytes,
49 ; blank the screen, empty the cartridge's four path
    buffers -- is done by bank
50 ; 0 at BSCOLD2, because the tail is 220 bytes and
    the shared transport and
51 ; text primitives want 160 of them. All that has to
    be here is what cannot be
52 ; anywhere else: the arming pair, because banking is
    a control-page operation
53 ; and that page decodes nothing until an ordered
    pair of stores carrying two
54 ; specific values arrives, and the bank switch
    itself.
55 ;
56 ; A cold stub in bank 0 would not do. This console
    has NO RESET LINE to the
57 ; cartridge: the RESET switch restarts the 6507 with
    whatever bank was last
58 ; selected still mapped, so the stub has to be
    somewhere every bank can see.
59 BSCOLD: sei
60         cld
61         ldx     #SFF
62         txs
63 ; FORCE A COLD ENTRY. This console does not clear
    its RAM on a reset, so
64 ; BSENT -- the byte that says what the bank being
    entered should do -- still
65 ; holds whatever the last screen set it to. A reset
    taken at the keyboard came
66 ; back into the LOBBY'S WARM PATH, which redraws a
    table list out of a reply
67 ; window that is holding a Game, and reads table
    names out of the middle of
68 ; lastResult.
69     lda     #ENCOLD
70     sta     BSENT
71     lda     #FNAM1
72     sta     FNRSSEL+FH_ARM1
73     lda     #FNAM2
74     sta     FNRSSEL+FH_ARM2
75     lda     #BANKLOB
76     jmp     BSGOTO
77
78     ORG     $1FFC
79     DW      BSCOLD
80     DW      BSCOLD
81
82

```

SRC/BSTRANS.INC

```

1 ; bstrans.inc -- the transport, assembled ONCE into
    the fixed tail.
2 ;
3 ; $1F20-$1FFB is the only region of the window that
    every bank sees at the
4 ; same address, so it is the only place shared code
    can live. Each 2K bank
5 ; otherwise carries its own copy of everything it
    calls, and the transport is
6 ; 137 bytes of that -- three times over, in a client
    whose game bank came in
7 ; exactly 2048 bytes full.
8 ;
9 ; The banks do not include this file. They include
    build/tail.inc, which
10 ; tools/mktail.py generates from THIS file's
    assembled addresses -- so the two
11 ; cannot drift: there is no hand-maintained address
    list to get wrong.
12 ;
13 ; TWO RULES ABOUT THE CONTROL PAGES, both enforced
    by tools/checkrom.py:
14 ;

```

```

15 ; A register write is TWO stores on this console,
    not one:
16 ;
17 ;     lda #value
18 ;     sta FNRSSEL+reg           ; arm; this store's
    data is ignored
19 ;     sta FNCMT               ; commit: register
    'reg' = value
20 ;
21 ; And never a read-modify-write instruction on the
    control or TX pages.
22 ; INC/DEC/ASL/LSR/ROL/ROR absolute are three bus
    cycles at one address --
23 ; read, write old, write new -- and the
    cartridge's "last value before the
24 ; address changed" sampling has no defined answer
    in that burst. Only
25 ; STA/STX/STY, and never an indirect store
    anywhere.
26 ;
27 ;
28 ; 'sta FNRSSEL,x' is safe for any X: the base low
    byte is $00 so the index can

```

```

29 ; never carry, which means the dummy read STA abs,X
    always performs lands on
30 ; the SAME address as the write.
31 FNFW: sta FNRSEL,x
32 sta FNCNT
33 rts
34
35 ; FNARM -- open the decode gate. Two stores, in
    order, carrying two specific
36 ; values; nothing in the control page decodes until
    they arrive.
37 FNARM: lda #FNAM1
38 sta FNRSEL+FH_ARM1
39 lda #FNAM2
40 sta FNRSEL+FH_ARM2
41 rts
42
43 ; FNCHK -- Z set if a FujiNet cartridge is
    answering.
44 FNCHK: lda FNMAG0
45 cmp #'F'
46 bne FNCHK1
47 lda FNMAG1
48 cmp #'N'
49 FNCHK1: rts
50
51 ; FNBEG -- begin a transaction described by FNDEV /
    FNCMD / FNNPR.
52 ; Rewinds the TX stream LAST, so parameters may be
    appended immediately.
53 FNBEG: lda #16 ; ~9s; longer than
    the cart's own 5s budget so
54 sta FNTMO ; a real timeout
    is reported as its error
55 lda FNDEV
56 ldx #FR_DEV
57 jsr FNRW
58 lda FNCMD
59 ldx #FR_CMD
60 jsr FNRW
61 lda FNNPR
62 ldx #FR_NPAR
63 jsr FNRW
64 lda #0
65 ldx #FR_DRST
66 jmp FNRW
67
68 ; FNBP -- append a one-byte parameter with value A.
    The stream is
69 ; NPARAM x { size byte, then that many value bytes }
    and then the payload.
70 FNBP: pha
71 lda #1
72 sta FNTX
73 pla
74 sta FNTX
75 rts
76
77 ; FNFW -- append a two-byte little-endian parameter.
    A = low, X = high.
78 FNFW: pha
79 lda #2
80 sta FNTX
81 pla
82 sta FNTX
83 txa
84 sta FNTX
85 rts
86
87 ; FNGO -- commit and wait. Returns A = FNE* (0 = the
    transaction completed).
88 ;
89 ; THE RULE EVERY PORT IN THIS FAMILY LEARNED THE
    HARD WAY: the next sequence
90 ; number comes from the CART'S OWN persisted ACKSEQ
    + 1, never from a counter
91 ; in RAM. A console RESET restarts this client and
    re-zeroes its variables but
92 ; does NOT reset the cartridge -- there is no reset
    line on the connector at
93 ; all, and the RESET switch is a bit in a RIOT
    register the cart never sees.
94 ; A program-local counter would restart at 1,
    collide with a sequence already
95 ; answered, and every later transaction would be
    dropped in silence.
96 FNGO: lda FNACKS
97 clc
98 adc #1
99 bne FNGO1
100 lda #1 ; 0 is reserved as
    "never used"
101 FNGO1: sta FNSEQ
102 ldx #FR_SEQ
103 jsr FNRW ; launching it is
    this single commit
104 lda FNTMO
105 sta FNCNT
106 FNGO2: ldy #0
107 FNGO3: ldx #0
108 ; ACKSEQ is published LAST, after the reply,
    its length and the error
109 ; code are all in place, so a match means
    everything behind it is
    already readable.
110 FNGO4: lda FNACKS
111 cmp FNSEQ
112 beq FNGO5
113 dex
114 bne FNGO4
115 dey
116 bne FNGO3
117 bne FNCNT
118 bne FNGO2
119 lda #FNWAIT
120 rts
121
122 FNGO5: lda FNERR
123 rts
124
125 ; FNACK -- 0 on success, FENAK if the server said
    no.
126 FNACK: lda FNRCMD
127 cmp #06
128 beq FNACK1
129 lda #FENAK
130 rts
131 FNACK1: lda #FNEOK
132 rts
133
134 ; -----
135 ; Text, composed by the cartridge. Shared for the
    same reason the transport
136 ; is: every bank draws.
137
138 ; FNROWA -- begin composing text row A.
139 FNROWA: sta FNRSEL+FH_TROW
140 rts
141
142 ; FNCHR -- append character A to the row being
    composed.
143 FNCHR: sta FNRSEL+FH_TCHR
144 rts
145
146 ; FNDNR -- render the composed row.
147 FNDNR: lda #0
148 sta FNRSEL+FH_TEND
149 rts
150
151 ; FNSPC -- append A spaces.
152 FNSPC: tax
153 FNSPC1: lda #1
154 sta FNRSEL+FH_TCHR
155 dex
156 bne FNSPC1
157 rts
158

```

SRC/BSLIB.INC

```
1 ; bslib.inc -- the mailbox transport, the text
2 ; ports, and the odd jobs.
3 ;
4 ; CODE. This file is included INSIDE the ORG; the
5 ; equates are fujinet.inc and
6 ; come before it.
7 ;
8 ; Transcribed from fujinet-firmware/pico/atari-2600/
9 ; testrom/fujilib.inc by way
10 ; of the 5 Card Stud port. Trimmed to what this
11 ; program uses -- there is no
12 ; boot, no ROM swap, no directory -- plus the two
13 ; waits (FNBURG, FNBLIT) that
14 ; make a burst of cartridge work safe on hardware as
15 ; well as in emulation.
16 ;
17 ; TWO RULES ABOUT THE CONTROL PAGES, both enforced
18 ; by tools/checkrom.py:
19 ;
20 ; A register write is TWO stores on this console,
21 ; not one:
22 ;
23 ;     lda #value
24 ;     sta FNRSel+reg          ; arm; this store's
25 ;     data is ignored
26 ;     sta FNCHT              ; commit: register
27 ;     reg = value
28 ;
29 ; And never a read-modify-write instruction on the
30 ; control or TX pages.
31 ; INC/DEC/ASL/LSR/ROL/ROR absolute are three bus
32 ; cycles at one address --
33 ; read, write old, write new -- and the
34 ; cartridge's "last value before the
35 ; address changed" sampling has no defined answer
36 ; in that burst. Only
37 ; STA/STX/STY, and never an indirect store
38 ; anywhere, because an indirect
39 ; store performs a read at the address it writes.
40 ;
41 ; The transport core -- FNRW, FNARM, FNCHK, FNBEQ,
42 ; FNPB, FNPW, FNCO and
43 ; FNACK -- is NOT here. It lives in the fixed tail
44 ; (src/bstrans.inc, assembled
45 ; by bstrail.asm) and the banks reach it through
46 ; build/tail.inc.
47 ;
48 ; That is worth the build machinery. Every bank is
49 ; 2048 bytes and carries its
50 ; own copy of everything it calls, so a byte in this
51 ; file costs three; the
52 ; transport is 137 of them and the game bank came in
53 ; exactly full without the
54 ; move it needed. The fixed tail at $1F20-$1F7B is
55 ; the one region every bank
56 ; sees at the same address, which is what makes
57 ; sharing possible at all.
58 ;
59 ; -----
60 ; Text, composed by the cartridge.
61 ;
62 ; FNROWA, FNCHCR, FNENDR and FNAPC are in the fixed
63 ; tail with the transport:
64 ;
65 ; every bank composes text, and four bytes apiece
66 ; times seven banks is worth
67 ; one copy. FNCLS is NOT -- it was, until the
68 ; trampoline needed three bytes
69 ; for the stack reset and the tail was exactly full.
70 ;
71 ; IF BSHASCLS
72 ; FNCLS -- blank every text row.
73 ;
74 ; THE TEXT PLANES ARE CARTRIDGE STATE AND SURVIVE A
75 ; BANK SWITCH, so a screen
76 ; that simply does not paint a row inherits whatever
77 ; the previous screen left
78 ; there. Every full redraw starts here rather than
79 ; each bank tracking which
80 ; rows it owns -- that is exactly the bookkeeping
81 ; that goes wrong when a menu
82 ; of a different length opens over a screen of a
83 ; different shape.
84 ; FNCLS: ldx #0
85 ; FNCLS1: txa
86 ; lsr FNROWA
87 ; lsr FNENDR
88 ; inx
89 ; cpx #FNTRW
90 ; bcc FNCLS1
91 ; rts
92 ;
93 ; ENDIF
94 ;
95 ; IF BSHASSTR
96 ; FNSTRA -- append the NUL-terminated string at
97 ; FNPTRL/FNPTRH to the row
98 ; being composed, without ending it. Clobbers Y.
99 ; FNSTRA: ldy #0
100 ; FNSTA1: lda (FNPTRL),y
101 ; beq FNSTA2
102 ; sta FNRSel+FH_TCHR
103 ; iny
104 ; cpy #FNTPCL
105 ; bne FNSTA1
106 ; FNSTA2: rts
107 ;
108 ; ENDIF
109 ;
110 ; FNSETP -- point FNPTRL/H at the string whose
111 ; address is Y (lo) Y (hi)
112 ; Not guarded: url.inc needs it, and url.inc is in
113 ; the bank that draws
114 ; nothing.
115 ; FNSETP: stx FNPTRL
116 ; sty FNPTRH
117 ; rts
118 ;
119 ; FNENDW -- render the composed row and WAIT for it
120 ; to land. FNBTXG is bumped
121 ; by the cartridge after the row is actually on the
122 ; planes; a client that
123 ; goes on to the next row's characters before then,
124 ; on hardware, hands the
125 ; render a row that is already being overwritten. In
126 ; emulation the render is
127 ; done inside the store and the loop exits at once.
128 ; Bounded, so a cartridge
129 ; that never answers costs a millisecond and not the
130 ; frame.
131 ; FNENDW: ldx FNBTXG
132 ; lda #0
133 ; sta FNRSel+FH_TEND
134 ; ldy #128
135 ; FNEW1: cpx FNBTXG
136 ; bne FNEW2
137 ; dey
138 ; bne FNEW1
139 ; FNEW2: rts
140 ;
141 ; FNBLIT -- fire blit transform A and wait for
142 ; FNNGEN to say it has landed,
143 ; for the same reason. A blit is a SINGLE-SLOT
144 ; request on the cartridge --
145 ; there is no queue -- so the next one's argument
146 ; stores would overwrite this
147 ; one's before it. Clobbers X and Y.
148 ; FNBLIT: ldx FNNGEN
149 ; ldy FNRSel+FH_BGO
150 ; lda #128
151 ; FNBL1: cpx FNNGEN
152 ; bne FNBL2
153 ; dey
154 ; bne FNBL1
155 ; FNBL2: rts
156 ;
157 ; FNBARG -- the blit arguments: A = src low, X = dst
158 ; low, Y = cnt; the two
159 ; high bytes are zero, which every playfield blit
160 ; and every text blit here
161 ; wants. Nothing is fired.
162 ; FNBARG: sta FNRSel+FH_BSL
163 ; stx FNRSel+FH_BDL
164 ; sty FNRSel+FH_BCN
165 ; lda #0
166 ; sta FNRSel+FH_BSH
167 ; sta FNRSel+FH_BDH
168 ;
169 ; BSHASRPL, not BSHASSTR: only the composer streams
170 ; the reply window into a
171 ; row. Every other bank that composes text composes
172 ; it out of its own strings.
173 ; IF BSHASRPL
174 ; FNRPPLA -- append up to Y bytes of the reply window
175 ; at offset X to the row
176 ; being composed, stopping at NUL. Pads the rest
177 ; with spaces so a shorter
178 ; value leaves no debris behind it.
179 ; FNRPPLA: ldx #1
180 ; rts
181 ;
182 ; This is the zero-copy path and on this console it
183 ; is not an optimisation:
184 ; the reply window is repainted only by a SEQ commit
185 ; or a slice select, so it
186 ; is stable while the row is streamed, and the bytes
187 ; go from the cartridge
188 ; back to the cartridge without ever occupying any
189 ; of the 128 bytes.
190 ; FNRPPLA: ldx FNRPPL,x
191 ; beq FNRP2A
192 ; sta FNRSel+FH_TCHR
193 ; inx
194 ; dey
195 ; bne FNRPPLA
196 ; rts
197 ;
198 ; FNRP2A: ldx #1
199 ; rts
200 ;
201 ; is left of the field
202 ; FNRP2A: ldx #1
203 ; sta FNRSel+FH_TCHR
204 ; dey
205 ; bne FNRP2A
206 ; rts
207 ;
208 ; ENDIF
209 ;
210 ; The cartridge's path buffers: where the player
211 ; name and the table id live,
212 ; because 128 bytes of console RAM cannot spare
213 ; nineteen for them. They
214 ; SURVIVE A CONSOLE RESET, so a cold start must
215 ; empty all four.
216 ; FNWSEL -- select path buffer A (FP_SEL0..SEL3).
217 ; FNWSEL: sta FNRSel+FH_PATHO
218 ; rts
219 ;
220 ; IF BSHASED
221 ; FNWRST -- empty the selected buffer.
222 ; FNWRST: ldx #FP_RST
223 ; sta FNRSel+FH_PATHO
224 ; rts
225 ;
226 ; FNWPOP -- the text editor's backspace: drop the
227 ; last character.
228 ; FNWPOP: ldx #FP_POPCH
229 ; sta FNRSel+FH_PATHO
230 ; rts
231 ;
232 ; ENDIF
233 ;
234 ; FNRW -- stream the selected buffer into TX
235 ; exactly as it stands, with no
```

```

173 ; padding. This is how a table id or a player name
174 reaches a URL without ever
175 ; crossing into console RAM.
176 FNRWR: lda #FP_TARAW
177 sta FNRSEL+FH_PATHO
178 rts
179
180 IF BSHASED
181 ; FNRWPL -- append up to Y bytes of the reply window
182 at offset X to the
183 ; selected buffer, stopping at NUL.
184 ;
185 ; Fixed-width wire fields are pushed up to their NUL
186 and never through it: the
187 ; padding in the middle of a URL corrupts the
188 request, which is the bug the
189 ; Arcadia port calls out by name.
190 FNRWPL: lda FNRPLY,x
191 beq FNRW2
192 sta FNRSEL+FH_PATHC
193 inx
194 dey
195 FNRW2: rts
196 FNRW2: ENDF
197 IF BSHASDEC
198 ;
199 ; BSDEC -- print BSNUM0/BSNUM1 as decimal into the
200 row being composed, in a
201 ; field of exactly A characters (1-5). Clobbers X
202 and Y, and NOT BSTMP.
203 ;
204 ; Sixteen bits, because a pot and a purse are u16
205 and the widest number any
206 ; sibling 2600 client had to print was a board
207 coordinate. Repeated
208 ; subtraction against a table of powers of ten: five
209 16-bit subtracts is
210 ; smaller than any division, and there is no
211 multiply on this machine.
212 ;
213 ; The field width picks the STARTING power, so a
214 four-wide field prints
215 ; thousands down to units and nothing above --
216 callers with a narrow field
217 ; call BSCL4 first so a value that would not fit
218 cannot reach here. Leading
219 ; zeros are spaces; the units digit always prints,
220 so a pot of zero reads "0"
221 ; and not as an empty field. The field is always
222 exactly A wide, which is what
223 ; makes a value that SHRINKS leave nothing behind
224 it.
225 BSDEC: eor #$FF ; A = 5 - width, so
226 the field width picks the ; first power of
227 ten to use. No scratch cell ; at all: BSDEC
228 ; adc ;
229 once borrowed BSTMP for this ; and silently ate
230 its caller's loop index.
231 tax
232 lda #0
233 sta BSDGT
234 BSD1: lda BSDGT
235 and #$80 ; a new digit;
236 significance is sticky
237 sta BSDGT
238 BSD2: lda BSNUM0 ; num -= power,
239 sixteen bits, while it fits
240 sec
241 sbc BSPOW10,x
242 tay
243 lda BSNUM1
244 sbc BSPOW10+1,x
245 bcc BSD3 ; it did not fit:
246 this digit is done
247 sta BSNUM1
248 sty BSNUM0
249 inc BSDGT
250 this cannot reach bit 7 ; a digit is 0-9, so
251 jmp BSD2
252 BSD3: lda BSDGT
253 and #$0F
254 bne BSD4 ; a significant
255 digit
256 cpx #8 ; the units digit
257 prints even as a zero
258 beq BSD4
259 bit BSDGT ; bit 7 into N
260 bpl BSD5 ; still leading:
261 emit a space
262 BSD4: lda BSDGT
263 ora #$80
264 sta BSDGT
265 and #$0F
266 clc
267 adc #'0'
268 jmp BSD6
269 BSD5: lda #' '
270 BSD6: sta FNRSEL+FH_TCHR
271 inx
272 inx
273 cpx #10
274 bne BSD1
275 rts
276
277 ; Powers of ten, little-endian, high to low.
278 BSPOW10: DW 10000
279 DW 1000
280 DW 100
281 DW 10
282 DW 1
283 ENDF
284
285 ; BSHASINP, not BSHASUI: reading the stick and
286 reading a player record out of

```

```

263 ; the reply window are wanted by different banks,
264 and 104 bytes of input
265 ; scanning in the composer -- whose APPVBL is an rts
266 ; was 104 bytes the
267 ; fleet rows needed: state.inc's helpers keep
268 BSHASUI.
269 IF BSHASINP
270 ;
271 ; INSCAN -- fold the joystick and the console
272 switches into one active-high
273 ; byte in INCUR, and return only what is NEWLY
274 pressed, so a held stick steps
275 ; once.
276 ; INCUR is left holding the LEVELS, which is what
277 the purse overlay reads:
278 ; "SELECT held" is a level and not an edge.
279 INSCAN: lda SWCHA
280 eor #$FF ; the port is active
281 low
282 and #$F0 ; right, left, down,
283 up in bits 7-4
284 lsr a
285 lsr a
286 lsr a
287 lsr a
288 sta INCUR
289 INP4 INSC1 ; bit 7 high = fire
290 not pressed
291 INSCAN: lda INCUR
292 ora #IN_FIRE
293 sta INCUR
294 INSC1: lda SWCHB
295 and #$01
296 bne INC2
297 lda INCUR
298 ora #IN_RST
299 sta INCUR
300 INSC2: lda SWCHB
301 and #$02
302 bne INC3
303 lda INCUR
304 ora #IN_SEL
305 sta INCUR
306 INSC3: lda INPREV
307 and NOT ; newly pressed =
308 eor #$FF
309 INCUR
310 pha
311 lda INCUR
312 INPREV
313 rts
314 ; INREPT -- INSCAN, plus auto-repeat on the four
315 DIRECTIONS only.
316 ; FIRE, SELECT and RESET are deliberately not
317 repeated: a repeating fire would
318 ; submit a bet and then submit whatever the cursor
319 landed on next.
320 ; Repeat is affordable only because moving a cursor
321 costs no round trip on any
322 ; screen here -- the reply window is stable between
323 transactions, so a redraw
324 ; reads names the cartridge is already holding.
325 INDLAY EQU 30 ; ~0.5s before the
326 first repeat
327 INRATE EQU 6 ; ~10 a second after
328 that
329 INREPT: jar INSCAN
330 beq INRP1 ; nothing newly
331 pressed: a repeat may be due
332 pha
333 the delay
334 restart #INDLAY
335 sta INRP1
336 pla
337 rts
338 INRP1: lda INCUR
339 and #INDIRS
340 INRP2: lda INRP1
341 and #INDIRS ; no direction held:
342 INRP3: lda INRP2
343 and #INDIRS
344 INRP4: lda INRP3
345 and #INDIRS
346 INRP5: lda INRP4
347 and #INDIRS
348 INRP6: lda INRP5
349 and #INDIRS
350 INRP7: lda INRP6
351 and #INDIRS
352 INRP8: lda INRP7
353 and #INDIRS
354 INRP9: lda INRP8
355 and #INDIRS
356 INRP10: lda INRP9
357 and #INDIRS
358 INRP11: lda INRP10
359 and #INDIRS
360 INRP12: lda INRP11
361 and #INDIRS
362 INRP13: lda INRP12
363 and #INDIRS
364 INRP14: lda INRP13
365 and #INDIRS
366 INRP15: lda INRP14
367 and #INDIRS
368 INRP16: lda INRP15
369 and #INDIRS
370 INRP17: lda INRP16
371 and #INDIRS
372 INRP18: lda INRP17
373 and #INDIRS
374 INRP19: lda INRP18
375 and #INDIRS
376 INRP20: lda INRP19
377 and #INDIRS
378 INRP21: lda INRP20
379 and #INDIRS
380 INRP22: lda INRP21
381 and #INDIRS
382 INRP23: lda INRP22
383 and #INDIRS
384 INRP24: lda INRP23
385 and #INDIRS
386 INRP25: lda INRP24
387 and #INDIRS
388 INRP26: lda INRP25
389 and #INDIRS
390 INRP27: lda INRP26
391 and #INDIRS
392 INRP28: lda INRP27
393 and #INDIRS
394 INRP29: lda INRP28
395 and #INDIRS
396 INRP30: lda INRP29
397 and #INDIRS
398 INRP31: lda INRP30
399 and #INDIRS
400 INRP32: lda INRP31
401 and #INDIRS
402 INRP33: lda INRP32
403 and #INDIRS
404 INRP34: lda INRP33
405 and #INDIRS
406 INRP35: lda INRP34
407 and #INDIRS
408 INRP36: lda INRP35
409 and #INDIRS
410 INRP37: lda INRP36
411 and #INDIRS
412 INRP38: lda INRP37
413 and #INDIRS
414 INRP39: lda INRP38
415 and #INDIRS
416 INRP40: lda INRP39
417 and #INDIRS
418 INRP41: lda INRP40
419 and #INDIRS
420 INRP42: lda INRP41
421 and #INDIRS
422 INRP43: lda INRP42
423 and #INDIRS
424 INRP44: lda INRP43
425 and #INDIRS
426 INRP45: lda INRP44
427 and #INDIRS
428 INRP46: lda INRP45
429 and #INDIRS
430 INRP47: lda INRP46
431 and #INDIRS
432 INRP48: lda INRP47
433 and #INDIRS
434 INRP49: lda INRP48
435 and #INDIRS
436 INRP50: lda INRP49
437 and #INDIRS
438 INRP51: lda INRP50
439 and #INDIRS
440 INRP52: lda INRP51
441 and #INDIRS
442 INRP53: lda INRP52
443 and #INDIRS
444 INRP54: lda INRP53
445 and #INDIRS
446 INRP55: lda INRP54
447 and #INDIRS
448 INRP56: lda INRP55
449 and #INDIRS
450 INRP57: lda INRP56
451 and #INDIRS
452 INRP58: lda INRP57
453 and #INDIRS
454 INRP59: lda INRP58
455 and #INDIRS
456 INRP60: lda INRP59
457 and #INDIRS
458 INRP61: lda INRP60
459 and #INDIRS
460 INRP62: lda INRP61
461 and #INDIRS
462 INRP63: lda INRP62
463 and #INDIRS
464 INRP64: lda INRP63
465 and #INDIRS
466 INRP65: lda INRP64
467 and #INDIRS
468 INRP66: lda INRP65
469 and #INDIRS
470 INRP67: lda INRP66
471 and #INDIRS
472 INRP68: lda INRP67
473 and #INDIRS
474 INRP69: lda INRP68
475 and #INDIRS
476 INRP70: lda INRP69
477 and #INDIRS
478 INRP71: lda INRP70
479 and #INDIRS
480 INRP72: lda INRP71
481 and #INDIRS
482 INRP73: lda INRP72
483 and #INDIRS
484 INRP74: lda INRP73
485 and #INDIRS
486 INRP75: lda INRP74
487 and #INDIRS
488 INRP76: lda INRP75
489 and #INDIRS
490 INRP77: lda INRP76
491 and #INDIRS
492 INRP78: lda INRP77
493 and #INDIRS
494 INRP79: lda INRP78
495 and #INDIRS
496 INRP80: lda INRP79
497 and #INDIRS
498 INRP81: lda INRP80
499 and #INDIRS
500 INRP82: lda INRP81
501 and #INDIRS
502 INRP83: lda INRP82
503 and #INDIRS
504 INRP84: lda INRP83
505 and #INDIRS
506 INRP85: lda INRP84
507 and #INDIRS
508 INRP86: lda INRP85
509 and #INDIRS
510 INRP87: lda INRP86
511 and #INDIRS
512 INRP88: lda INRP87
513 and #INDIRS
514 INRP89: lda INRP88
515 and #INDIRS
516 INRP90: lda INRP89
517 and #INDIRS
518 INRP91: lda INRP90
519 and #INDIRS
520 INRP92: lda INRP91
521 and #INDIRS
522 INRP93: lda INRP92
523 and #INDIRS
524 INRP94: lda INRP93
525 and #INDIRS
526 INRP95: lda INRP94
527 and #INDIRS
528 INRP96: lda INRP95
529 and #INDIRS
530 INRP97: lda INRP96
531 and #INDIRS
532 INRP98: lda INRP97
533 and #INDIRS
534 INRP99: lda INRP98
535 and #INDIRS
536 INRP100: lda INRP99
537 and #INDIRS
538 INRP101: lda INRP100
539 and #INDIRS
540 INRP102: lda INRP101
541 and #INDIRS
542 INRP103: lda INRP102
543 and #INDIRS
544 INRP104: lda INRP103
545 and #INDIRS
546 INRP105: lda INRP104
547 and #INDIRS
548 INRP106: lda INRP105
549 and #INDIRS
550 INRP107: lda INRP106
551 and #INDIRS
552 INRP108: lda INRP107
553 and #INDIRS
554 INRP109: lda INRP108
555 and #INDIRS
556 INRP110: lda INRP109
557 and #INDIRS
558 INRP111: lda INRP110
559 and #INDIRS
560 INRP112: lda INRP111
561 and #INDIRS
562 INRP113: lda INRP112
563 and #INDIRS
564 INRP114: lda INRP113
565 and #INDIRS
566 INRP115: lda INRP114
567 and #INDIRS
568 INRP116: lda INRP115
569 and #INDIRS
570 INRP117: lda INRP116
571 and #INDIRS
572 INRP118: lda INRP117
573 and #INDIRS
574 INRP119: lda INRP118
575 and #INDIRS
576 INRP120: lda INRP119
577 and #INDIRS
578 INRP121: lda INRP120
579 and #INDIRS
580 INRP122: lda INRP121
581 and #INDIRS
582 INRP123: lda INRP122
583 and #INDIRS
584 INRP124: lda INRP123
585 and #INDIRS
586 INRP125: lda INRP124
587 and #INDIRS
588 INRP126: lda INRP125
589 and #INDIRS
590 INRP127: lda INRP126
591 and #INDIRS
592 INRP128: lda INRP127
593 and #INDIRS
594 INRP129: lda INRP128
595 and #INDIRS
596 INRP130: lda INRP129
597 and #INDIRS
598 INRP131: lda INRP130
599 and #INDIRS
600 INRP132: lda INRP131
601 and #INDIRS
602 INRP133: lda INRP132
603 and #INDIRS
604 INRP134: lda INRP133
605 and #INDIRS
606 INRP135: lda INRP134
607 and #INDIRS
608 INRP136: lda INRP135
609 and #INDIRS
610 INRP137: lda INRP136
611 and #INDIRS
612 INRP138: lda INRP137
613 and #INDIRS
614 INRP139: lda INRP138
615 and #INDIRS
616 INRP140: lda INRP139
617 and #INDIRS
618 INRP141: lda INRP140
619 and #INDIRS
620 INRP142: lda INRP141
621 and #INDIRS
622 INRP143: lda INRP142
623 and #INDIRS
624 INRP144: lda INRP143
625 and #INDIRS
626 INRP145: lda INRP144
627 and #INDIRS
628 INRP146: lda INRP145
629 and #INDIRS
630 INRP147: lda INRP146
631 and #INDIRS
632 INRP148: lda INRP147
633 and #INDIRS
634 INRP149: lda INRP148
635 and #INDIRS
636 INRP150: lda INRP149
637 and #INDIRS
638 INRP151: lda INRP150
639 and #INDIRS
640 INRP152: lda INRP151
641 and #INDIRS
642 INRP153: lda INRP152
643 and #INDIRS
644 INRP154: lda INRP153
645 and #INDIRS
646 INRP155: lda INRP154
647 and #INDIRS
648 INRP156: lda INRP155
649 and #INDIRS
650 INRP157: lda INRP156
651 and #INDIRS
652 INRP158: lda INRP157
653 and #INDIRS
654 INRP159: lda INRP158
655 and #INDIRS
656 INRP160: lda INRP159
657 and #INDIRS
658 INRP161: lda INRP160
659 and #INDIRS
660 INRP162: lda INRP161
661 and #INDIRS
662 INRP163: lda INRP162
663 and #INDIRS
664 INRP164: lda INRP163
665 and #INDIRS
666 INRP165: lda INRP164
667 and #INDIRS
668 INRP166: lda INRP165
669 and #INDIRS
670 INRP167: lda INRP166
671 and #INDIRS
672 INRP168: lda INRP167
673 and #INDIRS
674 INRP169: lda INRP168
675 and #INDIRS
676 INRP170: lda INRP169
677 and #INDIRS
678 INRP171: lda INRP170
679 and #INDIRS
680 INRP172: lda INRP171
681 and #INDIRS
682 INRP173: lda INRP172
683 and #INDIRS
684 INRP174: lda INRP173
685 and #INDIRS
686 INRP175: lda INRP174
687 and #INDIRS
688 INRP176: lda INRP175
689 and #INDIRS
690 INRP177: lda INRP176
691 and #INDIRS
692 INRP178: lda INRP177
693 and #INDIRS
694 INRP179: lda INRP178
695 and #INDIRS
696 INRP180: lda INRP179
697 and #INDIRS
698 INRP181: lda INRP180
699 and #INDIRS
700 INRP182: lda INRP181
701 and #INDIRS
702 INRP183: lda INRP182
703 and #INDIRS
704 INRP184: lda INRP183
705 and #INDIRS
706 INRP185: lda INRP184
707 and #INDIRS
708 INRP186: lda INRP185
709 and #INDIRS
710 INRP187: lda INRP186
711 and #INDIRS
712 INRP188: lda INRP187
713 and #INDIRS
714 INRP189: lda INRP188
715 and #INDIRS
716 INRP190: lda INRP189
717 and #INDIRS
718 INRP191: lda INRP190
719 and #INDIRS
720 INRP192: lda INRP191
721 and #INDIRS
722 INRP193: lda INRP192
723 and #INDIRS
724 INRP194: lda INRP193
725 and #INDIRS
726 INRP195: lda INRP194
727 and #INDIRS
728 INRP196: lda INRP195
729 and #INDIRS
730 INRP197: lda INRP196
731 and #INDIRS
732 INRP198: lda INRP197
733 and #INDIRS
734 INRP199: lda INRP198
735 and #INDIRS
736 INRP200: lda INRP199
737 and #INDIRS
738 INRP201: lda INRP200
739 and #INDIRS
740 INRP202: lda INRP201
741 and #INDIRS
742 INRP203: lda INRP202
743 and #INDIRS
744 INRP204: lda INRP203
745 and #INDIRS
746 INRP205: lda INRP204
747 and #INDIRS
748 INRP206: lda INRP205
749 and #INDIRS
750 INRP207: lda INRP206
751 and #INDIRS
752 INRP208: lda INRP207
753 and #INDIRS
754 INRP209: lda INRP208
755 and #INDIRS
756 INRP210: lda INRP209
757 and #INDIRS
758 INRP211: lda INRP210
759 and #INDIRS
760 INRP212: lda INRP211
761 and #INDIRS
762 INRP213: lda INRP212
763 and #INDIRS
764 INRP214: lda INRP213
765 and #INDIRS
766 INRP215: lda INRP214
767 and #INDIRS
768 INRP216: lda INRP215
769 and #INDIRS
770 INRP217: lda INRP216
771 and #INDIRS
772 INRP218: lda INRP217
773 and #INDIRS
774 INRP219: lda INRP218
775 and #INDIRS
776 INRP220: lda INRP219
777 and #INDIRS
778 INRP221: lda INRP220
779 and #INDIRS
780 INRP222: lda INRP221
781 and #INDIRS
782 INRP223: lda INRP222
783 and #INDIRS
784 INRP224: lda INRP223
785 and #INDIRS
786 INRP225: lda INRP224
787 and #INDIRS
788 INRP226: lda INRP225
789 and #INDIRS
790 INRP227: lda INRP226
791 and #INDIRS
792 INRP228: lda INRP227
793 and #INDIRS
794 INRP229: lda INRP228
795 and #INDIRS
796 INRP230: lda INRP229
797 and #INDIRS
798 INRP231: lda INRP230
799 and #INDIRS
800 INRP232: lda INRP231
801 and #INDIRS
802 INRP233: lda INRP232
803 and #INDIRS
804 INRP234: lda INRP233
805 and #INDIRS
806 INRP235: lda INRP234
807 and #INDIRS
808 INRP236: lda INRP235
809 and #INDIRS
810 INRP237: lda INRP236
811 and #INDIRS
812 INRP238: lda INRP237
813 and #INDIRS
814 INRP239: lda INRP238
815 and #INDIRS
816 INRP240: lda INRP239
817 and #INDIRS
818 INRP241: lda INRP240
819 and #INDIRS
820 INRP242: lda INRP241
821 and #INDIRS
822 INRP243: lda INRP242
823 and #INDIRS
824 INRP244: lda INRP243
825 and #INDIRS
826 INRP245: lda INRP244
827 and #INDIRS
828 INRP246: lda INRP245
829 and #INDIRS
830 INRP247: lda INRP246
831 and #INDIRS
832 INRP248: lda INRP247
833 and #INDIRS
834 INRP249: lda INRP248
835 and #INDIRS
836 INRP250: lda INRP249
837 and #INDIRS
838 INRP251: lda INRP250
839 and #INDIRS
840 INRP252: lda INRP251
841 and #INDIRS
842 INRP253: lda INRP252
843 and #INDIRS
844 INRP254: lda INRP253
845 and #INDIRS
846 INRP255: lda INRP254
847 and #INDIRS
848 INRP256: lda INRP255
849 and #INDIRS
850 INRP257: lda INRP256
851 and #INDIRS
852 INRP258: lda INRP257
853 and #INDIRS
854 INRP259: lda INRP258
855 and #INDIRS
856 INRP260: lda INRP259
857 and #INDIRS
858 INRP261: lda INRP260
859 and #INDIRS
860 INRP262: lda INRP261
861 and #INDIRS
862 INRP263: lda INRP262
863 and #INDIRS
864 INRP264: lda INRP263
865 and #INDIRS
866 INRP265: lda INRP264
867 and #INDIRS
868 INRP266: lda INRP265
869 and #INDIRS
870 INRP267: lda INRP266
871 and #INDIRS
872 INRP268: lda INRP267
873 and #INDIRS
874 INRP269: lda INRP268
875 and #INDIRS
876 INRP270: lda INRP269
877 and #INDIRS
878 INRP271: lda INRP270
879 and #INDIRS
880 INRP272: lda INRP271
881 and #INDIRS
882 INRP273: lda INRP272
883 and #INDIRS
884 INRP274: lda INRP273
885 and #INDIRS
886 INRP275: lda INRP274
887 and #INDIRS
888 INRP276: lda INRP275
889 and #INDIRS
890 INRP277: lda INRP276
891 and #INDIRS
892 INRP278: lda INRP277
893 and #INDIRS
894 INRP279: lda INRP278
895 and #INDIRS
896 INRP280: lda INRP279
897 and #INDIRS
898 INRP281: lda INRP280
899 and #INDIRS
900 INRP282: lda INRP281
901 and #INDIRS
902 INRP283: lda INRP282
903 and #INDIRS
904 INRP284: lda INRP283
905 and #INDIRS
906 INRP285: lda INRP284
907 and #INDIRS
908 INRP286: lda INRP285
909 and #INDIRS
910 INRP287: lda INRP286
911 and #INDIRS
912 INRP288: lda INRP287
913 and #INDIRS
914 INRP289: lda INRP288
915 and #INDIRS
916 INRP290: lda INRP289
917 and #INDIRS
918 INRP291: lda INRP290
919 and #INDIRS
920 INRP292: lda INRP291
921 and #INDIRS
922 INRP293: lda INRP292
923 and #INDIRS
924 INRP294: lda INRP293
925 and #INDIRS
926 INRP295: lda INRP294
927 and #INDIRS
928 INRP296: lda INRP295
929 and #INDIRS
930 INRP297: lda INRP296
931 and #INDIRS
932 INRP298: lda INRP297
933 and #INDIRS
934 INRP299: lda INRP298
935 and #INDIRS
936 INRP300: lda INRP299
937 and #INDIRS
938 INRP301: lda INRP300
939 and #INDIRS
940 INRP302: lda INRP301
941 and #INDIRS
942 INRP303: lda INRP302
943 and #INDIRS
944 INRP304: lda INRP303
945 and #INDIRS
946 INRP305: lda INRP304
947 and #INDIRS
948 INRP306: lda INRP305
949 and #INDIRS
950 INRP307: lda INRP306
951 and #INDIRS
952 INRP308: lda INRP307
953 and #INDIRS
954 INRP309: lda INRP308
955 and #INDIRS
956 INRP310: lda INRP309
957 and #INDIRS
958 INRP311: lda INRP310
959 and #INDIRS
960 INRP312: lda INRP311
961 and #INDIRS
962 INRP313: lda INRP312
963 and #INDIRS
964 INRP314: lda INRP313
965 and #INDIRS
966 INRP315: lda INRP314
967 and #INDIRS
968 INRP316: lda INRP315
969 and #INDIRS
970 INRP317: lda INRP316
971 and #INDIRS
972 INRP318: lda INRP317
973 and #INDIRS
974 INRP319: lda INRP318
975 and #INDIRS
976 INRP320: lda INRP319
977 and #INDIRS
978 INRP321: lda INRP320
979 and #INDIRS
980 INRP322: lda INRP321
981 and #INDIRS
982 INRP323: lda INRP322
983 and #INDIRS
984 INRP324: lda INRP323
985 and #INDIRS
986 INRP325: lda INRP324
987 and #INDIRS
988 INRP326: lda INRP325
989 and #INDIRS
990 INRP327: lda INRP326
991 and #INDIRS
992 INRP328: lda INRP327
993 and #INDIRS
994 INRP329: lda INRP328
995 and #INDIRS
996 INRP330: lda INRP329
997 and #INDIRS
998 INRP331: lda INRP330
999 and #INDIRS
1000 INRP332: lda INRP331
1001 and #INDIRS
1002 INRP333: lda INRP332
1003 and #INDIRS
1004 INRP334: lda INRP333
1005 and #INDIRS
1006 INRP335: lda INRP334
1007 and #INDIRS
1008 INRP336: lda INRP335
1009 and #INDIRS
1010 INRP337: lda INRP336
1011 and #INDIRS
1012 INRP338: lda INRP337
1013 and #INDIRS
1014 INRP339: lda INRP338
1015 and #INDIRS
1016 INRP340: lda INRP339
1017 and #INDIRS
1018 INRP341: lda INRP340
1019 and #INDIRS
1020 INRP342: lda INRP341
1021 and #INDIRS
1022 INRP343: lda INRP342
1023 and #INDIRS
1024 INRP344: lda INRP343
1025 and #INDIRS
1026 INRP345: lda INRP344
1027 and #INDIRS
1028 INRP346: lda INRP345
1029 and #INDIRS
1030 INRP347: lda INRP346
1031 and #INDIRS
1032 INRP348: lda INRP347
1033 and #INDIRS
1034 INRP349: lda INRP348
1035 and #INDIRS
1036 INRP350: lda INRP349
1037 and #INDIRS
1038 INRP351: lda INRP350
1039 and #INDIRS
1040 INRP352: lda INRP351
1041 and #INDIRS
1042 INRP353: lda INRP352
1043 and #INDIRS
1044 INRP354: lda INRP353
1045 and #INDIRS
1046 INRP355: lda INRP354
1047 and #INDIRS
1048 INRP356: lda INRP355
1049 and #INDIRS
1050 INRP357: lda INRP356
1051 and #INDIRS
1052 INRP358: lda INRP357
1053 and #INDIRS
1054 INRP359: lda INRP358
1055 and #INDIRS
1056 INRP360: lda INRP359
1057 and #INDIRS
1058 INRP361: lda INRP360
1059 and #INDIRS
1060 INRP362: lda INRP361
1061 and #INDIRS
1062 INRP363: lda INRP362
1063 and #INDIRS
1064 INRP364: lda INRP363
1065 and #INDIRS
1066 INRP365: lda INRP364
1067 and #INDIRS
1068 INRP366: lda INRP365
1069 and #INDIRS
1070 INRP367: lda INRP366
1071 and #INDIRS
1072 INRP368: lda INRP367
1073 and #INDIRS
1074 INRP369: lda INRP368
1075 and #INDIRS
1076 INRP370: lda INRP369
1077 and #INDIRS
1078 INRP371: lda INRP370
1079 and #INDIRS
1080 INRP372: lda INRP371
1081 and #INDIRS
1082 INRP373: lda INRP372
1083 and #INDIRS
1084 INRP374: lda INRP373
1085 and #INDIRS
1086 INRP375: lda INRP374
1087 and #INDIRS
1088 INRP376: lda INRP375
1089 and #INDIRS
1090 INRP377: lda INRP376
1091 and #INDIRS
1092 INRP378: lda INRP377
1093 and #INDIRS
1094 INRP379: lda INRP378
1095 and #INDIRS
1096 INRP380: lda INRP379
1097 and #INDIRS
1098 INRP381: lda INRP380
1099 and #INDIRS
1100 INRP382: lda INRP381
1101 and #INDIRS
1102 INRP383: lda INRP382
1103 and #INDIRS
1104 INRP384: lda INRP383
1105 and #INDIRS
1106 INRP385: lda INRP384
1107 and #INDIRS
1108 INRP386: lda INRP385
1109 and #INDIRS
1110 INRP387: lda INRP386
1111 and #INDIRS
1112 INRP388: lda INRP387
1113 and #INDIRS
1114 INRP389: lda INRP388
1115 and #INDIRS
1116 INRP390: lda INRP389
1117 and #INDIRS
1118 INRP391: lda INRP390
1119 and #INDIRS
1120 INRP392: lda INRP391
1121 and #INDIRS
1122 INRP393: lda INRP392
1123 and #INDIRS
1124 INRP394: lda INRP393
1125 and #INDIRS
1126 INRP395: lda INRP394
1127 and #INDIRS
1128 INRP396: lda INRP395
1129 and #INDIRS
1130 INRP397: lda INRP396
1131 and #INDIRS
1132 INRP398: lda INRP397
1133 and #INDIRS
1134 INRP399: lda INRP398
1135 and #INDIRS
1136 INRP400: lda INRP399
1137 and #INDIRS
1138 INRP401: lda INRP400
1139 and #INDIRS
1140 INRP402: lda INRP401
1141 and #INDIRS
1142 INRP403: lda INRP402
1143 and #INDIRS
1144 INRP404: lda INRP403
1145 and #INDIRS
1146 INRP405: lda INRP404
1147 and #INDIRS
1148 INRP406: lda INRP405
1149 and #INDIRS
1150 INRP407: lda INRP406
1151 and #INDIRS
1152 INRP408: lda INRP407
1153 and #INDIRS
1154 INRP409: lda INRP408
1155 and #INDIRS
1156 INRP410: lda INRP409
1157 and #INDIRS
1158 INRP411: lda INRP410
1159 and #INDIRS
1160 INRP412: lda INRP411
1161 and #INDIRS
1162 INRP413: lda INRP412
1163 and #INDIRS
1164 INRP414: lda INRP413
1165 and #INDIRS
1166 INRP415: lda INRP414
1167 and #INDIRS
1168 INRP416: lda INRP415
1169 and #INDIRS
1170
```

SRC/NET.INC

```

1 ; net.inc -- the N: device round trip.
2 ;
3 ; A 6582 transcription of arcadia/net.inc's
4 ; sequence, which is the astrocade's,
5 ; which is the Odyssey 2's. The rules it carries
6 ; forward were each learned
7 ; somewhere else in this family and none of them is
8 ; optional:
9 ;
10 ; * The CLOSE belongs at the START of the next
11 ; request, never after the READ.
12 ; Every transaction repaints the whole reply
13 ; window, a CLOSE reply is
14 ; empty, and every screen here renders straight
15 ; out of that window BETWEEN
16 ; polls -- so closing after the read wipes the
17 ; thing being displayed.
18 ; * FNRLX0/FNRXHI belong to the MOST RECENT
19 ; transaction, so the reply length
20 ; is captured immediately after the READ and not
21 ; a transaction later.
22 ; * The ESP32 reports STATUS as soon as SOME of the
23 ; response has arrived.
24 ; Poll until two consecutive readings agree AND
25 ; the count has reached a
26 ; caller-supplied minimum. Two agreeing readings
27 ; alone is not enough: the
28 ; Channel F port watched a /state fetch settle at
29 ; 129 bytes of a 385-byte
30 ; response and hand back a plausible header
31 ; followed by forty bytes of
32 ; nothing.
33 ; * The 4th STATUS byte is nDevStatus_t. The HTTP
34 ; GET is DEFERRED until the
35 ; first STATUS call, so that byte is the only
36 ; place an HTTP error is
37 ; visible -- an error page has a perfectly
38 ; readable body and a perfectly
39 ; plausible length.
40 ; * Unsigned compares go through the CARRY, never
41 ; the sign bit. The Channel F
42 ; port's settle loop used the sign and 181 - 1
43 ; came out negative, so a
44 ; settled 181-byte reply read as short of a one-
45 ; byte minimum and the poll
46 ; never terminated.
47 ;
48 ; Every routine returns A = 0 on success and sets Z
49 ; accordingly.
50 ;
51 ; NCLOSE -- close the previous request's connection.
52 ; The first call has
53 ; nothing to close and earns a NAK; both results are
54 ; ignored on purpose.
55 NCLOSE: lda #NETDEV
56 sta #NDEV
57 lda #NCLOSE
58 sta #NCMD
59 lda #0
60 jsr FNPR
61 jsr FNBEQ
62 jsr NPGO
63 lda #0
64 rts
65 ;
66 ; NOPEN -- OPEN the devicespec as an HTTP GET with
67 ; no translation.
68 ; The devicespec is streamed by BLDURL straight into
69 ; the TX page; nothing is
70 ; assembled in RAM, because there is no RAM to
71 ; assemble it in.
72 NOPEN: lda #NETDEV
73 sta #NDEV
74 lda #NCOPEN
75 sta #NCMD
76 lda #2
77 sta #NPNR
78 jsr FNBEQ
79 jsr #NHHITPG
80 jsr FNBP
81 lda #NTRNONE
82 jsr FNBP
83 jsr BLDURL
84 jsr NPGO
85 cmp #FNEOK
86 bne NOBAD
87 jsr FNACK
88 cmp #FNEOK
89 bne NOBAD
90 lda #0
91 rts
92 NOBAD: sta BSERR
93 lda #1
94 sta BSSTEP
95 lda #SFF
96 rts
97 ;
98 ; NSTATUS -- one STATUS poll. On success BSAVL0/1 is
99 ; the byte count waiting.
100 NSTATUS: lda #NETDEV
101 sta #NDEV
102 lda #NCSTAT
103 sta #NCMD
104 lda #2
105 sta #NPNR
106 jsr FNBEQ
107 jsr FNBP
108 lda #0
109 jsr FNBP
110 lda #0
111 jsr FNBP
112 cmp #NPGO
113 cmp #FNEOK
114 bne NSBAD
115 jsr FNACK
116 cmp #FNEOK
117 bne NSBAD
118
119 lda FNRLPY+NSDEVST
120 cmp #1
121 ; anything else is an HTTP error,
122 ; NSBAD: bne NSBAD2
123 ; would read as a short reply
124 lda FNRLPY+NSAVLO
125 sta BSAVL0
126 lda FNRLPY+NSAVHI
127 sta BSAVL1
128 lda #FNEOK
129 rts
130 NSBAD2: lda FNRLPY+NSDEVST
131 ; server actually said
132 NSBAD: sta BSERR
133 lda #2
134 sta BSSTEP
135 lda #SFF
136 rts
137 ;
138 ; NRCAP -- A (low) / X (high) = min(BSAVL, the read
139 ; cap for this request).
140 ; The cap is not caution. A four-player Game is 509
141 ; bytes and the reply window
142 ; is 512, so a reply that fits is a reply that can
143 ; be read IN PLACE with no
144 ; slice paging at all -- which is the whole reason
145 ; the window is 512 and not
146 ; 256.
147 NRCAP: lda BSREQ
148 cmp #RQTABLE
149 bne NRC1
150 lda #1
151 ldx #1
152 jmp NRC2
153 NRC1: lda #1
154 ldx #1
155 NRC2: ldx #1
156 bcc NRC3
157 ; byte is lower: the cap wins
158 ; is available wins
159 cmp BSAVL0
160 bcc NRC3
161 NRC4: ldx BSAVL0
162 ldx BSAVL1
163 rts
164 ;
165 ; NREAD -- READ min(available, cap) bytes, then
166 ; capture the reply length.
167 NREAD: lda #NETDEV
168 sta #NDEV
169 lda #NREAD
170 sta #NCMD
171 lda #1
172 sta #NPNR
173 jsr FNBEQ
174 jsr NRCAP
175 jsr FNWP
176 ; Two bytes, not two of one
177 jsr NPGO
178 cmp #FNEOK
179 bne NRBAD
180 jsr FNACK
181 cmp #FNEOK
182 bne NRBAD
183 lda FNRLX0
184 ; next transaction's reply
185 sta BSXL0
186 ; replace it, and it is always 0
187 lda FNRLX1
188 sta BSXL1
189 lda #FNEOK
190 rts
191 NRBAD: sta BSERR
192 lda #3
193 sta BSSTEP
194 lda #SFF
195 rts
196 ;
197 ; NSMIN -- BSMin = the byte count this request will
198 ; not settle below.
199 ;
200 ; Every game request returns a state blob whose
201 ; fixed header is 38 bytes, so
202 ; 38 is the point at which every offset VALIDB reads
203 ; is real. A /tables
204 ; listing has no such floor -- an empty server is
205 ; one byte -- and neither has
206 ; /leave.
207 NSMIN: ldx #0
208 lda BSREQ
209 cmp #RQTABLE
210 beq NSM1
211 cmp #RQLEAVE
212 beq NSM1
213 lda #GMINLEN
214 jmp NSM2
215 NSM1: ldx #1
216 NSM2: sta BSMin0
217 sta BSMin1
218 rts
219 ;
220 ; NFRAME -- one frame of waiting, and the whole
221 ; reason this bank has a display
222 ; kernel in it.
223 ;
224 ; For a game request that is a REAL frame: the
225 ; playfield tables and the text
226 ; rows still hold what the game bank composed out of
227 ; the last reply, and the
228 ; reply window is not repainted until the READ, so
229 ; redrawing them is both

```

```

183 ; correct and free. For the lobby's one-off RQTABLE
184 and RQLEAVE it is the
185 ; old blind delay, because this bank carries the
186 BOARD kernel and only that
187 ; -- and those two happen on a screen change, where
188 nobody can see a blank.
189 ;
190 ; It clobbers X and Y on both paths, which is why
191 the settle loop counts its
192 ; repeats in BSOLV. This port's first caller counted
193 in Y as well and every
194 ; wait ran 256 times over -- seventy seconds where
195 seventeen milliseconds were
196 ; meant, which looks exactly like a server that
197 never answers.
198 NFRAME: lda BSREQ
199 cmp #RQTABLE
200 beq NFBLND
201 cmp #RQLEAVE
202 beq NFBLND
203 ldx DFRAME
204 jmp BSWAIT
205 NFBLND: jmp
206 ;
207 -----
208 ; NPGO -- FNGO, paced by the display instead of by a
209 spin loop.
210 ;
211 ; The fixed tail's FNGO waits on ACKSEQ in a three-
212 deep counted loop with the
213 ; screen dead. It cannot be changed there -- the
214 tail is full and it is shared
215 ; with bsname, which has no picture to keep during
216 an apkey call -- so this
217 ; is its twin, assembled into the one bank that
218 polls.
219 ;
220 ; The launch is identical and has to be: the next
221 sequence number comes from
222 ; the CART'S OWN persisted ACKSEQ + 1, never from a
223 counter in RAM, because a
224 ; console RESET restarts this client without
225 resetting the cartridge and a
226 ; program-local counter would collide with a
227 sequence already answered.
228 ;
229 ; The timeout is the same -9s in the same FNTMO
230 quanta, so a real stall is
231 ; still reported as the cartridge's error rather
232 than ours: sixteen quanta of
233 ; thirty-four frames is 544 frames, and FNTMO itself
234 is the inner counter --
235 ; FNBEG's value is read the moment it has been
236 copied into FNCONT, and there is
237 ; no spare byte of zero page in this program to use
238 instead.
239 NPQUANT EQU 34 ; frames per FNTMO
240 quantum, ~0.57s
241 ;
242 NPGO: lda FNACKS
243 clc
244 adc #1
245 bne NPGO1
246 #1 ; 0 is reserved as
247 "never used"
248 NPGO1: sta FNSEQ
249 ldx #FR_SEQ
250 jsr FNFW ; launching it is
251 this single commit
252 lda FNTMO
253 sta FNCONT
254 NPGO2: lda #NPQUANT
255 sta FNTMO
256 ;
257 ; ONE FRAME BEFORE THE FIRST LOOK. In emulation the
258 cartridge answers inside
259 ; the commit itself, so without this every step of a
260 transaction -- and the
261 ; setup of the next, a URL streamed a character at a
262 time -- piled into the
263 ; overscan of a single frame and ran it fifty lines
264 long. A frame per step
265 ; is what real hardware costs anyway, and a poll is
266 ninety frames apart.
267 jsr NFRAME
268 ; ACKSEQ is published LAST, after the reply, its
269 length and the error code are
270 ; all in place, so a match means everything behind
271 it is already readable.
272 NPGO3: lda FNACKS
273 cmp FNSEQ
274 beq NPGO5
275 jsr NFRAME
276 dec FNTMO
277 bne NPGO3
278 dec FNCONT
279 bne NPGO2
280 lda #FNWAIT
281 rts
282 NPGO5: lda FNERR
283 rts
284 ;
285 -----
286 ; APICALL -- A = the request (RQ*). One full round
287 trip; on success (A = 0)
288 ; the reply is in the window and stays valid until
289 the next APICALL.
290 APICALL: sta BSREQ
291 jsr NSMIN
292 jsr NCLOSE ; the PREVIOUS
293 request's connection
294 jsr NOPEN
295 beq APOK
296 rts ; the open failed,
297 and it already closed
298 APOK: lda #0
299 sta BSPRV0
300 sta BSPRV1
301 lda #20 ; the settle loop's
302 bound
303 ;
304 ;
305 ;
306 ;
307 ;
308 ;
309 ;
310 ;
311 ;
312 ;
313 ;
314 ;
315 ;
316 ;
317 ;
318 ;
319 ;
320 ;
321 ;
322 ;
323 ;
324 ;
325 ;
326 ;
327 ;
328 ;
329 ;
330 ;
331 ;
332 ;
333 ;
334 ;
335 ;
336 ;
337 ;
338 ;
339 ;
340 ;
341 ;
342 ;
343 ;
344 ;
345 ;
346 ;
347 ;
348 ;
349 ;
350 ;
351 ;
352 ;
353 ;
354 ;
355 ;
356 ;
357 ;
358 ;
359 ;
360 ;
361 ;
362 ;
363 ;
364 ;
365 ;
366 ;
367 ;
368 ;
369 ;
370 ;
371 ;
372 ;
373 ;
374 ;
375 ;
376 ;
377 ;
378 ;
379 ;
380 ;
381 ;
382 ;
383 ;
384 ;
385 ;
386 ;
387 ;
388 ;
389 ;
390 ;
391 ;
392 ;
393 ;
394 ;
395 ;
396 ;
397 ;
398 ;
399 ;
400 ;
401 ;
402 ;
403 ;
404 ;
405 ;
406 ;
407 ;
408 ;
409 ;
410 ;
411 ;
412 ;
413 ;
414 ;
415 ;
416 ;
417 ;
418 ;
419 ;
420 ;
421 ;
422 ;
423 ;
424 ;
425 ;
426 ;
427 ;
428 ;
429 ;
430 ;
431 ;
432 ;
433 ;
434 ;
435 ;
436 ;
437 ;
438 ;
439 ;
440 ;
441 ;
442 ;
443 ;
444 ;
445 ;
446 ;
447 ;
448 ;
449 ;
450 ;
451 ;
452 ;
453 ;
454 ;
455 ;
456 ;
457 ;
458 ;
459 ;
460 ;
461 ;
462 ;
463 ;
464 ;
465 ;
466 ;
467 ;
468 ;
469 ;
470 ;
471 ;
472 ;
473 ;
474 ;
475 ;
476 ;
477 ;
478 ;
479 ;
480 ;
481 ;
482 ;
483 ;
484 ;
485 ;
486 ;
487 ;
488 ;
489 ;
490 ;
491 ;
492 ;
493 ;
494 ;
495 ;
496 ;
497 ;
498 ;
499 ;
500 ;
501 ;
502 ;
503 ;
504 ;
505 ;
506 ;
507 ;
508 ;
509 ;
510 ;
511 ;
512 ;
513 ;
514 ;
515 ;
516 ;
517 ;
518 ;
519 ;
520 ;
521 ;
522 ;
523 ;
524 ;
525 ;
526 ;
527 ;
528 ;
529 ;
530 ;
531 ;
532 ;
533 ;
534 ;
535 ;
536 ;
537 ;
538 ;
539 ;
540 ;
541 ;
542 ;
543 ;
544 ;
545 ;
546 ;
547 ;
548 ;
549 ;
550 ;
551 ;
552 ;
553 ;
554 ;
555 ;
556 ;
557 ;
558 ;
559 ;
560 ;
561 ;
562 ;
563 ;
564 ;
565 ;
566 ;
567 ;
568 ;
569 ;
570 ;
571 ;
572 ;
573 ;
574 ;
575 ;
576 ;
577 ;
578 ;
579 ;
580 ;
581 ;
582 ;
583 ;
584 ;
585 ;
586 ;
587 ;
588 ;
589 ;
590 ;
591 ;
592 ;
593 ;
594 ;
595 ;
596 ;
597 ;
598 ;
599 ;
600 ;
601 ;
602 ;
603 ;
604 ;
605 ;
606 ;
607 ;
608 ;
609 ;
610 ;
611 ;
612 ;
613 ;
614 ;
615 ;
616 ;
617 ;
618 ;
619 ;
620 ;
621 ;
622 ;
623 ;
624 ;
625 ;
626 ;
627 ;
628 ;
629 ;
630 ;
631 ;
632 ;
633 ;
634 ;
635 ;
636 ;
637 ;
638 ;
639 ;
640 ;
641 ;
642 ;
643 ;
644 ;
645 ;
646 ;
647 ;
648 ;
649 ;
650 ;
651 ;
652 ;
653 ;
654 ;
655 ;
656 ;
657 ;
658 ;
659 ;
660 ;
661 ;
662 ;
663 ;
664 ;
665 ;
666 ;
667 ;
668 ;
669 ;
670 ;
671 ;
672 ;
673 ;
674 ;
675 ;
676 ;
677 ;
678 ;
679 ;
680 ;
681 ;
682 ;
683 ;
684 ;
685 ;
686 ;
687 ;
688 ;
689 ;
690 ;
691 ;
692 ;
693 ;
694 ;
695 ;
696 ;
697 ;
698 ;
699 ;
700 ;
701 ;
702 ;
703 ;
704 ;
705 ;
706 ;
707 ;
708 ;
709 ;
710 ;
711 ;
712 ;
713 ;
714 ;
715 ;
716 ;
717 ;
718 ;
719 ;
720 ;
721 ;
722 ;
723 ;
724 ;
725 ;
726 ;
727 ;
728 ;
729 ;
730 ;
731 ;
732 ;
733 ;
734 ;
735 ;
736 ;
737 ;
738 ;
739 ;
740 ;
741 ;
742 ;
743 ;
744 ;
745 ;
746 ;
747 ;
748 ;
749 ;
750 ;
751 ;
752 ;
753 ;
754 ;
755 ;
756 ;
757 ;
758 ;
759 ;
760 ;
761 ;
762 ;
763 ;
764 ;
765 ;
766 ;
767 ;
768 ;
769 ;
770 ;
771 ;
772 ;
773 ;
774 ;
775 ;
776 ;
777 ;
778 ;
779 ;
780 ;
781 ;
782 ;
783 ;
784 ;
785 ;
786 ;
787 ;
788 ;
789 ;
790 ;
791 ;
792 ;
793 ;
794 ;
795 ;
796 ;
797 ;
798 ;
799 ;
800 ;
801 ;
802 ;
803 ;
804 ;
805 ;
806 ;
807 ;
808 ;
809 ;
810 ;
811 ;
812 ;
813 ;
814 ;
815 ;
816 ;
817 ;
818 ;
819 ;
820 ;
821 ;
822 ;
823 ;
824 ;
825 ;
826 ;
827 ;
828 ;
829 ;
830 ;
831 ;
832 ;
833 ;
834 ;
835 ;
836 ;
837 ;
838 ;
839 ;
840 ;
841 ;
842 ;
843 ;
844 ;
845 ;
846 ;
847 ;
848 ;
849 ;
850 ;
851 ;
852 ;
853 ;
854 ;
855 ;
856 ;
857 ;
858 ;
859 ;
860 ;
861 ;
862 ;
863 ;
864 ;
865 ;
866 ;
867 ;
868 ;
869 ;
870 ;
871 ;
872 ;
873 ;
874 ;
875 ;
876 ;
877 ;
878 ;
879 ;
880 ;
881 ;
882 ;
883 ;
884 ;
885 ;
886 ;
887 ;
888 ;
889 ;
890 ;
891 ;
892 ;
893 ;
894 ;
895 ;
896 ;
897 ;
898 ;
899 ;
900 ;
901 ;
902 ;
903 ;
904 ;
905 ;
906 ;
907 ;
908 ;
909 ;
910 ;
911 ;
912 ;
913 ;
914 ;
915 ;
916 ;
917 ;
918 ;
919 ;
920 ;
921 ;
922 ;
923 ;
924 ;
925 ;
926 ;
927 ;
928 ;
929 ;
930 ;
931 ;
932 ;
933 ;
934 ;
935 ;
936 ;
937 ;
938 ;
939 ;
940 ;
941 ;
942 ;
943 ;
944 ;
945 ;
946 ;
947 ;
948 ;
949 ;
950 ;
951 ;
952 ;
953 ;
954 ;
955 ;
956 ;
957 ;
958 ;
959 ;
960 ;
961 ;
962 ;
963 ;
964 ;
965 ;
966 ;
967 ;
968 ;
969 ;
970 ;
971 ;
972 ;
973 ;
974 ;
975 ;
976 ;
977 ;
978 ;
979 ;
980 ;
981 ;
982 ;
983 ;
984 ;
985 ;
986 ;
987 ;
988 ;
989 ;
990 ;
991 ;
992 ;
993 ;
994 ;
995 ;
996 ;
997 ;
998 ;
999 ;

```

SRC/URL.INC

```

1 ; url.inc -- the devicespec, streamed straight into
2 ; the TX page a character at
3 ; a time.
4 ; Nothing is assembled in RAM. The cartridge counts
5 ; the payload itself, so
6 ; nothing here tracks a length -- which is the only
7 ; reason a console with 128
8 ; bytes can build
9 ; N:https://battleship.carr-designs.com/attack/47?
10 ; table=ai&player=VCS&bin=1&v=2
11 ; at all. The table id and the player name are not
12 ; in console RAM either: they
13 ; live in the cartridge's path buffers and FP_TXRAW
14 ; streams them from there
15 ; into the request without their ever crossing the
16 ; bus. The five placements
17 ; and the shot ARE in RAM -- seven bytes -- and go
18 ; out as decimal through UDEC,
19 ; which is what saves a twenty-character move buffer
20 ; this console has not got.
21 ;
22 ; A fixed-width wire field is pushed up to its NUL
23 ; and NEVER through it. The
24 ; server sends name[9] space-padded, and padding in
25 ; the middle of a URL
26 ; corrupts the request -- the Arcadia port calls
27 ; this out by name.
28 ;
29 ; All six requests are built here, in the one bank
30 ; that issues any of them.
31 ; UTXS -- append the NUL-terminated string at
32 ; FNPTL/FNPTRH to the TX stream.
33 UTXS: ldy #0
34 UTXS1: lda (FNPTL),y
35 beq UTXS2
36 sta FNTX
37 iny
38 bne UTXS1
39 UTXS2: rts
40 ; UPUT -- point at the string in X (lo) / Y (hi) and
41 ; append it.
42 UPUT: jsr FNSETP
43 jmp UTXS
44 ; UDEC -- append A (0-255) as decimal with no
45 ; leading zeros. Clobbers X.
46 UDEC: ldx #0 ; hundreds
47 UDC1: cmp #100
48 bcc UDC2
49 sbc #100
50 inx
51 bne UDC1
52 UDC2: pha
53 beq UDC3 ; no hundreds: no
54 clc
55 adc #10'
56 sta FNTX
57 pla
58 UDC4: cmp #10 ; tens, always
59 printed after a hundreds digit
60 bcc UDC5
61 sbc #10
62 inx
63 bne UDC4
64 UDC5: pha
65 txa
66 clc
67 adc #10'
68 sta FNTX
69 pla
70 jmp UDC8
71 UDC3: pha
72 ldx #0
73 cmp #10
74 bcc UDC7
75 sbc #10
76 inx
77 bne UDC6
78 UDC7: pha
79 txa
80 beq UDC9 ; a zero tens digit
81 prints only after hundreds
82 clc
83 adc #10'
84 sta FNTX
85 UDC9: pha
86 UDC8: clc ; the units digit
87 always prints
88 txa
89 sta FNTX
90 rts
91 ;
92 ; BLDURL -- stream the devicespec for the request in
93 ; BSREQ.
94 BLDURL: lda #'N'
95 sta FNTX
96 lda #':'
97 sta FNTX
98 ldx #(UENDPT)&$FF
99 ldy #(UENDPT)>>8
100 jsr UPUT
101 lda BSREQ
102 cmp #RQTABLE
103 bne
104 ; /tables carries its own query and takes neither a
105 ; table nor a player.
106 ldx # (UTABLS)&$FF
107 ldy # (UTABLS)>>8
108 jmp UPUT
109 cmp #RQLEAVE
110 bne BU2
111 ldx # (ULEAVE)&$FF
112 ldy # (ULEAVE)>>8
113 jsr UPUT
114 BUQ: cmp #RQREADY
115 bne BU3
116 ldx # (UREADY)&$FF ; a bare /ready
117 ldy # (UREADY)>>8 ; Intellivision
118 and the Astrocade
119 jsr UPUT
120 BUQ: cmp #RQPLACE
121 bne BU4
122 ; /place/P,P,P,P -- decimal, no leading zeros, no
123 ; trailing comma.
124 ldx # (UPLACE)&$FF
125 ldy # (UPLACE)>>8
126 jsr UPUT
127 ldx #0
128 BU3A: stx BSIDX
129 lda SHEPS,x
130 jsr UDEC
131 ldx BSIDX
132 inx
133 cpx #NSHIPS
134 beq BUQ
135 lda #1
136 FNTX: sta FNTX
137 jmp BU3A
138 BU4: cmp #RQATTCK
139 bne BU5
140 ; /attack/P, P = y*10 + x. Its reply IS the next
141 ; state.
142 ldx # (UATTCK)&$FF
143 ldy # (UATTCK)>>8
144 jsr UPUT
145 CELLNO: jsr UDEC
146 UDEC: jsr UPUT
147 BU5: ldy # (USTATE)&$FF
148 ldx # (USTATE)>>8
149 jsr UPUT
150 ; The query. Both halves come out of the cartridge.
151 BUQ: ldx # (UQTAB)&$FF
152 ldy # (UQTAB)>>8
153 jsr UPUT
154 ldx #PBTABLE
155 jsr FNWSEL
156 jsr FNWRW
157 ldx # (UQPLR)&$FF
158 ldy # (UQPLR)>>8
159 jsr UPUT
160 #PNAME: jsr FNWSEL
161 jsr FNWRW
162 ldx # (UQBIN)&$FF
163 ldy # (UQBIN)>>8
164 jsr UPUT
165 INCLUDE "../build/endpoint.inc"
166 UTABLES: DB "tables?bin=1",0
167 USTATE: DB "state",0
168 UREADY: DB "ready",0
169 UPLACE: DB "place/",0
170 UATTACK: DB "attack/",0
171 ULEAVE: DB "leave",0
172 UQTAB: DB "table=",0
173 UQPLR: DB "&player=",0
174 ; v=2: the winner and the ship reveal at game over.
175 ; No &bel: the Go server's
176 ; big-endian path is the road much less travelled.
177 UQBIN: DB "&bin=1&v=2",0
178

```

SRC/STATE.INC

```

1 ; state.inc -- what the reply window holds, and how
2 ; to find things in it.
3 ; Nothing here copies. The reply stays in the
4 ; cartridge's window and the
5 ; routines below turn a player index into an offset
6 ; or a pointer into it.
7
8 ; IF BSHASNET
9
10 ; VALID8 -- sanity-check the reply length against
11 ; what the header implies,
12 ; transcribed from arcadia/net.inc's (itself the
13 ; Intellivision's
14 ; validate_state), and note the four bytes every
15 ; bank keys on.
16 ; Minimum, not exact: a longer reply is harmless. A
17 ; SHORTER one means the
18 ; fixed offsets would read state bytes out of a
19 ; window that is never cleared,
20 ; which is the failure this exists to stop.
21 ;
22 ; tables any nonzero length
23 ; lobby 59 * 10 * playerCount
24 ; place 49 (records are name+status
25 ; only)
26 ;
27 ; in play 49 + 115 * playerCount
28
29 VALID8: lda BSRXL0
30 cmp #ROTABLE
31 bne VALG0
32 lda BSRXL0 ; tables: any
33 nonzero length will do
34 ora BSRXL1
35 bne VALT0
36 jmp VALBAD ; the count byte;
37 VALT0: lda FNRPV
38 the lobby shows MAXTBL
39 cmp #MAXTBL+1
40 bcc VALT1
41 lda #MAXTBL
42 VALT1: sta BSCNT
43 lda #FNEOK
44 rts
45 VALG0: cmp #RQLEAVE
46 bne VALG0A
47 lda #FNEOK ; /leave: we are
48 going whatever it said
49 VALG0A: lda BSRXL1 ; a header at
50 minimum, always
51 lda VALG1
52 cmp #GHDRLEN
53 bcc VALBAD
54 VALG1: lda FNRPV+GOPCNT
55 cmp #MAXPLYR+1
56 bcs VALBAD
57 sta BSCNT
58 lda FNRPV+GOSTAT
59 sta BSTAT
60 bne VALG2
61 ; lobby: 59 * 10 * playerCount
62 lda #0
63 ldx BSCNT
64 beq VALG1B
65 VALG1A: clc
66 adc #LBSTRID
67 dex
68 bne VALG1A
69 VALG1B: clc
70 adc #GLPLYRS
71 jmp VALMIN ; always < 256
72
73 VALG2: cmp #STPLACE
74 bne VALG3
75 ldx #GLPLYRS ; place: the fixed
76 header only
77 jmp VALMIN
78
79 VALG3: ; in play: 49 + 115 * playerCount, which
80 ; passes 256 at two players
81 lda #0
82 sta BSTM2 ; the high byte
83 accumulates here
84 ldx #GLPLYRS
85 bcc BSCNT
86 VALG3A: clc
87 adc #PLSTRID
88 bcc VALG3B
89 inc BSTM2
90 VALG3B: dex
91 bne VALG3A
92 ldx BSTM2
93 beq VALMIN
94 ; 16-bit compare: high byte first
95 cpx BSRXL1
96 beq VALLO
97 bcc VALOK
98 bcs VALBAD
99 VALLO: cmp BSRXL0
100 beq VALOK
101 bcc VALOK
102 bcs VALBAD
103 VALOK: lda FNRPV+GOPLST
104
105 sta BSMYST
106 lda FNRPV+GOACTPL
107 sta BSACT
108 lda #FNEOK
109 rts
110 VALBAD: lda #FNEBADL
111 sta BSERR
112 lda #4
113 sta BSSTEP
114 lda #FFF
115 rts
116 ENDF
117
118 ;
119 ;
120 ; CELLNO -- A = the cursor's cell, y+10 * x.
121 ; Clobbers BSTM2.
122 CELLNO: lda BSCURY
123 a
124 sta BSTM2
125 a
126 a
127 a
128 a
129 a
130 a
131 a
132 a
133 a
134 a
135 a
136 a
137 a
138 a
139 a
140 a
141 a
142 a
143 a
144 a
145 a
146 a
147 a
148 a
149 a
150 a
151 a
152 a
153 a
154 a
155 a
156 a
157 a
158 a
159 a
160 a
161 a
162 a
163 a
164 a
165 a
166 a
167 a
168 a
169 a
170 a
171 a
172 a
173 a
174 a
175 a
176 a
177 a
178 a
179 a
180 a
181 a
182 a
183 a
184 a
185 a
186 a
187 a
188 a
189 a
190 a
191 a
192 a
193 a
194 a
195 a
196 a
197 a
198 a
199 a
200 a
201 a
202 a
203 a
204 a
205 a
206 a
207 a
208 a
209 a
210 a
211 a
212 a
213 a
214 a
215 a
216 a
217 a
218 a
219 a
220 a
221 a
222 a
223 a
224 a
225 a
226 a
227 a
228 a
229 a
230 a
231 a
232 a
233 a
234 a
235 a
236 a
237 a
238 a
239 a
240 a
241 a
242 a
243 a
244 a
245 a
246 a
247 a
248 a
249 a
250 a
251 a
252 a
253 a
254 a
255 a
256 a
257 a
258 a
259 a
260 a
261 a
262 a
263 a
264 a
265 a
266 a
267 a
268 a
269 a
270 a
271 a
272 a
273 a
274 a
275 a
276 a
277 a
278 a
279 a
280 a
281 a
282 a
283 a
284 a
285 a
286 a
287 a
288 a
289 a
290 a
291 a
292 a
293 a
294 a
295 a
296 a
297 a
298 a
299 a
300 a
301 a
302 a
303 a
304 a
305 a
306 a
307 a
308 a
309 a
310 a
311 a
312 a
313 a
314 a
315 a
316 a
317 a
318 a
319 a
320 a
321 a
322 a
323 a
324 a
325 a
326 a
327 a
328 a
329 a
330 a
331 a
332 a
333 a
334 a
335 a
336 a
337 a
338 a
339 a
340 a
341 a
342 a
343 a
344 a
345 a
346 a
347 a
348 a
349 a
350 a
351 a
352 a
353 a
354 a
355 a
356 a
357 a
358 a
359 a
360 a
361 a
362 a
363 a
364 a
365 a
366 a
367 a
368 a
369 a
370 a
371 a
372 a
373 a
374 a
375 a
376 a
377 a
378 a
379 a
380 a
381 a
382 a
383 a
384 a
385 a
386 a
387 a
388 a
389 a
390 a
391 a
392 a
393 a
394 a
395 a
396 a
397 a
398 a
399 a
400 a
401 a
402 a
403 a
404 a
405 a
406 a
407 a
408 a
409 a
410 a
411 a
412 a
413 a
414 a
415 a
416 a
417 a
418 a
419 a
420 a
421 a
422 a
423 a
424 a
425 a
426 a
427 a
428 a
429 a
430 a
431 a
432 a
433 a
434 a
435 a
436 a
437 a
438 a
439 a
440 a
441 a
442 a
443 a
444 a
445 a
446 a
447 a
448 a
449 a
450 a
451 a
452 a
453 a
454 a
455 a
456 a
457 a
458 a
459 a
460 a
461 a
462 a
463 a
464 a
465 a
466 a
467 a
468 a
469 a
470 a
471 a
472 a
473 a
474 a
475 a
476 a
477 a
478 a
479 a
480 a
481 a
482 a
483 a
484 a
485 a
486 a
487 a
488 a
489 a
490 a
491 a
492 a
493 a
494 a
495 a
496 a
497 a
498 a
499 a
500 a
501 a
502 a
503 a
504 a
505 a
506 a
507 a
508 a
509 a
510 a
511 a
512 a
513 a
514 a
515 a
516 a
517 a
518 a
519 a
520 a
521 a
522 a
523 a
524 a
525 a
526 a
527 a
528 a
529 a
530 a
531 a
532 a
533 a
534 a
535 a
536 a
537 a
538 a
539 a
540 a
541 a
542 a
543 a
544 a
545 a
546 a
547 a
548 a
549 a
550 a
551 a
552 a
553 a
554 a
555 a
556 a
557 a
558 a
559 a
560 a
561 a
562 a
563 a
564 a
565 a
566 a
567 a
568 a
569 a
570 a
571 a
572 a
573 a
574 a
575 a
576 a
577 a
578 a
579 a
580 a
581 a
582 a
583 a
584 a
585 a
586 a
587 a
588 a
589 a
590 a
591 a
592 a
593 a
594 a
595 a
596 a
597 a
598 a
599 a
600 a
601 a
602 a
603 a
604 a
605 a
606 a
607 a
608 a
609 a
610 a
611 a
612 a
613 a
614 a
615 a
616 a
617 a
618 a
619 a
620 a
621 a
622 a
623 a
624 a
625 a
626 a
627 a
628 a
629 a
630 a
631 a
632 a
633 a
634 a
635 a
636 a
637 a
638 a
639 a
640 a
641 a
642 a
643 a
644 a
645 a
646 a
647 a
648 a
649 a
650 a
651 a
652 a
653 a
654 a
655 a
656 a
657 a
658 a
659 a
660 a
661 a
662 a
663 a
664 a
665 a
666 a
667 a
668 a
669 a
670 a
671 a
672 a
673 a
674 a
675 a
676 a
677 a
678 a
679 a
680 a
681 a
682 a
683 a
684 a
685 a
686 a
687 a
688 a
689 a
690 a
691 a
692 a
693 a
694 a
695 a
696 a
697 a
698 a
699 a
700 a
701 a
702 a
703 a
704 a
705 a
706 a
707 a
708 a
709 a
710 a
711 a
712 a
713 a
714 a
715 a
716 a
717 a
718 a
719 a
720 a
721 a
722 a
723 a
724 a
725 a
726 a
727 a
728 a
729 a
730 a
731 a
732 a
733 a
734 a
735 a
736 a
737 a
738 a
739 a
740 a
741 a
742 a
743 a
744 a
745 a
746 a
747 a
748 a
749 a
750 a
751 a
752 a
753 a
754 a
755 a
756 a
757 a
758 a
759 a
760 a
761 a
762 a
763 a
764 a
765 a
766 a
767 a
768 a
769 a
770 a
771 a
772 a
773 a
774 a
775 a
776 a
777 a
778 a
779 a
780 a
781 a
782 a
783 a
784 a
785 a
786 a
787 a
788 a
789 a
790 a
791 a
792 a
793 a
794 a
795 a
796 a
797 a
798 a
799 a
800 a
801 a
802 a
803 a
804 a
805 a
806 a
807 a
808 a
809 a
810 a
811 a
812 a
813 a
814 a
815 a
816 a
817 a
818 a
819 a
820 a
821 a
822 a
823 a
824 a
825 a
826 a
827 a
828 a
829 a
830 a
831 a
832 a
833 a
834 a
835 a
836 a
837 a
838 a
839 a
840 a
841 a
842 a
843 a
844 a
845 a
846 a
847 a
848 a
849 a
850 a
851 a
852 a
853 a
854 a
855 a
856 a
857 a
858 a
859 a
860 a
861 a
862 a
863 a
864 a
865 a
866 a
867 a
868 a
869 a
870 a
871 a
872 a
873 a
874 a
875 a
876 a
877 a
878 a
879 a
880 a
881 a
882 a
883 a
884 a
885 a
886 a
887 a
888 a
889 a
890 a
891 a
892 a
893 a
894 a
895 a
896 a
897 a
898 a
899 a
900 a
901 a
902 a
903 a
904 a
905 a
906 a
907 a
908 a
909 a
910 a
911 a
912 a
913 a
914 a
915 a
916 a
917 a
918 a
919 a
920 a
921 a
922 a
923 a
924 a
925 a
926 a
927 a
928 a
929 a
930 a
931 a
932 a
933 a
934 a
935 a
936 a
937 a
938 a
939 a
940 a
941 a
942 a
943 a
944 a
945 a
946 a
947 a
948 a
949 a
950 a
951 a
952 a
953 a
954 a
955 a
956 a
957 a
958 a
959 a
960 a
961 a
962 a
963 a
964 a
965 a
966 a
967 a
968 a
969 a
970 a
971 a
972 a
973 a
974 a
975 a
976 a
977 a
978 a
979 a
980 a
981 a
982 a
983 a
984 a
985 a
986 a
987 a
988 a
989 a
990 a
991 a
992 a
993 a
994 a
995 a
996 a
997 a
998 a
999 a
1000 a

```

```

196 FNRP1: lda    (FNTRL),y
197      beq      FNRP2
198      sta      FNRSel+FH_TCHR
199      iny
200      dex
201      bne      FNRP1
202      rts
203 FNRP2: lda      #1
204      sta      FNRSel+FH_TCHR
205      dex
206      bne      FNRP2
207      rts
208      ENDIF
209

```

SRC/DISPTEXT.INC

```

1 ; dispext.inc -- the 48-pixel text display, with
2 colour.
3
4 ; The cartridge composes the glyphs (firmware/src/
5 vcs_render.c) and publishes
6 ; them as six 128-byte planes at $1800-$1AFF; this
7 is the console half.
8
9 ; THE 53-CYCLE INK BODY BELOW IS TRANSCRIBED FROM
10 ; fujinet-firmware/pico/atari-2600/testrom/
11 fujidisp.inc AND MUST NOT BE
12 ; RETIRED. Its four constants are the coordinates of
13 a ONE-PIXEL-WIDE window
14 ; found by byte-comparing the raster against the
15 renderer with
16 ; emu/dispatch.py -- every wrong setting still
17 looks like text.
18
19 ; WHAT IS ADDED HERE: colour, for nothing.
20
21 ; vcs_render_row() writes ZERO to scanline 5 of
22 every cell -- the sixth line
23 ; of a 4x6 cell is blank leading and the renderer
24 does not store it. So the
25 ; loop is 21 rows of (5 cycle-exact ink lines + 1
26 SEAM LINE), the seam line is
27 ; a line on which no glyph can possibly be drawn,
28 and it is free for whatever
29 ; the kernel wants to do instead. That costs no
30 scanlines at all; the blank
31 ; line was already there, and the picture is still
32 21x6 = 126 lines.
33
34 ; "Free" means free of DEADLINES, not free of
35 cycles; the seam line still has
36 ; to finish inside its 76, or it quietly costs a
37 second scanline. The budget is
38 ; spelled out where it is spent, at the seam line
39 itself.
40
41 ; Three facts make the seam line's timing a non-
42 problem, which is the whole
43 ; trick:
44
45 ; * Its sprite writes are BLANKING writes. The
46 copies draw nothing whatever
47 ; cycle the stores land on.
48 ; * COLUBK has until the green playfield border
49 ends at pixel 31, about
50 ; cycle 32, because the playfield draws OVER the
51 background -- a
52 ; background change underneath the border cannot
53 be seen. That is a
54 ; 33-cycle window for a compare and a branch,
55 not the 22 of hblank.
56 ; * COLUPB/COLUP1 have NO deadline on this line.
57 The sprites are blank, so a
58 ; new ink colour cannot show until the next line
59 anyway.
60
61 ; What that buys: a sea-blue frame, a black panel, a
62 full-width red bar under
63 ; the selected row, and a per-row background -- with
64 the ink body left
65 ; byte-identical on every line that draws. The 5
66 Card Stud original also
67 ; picked a per-seat ink colour here; every text
68 screen in this client is
69 ; white ink, so that pick is gone and the seam line
70 is shorter for it.
71
72 ; ----- kernel timing constants
73
74 ; Not taste. See the file header.
75
76 RNOP EQU 16 ; NOPs before RESP0
77
78 R3 EQU 1 ; ...plus this, so
79 RESP0 lands on cycle 38
80
81 HMO EQU 15 ; HMP0 = $F0: move
82 P0 right one pixel...
83
84 HM1 EQU 0 ; ...and leave P1,
85 closing the gap to eight
86
87 VOEL EQU 1 ; vertical delay on
88 both players
89
90 ; ----- text plane bases -----
91
92 TP0 EQU FNTXT+$000
93 TP1 EQU FNTXT+$080
94 TP2 EQU FNTXT+$100
95 TP3 EQU FNTXT+$180
96 TP4 EQU FNTXT+$200
97 TP5 EQU FNTXT+$280
98
99 TINK EQU FNTCLH-1 ; 5 ink lines, then
100 the seam
101 TLINES EQU FNTROW+FNTCLH ; 126 scanlines of
102 text
103
104 NTSC: 3 vsync + 37 vblank + 192 visible + 30
105 overscan = 562.
106
107 VBLINES EQU 37
108
109 OSLINES EQU 30
110 PADLINE EQU (192-TLINES)/2 ; 33 above and below
111
112 ; THE BLANKED BANDS ARE TIMED, NOT COUNTED, and that
113 is not a refinement --
114 ; counting them is what made the picture jump.
115
116 ; The old loop burned VBLINES scanlines on WSYNC and
117 THEN called APPVBL, so
118 ; whatever the hook did was ADDED to the frame
119 instead of spent inside it. A
120 ; frame measured 265 scanlines rather than 262 just
121 from scanning the input,
122 ; wandered between 262, 264 and 265 depending on
123 what the hook touched, and
124 ; reached 413 on the frame a SELECT press recomposed
125 the seats. Every one of
126 ; those is a different vertical position on the
127 screen.
128
129 ; So the two blanked bands set the RIOT's 64-cycle
130 timer, do their work, and
131 ; then wait out whatever is left of it. The hook now
132 runs INSIDE the vblank
133 ; and costs the frame nothing at all, as long as it
134 fits.
135
136 ; THE BUDGET IT HAS TO FIT IN IS 37 SCANLINES --
137 2812 cycles, and no more.
138
139 ; DPADA and DPABD are not spare: they are the green
140 bands, and they are
141 ; visible. Work that overruns still lengthens the
142 frame, and the frame still
143 ; jumps; it is just that almost nothing overruns
144 now.
145
146 ; The two counts are in 64-cycle units and are one
147 less than the arithmetic
148 ; suggests, because the timer has to expire INSIDE
149 the last line rather than
150 ; on its boundary -- at 43 it lands 36 cycles into
151 scanline 36 of 37, and the
152 ; WSYNC that follows takes the band to exactly 37.
153 They were confirmed by
154 ; histogramming VSYNC-to-VSYNC with emu/blank.lua,
155 not derived.
156
157 VBIM EQU 43 ; ~37 scanlines of
158 vblank
159
160 OSTIM EQU 35 ; ~30 scanlines of
161 overscan
162
163 ; The side border, PFEDGE in bsdefc.inc: PF0 is
164 pixels 0-3 in its HIGH nibble
165 ; and PF1 pixels 4-11 drawn MSB first, so $F0 in
166 each lights pixels 0-7 --
167 ; clocks 0-31. With CTRLPF's reflect bit the right
168 half mirrors, putting the
169 ; same 32 clocks at 128-159. The 48-pixel text block
170 sits at pixels 46-94 and
171 ; never comes near either.
172
173 DINIT: TIA setup and player
174 positioning -----
175
176 DINIT:
177 ; Two players, three copies each, eight pixels
178 apart: P0 draws groups 0,2,4
179 ; and P1 groups 1,3,5, interleaving to 48 contiguous
180 pixels. VOELP is 37, and
181 ; makes the six-write sequence land in the right
182 copies.
183
184 lda #03 ; NUSIZ: three
185 copies, close
186
187 sta NUSIZ0
188 sta NUSIZ1
189 lda #VOEL
190 sta VOELP0
191 sta VOELP1
192 lda #CWHITE
193 sta COLUP0
194 sta COLUP1
195 lda #CWATER
196 sta COLUBK
197 sta COLUPB
198 lda #01 ; CTRLPF: reflect,
199 so one nibble draws both
200 ; edges
201
202 sta CTRLPF
203 lda #PFEDGE
204 sta PF0
205 sta PF1
206 lda #0
207 sta PF2
208 sta ENAM1 ; the board banks'
209
210 divider
211 sta BSBAR ; no red bar until a
212 menu asks for one
213
214 sta BSFRAME
215 sta GRP0
216 sta GRP1
217 sta GRP0
218

```

```

131 ; Player positioning, transcribed from batari
132 Basic's std_kernel.asm rather
133 ; than re-derived. RESP0 strobed at cycle 43, RESP1
134 three cycles later. (nine
135 ; pixels, since two stores cannot be closer), then
136 HMP0/HMP1 pull them to
137 ; eight apart. ROLY is built from NOPs plus an
138 optional 3-cycle zero-page
139 ; store, because the copies are 2.67 cycles apart
140 and NOPs alone only reach
141 ; even delays.
142 sta WSYNC
143 rept R3
144 nop
145 endm
146 sta PAD3 ; 3 cycles, so odd
147 delays are reachable
148 endif
149 sta RESP0
150 sta RESP1
151 lda #HMP0+16
152 hmp0
153 lda #HMP1+16
154 hmp1
155 ; An EARLY HMOVE -- WSYNC, then strobe -- rather
156 than batari's late one at cycle 43.
157 ; cycle 73. A late HMOVE does not apply the motion
158 nibbles cleanly: nudging
159 ; HMP0 by one there widened the player gap from 8
160 pixels to 9, ready for the
161 ; sliding the block, and the window this kernel
162 needs is one pixel wide.
163 sta WSYNC
164 sta HMOVE
165 rts
166
167 ; ----- DLOOP: draw frames, forever
168
169 ; DFRESH is the entry for a FRESH start -- a screen
170 change, not a frame
171 ; being finished or followed -- and arms the
172 overscan timer DFRAME waits
173 ; on, because a stale count could be seconds. See
174 dispgame.inc.
175 DFRESH: lda #OSTIM
176 sta TIM64T
177 DLOOP: jsr DFRAME
178 jmp DLOOP
179
180 ; ----- DFRAME: draw ONE frame, then
181 return
182 ; Split out of DLOOP so a bank that is not running
183 the display loop can still
184 ; keep the picture alive: net.inc waits for the
185 cartridge a frame at a time
186 ; through here, which is what stops a poll from
187 blanking the screen.
188 ; A JSR into this is safe BEFORE the kernel parking
189 a graphics byte in S. The
190 ; return address is pushed BEFORE that happens and
191 nothing pushes again until
192 ; SAVSP has been restored -- the same rule the
193 kernel already lives by, one
194 ; level further out.
195 ; THE OVERSCAN IS WAITED OUT HERE, at the start of
196 the next frame, and not
197 ; at the end of the one before. DFRAME2 arms the
198 timer and returns at once,
199 ; so whatever a bank does between two frames --
200 build a URL into the TX
201 ; page, recompute the boards, switch banks -- is
202 spent INSIDE the overscan
203 ; and moves nothing, as long as it fits in thirty
204 lines. Waiting at the end
205 ; instead put every one of those between the
206 overscan and the next VSYNC,
207 ; and a played game measured 96 frames in 2,400 that
208 were 273 to 310 lines.
209 DFRAME: bit TIMINT
210 bpl DFRAME
211 sta WSYNC
212 lda #2
213 sta VSYNC
214 sta WSYNC
215 sta WSYNC
216 lda #0
217 sta VSYNC
218 lda #2
219 sta VBLANK
220 sta #VBTIM
221 lda TIM64T
222 ; The client's per-frame hook, called while the
223 screen is still blanked and
224 ; the raster is nowhere near the text. Input
225 scanning belongs here; so does
226 ; STARTING a transaction, which is a bank switch out
227 of this loop -- the bank
228 ; that talks keeps the picture alive by calling
229 DFRAME below, one frame per
230 ; wait, rather than by returning here.
231
232 ; It runs ON the clock the store above started, so
233 the frame does not grow by
234 ; what it does. See VBIM for the 2812 cycles it has
235 to fit in.
236 inc BFRAME
237 jsr APPVBL
238
239 ; ----- DFRAME2: finish the frame the
240 hook was called in -----
241 ; A bank entered from another bank's hook carries on
242 from here, so the
243 ; switch interrupted is still 262 lines. See
244 dispgame.inc.
245 DFRAME2:

```

```

295 ; following line and waits for the end of THAT one.
296 The first cut of this
297 ; reached DROWL's 'lda #TINK' at cycle 77 on the
298 seat path and every seat row
299 ; was seven scanlines instead of six: 142 lines of
300 text, a 278-line frame, a
301 ; picture that rolls on anything that cares. Nothing
302 about it looked wrong on
303 ; screen, which is the whole problem with the
304 failure mode.
305 ;
306 ; The budget is therefore from this WSYNC to the
307 last cycle of the next one,
308 ; and it is 76. The path below reaches it at 58,
309 with 'lda BSBGT,x' costing a
310 ; fifth cycle in any bank whose twenty-two byte
311 table straddles a page and
312 ; 'bne DSEAM1' a third if its target ever lands on
313 another page.
314 sta WSYNC
315 lda #0 ; 2
316 sta GRP0 ; 3 -> 5 three
317 writes, not two: with
318 sta GRP1 ; 3 -> 8
319 VDELP set a write to GRP0
320 sta GRP0 ; 3 -> 11
321 reloads the DELAYED GRP1
322 ldx BSROW ; 3 -> 14
323 inx ; 2 -> 16 the
324 row these colours are FOR
325 lda BSBGT,x ; 4 -> 20
326 cpx BSBAR ; 3 -> 23
327 bne DSEAM1 ; 3 -> 26
328 lda #CREG ; 2 the
329 selected move's bar
330 DSEAM1: sta COLUBK ; 3 -> 29 by
331 cycle 30, pixel 22 at the
332 the green border ; latest, under
333 stx BSROW ; 3 -> 32 the
334 row counter, advanced here
335 a read-modify-write at ; rather than with
336 ; the bottom: X
337 still holds it, and 'inc'
338 ; costs five
339 cycles this line has not got
340 iny ; 2 -> 34 step
341 past the blank plane byte.
342 ; AFTER the COLUBK
343 store, not before: it has
344 ; to land before
345 the green border ends at
346 ; pixel 31 and two
347 cycles is two-thirds of
348 ; the margin that
349 had.
350 ; The ink colour: white, every row. (The original
351 chose a per-seat colour
352 ; here; this client's text screens have no seats.)
353 lda #WHITE ; 2 -> 36
354 sta COLUP0 ; 3 -> 39
355 sta COLUP1 ; 3 -> 42
356 lda BSROW ; 3 -> 45
357 cmp #FNTRW ; 2 -> 47
358 bne DROWL ; 3 -> 50 +
359 DROWL's 5 and the WSYNC's 3
360 ldx SAVSP
361 txs
362 ; The last seam line already blanked the sprites and
363 set the background back
364 ; to felt -- BSBGT's twenty-second entry exists for
365 exactly that -- so the
366 ; bottom band needs nothing but scanlines.
367 ldx #PADLINE
368 DPADB: sta WSYNC
369 dex
370 bne DPADB
371 lda #2
372 sta VBLANK
373 lda #OSTIM ; the next DFRAME
374 waits this out; see above
375 sta TIM64T
376 ; Returns with VBLANK SET and the overscan timer
377 running: a bank may do up to
378 ; about 2,200 cycles of work before the next DFRAME
379 for free.
380 rts
381 ;
382 -----
383 ; BSBGT is NOT here. Each bank supplies its own
384 twenty-two byte table, because
385 ; each bank is a different screen.
386 ;
387 ; Twenty-TWO entries, not twenty-one: the seam line
388 at the bottom of row 20
389 ; programmes row 21, which does not exist, and
390 making that entry felt is what
391 ; turns the handover to the bottom band into a
392 colour the kernel was going to
393 ; write anyway.
394 ;

```

SRC/DISPGAME.INC

```

1 ; dispgame.inc -- the four-quadrant board display.
2 ;
3 ; Up to four 10x10 boards as COLOURED CELLS, drawn
4 with the TIA playfield
5 ; rewritten twice a scanline out of tables the
6 cartridge composes (see
7 ; fujinet.inc, the playfield board). A cell is two
8 playfield bits -- eight

```

```

6 ; pixels -- by eight scanlines (twelve in the two-
7 player layout, which makes
8 ; it square), and the colour a cell shows is decided
9 BY LINE: the playfield
10 ; has one colour per scanline, so each line of the
11 cell draws a different
12 ; table in a different colour, and a hit, a miss and
13 a hull come out red,

```

```

10 ; white and yellow on one board with no flicker at
11 all.
12 ; line 0 AUX yellow your hulls; the cursor
13 ; line 1-2 HIT red a hit: the top of the
14 red block
15 ; line 3-4 MID white hit OR miss: a hit's
16 white core, a miss's dash
17 ; line 5-6 HIT red the bottom of the block
18 ; line 7 AUX yellow
19
20 ; Two boards side by side are the whole 160 pixels,
21 abutting at 79|80, with a
22 ; two-clock black missile down the seam. Text rows
23 -- the status line and a
24 ; name row over each pair -- use the 48-pixel player
25 kernel from
26 ; disptext.inc, whose 53-cycle body is TRANSCRIBED
27 FROM THE FIRMWARE'S
28 ; fuji disp.inc AND MUST NOT BE RETIMED: its
29 constants are the coordinates of a
30 ; one-pixel-wide window. The players are positioned
31 once, in DINIT, and never
32 ; move; there is no HMOVE anywhere in the frame.
33
34 ; THE FRAME IS ALWAYS 262 LINES. The blanked bands
35 are timed with the RIOT
36 ; timer, as in disptext.inc, and so is the VISIBLE
37 band: the kernel's own
38 ; lines add up to less than 192 and a timed pad at
39 the bottom absorbs the
40 ; difference -- and any overrun -- rather than
41 moving the whole picture down
42 ; the screen, which is what this family has paid for
43 twice already.
44
45 ; Visible, three or four players (192). Counted as
46 WSYNCS: which is what the
47 ; raster sees; tools/pfcheck.py holds the same
48 arithmetic and must move with
49 it.
50 ; 1 the unblank line
51 ; 6 status row (text row 0: 5 ink
52 + a seam)
53 ; 6 names over pair A (text row 1)
54 ; 3 fleet strip (text row 3: 'u'
55 afloat, '=' sunk)
56 ; 1 the strip's seam, WHICH IS ALSO the lead-in:
57 sprites blanked,
58 ; water on the divider on
59 ; 80 board pair A, rows 0-9
60 ; 1 lead-out: playfield off, the divider off
61 ; 2 gap
62 ; 6 names over pair B (text row 2)
63 ; 3 fleet strip (text row 4)
64 ; 1 seam and lead-in
65 ; 80 board pair B, rows 10-19
66 ; 1 lead-out
67 ; 1 timed pad
68 ; That is 191 lines of kernel in a band of 192, and
69 the pad is the last one.
70 ; The count is fixed -- nothing in the visible band
71 branches on data -- so
72 ; exact is safe where variable would not be, and
73 'make frames' is what says
74 ; so. The gap was seven lines and the pad four
75 before the strips; six of
76 ; those nine are what the two strips are made of.
77
78 ; Two players: one pair of 12-line cells (120), then
79 text rows 15-20, and no
80 ; strips -- the lower rows have carried the fleets
81 as pips all along.
82
83 ; ----- kernel timing constants
84
85 ; Not taste. See disptext.inc.
86 ; MNOP EQU 16 ; NOPs before RESP0
87 (2 cycles each).y.
88 ; R3 EQU 1 ; ...plus this, so
89 RESP0 lands on cycle 38
90 ; HMP0 = $F0: move
91 P0 right one pixel...
92 ; ...and leave P1,
93 HM1 EQU 0
94 closing the gap to eight
95 ; VDEL EQU 1 ; vertical delay on
96 both players
97
98 ; The divider: RESM1 completing at cycle 48 puts a
99 missile at clock 80 (a
100 ; player lands at three times the completion cycle
101 less 63, a missile one
102 ; clock earlier -- both measured), and one clock of
103 HM11 pulls it to 79-80
104 ; with the two-clock width NUSIZ1 gives it.
105 Confirmed against the raster by
106 ; tools/pfcheck.py.
107 ; MNOP EQU 21 ; NOPs before
108 RESM1...
109 ; M3 EQU 1 ; ...plus a 3-cycle
110 store: 45, so it lands at 48
111 ; then one clock
112 ; left
113
114 ; ----- text plane bases -----
115 ; TP0 EQU FNTXT+$000
116 ; TP1 EQU FNTXT+$080
117 ; TP2 EQU FNTXT+$100
118 ; TP3 EQU FNTXT+$180
119 ; TP4 EQU FNTXT+$200
120 ; TP5 EQU FNTXT+$280
121
122 ; TINK EQU FNTCLH-1 ; 5 ink lines per
123 text row
124
125 ; ----- the playfield tables
126
127 ; Plane r holds register r's tables: HIT at PFWO0,
128 MID and AUX after it.

```

```

88 ; Every one of these fits in one page for y in 0..19
89 (the highest is $1A09),
90 so 'lda table,y' is always four cycles. host_test
91 asserts it.
92 THIT0 EQU FNTXT+$000+PFWO0
93 THIT1 EQU FNTXT+$080+PFWO0
94 THIT2 EQU FNTXT+$100+PFWO0
95 THIT3 EQU FNTXT+$180+PFWO0
96 THIT4 EQU FNTXT+$200+PFWO0
97 THIT5 EQU FNTXT+$280+PFWO0
98 TMID0 EQU THIT0+PFKLEN
99 TMID1 EQU THIT1+PFKLEN
100 TMID2 EQU THIT2+PFKLEN
101 TMID3 EQU THIT3+PFKLEN
102 TMID4 EQU THIT4+PFKLEN
103 TMID5 EQU THIT5+PFKLEN
104 TAUX0 EQU THIT0+2*PFKLEN
105 TAUX1 EQU THIT1+2*PFKLEN
106 TAUX2 EQU THIT2+2*PFKLEN
107 TAUX3 EQU THIT3+2*PFKLEN
108 TAUX4 EQU THIT4+2*PFKLEN
109 TAUX5 EQU THIT5+2*PFKLEN
110
111 ; NTSC: 3 vsync + 37 vblank + 192 visible + 30
112 overscan = 262. The counts
113 ; are in 64-cycle units and one less than the
114 arithmetic suggests, because
115 ; the timer has to expire INSIDE the last line
116 rather than on its boundary;
117 ; VBITM and OSTIM are disptext.inc's, confirmed by
118 histogram.
119 VBITM EQU 43 ; ~37 scanlines of
120 vblank
121 OSTIM EQU 35 ; ~30 scanlines of
122 overscan
123 VITIM EQU 227 ; ~192 scanlines of
124 picture
125
126 ; ----- DINIT: TIA setup and positioning
127 -----
128 DINIT:
129 ; Every TIA register the kernel relies on, written
130 -- RESM1 above all, whose
131 ; power-on state on real silicon is undefined and
132 which, set, locks the
133 ; missile to its player and never draws it. The cold
134 start clears $00-$2C
135 ; too; this is for a bank entered warm.
136 lda #0
137 ; no reflect, no
138 priority, no score mode
139 sta CTRLPF
140 sta PF0
141 sta PF1
142 sta PF2
143 sta ENAM0
144 sta ENAM1
145 sta ENABL
146 sta RESMP0
147 sta RESMP1
148 sta REFP0
149 sta REFP1
150 sta HMM0
151 sta HMM1
152 sta HMBL
153 sta HMP0
154 sta HMP1
155 sta VOELBL
156 sta COLUBK
157 sta GRP0
158 sta GRP1
159 sta GRP2
160 sta BSFRAME
161 lda #0
162 ; P0: three copies,
163 close
164 lda NUSIZ0
165 ; P1: three copies,
166 lda $13
167 close, and a 2-clock
168 sta NUSIZ1 ; missile -- the
169 two fields are independent
170 lda #VOEL
171 sta VOELP0
172 sta VOELP1
173 lda #CWITE
174 sta COLUP0
175 sta COLUP1
176
177 ; The divider first, while HMP0/HMP1 are still zero
178 so its HMOVE moves
179 ; nothing else. The nudge is cleared straight after,
180 so the players' HMOVE
181 ; below leaves it where it is, and there are no
182 HMOVEs after that.
183 sta WSYNCR
184 REPT
185 nop
186 ENDM
187 IF M3
188 sta PAD3
189 ENDF
190 sta RESM1
191 lda #HMM1
192 sta HMM1
193 sta WSYNCR
194 sta HMOVE
195 sta WSYNCR
196 most of a line to apply;
197 lda #0 ; clearing the
198 nudge inside it cuts it short
199 sta HMM1
200
201 ; The players, transcribed from batari Basic's
202 std_kernel.asm by way of the
203 ; Firmware's fuji disp.inc: RESP0 at 38, RESP1 three
204 cycles later, HMP0 pulls
205 ; P0 right one pixel, and an EARLY HMOVE on the next
206 line applies it.
207 sta WSYNCR
208 REPT
209 nop
210 ENDM

```

```

183 IF R3
184 sta PAD3
185 ENDIF
186 sta RESP0
187 sta RESP1
188 lda #HM0*16
189 sta HMP0
190 lda # (HM1*16)&5FF
191 sta HMP1
192 sta WSYNC
193 sta HMOVE
194 rts
195
196 ; ----- DLOOP: draw frames, forever
197 ; -----
198 ; DFRESH is the entry for a FRESH start -- a bank
199 ; that is not finishing or
200 ; following a frame -- and arms the overscan timer
201 ; DFRAME waits on, because
202 ; a stale count could be seconds. A bank entered
203 ; between frames must NOT
204 ; arm it: the previous frame's DFRAME2 already did,
205 ; and arming it again in
206 ; DINIT cost thirty lines on every re-entry after a
207 poll.
208 DFRESH: lda #OSTIM
209 sta TIM64T
210 DLOOP: jsr DFRAME
211 jmp DLOOP
212
213 ; ----- DFRAME: draw ONE frame, then
214 ; return
215 ; Split out of DLOOP so a bank that is not running
216 ; the display loop can still
217 ; keep the picture alive: net.inc waits for the
218 ; cartridge a frame at a time
219 ; through here, which is what stops a poll from
220 ; blanking the screen.
221 ; THE OVERSCAN IS WAITED OUT HERE, at the start of
222 ; the next frame, and not
223 ; at the end of the one before. DFRAME2 arms the
224 ; timer and returns at once,
225 ; so whatever a bank does between two frames --
226 ; build a URL into the TX
227 ; page, recompose the boards, switch banks -- is
228 ; spent INSIDE the overscan
229 ; and moves nothing, as long as it fits in thirty
230 ; lines. Waiting at the end
231 ; instead put every one of those between the
232 ; overscan and the next VSYNC,
233 ; and a played game measured 96 frames in 2,400 that
234 ; were 273 to 310 lines.
235 DFRAME: bit TIMINT
236 bpl DFRAME
237 sta WSYNC
238 lda #2
239 sta WSYNC
240 sta WSYNC
241 sta WSYNC
242 sta WSYNC
243 lda #0
244 sta VSYNC
245
246 lda #2
247 sta VBLANK
248 sta #VBINT
249 sta TIM64T
250 ; The per-frame hook INSIDE the timed band: input,
251 ; sound, and the whole of a
252 ; recompose. It has 2812 cycles less what surrounds
253 ; it, and what it does costs
254 ; the frame nothing as long as it fits.
255 inc BSFRAME
256 jsr APPVBL
257
258 ; ----- DFRAME2: finish the frame the
259 ; hook was called in -----
260 A bank switch is taken from INSIDE the hook, and
261 the bank entered carries on
262 ; from here: the vblank timer is the RIOT's and does
263 ; not care which bank is
264 ; mapped, so the frame the switch interrupted is
265 ; completed at exactly 262
266 ; lines by whoever holds the kernel now. The
267 ; alternative -- switching between
268 ; frames -- puts the next VSYNC tens of cycles late,
269 ; once a poll, and a
270 ; switch that starts a fresh frame from the hook
271 ; throws a 40-line frame at
272 ; the set.
273 DFRAME2:
274 ; The status row's colours, set here where the
275 ; raster is nowhere near
276 ; anything, in case the hook -- or the bank before
277 ; it -- left them elsewhere.
278 lda #0
279 sta COLUBK
280 sta PF0
281 sta PF1
282 sta PF2
283 sta ENAM1
284 lda #CSTATUS
285 sta COLUP0
286 sta COLUP1
287
288 ; Wait out the band -- TIMINT, not INTIM: the latch
289 ; stays set, so an overrun
290 ; costs exactly an overrun -- and land the unblank
291 ; on a line boundary.
292 DVBLP: bit TIMINT
293 bpl DVBLP
294 sta WSYNC
295 lda #0
296 sta VBLANK
297 lda #VITIM ; the picture's own
298 clock sta TIM64T
299
300 ; ----- the status row -----
301 ; Every text row is six lines: five of ink and a
302 ; seam line that blanks the

```

```

272 ; sprites, which otherwise go on showing their last
273 ; two bytes down the
274 ; screen. TINKS returns early in its seam line, so a
275 ; colour change here lands
276 ; on a line nothing is drawn on.
277 lda #RSTAT*FNTCLH
278 jsr TINKS
279 lda #CLABEL
280 sta COLUP0
281 sta COLUP1
282
283 ; ---- pair A: names, the fleet strip, then the
284 ; boards ----
285 lda #RLABA*FNTCLH
286 jsr TINKS
287 lda #CAUX
288 ; the strip is gold,
289 like the hull brackets.
290 sta COLUP0 ; The name row's
291 seam line has the cycles
292 sta COLUP1 ; for this and
293 nothing is drawn on it.
294 lda BSMODE
295 beq DFA2
296 lda #RFLTA*FNTCLH
297 jsr PIPROW ; three ink lines
298 AND the lead-in
299 jmp DFA3
300 DFA2: jsr LEADIN ; two seats: the
301 lower rows carry the fleets
302 DFA3: lda #0
303 lda #10
304 sta ROWEND
305 lda BSMODE
306 beq DFA2
307 jsr BOARD8
308 jsr LEADOUT
309
310 ; ---- pair B ----
311 sta WSYNC ; the first gap line
312 takes the background back
313 lda #0
314 sta COLUBK
315 lda #CLABEL
316 sta COLUP0
317 sta COLUP1 ; the gap, shortened
318 ldx #1
319 to pay for the strips
320 DFgap: sta WSYNC
321 bne DFgap
322 lda #RLABB*FNTCLH
323 jsr TINKS
324 lda #CAUX
325 sta COLUP0
326 sta COLUP1
327 lda #RFLTB*FNTCLH
328 jsr PIPROW
329 lda #10
330 lda #20
331 sta ROWEND
332 jsr BOARD8
333 jsr LEADOUT
334 jmp DFpad
335
336 ; ---- two players: one pair of square cells, then
337 ; the lower text ----
338 DF2P: jsr BOARD12
339 jsr LEADOUT
340 sta WSYNC
341 lda #0
342 sta COLUBK
343 lda #CWHITE
344 sta COLUP0
345 sta COLUP1
346 ldx #3
347 DF2GAP: sta WSYNC
348 bne DF2GAP
349 lda #RLOW*FNTCLH
350 ldx #NLOWROW
351 DF2ROW: stx BSTM2
352 jsr TINKS
353 ldx #BSTM2
354 bne DF2ROW
355
356 ; ---- the pad: whatever is left of 192, timed ----
357 DFpad: sta WSYNC ; the lead-out's
358 water stays whole
359 lda #0
360 sta COLUBK
361 DFpad1: bit TIMINT
362 bpl DFpad1
363 sta WSYNC
364
365 lda #2
366 sta VBLANK
367 lda #OSTIM ; the next DFRAME
368 waits this out; see above
369 sta TIM64T
370 ; Returns with VBLANK SET and the overscan timer
371 ; running: a bank may do up to
372 ; about 2,200 cycles of work before the next DFRAME
373 ; for free.
374 rts
375
376 ; -----
377 ; TINKS -- the text row whose first plane byte is V:
378 ; five ink lines and a
379 ; SEAM LINE. Leaves V at the next row's first byte
380 ; and returns early in the
381 ; seam line, so the caller has that line for colour
382 ; changes.
383 ; The body is disptext.inc's, with the stack pointer
384 ; parked as a register
385 ; and restored before the return, and the 'iny /
386 ; cpy / bne' loop is what
387 ; keeps an ink line inside 76 cycles. The seam line
388 ; is not optional: the

```

```

371 ; copies go on displaying the last two bytes written
372 until something blanks
373 ; them, and the exit path is too long to do it on
374 the last ink line.
375 ; jsr after this routine's 'rts' lands at cycle
376 79, which is a line lost
377 ; without anything looking wrong.
378 ;
379 ; TINKN is the same row with A ink lines instead of
380 five, and it hands the
381 ; seam line to its caller WITHOUT blanking; PIPROW
382 needs the water up before
383 ; pixel 0, which is earlier than the blanking has to
384 be, and only the caller
385 ; knows that. TINK5 is TINKN plus the blanking,
386 which is what every other row
387 ; wants. The extra jsr/rts costs twelve cycles of a
388 line that has fifty-seven
389 ; spare.
390 TINK5: lda #TINK
391 jsr TINKN
392 lda #0
393 sta GRP0
394 sta GRP1
395 sta GRP0
396 iny
397 rts
398 ; past the blank
399
400 TINKN: tsx
401 stx SAVSP
402 sta BSLINE
403 ; the ink count,
404 until Y is added to it
405 tya
406 clc
407 adc BSLINE
408 sta BSLINE
409 ; the first byte
410
411 past the ink
412 TII: sta WSYNC
413 lda TP0,y
414 sta GRP0
415 GRP0 = group 0
416 lda TP1,y
417 sta GRP1
418 displays group 0
419 lda TP2,y
420 sta GRP0
421 displays group 1
422 lda TP4,y
423 tax
424 txs
425 ; 2 -> 25
426 ; 2 -> 27
427 ; 2 -> 29 park
428
429 group 4 in S
430 lda TP3,y
431 tax
432 lda TP5,y
433 stx GRP1
434 displays group 2
435 tsx
436 recover group 4
437 stx GRP0
438 displays group 3
439 sta GRP1
440 displays group 4
441 sty GRP0
442 displays group 5; value unused
443 iny
444 cpy BSLINE
445 bne T11
446 ldx SAVSP
447 txs
448 ldx WSYNC
449 rts
450 ; the seam line, from cycle 6
451 ;
452 ;-----
453 ; PIPROW -- the fleet strip whose first plane byte
454 is Y: three ink lines, and
455 ; a seam line that is ALSO the board's lead-in.
456 Replaces LEADIN in the
457 ; quadrant layout, so the strip costs three
458 scanlines and not four.
459 ;
460 ; Three ink lines, because that is what the gap and
461 the pad between them can
462 ; pay for. The font is a 3x5 glyph, so this is its
463 top three rows: 'l' is
464 ; {5,7,5} and '=' is {0,7,0} -- a hash and a bar,
465 which is the whole
466 ; vocabulary the strip needs. Digits would NOT
467 survive the clip; '0' and '9'
468 ; are both {7,5,7}.
469 ;
470 ; The order of this line is not taste. COLUBK must
471 be written before pixel 0
472 ; at cycle 22.67; the sprites only have to be
473 blanked before the text block
474 ; at cycle 52, which is cycle 39. Doing the blanking
475 first -- as LEADIN does,
476 ; where it is free -- puts the water in four clocks
477 late and leaves a black
478 ; sliver down the left of the line.
479 PIPROW: lda #3
480 jsr TINKN
481 lda #CWATER
482 sta COLUBK
483 ; 5 before pixel
484
485 0 at 22
486 lda #0
487 sta GRP0
488 text block at 39
489 sta GRP1
490 sta GRP0
491 sta COLUP1
492 colour
493 lda $S10
494 clocks wide
495 lda NUSIZ1
496 lda #2
497 sta ENAM1
498 divider at 49
499
500 rts
501 ;-----
502 ; LEADIN -- the line before a board: the water on,
503 the divider black, ONE
504 ; copy and on. NUSIZ1's copy count is P1's AND its
505 missile's, so the three
506 ; close copies the text needs would put three
507 dividers across the boards;
508 ; it is switched here and back in LEADOUT, both
509 while noting of P1's is
510 ; visible. The sprites are blanked again for the
511 bank that entered warn.
512 LEADIN: sta WSYNC
513 lda #0
514 sta GRP0
515 sta GRP1
516 sta GRP0
517 sta COLUP1
518 colour
519 lda #CWATER
520 sta COLUBK
521 0 at 22
522 lda $S10
523 clocks wide
524 lda NUSIZ1
525 lda #2
526 sta ENAM1
527 divider at 49
528 rts
529 ;-----
530 ; LEADOUT -- the line after a board: the playfield
531 and the divider off before
532 ; pixel 0, the copies back for the text, the water
533 left on for one more
534 ; line. The line after this one takes the background
535 back to black in its
536 ; hblank; a mid-line COLUBK change is a visible
537 seam.
538 LEADOUT: sta WSYNC
539 lda #0
540 sta PF0
541 sta PF1
542 sta PF2
543 sta ENAM1
544 lda $S13
545 sta NUSIZ1
546 rts
547 ;-----
548 ; The three line bodies. Each is one scanline of one
549 kind, cycle-exact:
550
551 PF0/PF1/PF2 for the LEFT half must be written by
552 cycles 22/28/38, before
553 ; the beam reaches pixels 0/16/48. For the RIGHT
554 half PF0 must land in
555 ; 28, sta (after pixel 15 has been fetched, before
556 80), PF1 in 39..54, PF2 in
557 ; 50..65. The NOPs are not padding: without them
558 the right-half PF2 lands at
559 ; 49 and its first colour clock is inside pixel
560 79, the left board's last.
561 ; Every window here has at least ten colour clocks
562 of margin.
563
564 ; Y is the cell row; A is free; X is untouched. Each
565 ends at cycle 59 with the
566 ; RTS, and a 'jsr' to the next body puts its WSYNC
567 at 68.
568 LAUX: sta WSYNC
569 lda #CAUX
570 sta COLUPF
571 lda TAUX0,y
572 sta PF0
573 lda TAUX1,y
574 sta PF1
575 lda TAUX2,y
576 sta PF2
577 lda TAUX3,y
578 sta PF0
579 lda TAUX4,y
580 nop
581 lda PF1
582 lda TAUX5,y
583 nop
584 lda PF2
585 rts
586
587 LHIT: sta WSYNC
588 lda #CHIT
589 sta COLUPF
590 lda THIT0,y
591 sta PF0
592 lda THIT1,y
593 sta PF1
594 lda THIT2,y
595 sta PF2
596 lda THIT3,y
597 sta PF0
598 lda THIT4,y
599 nop
600 lda PF1
601 lda THIT5,y
602 nop
603 lda PF2
604 rts
605
606 LMID: sta WSYNC
607 lda #CMISS
608 sta COLUPF
609 lda TMID0,y
610 sta PF0
611 lda TMID1,y
612 sta PF1
613 lda TMID2,y
614 sta PF2

```

```

550     lda     TMID3,y
551     sta     PF0
552     lda     TMID4,y
553     nop
554     sta     PF1
555     lda     TMID5,y
556     nop
557     nop
558     sta     PF2
559     rts
560
561 ;
-----
562 ; BOARD8 -- cell rows Y .. ROWEND-1, eight lines
563 ; each. The last line of the
564 ; row is the AUX body inline with the loop test:
565 ; iny / cpy / bne' reaches
566 ; the next row's first WSYNC at 70.
567 BOARD8: jsr     LAUX          ; 0
568         jsr     LHIT         ; 1
569         jsr     LHIT         ; 2
570         jsr     LMID         ; 3
571         jsr     LMID         ; 4
572         jsr     LHIT         ; 5
573         jsr     LHIT         ; 6
574         sta     WSYNC        ; 7
575     lda     #CAUX
576     sta     COLUPF
577     lda     TAUX0,y
578     sta     PF0
579     lda     TAUX1,y
580     sta     PF1
581     lda     TAUX2,y
582     sta     PF2
583     lda     TAUX3,y
584     nop
585     sta     PF0
586     lda     TAUX4,y
587     nop
588     nop
589     sta     PF1          ; 53
590     iny          ; 55
591     cpy     ROWEND      ; 58
592     bne     BOARD8     ; 61
593     rts
594
595 ; BOARD12 -- the two-player layout's square cells:
596 ; three lines of red, four
597 ; of white, three of red, between the yellow lines.
598 BOARD12: jsr     LAUX          ; 0
599         jsr     LHIT         ; 1
600         jsr     LHIT         ; 2
601         jsr     LHIT         ; 3
602         jsr     LMID         ; 4
603         jsr     LMID         ; 5
604         jsr     LMID         ; 6
605         jsr     LMID         ; 7
606         jsr     LHIT         ; 8
607         jsr     LHIT         ; 9
608         jsr     LHIT         ; 10
609         sta     WSYNC        ; 11
610     lda     #CAUX
611     sta     COLUPF
612     lda     TAUX0,y
613     sta     PF0
614     lda     TAUX1,y
615     sta     PF1
616     lda     TAUX2,y
617     sta     PF2
618     lda     TAUX3,y
619     sta     PF0
620     lda     TAUX4,y
621     nop
622     sta     PF1
623     lda     TAUX5,y
624     nop
625     nop
626     sta     PF2
627     iny
628     cpy     ROWEND
629     bne     BOARD12
630     rts
631
632

```

SRC/SOUND.INC

```

1 ; sound.inc -- the cues, on one TIA channel.
2 ;
3 ; A cue is a SCRIPT: steps of (frames, AUDC, AUDF,
4 ; AUDV), ended by a bank's copy.
5 ; zero frames. SNDIFIRE starts one and SNDTICK, once
6 ; a frame from APPVBL.
7 ; advances it. Nothing blocks: the display loop
8 ; cannot afford to, and the
9 ; whole point of a cue is to announce something the
10 ; picture is showing NOW.
11 ;
12 ; The vocabulary is the family's (src/platform-
13 ; specific/sound.h), with the
14 ; IIA's own character: noise for the water and the
15 ; explosions, pure tones for
16 ; the interface. Two rules from the siblings: the
17 ; result cue rides the STATUS
18 ; edge, so an opponent's shot is heard too; and the
19 ; turn cue plays before the
20 ; draw, because it is an alert and an alert after
21 ; the fact is not one.
22 ;
23 ; Two cells of zero page: SNDPTR, the offset of the
24 ; current step in SNDATA
25 ; (0 = silent, so the table's first byte is a
26 ; terminator and offset 0 is never
27 ; a real step), and SNDCHT, the frames left on it.
28 ;
29 ; THE ORDER IS NOT ARBITRARY: a bank carries its own
30 ; copy of every byte it
31 ; includes, and the cues are 160 of them. They are
32 ; numbered so that each
33 ; bank's are a PREFIX, and SNDLAST says where that
34 ; bank's copy stops -- the
35 ; place bank needs four cues and used to carry
36 ; twelve, which is 148 bytes it
37 ; did not have.
38 ; SNDMOVE EQU 0 ; the cursor stepped
39 ; one cell: a click ; a choice made, a
40 ; SNDSEL EQU 1 ; a ship committed,
41 ; SNDERR EQU 2 ; refused: low,
42 ; short, deliberately ugly
43 ; SNDSHOT EQU 3 ; the shot went out:
44 ; a falling swoosh
45 ; SNDCLK EQU 4 ; the clock's last
46 ; seconds
47 ; SNDMISS EQU 5 ; a splash, low and
48 ; short
49 ; SNDHIT EQU 6 ; an explosion,
50 ; brighter and longer
51 ; SNDUNK EQU 7 ; the explosion,
52 ; then three falling tones
53 ; SNDTURN EQU 8 ; your turn: three
54 ; rising, carries a room
55 ; SNDOVER EQU 9 ; game over: three
56 ; long falling tones
57 ; SNDJOIN EQU 10 ; the start
58 ; countdown began

```

```

35
36 ; SNDIFIRE -- start cue A, replacing whatever is
   playing.
37 SNDIFIRE: tax
38           lda     SNDTAB,x
39           sta     SNDPTR
40           jmp     SNDSTEP
41
42           IF      SNDFULL
43 ; SNDISOFT -- start cue A only if nothing is playing.
   For the cursor click,
44 ; which must never cut a hit short.
45 SNDISOFT: ldx     SNDPTR
46           bne     SNDX
47           jmp     SNDIFIRE
48
49 ; SNDTICK -- once a frame.
50 SNDTICK: lda     SNDPTR
51           beq     SNDX
52           dec     SNDCNT
53           bne     SNDX
54           clc
55           adc     #4
56           sta     SNDPTR
57           ENDF
58 ; SNDSTEP -- load the step at SNDPTR; a zero-frame
   step ends the cue.
59 SNDSTEP: ldx     SNDPTR
60           lda     SNDDATA,x
61           beq     SNDOFF
62           sta     SNDCNT
63           lda     SNDDATA+1,x
64           lda     AUDC0
65           lda     SNDDATA+2,x
66           sta     AUDF0
67           lda     SNDDATA+3,x
68           sta     AUDV0
69 SNDX:     rts
70 SNDOFF:  sta     AUDV0
71           sta     SNDPTR
72           rts
73
74 ; The scripts. Offsets into SNDDATA, one per cue, up
   to SNDLAST.
75 SNDTAB: DB      SMOVE-SNDDATA, SSEL-SNDDATA,
   SDPLC-SNDDATA, SDERR-SNDDATA
76           IF      SNDLAST>=5
77           DB      SDSHOT-SNDDATA, SDTICK-SNDDATA
78           ENDF
79           IF      SNDLAST>=11
80           DB      SMISS-SNDDATA, SDHIT-SNDDATA,
   SDTURN-SNDDATA, SDOVER-SNDDATA, SDJOIN-SNDDATA
81           ENDF
82
83 SNDDATA: DB      0          ; offset 0 is
   "silent"
84
85 SMOVE: DB      1, 12, 3, 4, 0
86 SSEL:  DB      3, 12, 14, 6, 3, 12, 9, 6, 0
87 SDPLC: DB      2, 12, 11, 5, 3, 12, 7, 5, 0
88 SDERR: DB      10, 1, 28, 5, 0
89           IF      SNDLAST>=5
90           DB      2, 1, 6, 8, 2, 1, 9, 7, 2, 1, 12,
   6, 2, 1, 16, 5
91           DB      2, 1, 20, 4, 3, 1, 26, 3, 3, 1,
   31, 2, 0
92 SDTICK: DB      1, 8, 4, 4, 0
93           ENDF
94           IF      SNDLAST>=11
95           DB      2, 8, 24, 6, 3, 8, 28, 4, 4, 8,
   31, 2, 0
96 SDHIT: DB      3, 8, 6, 12, 3, 8, 10, 11, 4, 8,
   15, 9, 4, 8, 20, 7
97           DB      5, 8, 26, 5, 6, 8, 31, 3, 0
98 SDSUNK: DB      3, 8, 6, 12, 3, 8, 10, 11, 4, 8,
   15, 9, 4, 8, 22, 6
99           DB      5, 12, 9, 8, 5, 12, 12, 8, 10, 12,
   16, 8, 0
100 SDTURN: DB      4, 12, 15, 6, 4, 12, 11, 6, 7, 12,
   8, 7, 0
101 SDOVER: DB      10, 12, 8, 7, 10, 12, 12, 7, 20,
   12, 18, 7, 0
102 SDJOIN: DB      4, 12, 12, 5, 4, 12, 10, 5, 6, 12,
   8, 5, 0
103           ENDF
104

```

SRC/BSTAIL.ASM

```

1 ; bstail.asm -- the fixed half: the trampoline, the
   shared transport, and the
2 ; cold stub
3 ;
4 ; $1800-$1F1F is the cartridge's mailbox -- text
   planes, reply window, control
5 ; page, TX page, status -- and the cartridge paints
   it. The client owns
6 ; $1F20-$1F7B, which fuji_mailbox.h calls the fixed
   tail, plus the vectors.
7 ;
8 ; Everything here has to be at an address that does
   not move, for three
9 ; different reasons:
10 ;
11 ; * The store that switches bank is the LAST
   instruction fetched from the
12 ; old bank and the very next fetch comes from
   the new one, so the jump
13 ; after it cannot live in a bank.
14 ; * This console has no reset line to the
   cartridge. The RESET switch
15 ; restarts the 6507 with whatever bank was last
   selected still mapped, so
16 ; a cold stub living in bank 0 would simply not
   be there when it was
17 ; needed.
18 ; * The transport is the same 137 bytes in all
   three banks, and a bank is

```

```

19 ;      2048 bytes. Here it is one copy that all of
   them can reach.
20
21 CPU      6502
22 INCLUDE  "vcs.inc"
23 INCLUDE  "fujiinet.inc"
24 INCLUDE  "bsdefs.inc"
25 ; bstrans.inc has no bank identity -- it IS the
   shared copy -- but it is
26 ; assembled next to the same equates every bank
   uses.
27 BSANK    EQU      BANKLOB
28 BSHASINP EQU      0
29 BSHASUI  EQU      0
30 BSHASED  EQU      0
31 BSHASNET EQU      0
32 BSHASCLS EQU      0
33 BSHASSTR EQU      0
34 BSHASRPL EQU      0
35 BSHASDEC EQU      0
36
37 INCLUDE  "bscore.inc"
38
39 END
40

```

SRC/BSLOBBY.ASM

```

1 ; bslobby.asm -- bank 0: the cold start and the
2 ; table list.
3 ; Bank 0 by construction: the cold stub in the fixed
4 ; tail sends the console
5 ; here, because it is the only bank whose number the
6 ; stub can know before
7 ; anything has run.
8
9 CPU 6502
10 INCLUDE "vcs.inc"
11 INCLUDE "fujinet.inc"
12 INCLUDE "bsdefs.inc"
13
14 BSBANK EQU BANKLOB
15 BSHASINP EQU 1
16 BSHASUI EQU 0
17 BSHASED EQU 1
18 BSHASNET EQU 0
19 BSHASSTR EQU 1
20 BSHASRPL EQU 0
21 BSHASDEC EQU 0
22
23 INCLUDE ".../build/tail.inc"
24
25 RLITL EQU 0
26 RLST0 EQU 3
27 RLSEAT EQU 15
28 RLFOOT EQU 19
29
30 ORG $1000
31
32 ;
33 LENTRY: lda #2
34 sta VBLANK
35 lda BSENT
36 cmp #ENCOLD
37 beq LCOLD
38 jmp LWARM
39
40 ;
41 ; The cold start proper. The stub in the tail does
42 ; only what has to be done
43 ; from an address every bank can see -- the arming
44 ; pair and the bank switch --
45 ; because the tail is 220 bytes and the shared
46 ; transport wants 160 of them.
47 ;
48 ; EVERY TIA REGISTER IS CLEARED, not just RAM:
49 ; RESMP1's power-on state on
50 ; real silicon is undefined, and set it locks the
51 ; divider missile to its
52 ; player and never draws it -- an emulator starts it
53 ; at zero and would never
54 ; show the difference.
55 LCOLD: ldx #FFF
56 txs
57 lda #0
58 LCL1: sta $00,x
59 TIA, $00-$FF is RAM
60
61 dex
62 bne LCL1
63 sta $00
64 lda #2
65 sta VBLANK
66
67 ; THE CARTRIDGE'S PATH BUFFERS SURVIVE A CONSOLE
68 ; RESET: there is no reset
69 ; line on this connector and the cart never sees the
70 ; switch -- so a client
71 ; that assumes they are empty comes back from a
72 ; RESET with the last session's
73 ; player name still in its URLs. Empty all four and
74 ; leave a known one
75 ; selected.
76 LCL2: txa #FP_SEL3
77 sta FNRSSEL+FH_PATHO
78 lda #FP_RST
79 sta FNRSSEL+FH_PATHO
80 dex
81 cpx #FP_SEL0-1
82 bne LCL2
83
84 jsr FNARM ; harmless twice,
85 and cheap insurance
86 jsr FNCHK
87 beq LGOT
88
89 ; No cartridge means no character generator either,
90 ; so there is nothing to say
91 ; it with. A red screen is the whole message.
92 lda #CRED
93 sta COLUBK
94 lda #0
95 sta VBLANK
96 LHALT: jmp LHALT
97
98 LGOT: jsr DINIT
99 jsr FNCLS
100 lda #ENNAME
101 sta BSENT
102 lda #BANKGAM
103 jmp BSGOTO
104
105 ;
106 ; Back from a /tables fetch, or from a redraw.
107 ; THE TEXT PLANES SURVIVE A BANK SWITCH. Coming back
108 ; here from a table
109
110 ; without clearing would leave the game's rows
111 ; showing under the lobby's
112 ; title and footer -- and the lobby paints only the
113 ; rows it uses.
114 LWARM: jsr DINIT
115 jsr FNCLS
116 lda #0
117 sta BSSEL
118 jsr LDRAW ; the WHOLE list.
119
120 LDSEL is the cursor only and ; FNCLS above has
121 just blanked every row.
122 lda #0
123 sta VBLANK
124 jmp DFRESH
125
126 ;
127 APPVBL: jsr INREPT
128 sta BSINP
129 and #IN_UP
130 beq LAV1
131 lda BSSEL
132 beq LAV1
133 dec BSSEL
134 jsr LDSEL
135 lda BSINP
136 and #IN_DOWN
137 beq LAV2
138 lda BSSEL
139 clc
140 adc #1
141 BSCNT
142 cmp LAV2
143 bcs LAV2
144 sta BSSEL
145 jsr LDSEL
146 lda BSINP
147 and #IN_RIGHT
148 beq LAV3
149 lda #ENTABLE ; list again
150 sta BSENT
151 lda #BANKNET
152 jmp BSGOTO
153
154 LAV3: lda BSINP
155 and #IN_SEL
156 beq LAV4
157 lda #ENNAME ; change the name
158 sta BSENT
159 lda #BANKGAM
160 jmp BSGOTO
161
162 LAV4: and BSINP
163 and #IN_FIRE
164 beq LAV5
165 lda BSCNT
166 beq LAV5
167 lda LJOIN
168 jmp
169
170 LAV5: rts
171
172 ;
173 ; LJOIN -- take the selected table's id into path
174 ; buffer 0 and sit down.
175
176 ; This is one of exactly two strings this client
177 ; copies out of a reply, and it
178 ; goes into the CARTRIDGE, not into console RAM: it
179 ; has to survive every
180 ; /state poll after it, and each poll repaints the
181 ; window it came from.
182
183 ; The field is fixed-width and space-padded, so it
184 ; is pushed up to its NUL and
185 ; never through it -- padding in the middle of a URL
186 ; corrupts the request.
187
188 ; Joining a table is not a call: retrieving the
189 ; state for one IS the join,
190 ; and that is the game bank's first poll.
191 LJOIN: lda #PBTABLE
192 jsr FNMSL
193 jsr FNWRST
194 jsr LTBPTR ; FNPTRL/H -> the
195 record
196 ldy #TBD
197 ldx #9
198 LJ1: lda (FNPTRL),y
199 beq LJ2
200 cmp #1
201 beq LJ2
202 sta FNRSSEL+FH_PATHC
203 iny
204 dex
205 bne LJ1
206 LJ2: lda #ENGOLD
207 sta BSENT
208 lda #BANKGAM
209 jmp BSGOTO
210
211 ; LTBPTR -- FNPTRL/H points at table BSSEL's record;
212 ; LTBPTR2 at table
213 ; BSIDX's, for the row being drawn.
214
215 ; The records start at offset ONE, after the count
216 ; byte, and are 36 bytes
217 ; apart -- so the eighth one is past 256 and this
218 ; has to be a pointer.
219
220 ; Dividing the reply length by the stride to get the
221 ; count looks equivalent
222 ; to reading that first byte and is not: 181 bytes
223 ; is 1 + 5*36, and the
224 ; division says 5 only by accident of rounding.
225 LTBPTR: ldx BSSEL
226 ldy LTB
227 LTBPTR2: ldx BSIDX
228 LTB0: lda #((FNPLY+1)&5FF)
229 sta FNPTRL
230 lda #((FNPLY+1)>>8)

```

```

191      sta      FNPTRH
192      cpx      #0
193      beq      LT82
194 LT81:  lda      FNPTRL
195      clc
196      adc      #TBSTRID
197      sta      FNPTRL
198      bcc      LT81A
199      inc      FNPTRH
200 LT81A: dex
201      bne      LT81
202 LT82:  rts
203
204 ; -----
205 ; LDRAW -- the whole lobby.
206 LDRAW: lda      #RLITL
207        jsr      FNROWA
208        ldx      # (LSITL)&$$FF
209        ldy      # (LSITL)>>8
210        jsr      FNSETP
211        jsr      FNSTRA
212        jsr      FNENDW
213
214        lda      BSCNT
215        bne      LD1
216 ; An empty list is a real answer, not a failure. Say
217 ; so and offer the retry.
218        lda      #RLIST0
219        jsr      FNROWA
220        ldx      # (LSNONE)&$$FF
221        ldy      # (LSNONE)>>8
222        jsr      FNSETP
223        jsr      FNSTRA
224        jsr      FNENDW
225        lda      #$$FF
226        sta      BSBAR
227        jmp      LDFOOT
228 LD1:  lda      #0
229        sta      BSIDX
230 LD2:  lda      BSIDX
231        clc
232        adc      #RLIST0
233        jsr      FNROWA
234        lda      BSIDX
235        cmp      BSCNT
236        bcs      LDBLK
237 ; the row is the table's name, twelve of the
238 ; twenty-one bytes it sends
239        jsr      LTBPTR2
240        ldy      #FNTCOL
241 LD3:  lda      (FNPTRL),y
242        beq      LD4
243        sta      FNRSEL+FH_TCHR
244        iny
245        dex
246        bne      LD3
247        jmp      LD5
248 LD4:  lda      # ' '
249        sta      FNRSEL+FH_TCHR
250        dex
251        bne      LD4
252 LD5:  jsr      FNENDW
253        jmp      LDNXT
254 LDBLK: lda      #FNTCOL
255        jsr      FNSPC
256        jsr      FNENDW
257 LDNXT: inc      BSIDX
258        lda      BSIDX
259        cmp      #MAXTBL
260        bne      LD2
261        jsr      LDSEL
262
263 LDFOOT: lda      #RLFOOT
264        jsr      FNROWA
265        ldx      # (LSFOOT)&$$FF
266        ldy      # (LSFOOT)>>8
267        jsr      FNSETP
268        jsr      FNSTRA
269        jmp      FNENDW
270
271 ; -----
272 ; LDSEL -- the cursor moved, and NOTHING ELSE DID:
273 ; the bar, and the seat
274 ; count of the selected table. Redrawing the whole
275 ; list for a cursor key is
276 ; about 3,800 cycles against a vblank budget of
277 ; 2,812, and the overrun pushes
278 ; the picture down the screen for a frame -- the
279 ; list bounced on every press.
280 ;
281 ; The cursor is a whole-row red bar: there is no
282 ; inverse video and no
283 ; per-cell colour on this machine. The kernel reads
284 ; BSBAR on its seam line,
285 ; so moving the bar really is just the store.
286 LDSEL: lda      BSSEL
287        clc
288        adc      #RLIST0
289        sta      BSBAR
290 ; The seat count the server pre-formats as "cur /
291 ; max". It is a literal
292 ; string, not two numbers, so it is printed
293 ; verbatim.
294        lda      #RLSEAT
295        jsr      FNROWA
296        ldx      # (LSSEAT)&$$FF
297        ldy      # (LSSEAT)>>8
298        jsr      FNSETP
299        jsr      FNSTRA
300        jsr      LTBPTR
301        ldy      #TBPLVRS
302        ldx      #6
303 LD6:  lda      (FNPTRL),y
304        beq      LD7
305        sta      FNRSEL+FH_TCHR
306
307        iny
308        dex
309        bne      LD6
310        jmp      LD6W
311
312 ; The lobby is a list, so its rows are plain.
313 BS8GT: DB
314        CBLACK,CBLACK,CBLACK,CBLACK,CBLACK,CBLACK
315        DB
316        CBLACK,CBLACK,CBLACK,CBLACK,CBLACK,CBLACK
317        DB
318        CBLACK,CBLACK,CBLACK,CBLACK,CBLACK,CBLACK
319        DB
320        CWATER
321
322 back to the sea
323
324 INCLUDE "bslib.inc"
325 INCLUDE "state.inc"
326 INCLUDE "disptext.inc"
327
328 END

```

SRC/BSGAME.ASM

```

1 ; bsgame.asm -- bank 1: the game.
2
3 ; Every board on screen at once, composed BY THE
4 CARTRIDGE out of the reply
5 ; window into playfield tables (fujinet.inc, the
6 playfield board) and drawn
7 ; by dispgame.inc. The console never touches a cell.
8
9 ; THE COMPOSER IS ANOTHER BANK. This one holds the
10 kernel, the input, the
11 ; cues and the cursor, and with the composer in it
12 too it came in 961 bytes
13 ; over. A poll's recompose -- a dozen blit ops and
14 five text rows -- runs in
15 ; bscmp in the vblank of the first frame back from
16 the network, and bscmp
17 ; hands the frame here to be finished, so no frame
18 is ever anything but 262
19 ; lines. What this bank holds is a cursor, the four
20 bytes of state every
21 ; screen keys on, the previous poll's values for the
22 edges the cues ride on,
23 ; and a local move clock.
24
25 ; Three rules from the sibling clients, each learned
26 somewhere else:
27
28 ; * A shot lands on EVERY live enemy at once, so
29 the cursor is drawn on
30 ; every live enemy board and the shot is
31 refused, with a tone, at a cell
32 ; already resolved on all of them -- the server
33 refuses it silently and
34 ; the turn never passes.
35 ; * The result cue rides the STATUS edge paired
36 with lastAttackPos, so an
37 ; opponent's shot is heard too and a result that
38 repeats across polls is
39 ; heard once.
40 ; * The turn cue rides activePlayer's edge, and
41 PRVAC advances on every
42 ; poll including the lobby's, so the lobby-to-
43 play transition is caught
44 ; by the same comparison as any other turn
45 change.
46
47 CPU 6502
48 INCLUDE "vcs.inc"
49 INCLUDE "fujinet.inc"
50 INCLUDE "bsdefs.inc"
51
52 BSBANK EQU BANKGAM
53 BSHASINP EQU 1
54 BSHASUT EQU 1
55 BSHASED EQU 0
56 BSHASNET EQU 0 ; BANKNET polls for
57 it
58 BSHASCLS EQU 0
59 BSHASSTR EQU 0 ; BANKCMP composes
60 for it
61 BSHASRPL EQU 0
62 BSHASDEC EQU 0
63 SNDFULL EQU 1
64 SNDLAST EQU 5 ; ..and the shot
65 and the clock
66
67 INCLUDE "../build/tail.inc"
68
69 ORG $1000
70
71 ; Four ways in. ENGNEXT is from a poll by way of the
72 composer,
73 ; frames: the next one is started. ENGRUN is from
74 the composer's text pass,
75 ; mid-frame: that frame is finished. ENGRUND is the
76 same from the menu, by
77 ; way of the composer, and the menu's kernel left
78 the TIA set up for text,
79 ; so DINIT first. ENGCOLD is from the lobby's hook.
80 DINIT costs five lines
81 ; which is why it is not done on every entry: the
82 overscan has thirty.
83 GENTRY: lda BSENT
84 cmp #ENGCOLD
85 beq GCOLD
86 cmp #ENGNEXT
87 beq GRUN
88 ; between frames,
89 from a poll: the TIA is as
90 ; this kernel left
91 it
92 cmp #ENGRUN
93 beq GRUN2 ; mid-frame, from
94 the text pass: likewise
95 jsr DINIT ; ENGRUND: mid-
96 frame, from the menu's kernel
97 GRUN2: jsr DFRAME2
98 GRUN: jsr DFRAME
99 jmp GRUN
100
101 ; A table was just joined. Nothing is known: set the
102 state up and let the
103 ; composer clear the screen, then poll on the first
104 frame -- retrieving the
105 ; state IS the join.
106 GCOLD: lda #0
107 sta BSPOLL
108 sta BSMODE
109 sta BSLIVE
110 sta BSCLCK
111 sta BSERR2
112 sta SNDPTR
113 sta AUDV0
114 BSREDRW ; the composer's
115 previous seat count
116 lda #4
117
118 sta BSCURX
119 sta BSCURY
120 lda #SF
121 sta BSPRVAC
122 sta BSPRVST
123 sta BSPRVLP
124 sta BSCLASS
125 #RQSTATE
126 sta BSREQ2
127 lda #1
128 sta BSMODE
129 DINIT ; finish the lobby's
130 frame with blank boards,
131 jsr DFRAME2 ; then the
132 composer clears, in the overscan
133 #BANKCMP ; ENGCOLD still:
134 back as ENGNEXT
135 jmp BSGOTO
136
137 ; -----
138 100 ; The per-frame hook:
139 APPVBL: jsr SNOTICK
140 ; The composer's text pass, owed from the last poll:
141 taken here, at the top
142 ; of a vblank, and the composer finishes this frame.
143
144 103 bit BSREDRW
145 104 bpl GA0
146 105 lda #ENTEXT
147 106 bset BSENT
148 107 sta #BANKCMP
149 108 lda BSGOTO
150 109 jmp INREPT
151 110 GA0: jsr BSINP
152 111 and #IN_RST
153 112 beq GA1
154 113 lda #ENMENU
155 114 bset BSENT
156 115 lda #BANKMNU
157 116 jmp BSGOTO
158 117 GA1: lda BSINP
159 118 and #IN_SEL
160 119 beq GA2
161 120 lda #0
162 121 BSPOLL ; poll now
163 122 sta BSCLASS
164 123 GA2: lda #CLPLAY
165 124 cmp #CLPLAY
166 125 bne GA5
167 126 lda BSINP
168 127 bne GA6 ; defeated, or
169 watching: no cursor, no shot
170 128 lda BSINP
171 129 and #INDIRS
172 130 beq GA3
173 131 jsr GMOVE
174 132 GA3: lda BSINP
175 133 and #IN_FIRE
176 134 beq GA6
177 135 jsr GFIRE
178 136 jmp GA6
179 137 GA5: lda BSCLASS ; the lobby: FIRE
180 181 readies (a toggle)
182 182 bne GA6
183 183 lda BSINP
184 184 and #IN_FIRE
185 185 beq GA6
186 186 lda #RQREADY
187 187 BSREQ2
188 188 lda #0
189 189 sta BSPOLL
190 190 #SNDSEL
191 191 jsr SNDPTR
192 192 GA6: jsr GLOCK
193 193 ; The poll clock stops AT zero and the switch is
194 taken here, inside the
195 ; vblank: the network bank finishes this frame. The
196 test comes before the
197 ; decrement, or a due poll would wrap the clock to
198 255.
199 199 lda BSPOLL
200 200 beq GA7
201 201 dec BSPOLL
202 202 rts
203 203 GA7: lda #ENFETCH
204 204 sta BSENT
205 205 lda #BANKNET
206 206 jmp BSGOTO
207
208 ; -----
209 162 ; GMOVE -- the stick moved the cursor: off every
210 live board, step, back on.
211 163 GMOVE: lda #PFM_AUX|PFM_CLR
212 164 jsr GCURS
213 165 lda BSINP
214 166 lsr a
215 167 bcc GM1 ; up
216 168 lda BSCURY
217 169 beq GM4
218 170 dec BSCURY
219 171 GM1: jmp GM4
220 172 lsr a
221 173 bcc GM2 ; down
222 174 lda BSCURY
223 175 cmp #BRDDIM-1
224 176 bcs GM4
225 177 inc BSCURY
226 178 jmp GM4
227 179 a
228 180 bcc GM3 ; left
229 181 lda BSCURX
230 182 beq GM4
231 183 dec BSCURX
232 184 jmp GM4
233 185 GM3: lda BSCURX ; right
234 186 cmp #BRDDIM-1

```

```

187      bcs      GM4          292 GDCCEL: ldy      #0
188      inc      BSCURX      293      jsr      FNBARG
189 GM4:   lda      #SNDMOVE   294      lda      #FB_TCELL
190      jsr      SNDISOFT     295      jmp      FNBLLIT
191      lda      #PPM_AUX     296
192      jmp      GCURS        297 INCLUDE "bslib.inc"
193      ;                    298 INCLUDE "state.inc"
                                299 INCLUDE "sound.inc"
                                300 INCLUDE "dispgame.inc"
194 ; GFIRE -- FIRE in play: stage /attack at the
195 ; cursor, if it is your turn and
196 ; the cell is still unresolved on at least one live
197 ; enemy.
198 GFIRE:  lda      BSACT
199      bne      GFERR
200      jsr      CELLNO
201      sta      BSTMP2
202      ldx      #1
203 GF1:    cpx      BSPCNT
204      bcs      GFERR      ; nowhere it would
205      land: refuse it
206      stx      BSIDX
207      jsr      PLRECP
208      ldy      #PLSTAT
209      lda      (FNPTRL),y
210      bne      GF2      ; defeated or
211      watching: not a target
212      FNPTRL
213      clc
214      adc      #PLFIELD
215      sta      FNPTRL
216      bcc      GF1A
217      inc      FNPTRH
218 GF1A:   ldy      BSTMP2
219      lda      (FNPTRL),y
220      beq      GFOK      ; open sea here: it
221      lands
222 GF2:    ldx      BSIDX
223      tnx
224      jmp      GF1
225 GFOK:   lda      #RQATTCK
226      sta      BSREQ2
227      lda      #0
228      sta      BSPOLL
229      lda      #SNDSHOT
230      jmp      SNDFIRE
231 GFERR:  lda      #SNDERR
232      jmp      SNDFIRE
233      ;
234 -----
235 ; GLOCK -- the move clock, counted down locally a
236 ; second at a time from the
237 ; server's moveTime (already net of a round trip),
238 ; ticking in its last
239 ; seconds, and polling when it runs out: the server
240 ; has moved on by then and
241 ; a client that waited for a button would never
242 ; learn.
243 GLOCK:  lda      BSCLASS
244      cmp      #CLPLAY
245      bne      GCKX
246      lda      BSCLK
247      beq      GCKX
248      dec      BSTICK
249      bne      GCKX
250      lda      #60
251      sta      BSTICK
252      dec      BSCLK
253      lda      BSACT
254      bne      GCKX      ; only your own
255      clock shows
256      jsr      GCLKDR
257      lda      BSCLK
258      beq      GCK0
259      cmp      #6
260      bcs      GCKX
261      lda      #SNDCLK
262      jmp      SNDFIRE
263 GCK0:   sta      BSPOLL
264      rts
265 -----
266 ; GCLKDR -- the three clock digits at the end of the
267 ; status row, a cell
268 ; each through FB_TCELL: the only part of the row a
269 ; second changes, and the
270 ; composer that draws the rest lives in another
271 ; bank
272 GCLKDR: lda      BSCLK
273      ldx      #0
274 GCD1:   cmp      #100
275      bcc      GCD2
276      sbc      #100
277      tnx
278      bne      GCD1
279      pha
280      txi
281      beq      GCD2A
282      clc
283      adc      #'0'
284      jmp      GCD2B
285 GCD2A:  lda      #RSTAT*FNTCOL+9      ; no leading zero
286 GCD2B:  ldx      #RSTAT*FNTCOL+9
287      jsr      GDCCEL
288      pla
289      ldx      #0
290      cmp      #10
291      bcc      GCD4
292      sbc      #10
293      tnx
294      bne      GCD3
295      pha
296      txi
297      beq      GCD4A
298      clc
299      adc      #'0'
300      lda      #RSTAT*FNTCOL+10
301      jsr      GDCCEL
302      pla
303      ldx      #0
304      cmp      #10
305      bcc      GCD4
306      sbc      #10
307      tnx
308      bne      GCD3
309      pha
310      txi
311      beq      GCD4A
312      clc
313      adc      #'0'
314      lda      #RSTAT*FNTCOL+11
315      jsr      GDCCEL
316      pla
317      ldx      #0
318      cmp      #10
319      bcc      GCD4
320      sbc      #10
321      tnx
322      bne      GCD3
323      pha
324      txi
325      beq      GCD4A
326      clc
327      adc      #'0'
328      lda      #RSTAT*FNTCOL+12
329      jsr      GDCCEL
330      pla
331      ldx      #0
332      cmp      #10
333      bcc      GCD4
334      sbc      #10
335      tnx
336      bne      GCD3
337      pha
338      txi
339      beq      GCD4A
340      clc
341      adc      #'0'
342      lda      #RSTAT*FNTCOL+13
343      jsr      GDCCEL
344      pla
345      ldx      #0
346      cmp      #10
347      bcc      GCD4
348      sbc      #10
349      tnx
350      bne      GCD3
351      pha
352      txi
353      beq      GCD4A
354      clc
355      adc      #'0'
356      lda      #RSTAT*FNTCOL+14
357      jsr      GDCCEL
358      pla
359      ldx      #0
360      cmp      #10
361      bcc      GCD4
362      sbc      #10
363      tnx
364      bne      GCD3
365      pha
366      txi
367      beq      GCD4A
368      clc
369      adc      #'0'
370      lda      #RSTAT*FNTCOL+15
371      jsr      GDCCEL
372      pla
373      ldx      #0
374      cmp      #10
375      bcc      GCD4
376      sbc      #10
377      tnx
378      bne      GCD3
379      pha
380      txi
381      beq      GCD4A
382      clc
383      adc      #'0'
384      lda      #RSTAT*FNTCOL+16
385      jsr      GDCCEL
386      pla
387      ldx      #0
388      cmp      #10
389      bcc      GCD4
390      sbc      #10
391      tnx
392      bne      GCD3
393      pha
394      txi
395      beq      GCD4A
396      clc
397      adc      #'0'
398      lda      #RSTAT*FNTCOL+17
399      jsr      GDCCEL
400      pla
401      ldx      #0
402      cmp      #10
403      bcc      GCD4
404      sbc      #10
405      tnx
406      bne      GCD3
407      pha
408      txi
409      beq      GCD4A
410      clc
411      adc      #'0'
412      lda      #RSTAT*FNTCOL+18
413      jsr      GDCCEL
414      pla
415      ldx      #0
416      cmp      #10
417      bcc      GCD4
418      sbc      #10
419      tnx
420      bne      GCD3
421      pha
422      txi
423      beq      GCD4A
424      clc
425      adc      #'0'
426      lda      #RSTAT*FNTCOL+19
427      jsr      GDCCEL
428      pla
429      ldx      #0
430      cmp      #10
431      bcc      GCD4
432      sbc      #10
433      tnx
434      bne      GCD3
435      pha
436      txi
437      beq      GCD4A
438      clc
439      adc      #'0'
440      lda      #RSTAT*FNTCOL+20
441      jsr      GDCCEL
442      pla
443      ldx      #0
444      cmp      #10
445      bcc      GCD4
446      sbc      #10
447      tnx
448      bne      GCD3
449      pha
450      txi
451      beq      GCD4A
452      clc
453      adc      #'0'
454      lda      #RSTAT*FNTCOL+21
455      jsr      GDCCEL
456      pla
457      ldx      #0
458      cmp      #10
459      bcc      GCD4
460      sbc      #10
461      tnx
462      bne      GCD3
463      pha
464      txi
465      beq      GCD4A
466      clc
467      adc      #'0'
468      lda      #RSTAT*FNTCOL+22
469      jsr      GDCCEL
470      pla
471      ldx      #0
472      cmp      #10
473      bcc      GCD4
474      sbc      #10
475      tnx
476      bne      GCD3
477      pha
478      txi
479      beq      GCD4A
480      clc
481      adc      #'0'
482      lda      #RSTAT*FNTCOL+23
483      jsr      GDCCEL
484      pla
485      ldx      #0
486      cmp      #10
487      bcc      GCD4
488      sbc      #10
489      tnx
490      bne      GCD3
491      pha
492      txi
493      beq      GCD4A
494      clc
495      adc      #'0'
496      lda      #RSTAT*FNTCOL+24
497      jsr      GDCCEL
498      pla
499      ldx      #0
500      cmp      #10
501      bcc      GCD4
502      sbc      #10
503      tnx
504      bne      GCD3
505      pha
506      txi
507      beq      GCD4A
508      clc
509      adc      #'0'
510      lda      #RSTAT*FNTCOL+25
511      jsr      GDCCEL
512      pla
513      ldx      #0
514      cmp      #10
515      bcc      GCD4
516      sbc      #10
517      tnx
518      bne      GCD3
519      pha
520      txi
521      beq      GCD4A
522      clc
523      adc      #'0'
524      lda      #RSTAT*FNTCOL+26
525      jsr      GDCCEL
526      pla
527      ldx      #0
528      cmp      #10
529      bcc      GCD4
530      sbc      #10
531      tnx
532      bne      GCD3
533      pha
534      txi
535      beq      GCD4A
536      clc
537      adc      #'0'
538      lda      #RSTAT*FNTCOL+27
539      jsr      GDCCEL
540      pla
541      ldx      #0
542      cmp      #10
543      bcc      GCD4
544      sbc      #10
545      tnx
546      bne      GCD3
547      pha
548      txi
549      beq      GCD4A
550      clc
551      adc      #'0'
552      lda      #RSTAT*FNTCOL+28
553      jsr      GDCCEL
554      pla
555      ldx      #0
556      cmp      #10
557      bcc      GCD4
558      sbc      #10
559      tnx
560      bne      GCD3
561      pha
562      txi
563      beq      GCD4A
564      clc
565      adc      #'0'
566      lda      #RSTAT*FNTCOL+29
567      jsr      GDCCEL
568      pla
569      ldx      #0
570      cmp      #10
571      bcc      GCD4
572      sbc      #10
573      tnx
574      bne      GCD3
575      pha
576      txi
577      beq      GCD4A
578      clc
579      adc      #'0'
580      lda      #RSTAT*FNTCOL+30
581      jsr      GDCCEL
582      pla
583      ldx      #0
584      cmp      #10
585      bcc      GCD4
586      sbc      #10
587      tnx
588      bne      GCD3
589      pha
590      txi
591      beq      GCD4A
592      clc
593      adc      #'0'
594      lda      #RSTAT*FNTCOL+31
595      jsr      GDCCEL
596      pla
597      ldx      #0
598      cmp      #10
599      bcc      GCD4
600      sbc      #10
601      tnx
602      bne      GCD3
603      pha
604      txi
605      beq      GCD4A
606      clc
607      adc      #'0'
608      lda      #RSTAT*FNTCOL+32
609      jsr      GDCCEL
610      pla
611      ldx      #0
612      cmp      #10
613      bcc      GCD4
614      sbc      #10
615      tnx
616      bne      GCD3
617      pha
618      txi
619      beq      GCD4A
620      clc
621      adc      #'0'
622      lda      #RSTAT*FNTCOL+33
623      jsr      GDCCEL
624      pla
625      ldx      #0
626      cmp      #10
627      bcc      GCD4
628      sbc      #10
629      tnx
630      bne      GCD3
631      pha
632      txi
633      beq      GCD4A
634      clc
635      adc      #'0'
636      lda      #RSTAT*FNTCOL+34
637      jsr      GDCCEL
638      pla
639      ldx      #0
640      cmp      #10
641      bcc      GCD4
642      sbc      #10
643      tnx
644      bne      GCD3
645      pha
646      txi
647      beq      GCD4A
648      clc
649      adc      #'0'
650      lda      #RSTAT*FNTCOL+35
651      jsr      GDCCEL
652      pla
653      ldx      #0
654      cmp      #10
655      bcc      GCD4
656      sbc      #10
657      tnx
658      bne      GCD3
659      pha
660      txi
661      beq      GCD4A
662      clc
663      adc      #'0'
664      lda      #RSTAT*FNTCOL+36
665      jsr      GDCCEL
666      pla
667      ldx      #0
668      cmp      #10
669      bcc      GCD4
670      sbc      #10
671      tnx
672      bne      GCD3
673      pha
674      txi
675      beq      GCD4A
676      clc
677      adc      #'0'
678      lda      #RSTAT*FNTCOL+37
679      jsr      GDCCEL
680      pla
681      ldx      #0
682      cmp      #10
683      bcc      GCD4
684      sbc      #10
685      tnx
686      bne      GCD3
687      pha
688      txi
689      beq      GCD4A
690      clc
691      adc      #'0'
692      lda      #RSTAT*FNTCOL+38
693      jsr      GDCCEL
694      pla
695      ldx      #0
696      cmp      #10
697      bcc      GCD4
698      sbc      #10
699      tnx
700      bne      GCD3
701      pha
702      txi
703      beq      GCD4A
704      clc
705      adc      #'0'
706      lda      #RSTAT*FNTCOL+39
707      jsr      GDCCEL
708      pla
709      ldx      #0
710      cmp      #10
711      bcc      GCD4
712      sbc      #10
713      tnx
714      bne      GCD3
715      pha
716      txi
717      beq      GCD4A
718      clc
719      adc      #'0'
720      lda      #RSTAT*FNTCOL+40
721      jsr      GDCCEL
722      pla
723      ldx      #0
724      cmp      #10
725      bcc      GCD4
726      sbc      #10
727      tnx
728      bne      GCD3
729      pha
730      txi
731      beq      GCD4A
732      clc
733      adc      #'0'
734      lda      #RSTAT*FNTCOL+41
735      jsr      GDCCEL
736      pla
737      ldx      #0
738      cmp      #10
739      bcc      GCD4
740      sbc      #10
741      tnx
742      bne      GCD3
743      pha
744      txi
745      beq      GCD4A
746      clc
747      adc      #'0'
748      lda      #RSTAT*FNTCOL+42
749      jsr      GDCCEL
750      pla
751      ldx      #0
752      cmp      #10
753      bcc      GCD4
754      sbc      #10
755      tnx
756      bne      GCD3
757      pha
758      txi
759      beq      GCD4A
760      clc
761      adc      #'0'
762      lda      #RSTAT*FNTCOL+43
763      jsr      GDCCEL
764      pla
765      ldx      #0
766      cmp      #10
767      bcc      GCD4
768      sbc      #10
769      tnx
770      bne      GCD3
771      pha
772      txi
773      beq      GCD4A
774      clc
775      adc      #'0'
776      lda      #RSTAT*FNTCOL+44
777      jsr      GDCCEL
778      pla
779      ldx      #0
780      cmp      #10
781      bcc      GCD4
782      sbc      #10
783      tnx
784      bne      GCD3
785      pha
786      txi
787      beq      GCD4A
788      clc
789      adc      #'0'
790      lda      #RSTAT*FNTCOL+45
791      jsr      GDCCEL
792      pla
793      ldx      #0
794      cmp      #10
795      bcc      GCD4
796      sbc      #10
797      tnx
798      bne      GCD3
799      pha
800      txi
801      beq      GCD4A
802      clc
803      adc      #'0'
804      lda      #RSTAT*FNTCOL+46
805      jsr      GDCCEL
806      pla
807      ldx      #0
808      cmp      #10
809      bcc      GCD4
810      sbc      #10
811      tnx
812      bne      GCD3
813      pha
814      txi
815      beq      GCD4A
816      clc
817      adc      #'0'
818      lda      #RSTAT*FNTCOL+47
819      jsr      GDCCEL
820      pla
821      ldx      #0
822      cmp      #10
823      bcc      GCD4
824      sbc      #10
825      tnx
826      bne      GCD3
827      pha
828      txi
829      beq      GCD4A
830      clc
831      adc      #'0'
832      lda      #RSTAT*FNTCOL+48
833      jsr      GDCCEL
834      pla
835      ldx      #0
836      cmp      #10
837      bcc      GCD4
838      sbc      #10
839      tnx
840      bne      GCD3
841      pha
842      txi
843      beq      GCD4A
844      clc
845      adc      #'0'
846      lda      #RSTAT*FNTCOL+49
847      jsr      GDCCEL
848      pla
849      ldx      #0
850      cmp      #10
851      bcc      GCD4
852      sbc      #10
853      tnx
854      bne      GCD3
855      pha
856      txi
857      beq      GCD4A
858      clc
859      adc      #'0'
860      lda      #RSTAT*FNTCOL+50
861      jsr      GDCCEL
862      pla
863      ldx      #0
864      cmp      #10
865      bcc      GCD4
866      sbc      #10
867      tnx
868      bne      GCD3
869      pha
870      txi
871      beq      GCD4A
872      clc
873      adc      #'0'
874      lda      #RSTAT*FNTCOL+51
875      jsr      GDCCEL
876      pla
877      ldx      #0
878      cmp      #10
879      bcc      GCD4
880      sbc      #10
881      tnx
882      bne      GCD3
883      pha
884      txi
885      beq      GCD4A
886      clc
887      adc      #'0'
888      lda      #RSTAT*FNTCOL+52
889      jsr      GDCCEL
890      pla
891      ldx      #0
892      cmp      #10
893      bcc      GCD4
894      sbc      #10
895      tnx
896      bne      GCD3
897      pha
898      txi
899      beq      GCD4A
900      clc
901      adc      #'0'
902      lda      #RSTAT*FNTCOL+53
903      jsr      GDCCEL
904      pla
905      ldx      #0
906      cmp      #10
907      bcc      GCD4
908      sbc      #10
909      tnx
910      bne      GCD3
911      pha
912      txi
913      beq      GCD4A
914      clc
915      adc      #'0'
916      lda      #RSTAT*FNTCOL+54
917      jsr      GDCCEL
918      pla
919      ldx      #0
920      cmp      #10
921      bcc      GCD4
922      sbc      #10
923      tnx
924      bne      GCD3
925      pha
926      txi
927      beq      GCD4A
928      clc
929      adc      #'0'
930      lda      #RSTAT*FNTCOL+55
931      jsr      GDCCEL
932      pla
933      ldx      #0
934      cmp      #10
935      bcc      GCD4
936      sbc      #10
937      tnx
938      bne      GCD3
939      pha
940      txi
941      beq      GCD4A
942      clc
943      adc      #'0'
944      lda      #RSTAT*FNTCOL+56
945      jsr      GDCCEL
946      pla
947      ldx      #0
948      cmp      #10
949      bcc      GCD4
950      sbc      #10
951      tnx
952      bne      GCD3
953      pha
954      txi
955      beq      GCD4A
956      clc
957      adc      #'0'
958      lda      #RSTAT*FNTCOL+57
959      jsr      GDCCEL
960      pla
961      ldx      #0
962      cmp      #10
963      bcc      GCD4
964      sbc      #10
965      tnx
966      bne      GCD3
967      pha
968      txi
969      beq      GCD4A
970      clc
971      adc      #'0'
972      lda      #RSTAT*FNTCOL+58
973      jsr      GDCCEL
974      pla
975      ldx      #0
976      cmp      #10
977      bcc      GCD4
978      sbc      #10
979      tnx
980      bne      GCD3
981      pha
982      txi
983      beq      GCD4A
984      clc
985      adc      #'0'
986      lda      #RSTAT*FNTCOL+59
987      jsr      GDCCEL
988      pla
989      ldx      #0
990      cmp      #10
991      bcc      GCD4
992      sbc      #10
993      tnx
994      bne      GCD3
995      pha
996      txi
997      beq      GCD4A
998      clc
999      adc      #'0'
1000     lda      #RSTAT*FNTCOL+60
1001     jsr      GDCCEL
1002     pla
1003     ldx      #0
1004     cmp      #10
1005     bcc      GCD4
1006     sbc      #10
1007     tnx
1008     bne      GCD3
1009     pha
1010     txi
1011     beq      GCD4A
1012     clc
1013     adc      #'0'
1014     lda      #RSTAT*FNTCOL+61
1015     jsr      GDCCEL
1016     pla
1017     ldx      #0
1018     cmp      #10
1019     bcc      GCD4
1020     sbc      #10
1021     tnx
1022     bne      GCD3
1023     pha
1024     txi
1025     beq      GCD4A
1026     clc
1027     adc      #'0'
1028     lda      #RSTAT*FNTCOL+62
1029     jsr      GDCCEL
1030     pla
1031     ldx      #0
1032     cmp      #10
1033     bcc      GCD4
1034     sbc      #10
1035     tnx
1036     bne      GCD3
1037     pha
1038     txi
1039     beq      GCD4A
1040     clc
1041     adc      #'0'
1042     lda      #RSTAT*FNTCOL+63
1043     jsr      GDCCEL
1044     pla
1045     ldx      #0
1046     cmp      #10
1047     bcc      GCD4
1048     sbc      #10
1049     tnx
1050     bne      GCD3
1051     pha
1052     txi
1053     beq      GCD4A
1054     clc
1055     adc      #'0'
1056     lda      #RSTAT*FNTCOL+64
1057     jsr      GDCCEL
1058     pla
1059     ldx      #0
1060     cmp      #10
1061     bcc      GCD4
1062     sbc      #10
1063     tnx
1064     bne      GCD3
1065     pha
1066     txi
1067     beq      GCD4A
1068     clc
1069     adc      #'0'
1070     lda      #RSTAT*FNTCOL+65
1071     jsr      GDCCEL
1072     pla
1073     ldx      #0
1074     cmp      #10
1075     bcc      GCD4
1076     sbc      #10
1077     tnx
1078     bne      GCD3
1079     pha
1080     txi
1081     beq      GCD4A
1082     clc
1083     adc      #'0'
1084     lda      #RSTAT*FNTCOL+66
1085     jsr      GDCCEL
1086     pla
1087     ldx      #0
1088     cmp      #10
1089     bcc      GCD4
1090     sbc      #10
1091     tnx
1092     bne      GCD3
1093     pha
1094     txi
1095     beq      GCD4A
1096     clc
1097     adc      #'0'
1098     lda      #RSTAT*FNTCOL+67
1099     jsr      GDCCEL
1100     pla
1101     ldx      #0
1102     cmp      #10
1103     bcc      GCD4
1104     sbc      #10
1105     tnx
1106     bne      GCD3
1107     pha
1108     txi
1109     beq      GCD4A
1110     clc
1111     adc      #'0'
1112     lda      #RSTAT*FNTCOL+68
1113     jsr      GDCCEL
1114     pla
1115     ldx      #0
1116     cmp      #10
1117     bcc      GCD4
1118     sbc      #10
1119     tnx
1120     bne      GCD3
1121     pha
1122     txi
1123     beq      GCD4A
1124     clc
1125     adc      #'0'
1126     lda      #RSTAT*FNTCOL+69
1127     jsr      GDCCEL
1128     pla
1129     ldx      #0
1130     cmp      #10
1131     bcc      GCD4
1132     sbc      #10
1133     tnx
1134     bne      GCD3
1135     pha
1136     txi
1137     beq      GCD4A
1138     clc
1139     adc      #'0'
1140     lda      #RSTAT*FNTCOL+70
1141     jsr      GDCCEL
1142     pla
1143     ldx      #0
1144     cmp      #10
1145     bcc      GCD4
1146     sbc      #10
1147     tnx
1148     bne      GCD3
1149     pha
1150     txi
1151     beq      GCD4A
1152     clc
1153     adc      #'0'
1154     lda      #RSTAT*FNTCOL+71
1155     jsr      GDCCEL
1156     pla
1157     ldx      #0
1158     cmp      #10
1159     bcc      GCD4
1160     sbc      #10
1161     tnx
1162     bne      GCD3
1163     pha
1164     txi
1165     beq      GCD4A
1166     clc
1167     adc      #'0'
1168     lda      #RSTAT*FNTCOL+72
1169     jsr      GDCCEL
1170     pla
1171     ldx      #0
1172     cmp      #10
1173     bcc      GCD4
1174     sbc      #10
1175     tnx
1176     bne      GCD3
1177     pha
1178     txi
1179     beq      GCD4A
1180     clc
1181     adc      #'0'
1182     lda      #RSTAT*FNTCOL+73
1183     jsr      GDCCEL
1184     pla
1185     ldx      #0
1186     cmp      #10
1187     bcc      GCD4
1188     sbc      #10
1189     tnx
1190     bne      GCD3
1191     pha
1192     txi
1193     beq      GCD4A
1194     clc
1195     adc      #'0'
1196     lda      #RSTAT*FNTCOL+74
1197     jsr      GDCCEL
1198     pla
1199     ldx      #0
1200     cmp      #10
1201     bcc      GCD4
1202     sbc      #10
1203     tnx
1204     bne      GCD3
1205     pha
1206     txi
1207     beq      GCD4A
1208     clc
1209     adc      #'0'
1210     lda      #RSTAT*FNTCOL+75
1211     jsr      GDCCEL
1212     pla
1213     ldx      #0
1214     cmp      #10
1215     bcc      GCD4
1216     sbc      #10
1217     tnx
1218     bne      GCD3
1219     pha
1220     txi
1221     beq      GCD4A
1222     clc
1223     adc      #'0'
1224     lda      #RSTAT*FNTCOL+76
1225     jsr      GDCCEL
1226     pla
1227     ldx      #0
1228     cmp      #10
1229     bcc      GCD4
1230     sbc      #10
1231     tnx
1232     bne      GCD3
1233     pha
1234     txi
1235     beq      GCD4A
1236     clc
1237     adc      #'0'
1238     lda      #RSTAT*FNTCOL+77
1239     jsr      GDCCEL
1240     pla
1241     ldx      #0
1242     cmp      #10
1243     bcc      GCD4
1244     sbc      #10
1245     tnx
1246     bne      GCD3
1247     pha
1248     txi
1249     beq      GCD4A
1250     clc
1251     adc      #'0'
1252     lda      #RSTAT*FNTCOL+78
1253     jsr      GDCCEL
1254     pla
1255     ldx      #0
1256     cmp      #10
1257     bcc      GCD4
1258     sbc      #10
1259     tnx
1260     bne      GCD3
1261     pha
1262     txi
1263     beq      GCD4A
1264     clc
1265     adc      #'0'
1266     lda      #RSTAT*FNTCOL+79
1267     jsr      GDCCEL
1268     pla
1269     ldx      #0
1270     cmp      #10
1271     bcc      GCD4
1272     sbc      #10
1273     tnx
1274     bne      GCD3
1275     pha
1276     txi
1277     beq      GCD4A
1278     clc
1279     adc      #'0'
1280     lda      #RSTAT*FNTCOL+80
1281     jsr      GDCCEL
1282     pla
1283     ldx      #0
1284     cmp      #10
1285     bcc      GCD4
1286     sbc      #10
1287     tnx
1288     bne      GCD3
1289     pha
1290     txi
1291     beq      GCD4A
1292     clc
1293     adc      #'0'
1294     lda      #RSTAT*FNTCOL+81
1295     jsr      GDCCEL
1296     pla
1297     ldx      #0
1298     cmp      #10
1299     bcc      GCD4
1300     sbc      #10
1301     tnx
1302     bne      GCD3
1303     pha
1304     txi
1305     beq      GCD4A
1306     clc
1307     adc      #'0'
1308     lda      #RSTAT*FNTCOL+82
1309     jsr      GDCCEL
1310     pla
1311     ldx      #0
1312     cmp      #10
1313     bcc      GCD4
1314     sbc      #10
1315     tnx
1316     bne      GCD3
1317     pha
1318     txi
1319     beq      GCD4A
1320     clc
1321     adc      #'0'
1322     lda      #RSTAT*FNTCOL+83
1323     jsr      GDCCEL
1324     pla
1325     ldx      #0
1326     cmp      #10
1327     bcc      GCD4
1328     sbc      #10
1329     tnx
1330     bne      GCD3
1331     pha
1332     txi
1333     beq      GCD4A
1334     clc
1335     adc      #'0'
1336     lda      #RSTAT*FNTCOL+84
1337     jsr      GDCCEL
1338     pla
1339     ldx      #0
1340     cmp      #10
1341     bcc      GCD4
1342     sbc      #10
1343     tnx
1344     bne      GCD3
1345     pha
1346     txi
1347     beq      GCD4A
1348     clc
1349     adc      #'0'
1350     lda      #RSTAT*FNTCOL+85
1351     jsr      GDCCEL
1352     pla
1353     ldx      #0
1354     cmp      #10
1355     bcc      GCD4
1356     sbc      #10
1357     tnx
1358     bne      GCD3
1359     pha
1360     txi
1361     beq      GCD4A
1362     clc
1363     adc      #'0'
1364     lda      #RSTAT*FNTCOL+86
1365     jsr      GDCCEL
1366     pla
1367     ldx      #0
1368     cmp      #10
1369     bcc      GCD4
1370     sbc      #10
1371     tnx
1372     bne      GCD3
1373     pha
1374     txi
1375     beq      GCD4A
1376     clc
1377     adc      #'0'
1378     lda      #RSTAT*FNTCOL+87
1379     jsr      GDCCEL
1380     pla
1381     ldx      #0
1382     cmp      #10
1383     bcc      GCD4
1384     sbc      #10
1385     tnx
1386     bne      GCD3
1387     pha
1388     txi
1389     beq      GCD4A
1390     clc
1391     adc      #'0'
1392     lda      #RSTAT*FNTCOL+88
1393     jsr      GDCCEL
1
```

SRC/BSNET.ASM

```

1 ; bsnet.asm -- bank 2: one request, with the picture
  up, and back.
2 ;
3 ; This bank exists for a size reason. net.inc and
  url.inc are about 550 bytes
4 ; together, every bank carries its own copy of
  everything it calls, and the
5 ; game bank needs: the board kernel, the composer and
  the input. A poll
6 ; happens every ninety frames, so paying a bank
  switch for one costs nothing.
7 ;
8 ; IT CARRIES THE BOARD KERNEL, though it composes
  nothing. A transaction is
9 ; several socket round trips and the transport's
  wait for ACKSEQ is a spin
10 ; loop; spun blind, the picture stops for the
  duration and the set sees a
11 ; Flash once a poll. net.inc spends every wait in
  DFRAME instead, redrawing
12 ; the boards out of the tables the cartridge is
  still holding -- the reply
13 ; window is not repainted until the READ, so what is
  on screen stays true.
14 ;
15 ; THE SWITCH IN IS TAKEN INSIDE THE VBLANK HOOK: the
  game bank switches here
16 ; from its hook and this bank finishes that frame
  (DFRAME2) before it starts
17 ; talking. THE SWITCH OUT IS TAKEN BETWEEN FRAMES,
  in the overscan the last
18 ; frame's DFRAME2 armed: the composer's work and the
  game bank's re-entry
19 ; both fit in it. Every frame across a poll is 262
  lines, which is what "no
20 ; shudder" means.
21 ;
22 ; The lobby's /tables and the menu's /leave come in
  blanked, on a screen
23 ; change, and are spun blind: there is nothing to
  keep up.
24 ;
25 CPU 6502
26 INCLUDE "vcs.inc"
27 INCLUDE "fujinet.inc"
28 INCLUDE "bsdefs.inc"
29 ;
30 BSBANK EQU BANKNET
31 BSHASIMP EQU 0 ; it draws nothing
  and reads no input
32 BSHASUT EQU 0 ; ...and touches no
  player record
33 BSHASED EQU 0
34 BSHASNET EQU 1 ; it is the only
  bank that issues requests
35 BSHASCLS EQU 0
36 BSHASSTR EQU 0
37 BSHASRPL EQU 0
38 BSHASDEC EQU 0
39 ;
40 INCLUDE "../build/tail.inc"
41 ;
42 ORG $1000
43 ;
44 ;-----
45 NENTRY: lda BSENT
46 cmp #ENTABLE
47 bne NE1
48 jmp NTABLE
49 NE1: cmp #ENLEAVE
50 bne NE2
51 jmp NLEAVE
52 NE2:
53 ;
54 ;-----
55 ; A game request. The screen is mid-frame: finish it
  first.
56 jsr DFRAME2
57 ;
58 ; A STAGED REQUEST RIDES THE POLL: /attack, /ready
  and /place all return the
59 ; next state, so there is no separate submit
  anywhere in this client.
60 lda BSREQ2
61 jsr APICALL
62 sta BSERR2
63 lda #RQSTATE
64 sta BSREQ2 ; sent, or
  abandoned; either way not re-sent
65 ;
66 lda BSERR2
67 beq NGOOD
68 ; A failure leaves the reply window alone, so the
  boards on screen are still
69 ; the last good ones. Back off and let the game bank
  say so.
70 lda #FAILFRM
71 sta BSPOLL
72 lda BSREQ
73 cmp #RQPLACE
74 bne NBACKG
75 lda #ENPLFAIL ; the placement was
  refused: place again
76 sta BSENT
77 lda #BANKPLC
78 jmp BSGOTO
79 ;
80 ; The poll cadence follows the phase: the lobby is
  slow, play is brisk, the
81 ; game-over screen lingers until the server resets
  the table.
82 NGOOD: lda #0
83 sta BSERR
84 lda #POLLFRM
85 ldx BSSTAT
86 beq NG1 ; the lobby
87 cpx #STOVER
88 bne NG2
89 lda #OVERFRM
90 jmp NG2
91 NG1: lda #LOBFRM
92 NG2: sta BSPOLL
93 ; Placement is not a choice, so it does not wait for
  a button. The test is
94 ; BOTH the game's phase and YOUR status: the phase
  stays STPLACE until
95 ; everyone has placed, so the phase alone would send
  you back to the
96 ; placement screen the moment you had finished.
97 lda BSSTAT
98 cmp #STPLACE
99 bne NBACKG
100 lda BSMYST
101 cmp #PSPLACE
102 bne NBACKG
103 lda #ENPLACE
104 sta BSENT
105 lda #BANKPLC
106 jmp BSGOTO
107 ; The reply came in during the last frame's overscan
  -- NPGO checks once a
108 ; frame, after DFRAME has returned -- so the switch
  is taken HERE, between
109 ; frames, and the composer's work lands inside that
  overscan.
110 NBACKG: lda #ENGAME
111 sta BSENT
112 lda #BANKCMP
113 jmp BSGOTO
114 ;
115 ;-----
116 ; The lobby's table list, blanked.
117 NTABLE: lda #2
118 sta VBLANK
119 lda #RQTABLE
120 jsr APICALL
121 beq NTOK
122 lda #0
123 sta BSCNT ; no tables to show
124 jmp NBACKL
125 NTOK: lda #0
126 sta BSERR
127 NBACKL: lda #ENLOBBY
128 sta BSENT
129 lda #BANKLOB
130 jmp BSGOTO
131 ;
132 ;-----
133 ; Leaving. The reply is a game and is thrown away;
  what matters is that the
134 ; server frees the seat. Then STRAIGHT ON TO A
  FRESH /tables, because the
135 ; list the lobby is still holding has us sitting at
  one of them -- and
136 ; falling into NTABLE is also what puts the lobby up
  WARM rather than cold.
137 NLEAVE: lda #2
138 sta VBLANK
139 lda #RQLEAVE
140 jsr APICALL
141 jmp NTABLE
142 ;
143 ;-----
144 ; APPVBL -- nothing. Every frame this bank draws is
  drawn from INSIDE a
145 ; transaction, and polling for another here would
  re-enter APICALL through
146 ; its own settle loop.
147 APPVBL: rts
148 ;
149 INCLUDE "bslib.inc"
150 INCLUDE "net.inc"
151 INCLUDE "url.inc"
152 INCLUDE "state.inc"
153 INCLUDE "dispgame.inc"
154 END
155 ;
156 ;

```

SRC/BSMENU.ASM

```

1 ; bsmenu.asm -- bank 3: the in-game menu and the
2 help screen.
3 ; Reached by the RESET switch during a game. The
4 2600 has one button and four
5 ; directions and no keypad; the stick aims and FIRE
6 shoots, SELECT polls,
7 ; and RESET is the only labelled button going spare.
8 It is a SWITCH on this
9 ; console, SWCHB bit 0, which the program reads: it
10 restarts nothing, and what
11 ; it means is the client's to choose. RESET again
12 backs out of here, for the
13 ; same reason a door opens both ways.
14
15 ; The game bank switched here from inside its vblank
16 hook, so this bank
17 ; finishes that frame after it has drawn the menu;
18 the menu's rows take more
19 ; than a vblank to compose, so that one frame runs
20 long, once, on a screen
21 ; change.
22
23 CPU 6502
24 INCLUDE "vcs.inc"
25 INCLUDE "fujinet.inc"
26 INCLUDE "bsdef.inc"
27
28 BSBANK EQU BANKNMU
29 BSHASINP EQU 1
30 BSHASUI EQU 0 ; no player record
31
32 is read here
33 BSHASED EQU 0
34 BSHASNET EQU 0 ; /leave is
35
36 BANKNET's, like every request
37 BSHASCLS EQU 1
38 BSHASSTR EQU 1
39 BSHASRPL EQU 0
40 BSHASDEC EQU 0
41
42 INCLUDE "../build/tail.inc"
43
44 RMITIL EQU 0
45 RMITEM0 EQU 3
46 RMITEMS EQU 3
47 RMHELP0 EQU 9
48
49 MIRE EQU 0 ; resume
50 MIHE EQU 1 ; how to play
51 MILEAVE EQU 2 ; leave the table
52
53 ORG $1000
54
55 MENTRY: jsr DINIT
56 lda FNCLS
57 lda #0
58 sta BSSEL
59 sta BSMHELP
60 sta AUDV0 ; whatever cue was
61 playing
62 sta SNDPTR
63 jsr MDRAW
64 jsr DFRAME2
65 jmp DLOOP
66
67 APPVBL: jsr INREPT
68 sta BSINP
69
70 ; The help screen swallows everything but a button,
71 which closes it.
72
73 lda BSMHELP
74 beq MAV0
75 lda BSINP
76 and #IN_FIRE|IN_RST
77 beq MAVR
78 lda #0
79 sta BSMHELP
80 jsr FNCLS
81 jmp MDRAW
82
83 MAV0: lda BSINP
84 and #IN_UP
85 beq MAV1
86 lda BSSEL
87 beq MAV1
88 dec BSSEL
89 jsr MDRAW
90 lda BSINP
91 and #IN_DOWN
92 beq MAV2
93 lda BSSEL
94 cmp #RMITEMS-1
95 bcs MAV2
96 tnc BSSEL
97 jsr MDRAW
98 MAV2: lda BSINP
99 and #IN_RST ; RESET again backs
100 out, as it came
101 bne MRESUME
102 lda BSINP
103 and #IN_FIRE
104 beq MAVR
105 lda BSSEL
106 cmp #MIHELP
107 beq MHLEAVE
108 cmp #MILEAVE
109 beq MLEAVE
110
111 ; Resuming goes back to the game bank as if from a
112 poll: it recomposes the
113 ; boards from the reply window, which still holds
114 the last state, and its
115 ; poll clock is left at zero so the first frame back
116 fetches a fresh one.
117 MRESUME: lda #MRESUME
118 sta BSNT
119 lda BSENT
120
121 sta BSERR2 ; resuming is not a
122 failed poll
123 sta BSPOLL
124 lda #BANKCMP
125 jmp BSGOTO
126 MAVR: rts
127
128 MHLEP: lda #1
129 sta BSMHELP
130 lda #FFF
131 sta BSBAR
132 jsr FNCLS
133 jmp MDRAW
134
135 ; Leaving is a request, and requests live in
136 BANKNET. It sends /leave and goes
137 ; on to the lobby whatever the server says: the seat
138 is given up either way,
139 ; and a client that refused to leave because the
140 link hiccupped would be stuck.
141 MLEAVE: lda #ENLEAVE
142 sta BSENT
143 lda #BANKNET
144 jmp BSGOTO
145
146 MDRAW: lda BSMHELP
147 bne MHLEP
148
149 lda #RMITIL
150 jsr FNROWA
151 lda # (RMITIL)&FFF
152 ldy # (RMITIL)>>8
153 jsr FNSETP
154 jsr FNSTRA
155 jsr FNENDW
156
157 lda #0
158 sta BSIDX
159 lda BSIDX
160 clc
161 adc #RMITEM0
162 jsr FNROWA
163 lda BSIDX
164 asl a
165 asl a
166 sta BSTMP
167
168 item
169 asl a
170 clc
171 adc BSTMP
172 tax
173 ldy #FNTOCOL
174 lda MSITEMS,x
175 sta FNSEL+FH_TCHR
176
177 inx
178 dey
179 bne MD2
180 jsr FNENDW
181 tnc BSIDX
182 lda BSIDX
183 cmp #RMITEMS
184 bne MD1
185
186 lda BSSEL
187 clc
188 adc #RMITEM0
189 sta BSBAR
190 rts
191
192 ; The help screen: a list of rows in ROM, terminated
193 by a zero length.
194 MDHELP: lda #0
195 sta BSIDX
196 MDH1: lda BSIDX
197
198 adc #RMHELP0-RMHELP0
199 sta BSTMP
200 lda BSIDX
201 asl a
202 sta BSNUM0
203 asl a
204 clc
205 adc BSNUM0 ; twelve bytes a row
206 tax
207 lda MSHELP,x
208 beq MDH2 ; a zero first byte
209
210 ends the list
211 lda BSIDX
212 jsr FNROWA
213 ldy #FNTOCOL
214 lda MSHELP,x
215 sta FNSEL+FH_TCHR
216 tnx
217 dey
218 bne MDH1A
219 jsr FNENDW
220 tnc BSIDX
221 lda BSIDX
222 cmp #FNTRW
223 bne MDH1
224 rts
225
226 MDH2: DB "GAME MENU",0
227 MSITEMS: DB "RESUME"
228 DB "HOW TO PLAY"
229 DB "LEAVE TABLE"
230
231 ; Twelve columns a row, padded, terminated by a row
232 that starts with a NUL.
233
234 ; It exists for one sentence: A SHOT LANDS ON EVERY
235 ENEMY AT ONCE. That is
236 ; the rule this game does not share with the board
237 game, and nothing on the
238 ; play screen can teach it.
239 MSHELP: DB "HOW TO PLAY"

```

SRC/BSNAME.ASM

172

```

136 NMCHR:   jsr    NMGLYPH
137 NMAADD:  ldx    BSNLEN
138          cpx    #NAMEMAX
139          bcs    NMR
140          sta    FNRSEL+FH_PATHC
141          tnc    BSNLEN
142          jmp    NMDRAW
143 NMBS:     ldx    BSNLEN
144          beq    NMR
145          jsr    FNWPOP
146          dec    BSNLEN
147          jmp    NMDRAW
148 NMR:      rts
149
150 ; OK with nothing typed is not OK: the server would
151 ; see an empty player.
152 NMOK:     ldx    BSNLEN
153          jsr    AKWRITE
154          table works without it
155          ldx    #ENTABLE
156          sta    BSENT
157          lda    #BANKNET
158          jmp    BSGOTO
159
160 ; NMGLYPH -- A = the character at grid cell BSSEL.
161 NMGLYPH:  ldx    BSSEL
162          lda    KCHARS,x
163          rts
164 ;
165 ; ----- the whole keyboard. -----
166 NMDRAW:   ldx    #RNTITL
167          jsr    FNROWA
168          ldx    #(NSTITL)&$FF
169          # (NSTITL)>>8
170          jsr    FNSETP
171          jsr    FNSTRA
172          jsr    FNENDW
173
174 ; What has been typed, rendered by the CARTRIDGE out
175 ; of the buffer holding it.
176 ; The console cannot read that buffer back -- the
177 ; page it is written through is
178 ; write-only -- so FB_PATH is the only way to see it
179 ; at all.
180          ldx    #PBNAME
181          jsr    FNWSL
182          ldx    #0
183          sta    FNRSEL+FH_BSL
184          sta    FNRSEL+FH_BSH
185          sta    FNRSEL+FH_BDH
186          ldx    #RNRNAME
187          sta    FNRSEL+FH_BDL
188          ldx    #FNTCOL
189          sta    FNRSEL+FH_BCNT
190          ldx    #FNB_PATH
191          jsr    FNBLIT
192
193 ; The grid.
194          ldx    #0
195          sta    BSIDX
196          ldx    BSIDX
197          clc
198          adc    #NRGRID
199          jsr    FNROWA
200          ldx    BSIDX
201          asl    a
202          sta    BSTMP
203          asl    a
204          clc
205          adc    BSTMP
206          ldx    #KGRIDW
207          sta    KCHARS,x
208          sta    FNRSEL+FH_TCHR
209          tnc    NMD2
210          jsr    FNENDW
211          tnc    BSIDX
212          ldx    BSIDX
213          cmp    #KGRIDH
214          bne    NMD1
215
216          ldx    #RNRCTRL
217          jsr    FNROWA
218          ldx    #(NSCTRL)&$FF
219          # (NSCTRL)>>8
220          jsr    FNSETP
221          jsr    FNSTRA
222          jsr    FNENDW
223
224 ; The cursor. A whole-row red bar says WHICH ROW,
225 ; and this row says which cell
226 ; in it -- there is no inverse video on this machine
227 ; and no per-cell colour,
228 ; so one highlight cannot say both.
229          ldx    #RNPICK
230          jsr    FNROWA
231          ldx    #>
232          jsr    FNCHR
233          ldx    BSSEL
234          cmp    #KCELLS
235          bcs    NMD3
236          jsr    NMGLYPH
237          jsr    FNCHR
238          ldx    #'<
239          jsr    FNCHR
240          jmp    NMD4
241
242 NMD3:     sec
243          sbc    #KCELLS
244          asl    a
245          sta    a
246          ; four bytes a
247          control word
248          tax
249          ldy    #4
250          NMD3A: ldx    NSCTLW,x
251          sta    FNRSEL+FH_TCHR
252
253          tnx
254          dey
255          bne    NMD4
256          jsr    FNENDW
257          bcc    NMD5
258          ldx    #0
259          cmp    #KGRIDW
260          bcc    NMD5B
261          sec
262          sbc    #KGRIDW
263          tnx
264          jmp    NMD5A
265          clc
266          adc    #RNRGRID
267          sta    BSBAR
268          ldx    #RNFOT
269          jsr    FNROWA
270          ldx    #(NSFOOT)&$FF
271          # (NSFOOT)>>8
272          jsr    FNSETP
273          jsr    FNSTRA
274          jmp    FNENDW
275
276 ; -----
277 ; AKOPEN -- OPEN_APPKEY with mode A. Returns 0 if
278 ; the adapter took it.
279 ; The payload is a SIX-BYTE PACKED STRUCT and not
280 ; four parameters:
281 ; AppKeyMixin::appkey_open() does one
282 ; transaction_get() of sizeof(appkey),
283 ; which is creator as a ul6, then app, key, mode and
284 ; a reserved byte. The
285 ; Intellivision port documents what happens when
286 ; that last byte is left off --
287 ; transaction_get() waits for a byte that never
288 ; arrives, and it reads back as
289 ; a timeout rather than as a protocol error.
290 ;
291 ; It also fails outright when no SD card is mounted
292 ; on the adapter, which is
293 ; not a client bug and is why the keyboard is the
294 ; fallback rather than an
295 ; error screen.
296 AKOPEN:   pha
297          ldx    #FNDEVF
298          jsr    FNDEV
299          ldx    #FCAKOPN
300          sta    FNCD
301          ldx    #0
302          jsr    FNPNR
303          ldx    #FNBE
304          ldx    #(AKCREAT)&$FF
305          sta    FNTX
306          ldx    #(AKCREAT)>>8
307          sta    FNTX
308          ldx    #AKAPP
309          sta    FNTX
310          ldx    #AKKEY
311          sta    FNTX
312          ; optional
313          jsr    FNGO
314          bne    AKOX
315          jmp    FNACK
316          rts
317
318 ; AKREAD -- the shared username into path buffer 1.
319 ; Z set if there was one.
320 ; The name goes straight from the reply into a
321 ; cartridge path buffer: 128
322 ; bytes of console RAM, of which the stack owns the
323 ; top, cannot hold nine
324 ; bytes it does not have to.
325 AKREAD:   ldx    #AKMRD
326          jsr    AKOPEN
327          bne    AKBAD
328          ldx    #FNDEVF
329          jsr    FNDEV
330          ldx    #FCAKRD
331          sta    FNCD
332          ldx    #0
333          jsr    FNPNR
334          ldx    #FNBE
335          jsr    FNGO
336          bne    AKBAD
337          jsr    FNACK
338          bne    AKBAD
339          ; The reply is a length word and then the value.
340          ldx    #FNRPLY+0
341          ldx    #FNRPLY+1
342          beq    AKBAD
343          ; the slot is empty:
344          type one
345          ldx    #PBNAME
346          jsr    FNWSL
347          ldx    #FNRWST
348          ldx    #2
349          ldy    #NAMEMAX+1
350          jsr    FNWRPL
351          ldx    #0
352          rts
353          ; AKBAD: rts
354          # $FF
355          rts
356
357 ; AKWRITE -- store what was typed, so the next
358 ; FujiNet game finds it.

```

```

353 ; Best effort: a table works perfectly well without
    the slot being written.
354 AKWRITE: lda    #AKMWR
355           jsr    AKOPEN
356           bne    AKWX
357           lda    #FNDEVF
358           sta    FNDEV
359           lda    #FCAKWR
360           sta    FNCMD
361           lda    #1
362           sta    FNNPR
363           jsr    FNBEG
364           lda    BSNLEN
365           jsr    FNPB
366           lda    #PBNAME
367           jsr    FNNSEL
368           jsr    FNNRAW
369           jsr    FNGO
370 AKWX:     rts
371
372 NSTITL: DB    "YOUR NAME",0
373 NSCTRL: DB    "OK DEL SPC",0
374 NSCTLW: DB    "OK DEL SPC "
375 NSFOOT: DB    "FIRE PICKS",0
376 ; A-Z then 0-9: thirty-six cells in three rows of
    twelve, which is the one
377 ; shape that comes out even in this many columns.
378 KCHARS: DB    "ABCDEFHIJKL"
379           DB    "MNOPQRSTUVWXYZ"
380           DB    "YZ0123456789"
381
382 BS8GT: DB
    CBLACK,CBLACK,CBLACK,CBLACK,CBLACK,CBLACK
383 DB
    CBLACK,CBLACK,CBLACK,CBLACK,CBLACK,CBLACK
384 DB
    CBLACK,CBLACK,CBLACK,CBLACK,CBLACK,CBLACK
385 DB
    CBLACK,CBLACK,CBLACK
386 DB
    CWATER
387
388 INCLUDE "bslib.inc"
389 INCLUDE "state.inc"
390 INCLUDE "disptext.inc"
391
392 END
393

```

SRC/BSPLACE.ASM

```

1 ; bsplace.asm -- bank 5: put your five ships on the
    water.
2 ;
3 ; The five placements are staged in console RAM --
    five bytes at SHIPS, each
4 ; pos + 100*dir, the server's own encoding -- and
    sent when all five are
5 ; down, through BANKNET like every request. The
    board shows them as the
6 ; cartridge composes them: every hull as a bracket,
    the one being placed
7 ; blinking, all through FB_PFCEL one cell at a time.
8 ;
9 ; A RANDOM LEGAL FLEET IS ROLLED FIRST AND THEN
    ADJUSTED, which is what every
10 ; sibling does and is the right way round: the
    player is never made to solve
11 ; the packing problem from an empty board, the whole
    fleet is on the board
12 ; before a finger moves, and five presses accept the
    roll as it stands. The
13 ; stick moves the ship whose turn it is FROM WHERE
    THE ROLL PUT IT.
14 ;
15 ; THE ROLL IS ONE CANDIDATE A FRAME, in the vblank
    hook, not a loop in
16 ; PENTRY. A candidate costs up to about 1,650 cycles
    -- the hunt has to walk
17 ; every hull already down -- and a whole fleet is
    thousands more than the
18 ; two thousand two hundred an entry between frames
    has. Spread, the worst
19 ; frame is one candidate and the picture never
    moves; a fleet takes about ten
20 ; frames to land, which is a sixth of a second and
    looks like a deal. Nothing
21 ; reads the stick until it is down.
22 ;
23 ; Bounds AND overlaps are checked here now. Bounds
    always were -- a hull
24 ; running off the edge is a thing the player can see
    going wrong. Overlaps
25 ; used to be left to the server, which cost a
    refusal, a round trip, and all
26 ; five ships redone; the roll has to know about them
    anyway, so the same test
27 ; refuses a move that would sit on another hull,
    with a tone, for free.
28
29 CPU    6502
30 INCLUDE "vcs.inc"
31 INCLUDE "fujinet.inc"
32 INCLUDE "bsdefs.inc"
33
34 BSBANK EQU    BANKPLC
35 BSHASINP EQU 1
36 BSHASUIT EQU 0 ; your own board
    only; no PLRECP, no GCURS
37 BSHASED EQU 0
38 BSHASNET EQU 0
39 BSHASCLS EQU 0
40 BSHASSTR EQU 1
41 BSHASRPL EQU 0
42 BSHASDEC EQU 0
43 SNOFULL EQU 1
44 SNOLAST EQU 3 ; move, select,
    place, refused
45
46 INCLUDE "../build/tail.inc"
47
48 ; The hull walk's cells, live only inside PHULLS and
    POVL -- one walk at a
49 ; time, which is why three are enough. The game
    bank's clock cells, which it
50 ; reloads on its next compose -- NOT its edge memory
    at $AE-$80, which has to
51 ; survive placement or the first poll after it hears
    a phantom shot.
52 BSPOS EQU    BSLCK
53 BSSEG EQU    BSTICK
54 BSPDIR EQU    BSPOLL
55 ; The roll's own cells, on the same terms. BSSEL is
    the lobby's list cursor
56 ; and the keyboard's cell; BSERR2 is APICALL's
    result, which the network bank
57 ; writes before anything reads it again; BSBLINK is
    zeroed after the roll.
58 BSRNDL EQU    BSSEL
59 BSRNDH EQU    BSERR2
60 BSCND EQU    BSTMP2 ; the candidate, pos
    + 100*dir. CELLNO ; clobbers BSTMP2
61
62 and is called before it
63 ORG    $1000
64
65 ; The seed. FNACKS is the cartridge's sequence echo
    -- transactions have run
66 ; by the time anyone places, so it differs from to
    run -- and BSFRAME counts
67 ; how long the lobby took, which is human. INTIM
    would be the obvious third,
68 ; and is not read here: reading it CLEARS the TIMINT
    latch the kernel waits
69 ; on, and this runs inside an overscan that is
    already armed.
70 PENTRY: lda    FNACKS
71           eor    BSFRAME
72           sta    BSRNDL
73           lda    FNRPY+GOMVTIM
74           eor    #SAS
75           ora    #1 ; an all-zero LFSR
    never leaves zero
76           lda    BSRNDH
77           ldx    #1 ; the layout follows
    the seat count
78           ldx    BSPCNT
79           cpx    #3
80           bcs    PE2
81           ldx    #0
82 PE2:     sta    BSMODE ; no DINIT: the
    network bank's kernel is this
83           ldx    #NSHIPS-1 ; one, and the TIA
    is as it left it
84           lda    #$FF ; an empty slate for
    the roll to fill, a ship
85 PE1:     sta    SHIPS,x ; a frame
    only; no PLRECP, no GCURS
86           dex
87           bpl    PE1
88           ldx    #0
89           sta    BSHSHIP
90           sta    BSDIR
91           sta    BSBLINK
92           sta    BSCURX
93           sta    BSCURY
94           sta    BSLLIVE
95           jsr    PCLEAR
96           jsr    PTEXT
97           ldx    BSENT
98           cmp    #ENPLFAIL

```

```

99     bne      PE3
100     ldx      # (TAGAIN)&$FF ; the server refused
101     the last set
102     ldy      # (TAGAIN)>>8
103     jsr      PSTAT
104     PE3: jsr      PHULLS ; all of it inside
105     the overscan the network
106     PRUN: jsr      DFRAME ; bank's last
107     frame armed
108     jmp      PRUN
109     ;
110     -----
111     APPVBL: jsr      SNTDICK
112     ldx      SHIPS-SHIPS-1 ; the last slot
113     filled is the roll's flag
114     cmp      # $FF
115     bne      PA0
116     jmp      PRORLF
117     PA0: jsr      INREPT
118     sta      BSINP
119     and      #IN_RST
120     lda      #ENMENU
121     sta      BSENT
122     lda      #BANKMENU
123     jmp      BSGOTO
124     PA1: ldx      BSINP
125     and      #INDIRS
126     beq      PA2
127     jmp      PMOVE
128     PA2: ldx      PHULLS
129     bne      BSINP
130     #IN_SEL
131     beq      PA3
132     lda      BSDIR ; rotate
133     eor      #1
134     sta      BSDIR
135     lda      #SNDMOVE
136     jsr      SndsOFT
137     jmp      PHULLS
138     PA3: ldx      BSINP
139     and      #IN_FIRE
140     beq      PA4
141     jmp      PPLACE
142     PA4: ldx      BSBLINK ; the pending ship
143     bne      #BSLINKFR-1
144     PAX: jsr      PHULLS
145     rts
146     PMOVE: ldx      BSINP
147     lsr      a
148     bcc      PM1
149     lda      BSCURY
150     beq      PM4
151     dec      BSCURY
152     jmp      a
153     PM1: lsr      a
154     bcc      PM2
155     lda      BSCURY
156     cmp      #BRDDIM-1
157     bcs      PM4
158     inc      BSCURY
159     jmp      PM4
160     PM2: lsr      a
161     bcc      PM3
162     lda      BSCURX
163     beq      PM4
164     dec      BSCURX
165     jmp      PM4
166     PM3: ldx      BSCURX ; right
167     cmp      #BRDDIM-1
168     bcs      PM4
169     inc      BSCURX
170     PM4: ldx      #SNDMOVE
171     jmp      SndsOFT
172     ;
173     -----
174     PPLACE -- stage the current ship, and send them
175     all once the last is down.
176     PPLACE: jsr      FITS
177     bne      PBAD ; off the edge:
178     leave it where it was
179     jsr      PCAND
180     sta      BSCND
181     bne      PBAD0
182     likewise
183     ldx      BSCND
184     ldx      SHIPS,x
185     tnx
186     cpx      #NSHIPS
187     bcs      PSEND
188     stx      BSSHIP
189     jsr      PLOAD ; the next ship,
190     where the roll put it
191     ldx      #SNDPLC
192     jsr      SNDFIRE
193     jsr      PTEXT
194     jmp      PHULLS
195     PSEND: ldx      #SNDSEL
196     jsr      SNDFIRE
197     ldx      #RQPLACE
198     sta      BSREQ2
199     lda      #ENFETCH
200     sta      BSENT
201     lda      #BANKNET
202     jmp      BSGOTO
203     PBAD0: ldx      # (TLAP)&$FF
204     ldy      # (TLAP)>>8
205     bne      PBADS ; always: neither
206     string is at $xx00
207     PBAD: ldx      # (TOFF)&$FF
208     ldy      # (TOFF)>>8
209     -----
210     PBADS: sta      BSTMP ; SNDFIRE keeps Y
211     and      BSTMP, not X
212     ldx      #SNDERR
213     jsr      SNDFIRE
214     ldx      BSTMP
215     jmp      PSTAT
216     ;
217     -----
218     FITS -- Z set if the current ship fits on the
219     board from the cursor.
220     ldx      BSSHIP
221     ldx      SHPSIZ,x
222     clc
223     adc      # $FF ; length - 1, the
224     last segment's offset
225     bsdior
226     bne      FIT1
227     clc
228     adc      BSCURX
229     jmp      FIT2
230     FIT1: ldx      BSCURX
231     adc      BSCURY
232     cmp      #BRDDIM
233     bcs      #0
234     lda      #0
235     rts
236     FIT2: cmp      #BRDDIM
237     bcs      #0
238     lda      #1
239     rts
240     ;
241     -----
242     PHULLS -- your board's AUX plane: every staged
243     ship EXCEPT the one being
244     placed, and that one on the blink's lit phase at
245     wherever the cursor has
246     ; it. Every slot is filled from the first frame now,
247     so this paints the whole
248     ; fleet -- and painting "all but the one being
249     moved" is exactly the picture
250     ; POVL reasons about.
251     PHULLS: jsr      PSLOT0
252     tax
253     stx      BSLIVE ; your slot, for the
254     cells below
255     ldx      #PFM_AUX
256     ldy      #0
257     jsr      FNBARG
258     lda      #FB_PFLCLR
259     jsr      FNBILT
260     ldx      #0
261     PH1: cpx      #NSHIPS
262     bcs      PH2
263     cpx      BSSHIP
264     beq      PH1A ; the pending one:
265     the blink below draws it
266     stx      BSIDX
267     ldx      SHIPS,x
268     jsr      HULL
269     ldx      BSIDX
270     PH1A: tnx
271     bne      PH1 ; always
272     PH2: ldx      BSBLINK
273     and      #BLINKFR
274     bne      PH9
275     PH9: tnx
276     bne      PH9 ; off the edge: show
277     nothing rather than a
278     ; PCAND ; hull wrapping
279     into the next row
280     ldx      BSSHIP
281     stx      BSIDX
282     jmp      HULL
283     PH9: rts
284     ;
285     PCAND -- A = the cursor as pos + 100*dir, the
286     server's own encoding.
287     PCAND: jsr      CELLNO ; A = y*10 + x;
288     clobbers. BSTMP2
289     ldx      BSDIR
290     beq      PCA1
291     clc
292     adc      #BRDCELL
293     PCA1: rts
294     ;
295     PDEC -- start a hull walk: BSPOS = pos and BSPDIR
296     = dir from A, which is
297     ; pos + 100*dir, and BSSEG = the length of ship X.
298     The head HULL used to
299     ; carry inline, so POVL can walk a hull too.
300     PDEC: ldy      #0
301     cmp      #BRDCELL
302     bcc      PDC1
303     sbc      #BRDCELL ; the compare left C
304     set      ldy      #1
305     PDC1: sty      BSPDIR
306     sta      BSPOS
307     lda      SHPSIZ,x
308     sta      BSSEG
309     rts
310     ;
311     PSTEP -- the walk along one cell and one segment
312     down. Z set when it is
313     ; finished. BSPOS cannot wrap: the hull was bounds-
314     checked before the walk.
315     PSTEP: ldx      BSPOS
316     clc
317     ldy      BSPDIR
318     beq      PST1
319     adc      #BRDDIM
320     bne      PST2 ; down a row
321     PST1: adc      #1 ; always: pos + 10
322     is at least 10
323     PST2: sta      BSPOS ; along
324     dec      BSSEG
325     rts
326     ;
327     HULL -- the cells of ship BSIDX placed at A (pos +
328     100*dir), into AUX.
329     HULL: ldx      BSIDX
330     jsr      PDEC
331     ldx      #PFM_AUX
332     HU2: ldx      #PFM_AUX

```

```

308     ldx     BSLIVE
309     ldy     BSPOS
310     jsr     FNBARG
311     lda     #FB_PFCCL
312     fmbli   jsr
313     jsr     PSTEP
314     bne     HU2
315     rts
316
317 ; -----
318 ; PVL: -- Z set if the candidate hull -- ship BSSHIP
319 ; at cell BSCND in
320 ; direction BSDIR -- is clear of every OTHER staged
321 ; ship.
322 ; No occupancy map: a hundred bytes is most of this
323 ; console's RAM. Instead
324 ; each staged hull is WALKED and each of its cells
325 ; asked whether the
326 ; candidate covers it, which is arithmetic. One walk
327 ; is live at a time, so
328 ; PHULLS's three cells are all it costs.
329 PVL: ldx     #NSHIPS-1
330 PVL1: cpx     BSSHIP
331     beq     PVL7
332     lda     SHIPS,x
333     cmp     #$FF
334     beq     PVL7
335 ; nothing to hit
336     stx     BSDIX
337     jsr     PDEC
338     lda     BSPPOS
339     jsr     PINC
340     bne     PVL9
341     jsr     PSTEP
342     bne     PVL2
343     ldx     BSDIX
344     dex     PVL7
345     bpl     PVL1
346     lda     #0
347 ; Z set: clear
348 PVL9: lda     #1
349     rts
350 ; PINC -- Z CLEAR if cell A is one of the
351 ; candidate's. Across, the candidate
352 ; owns BSCND .. BSCND+len-1 and cannot wrap, because
353 ; FITS said so; own it
354 ; owns BSCND plus ten times each segment.
355 PINC: sec
356     sbc     BSCND
357     bcc     PIN8
358 ; before its first
359 ; cell
360     ldx     BSSHIP
361     ldy     SHPSIZ,x
362 ; the candidate's
363 ; length, in a cell because
364 ; sty     BSTMP
365 ; there is no CPY
366 ; absolute,X
367     ldy     BSDIR
368     beq     PIN5
369     ldy     #0
370 PIN1: cmp     #BRDDIM
371     bcc     PIN4
372     sbc     #BRDDIM
373     iny
374     cpy     BSTMP
375     pinl    PIN8
376     bcs     PIN8
377 ; past the last
378 ; segment
379 PIN4: cmp     #0
380 ; under ten: only an
381 ; exact multiple counts,
382 ; beq     PIN9
383 ; and Y < length
384 ; is already known
385 PIN5: bne     PIN8
386 PIN6: cmp     BSTMP
387     bcs     PIN8
388 PIN7: lda     #1
389 ; Z clear: covered
390 PIN8: lda     #0
391     rts
392 ; -----
393 ; PRND -- sixteen bits of Galois LFSR, poly $B400,
394 ; the Channel F port's,
395 PRND: lsr     BSRNDH
396     ror     BSRNDL
397     prni    BSRNDH
398     lda     BSRNDH
399     eor     #B4
400     sta     BSRNDH
401 PRN1: lda     BSRNDL
402     rts
403 ; PRND10 -- 0.9, by rejection on four bits: cheaper
404 ; than a divide, and the
405 ; six rejected values cost a shift each.
406 PRND10: jsr    PRND
407     and     #B0F
408     cmp     #BRDDIM
409     bcs     PRND10
410     rts
411 ; PROLL -- the cursor and the direction, rolled.
412 PROLL: jsr    PRND10
413     sta     BSCURX
414     jsr     PRND10
415     sta     BSCURY
416     jsr     PRND
417     and     #1
418     sta     BSDIR
419     rts
420 ; PROLLF -- ONE candidate for ship BSSHIP, this
421 ; frame. A miss costs nothing
422 ; but the frame; there is no bound and none is
423 ; needed, because a ten by ten
424 ; board always has room for the five and the
425 ; expected wait is two candidates
426 ; a ship. When the last lands the fleet is handed
427 ; over: the cursor onto the
428 ; first ship, the row redrawn, and the hulls painted
429 ; -- once, not five times,
430 ; because a paint and a candidate together would not
431 ; fit the same vblank.
432 PROLLF: jsr    PROLL
433     fits    FITS
434     bne     PRFX
435     jsr     PCAND
436     bne     BSCND
437     sta     POVL
438     bne     PRFX
439     lda     BSCND
440     ldx     BSSHIP
441     jsr     SHIPS,x
442     inx
443     cpx     #NSHIPS
444     bcs     PRF4
445     jsr     BSSHIP
446 PRFX: rts
447 PRF4: lda     #0
448     jsr     BSSHIP
449     jsr     PLOAD
450     jsr     PTEXT
451     jmp     PHULLS
452 ; PLOAD -- the cursor onto the seeded placement of
453 ; ship BSSHIP, so the stick
454 ; starts where the roll put it rather than at the
455 ; origin.
456 PLOAD: ldx     BSSHIP
457     lda     SHIPS,x
458     pdec
459     lda     BSPDIR
460     sta     BSDIR
461     lda     BSPOS
462     ldx     #$FF
463 PLD1: inx
464     sec
465     sbc     PLD1
466     bcs     #BRDDIM
467     adc     #BRDDIM
468 ; C is clear: this
469 ; adds ten back
470     sbcurx
471     sbcury
472     stx
473 ; PSLOT0 -- A = your slot: top-right with two
474 ; players, bottom-left with more.
475 PSLOT0: lda     #SLTR
476     ldx     BSMODE
477     beq     PSL1
478     lda     #SLBL
479     rts
480 ; PTEXT -- the status row and the name over your
481 ; board.
482 PTEXT: lda     #RSTAT
483     jsr     FNROWA
484     ldx     #(TPLACE)&$FF
485     lda     #(TPLACE)>>8
486     jsr     FNSETP
487     jsr     FNSTRA
488     jsr     BSSHIP
489     clc
490     adc     #'1'
491     jsr     FNCHR
492     jsr     FNENDW
493     lda     #RLABA
494     ldx     BSMODE
495     beq     PT1
496     lda     #RLABB
497     jsr     FNROWA
498     lda     BSMODE
499     pt2
500     bne     #7
501 ; two players: you
502 ; are on the right
503     jsr     FNNSPC
504     ldx     #(TYOU)&$FF
505     ldy     #(TYOU)>>8
506     jsr     FNSETP
507     jsr     FNSTRA
508     jsr     FNENDW
509 ; PSTAT -- the status row from the string at X/Y.
510 PSTAT: jsr     FNSETP
511     jsr     RSTAT
512     jsr     FNROWA
513     jsr     FNSTRA
514     jsr     FNENDW
515 ; PCLEAR -- the rows this screen owns blank, every
516 ; slot cleared.
517 PCLEAR: ldx     #0
518     stx     BSDIX
519     lda     PCLROWS,x
520     jsr     FNROWA
521     jsr     FNENDW
522     ldx     BSDIX
523     inx
524     cpx     #11
525     bne     PCL1
526     ldx     #PFSLOTS-1
527     stx     BSDIX
528     lda     #PFM_ALL
529     ldy     #0
530     jsr     FNBARG
531     lda     #FB_PFCCLR
532     jsr     FMBLIT
533     ldx     BSDIX
534     dex
535     bpl     PCL2
536     lda     BSMODE
537     bne     PCLX
538     lda     #RLOW0
539 ; the two-player
540 ; layout's hint rows
541     jsr     FNROWA
542     ldx     #(THINT1)&$FF

```

```

516     ldy     #(THINT1)>>8
517     jsr     FNSETP
518     jsr     FNSTRA
519     jsr     FNEWOW
520     lda     #RLOW*1
521     jsr     FNROWA
522     ldx     #(THINT2)&$$F
523     ldy     #(THINT2)>>8
524     jsr     FNSETP
525     jsr     FNSTRA
526     jsr     FNEWOW
527 PCLX:  rts
528
529 PCLROWS: DB    0, 1, 2, 3, 4, 15, 16, 17, 18, 19,
530            20
531 ; The classic five, longest first, matching the
532 ; server's myShips[] order.
533 SHPSIZ: DB    5, 4, 3, 3, 2
534
535 TPLACE: DB    "PLACE SHIP ",0
536 TYOU:   DB    "YOU",0
537 TOFF:   DB    "WON'T FIT",0
538 TLAP:   DB    "SHIPS TOUCH",0
539 TAGAIN: DB    "REFUSED-AGAIN",0
540 THINT1: DB    "SEL. TURNS",0
541 THINT2: DB    "FIRE PLACES",0
542
543 INCLUDE "bslib.inc"
544 INCLUDE "state.inc"
545 INCLUDE "sound.inc"
546 INCLUDE "dispgame.inc"
547
548 END

```

SRC/BSCOMP.ASM

```

1 ; bscomp.asm -- bank 6: compose the screen, in one
2 ; vblank, and hand it on.
3 ; This bank is here for SIZE: the game bank holds
4 ; the board kernel, the
5 ; input and the cues, and with the composer in it
6 ; too it came in 961 bytes
7 ; over. So a fresh state's recompose -- the cue
8 ; edges, four playfield boards
9 ; through the cartridge's transforms, the status
10 ; row, the name rows and the
11 ; two-player layout's lower rows -- runs here, in
12 ; two passes that each fit
13 ; the blanked time they run in (see CENTRY). Nothing
14 ; here draws, so there is
15 ; no kernel here.
16 ;
17 ; It is also the cold path's clear, and the failed
18 ; poll's status line.
19
20 CPU 6502
21 INCLUDE "vcs.inc"
22 INCLUDE "fuinet.inc"
23 INCLUDE "bsdefs.inc"
24
25 BSBANK EQU BANKCMP
26 BSHASINP EQU 0 ; it draws nothing
27 and reads no input
28 BSHASUI EQU 1
29 BSHASED EQU 0
30 BSHASNET EQU 0
31 BSHASCLS EQU 0
32 BSHASSTR EQU 1
33 BSHASRPL EQU 1
34 BSHASDEC EQU 0
35 SNDFULL EQU 0 ; it fires cues; the
36 game bank ticks them
37 SNDLAST EQU 11 ; every result cue
38
39 INCLUDE "../build/tail.inc"
40
41 ; The previous poll's playerCount, for the full
42 ; clear when a seat count
43 ; changes: the server compacts its list when someone
44 ; leaves. BSREDRW is the
45 ; flag that forces everything to be composed again.
46 BSPPCNT EQU BSBLINK
47
48 ORG $1000
49
50 ; A poll's recompose is about 3,900 cycles, and the
51 ; overscan a bank has
52 ; between two frames is about 2,200. So it is
53 ; PASSES: the cue edges and the
54 ; gamefields between frames, in the overscan the
55 ; network bank's last frame
56 ; armed; then the AUX plane -- the hulls and the
57 ; cursor -- and then the text,
58 ; a frame each, inside the vblank the game bank's
59 ; hook hands over from
60 ; (ENEXT). The status row is two frames behind the
61 ; boards, which nobody can
62 ; see.
63 ;
64 ; The AUX plane used to ride the overscan with the
65 ; gamefields, and at FOUR
66 ; seats the two did not fit: two more records to
67 ; walk and two more blits, and
68 ; the poll's frame measured 265 lines instead of
69 ; 262. It had always done
70 ; that; 'make frames' was AI1, which is two seats,
71 ; and nothing ran the
72 ; quadrant layout past a VSYNC tap until 'make
73 ; frames'
74 ;
75 ; ENGAME -- between frames, from a poll:
76 ; boards, then ENGNEXT
77 ; ENTEXT -- mid-frame, from the game's hook: the
78 ; AUX plane, then the
79 ; text, a frame each, then ENGRUN
80
81 ENRESUME mid-frame, from the menu:
82 everything's boards, ENGRUN
83 ; ENGCOLD -- between frames, a table joined:
84 clear, then ENGNEXT
85 CENTRY: lda BSNT
86 cmp #ENGCOL
87 beq CCOLD
88 cmp #ENRESUME
89 beq CRESUM
90 cmp #ENEXT
91 beq CTEXT
92 ; ENGAME: back from a poll. A failed poll leaves the
93 ; reply window alone, so
94 ; the boards are still the last good ones; only the
95 ; status row says.
96
97 BSERR2
98 CW1
99 GFAIL
100 CNEXT
101 GCOMPB
102 CNEXT: lda #ENGNEXT
103 sta BSNT
104 lda #BANKGAM
105 jmp BSGOTO
106
107 CTEXT: jmp GCOMPT
108 CRUN: lda #ENGRUN
109 sta BSNT
110 lda #BANKGAM
111 jmp BSGOTO
112
113 CRESUM: lda #1
114 sta BSREDRW ; everything: the
115 menu drew over it all
116 lda GCOMPB
117 lda #ENGRUND ; the menu's kernel
118 was a text one
119 BSNT
120 lda #BANKGAM
121 jmp BSGOTO
122
123 CCOLD: jsr GCLEAR
124 jmp CNEXT
125
126 APPVBL -- never called: this bank draws no frame.
127 APPVBL: rts
128
129 -----
130 ; GCOMPB -- the boards pass: the class, the layout,
131 ; the cue edges and the
132 ; four slots. Leaves BSREDRW saying what the text
133 ; pass owes: bit 7 always
134 ; (a pass is due), bit 1 the names, bit 0
135 ; everything
136 GCOMPB: ldx #CLLOBBY
137 lda BSSTAT
138 beq GCPI
139 lda #CLPLACE
140 cmp #STPLACE
141 beq GCPI
142 ldx #CLOVER
143 cmp #STOVER
144 beq GCPI
145 ldx #CLPLAY
146 cmp #BSTMP ; the class
147 lda #1
148 ldy BSPCNT
149 cpy #3
150 bcs GSP2
151 lda #0
152 GCP2: sta BSTMP2 ; the mode
153 ; A change of class, layout or seat count clears
154 ; everything: opaque
155 ; overdraw cannot repair content that shrank.
156 cmp BSMODE
157 bne GCPLR
158 lda BSTMP
159 cpx BSCLASS
160 bne GCPLR
161 lda BSPCNT

```

```

126      cmp      BSPPCNT
127      beq      GCP3
128      GCPCLR: lda      BSTMP2
129              sta      BSMODE
130              lda      BSTMP
131              sta      BSCCLASS
132              lda      BSPPCNT
133              sta      BSPPCNT
134              lda      #1
135              sta      BSREDRW
136              jsr      GCLRPF
137      GCP3:  lda      FNRPLY+GOMVTIM ; the clock first:
the status row shows it
138              sta      BSCLK
139              lda      #60
140              sta      BSTICK
141      ; The names are owed when something on that row can
have changed -- the
142      ; seats, the player to move, or the lobby's ready
marks. Decided here,
143      ; before GEDGES advances BSPRVAC, for the text pass.
144      lda      BSREDRW
145      and      #1
146      bne      GCP3A
147      lda      BSCCLASS
148      beq      GCP3A
149      lda      BSACT
150      cmp      BSPRVAC
151      beq      GCP3B
152      GCP3A: lda      BSREDRW
153              ora      #2
154              sta      BSREDRW
155      GCP3B: jsr      GEDGES
156              lda      GBOARD$
157              lda      BSREDRW
158              ora      #5A0 ; bit 7 a pass is
due, bit 5 the AUX plane
159              sta      BSREDRW
160              rts
161
162      ; GCOMPT -- the text pass, a frame later: the rows
the boards pass said.
163      ; "Everything" -- a screen change -- is three passes
of its own: the rows
164      ; blanked in one frame, the status and the names in
the next, the lower
165      ; rows in a third, with bits 2 and 3 carrying the
rest. Eleven blank rows
166      ; and ten composed ones in one vblank was an
eighteen-line jump once a
167      ; phase.
168      ; The AUX plane is tested BEFORE anything else, and
its bit cleared without
169      ; disturbing the others: the blank-everything stage
below rewrites BSREDRW
170      ; wholesale, so an AUX pass staged behind it would
be dropped on exactly the
171      ; screen change that needs it most.
172      GCOMPT: lda      BSREDRW
173              and      #520
174              beq      GCT0
175              jsr      GBAUX
176              lda      BSREDRW
177              and      #50F
178              and      #50F
179              sta      BSREDRW ; bit 7 still set:
come back next frame
180              rts
181      GCT0:  lda      BSREDRW
182              and      #1
183              beq      GCT1
184              jsr      GCLRTX ; everything: blank
the rows first...
185              lda      #584 ; ...and come back
for the rest
186              sta      BSREDRW
187              rts
188      GCT1:  lda      BSREDRW
189              and      #510
190              bne      GCTFLT ; the fleet strips,
a pass of their own
191              lda      BSREDRW
192              and      #8
193              bne      GCTL0W ; the third pass:
the lower rows
194              jsr      GSTAT
195              lda      BSREDRW
196              and      #4
197              bne      GCT1A ; everything: the
whole name rows...
198              lda      BSREDRW
199              and      #2
200              beq      GCT2
201              jsr      GLMARKS ; only the markers
moved: two cells a row
202              jmp      GCT2
203      GCT1A: jsr      GLABELS
204              lda      #58C ; ...and, next
frame, whichever of the two
205              ldx      BSMODE ; lower halves
this layout has
206              beq      GCT1B
207              lda      #590
208      GCT1B: sta      BSREDRW
209              rts
210      ; The last pass of each layout. Two seats compose
the lower rows every poll,
211      ; because the fleets live in them; the quadrant
layout stages the strips
212      ; instead, a frame later again -- two twelve-
character rows on the back of
213      ; the status row and the markers is the eighteen-
line jump this file already
214      ; paid for once.
215      GCT2:  lda      BSMODE
216              beq      GCTL0W
217              lda      #590
218              sta      BSREDRW
219              rts
220      GCTL0W: jsr      GLOWER
221
222      jmp      GCT3
223      GCT3:  lda      #0
224              sta      BSREDRW
225              rts
226      ;
227      ;
-----
228      ; GEDGES -- the cues, on edges only, decided BEFORE
the draw.
229      GEDGES: lda      BSCCLASS
230              cmp      #CLPLAY
231              bne      GE5
232      ; the result: (status, lastAttackPos) as a pair,
because the status repeats
233      ; across polls when nobody has fired since
234      lda      BSSTAT
235      cmp      #STMISS
236      bcc      GE4 ; GAMESTART: no
result yet
237      cmp      BSPRVST
238      bne      GE2
239      lda      FNRPLY+GLASTP
240      cmp      BSPRVLP
241      beq      GE4 ; the same event as
last poll
242      GE2:  lda      BSSTAT
243              cmp      #STMISS
244              bne      GE3
245              lda      #SNDMISS
246              jmp      GE3A
247      GE3:  cmp      #STHIT
248              bne      GE3B
249              lda      #SNDHIT
250              jmp      GE3A
251      GE3B: lda      #SND$UNK
252      GE3A: jsr      #SND$FIRE
253      GE4:  lda      FNRPLY+GLASTP
254              sta      BSPRVLP
255      ; the turn, last, so it is what is heard when both
happen at once
256              lda      BSACT
257              cmp      BSPRVAC
258              beq      GE9
259              lda      BSACT
260              bne      GE9
261              lda      #SNDTURN
262              jsr      #SND$FIRE
263              jmp      GE9
264      GE5:  cmp      #CLOVER
265              bne      GE6
266              lda      BSPRVST
267              cmp      #STOVER
268              beq      GE9
269              lda      #SND$OVER
270              jsr      #SND$FIRE
271              jmp      GE9
272      GE6:  cmp      #CLLOBBY
273              bne      GE9
274      ; the countdown: the server's prompt starts with
"starting in"
275              lda      FNRPLY+GOPRMP
276              cmp      #1's
277              bne      GE7
278              cmp      BSPRVLP
279              beq      GE8 ; already counting:
a tick a poll
280              #SNDJOIN
281              jsr      #SND$FIRE
282              jmp      GE7
283      GE8:  lda      #SNDCLK
284              jsr      #SND$FIRE
285      GE7:  lda      FNRPLY+GOPRMP
286              sta      BSPRVLP
287      GE9:  lda      BSACT
288              sta      BSPRVAC
289              lda      BSSTAT
290              sta      BSPRVST
291              rts
292      ;
-----
293      ; GBOARDS -- the HIT and MID planes of the four
slots, from the records.
294      ; The AUX plane is GBAUX and a pass of its own. The
two together are about
295      ; 2,100 cycles at four seats and the overscan a bank
has between frames is
296      ; about 2,240 -- of which the network bank has
already spent four lines by
297      ; the time it hands over, so the poll's frame came
out three lines long, once
298      ; a poll, for as long as the quadrant layout has
existed. Two seats fitted
299      ; and "make frames" was only ever run at two seats.
300      GBOARDS: lda      BSCCLASS
301              cmp      #CLPLAY
302              bcs      GB0 ; lobby and
placement: no gamefields exist
303      GB0:  ldx      BPCNT
304              cpx      BPCNT
305              bcs      GB1X
306              stx      BSIOX
307              jsr      PLRECP
308              in the window
309              lda      FNPTRL ; FNPTRL/H = the
record, sixteen bits
310              and      #15
311              adc      #PLFIELD
312              sta      FNRSEL+FH_BSL
313              lda      FNRTRH
314              and      #0
315              sec
316              sbc      #FNRPLY>>8
317              sta      FNRSEL+FH_BSH
318              ldx      BSIOX
319              jsr      PSLOT
320              sta      FNRSEL+FH_BDL

```

```

323      lda      #0
324      sta      FNRSEL+FH_BDH
325      lda      #FB_PFLD
326      jsr      FNBLLIT
327      ldx      BSIDX
328      tnx
329      jmp      GB1
330 GB1X: rts
331
332 ; GBAUX -- the AUX plane, in the vblank of the frame
333 ; after: cleared
334 ; everywhere, then your hulls, the winner's at game
335 ; over, and the cursor on
336 ; every live enemy. One frame behind the boards,
337 ; which nobody can see, in
338 ; exchange for a frame that is 262 lines.
339 GBAUX: lda      BSCLASS
340      cmp      #CLPLAY
341      bcs      GB2
342      rts
343 GB2: ldx      #PFSLOTS-1
344 GB3: stx      BSIDX
345      lda      #PFM_AUX
346      ldy      #0
347      jsr      FNBARG
348      lda      #FB_PFLCLR
349      jsr      FNBLLIT
350      ldx      BSIDX
351      dex
352      bpl      GB3
353      ldx      #0
354      jsr      PSLOT
355      tax
356      ldy      #GIMYSHP
357      ldy      #NSHIPS
358      jsr      FNBARG
359      lda      #FB_PFHUL
360      jsr      FNBLLIT
361      ldx      BSCLASS
362      cmp      #CLOVER
363      bne      GB4
364      lda      BSACT
365      beq      GB4
366      ; the winner, whose
367      ; layout came with v=2
368      ; you: yours are up
369      cpx      BSPCNT
370      bcs      GB4
371      jsr      PSLOT
372      lda      #GIMYSHP+NSHIPS
373      ldy      #NSHIPS
374      jsr      FNBARG
375      lda      #FB_PFHUL
376      jsr      FNBLLIT
377      ldx      #0
378      sta      BSLIVE
379      lda      BSCLASS
380      cmp      #CLPLAY
381      bne      GB9
382      lda      BSMYST
383      bne      GB9
384      ; defeated or
385      watching: no cursor
386      ldx      #1
387      cpx      BSPCNT
388      bcs      GB6
389      stx      BSIDX
390      jsr      PLRECP
391      ldy      #PLSTAT
392      lda      (FNPTL),y
393      bne      GB5A
394      ; not playing: a
395      shot does not reach them
396      ldx      BSIDX
397      jsr      PSLOT
398      tax
399      lda      SLOTBIT,x
400      bslvive      BSLIVE
401      sta      BSIDX
402      tnx
403      jmp      GB5
404      lda      #PFM_AUX
405      jmp      GCURS
406      rts
407
408 ; PSLOT -- A = the slot of player X, in the current
409 ; layout. X survives.
410 ; Two players: you top-right, the enemy top-left.
411 ; More: the family's
412 ; quadrants, you bottom-left and the others
413 ; clockwise from top-left.
414 PSLOT: tax
415      ldy      BSMODE
416      beq      PSL1
417      clc
418      adc      #4
419      tax
420      lda      SLOTMAP,x
421      tay
422      pla
423      tax
424      tya
425      rts
426
427 ;
428
429      cmp      #CLOVER
430      beq      GSOVR
431 ; play: the last result, whose turn, and your clock
432      ldx      # (TONE)&FFF
433      ldy      # (TONE)>>8
434      lda      BSSTAT
435      cmp      #STMISS
436      bne      GS1
437      ldx      # (TMISS)&FFF
438      ldy      # (TMISS)>>8
439      cmp      #STHIT
440      bne      GS2
441      ldx      # (THIT)&FFF
442      ldy      # (THIT)>>8
443      cmp      #STSUNK
444      bne      GS3
445      ldx      # (TSUNK)&FFF
446      ldy      # (TSUNK)>>8
447      jsr      FNSETP
448      jsr      FNSTR
449      lda      BSACT
450      bne      GSFOE
451      ldy      # (TYOU)&FFF
452      ldy      # (TYOU)>>8
453      jsr      FNSETP
454      jsr      FNSTR
455      jsr      DEC3
456      jmp      FNENDW
457      ldx      # (TFOE)&FFF
458      ldy      # (TFOE)>>8
459      jsr      FNSETP
460      jsr      FNSTR
461      jmp      FNENDW
462      ldx      # (TWAITS)&FFF
463      ldy      # (TWAITS)>>8
464      jsr      GSSTR
465      lda      FNRPLY+GOPRMT
466      beq      GSLOB1
467      ldx      #GOPRMT
468      lda      #FNTCOL
469      jsr      FNPLA
470      jmp      FNENDW
471      ldx      # (TREADY)&FFF
472      ldy      # (TREADY)>>8
473      lda      BSMYST
474      cmp      #PSREADY
475      bne      GSSTR
476      ldx      # (TWAITP)&FFF
477      ldy      # (TWAITP)>>8
478      jsr      GSSTR
479      jsr      BSACT
480      bne      GSOVR1
481      ldx      # (TWIN)&FFF
482      ldy      # (TWIN)>>8
483      jmp      GSSTR
484      cpx      BSPCNT
485      bcs      GSOVR2
486      jsr      PLRECP
487      #5
488      jsr      FNRPLP
489      ldx      # (TWINS)&FFF
490      ldy      # (TWINS)>>8
491      jmp      GSSTR
492      ldx      # (TOVER)&FFF
493      ldy      # (TOVER)>>8
494      jmp      GSSTR
495      ;
496
497 ; LABELS -- the name row over each pair: the left
498 ; board's owner in the
499 ; first five columns, a marker each, the right
500 ; board's owner in the last
501 ; five. The 48-pixel text block sits at clocks
502 ; 52-99, so column 7 starts
503 ; exactly at the seam.
504 GLABELS: lda      #RLAB
505      lda      #RLAB
506      ldx      #SLTL
507      jsr      GLNAME
508      ldx      #SLTL
509      jsr      GLMARK
510      ldx      #SLTR
511      jsr      GLNAME
512      ldx      #SLTR
513      jsr      GLMARK
514      lda      BSMODE
515      beq      GLX
516      lda      #RLABB
517      ldx      #SLRB
518      jsr      GLNAME
519      ldx      #SLRB
520      jsr      GLMARK
521      ldx      #SLBR
522      jsr      GLNAME
523      jmp      FNENDW
524      GLX: rts
525
526 ; GLNAME -- five columns: the name of the player in
527 ; slot X- you for you,
528 ; blank for an empty slot.
529 GLNAME: jsr      SLPLYR
530      beq      GLN1
531      tax
532      jsr      PLRECP
533      jsr      FNRPLP
534      jmp      GLN1
535      ldx      # (TYOVS)&FFF
536      ldy      # (TYOVS)>>8
537      jsr      FNSETP
538      jsr      FNSTR
539      lda      #5
540      jmp      FNSPC

```

```

542 ; GLMARK -- one column, appended: '+' ready in the
543 lobby, '>' the player
544 ; to move, 'X' sunk, else a space. GLMARKC is the
545 choice alone, in A.
546 GLMARK: jsr GLMARKC
547 jsr FNCHR
548 bmi GLMSP
549 sta BSTMP
550 tax
551 jsr PLRECP
552 ldy #PLSTAT ; LBSTAT and PLSTAT
553 are both offset 9
554 ldx (FNPTL),y
555 bne GLM1
556 cmp #PSREADY ; the lobby: ready
557 or not bne GLMSP
558 ldx #'*'
559 GLM1: rts
560 bne GLM2
561 ldx #*'
562 GLM2: rts
563 ldx BSTMP
564 cmp BSACT
565 bne GLMSP
566 ldx #'>'
567 GLMSP: rts
568 ldx #' '
569 rts
570 ; GLMARKS -- the four marker cells, in place through
571 FB_TCELL: what a poll
572 ; that only moved the turn changes on the name rows,
573 at a tenth of the cost
574 ; of composing them again.
575 GLMARKS: ldx #SLTL
576 jsr GLMARKC
577 ldx #RLAB*FNTCOL+5
578 jsr GLMCEL
579 ldx #SLTR
580 jsr GLMARKC
581 ldx #RLAB*FNTCOL+6
582 jsr GLMCEL
583 ldx BSMODE
584 beq GLMX
585 ldx #SLBL
586 jsr GLMARKC
587 ldx #RLAB*FNTCOL+5
588 jsr GLMCEL
589 ldx #SLBR
590 jsr GLMARKC
591 ldx #RLAB*FNTCOL+6
592 jsr GLMCEL
593 GLMX: rts
594 GLMCEL: ldy #0
595 jsr FNBARG
596 ldx #FB_TCELL
597 jmp FNBLLT
598 ;
599 -----
600 ; GLFLEET -- text rows 3 and 4: the fleet of each of
601 the four seats as five
602 ; pips, under the name row of its pair. The kernel
603 draws these three ink
604 ; lines high (disgame.inc, PIPROW), which is why
605 the sunk mark is '!' and
606 ; not 'X'.
607 ; GPIPS already emits a leading space and five pips
608 -- six columns -- so two
609 ; of them fill the row exactly and the right-hand
610 seat's pips land in columns
611 ; 7-11. Column 7 starts at clock 80, the board seam,
612 so each seat's pips sit
613 ; over its own half.
614 GLFLEET: ldx #RFLTA
615 jsr FNROWA
616 ldx #SLTL
617 jsr GFSLT
618 ldx #SLTR
619 jsr GFSLT
620 ldx #SLBR
621 jsr GFSLT
622 jmp FNENDW
623 ; GFSLT -- six columns for the seat in slot X: a
624 space and five pips, or six
625 ; spaces for an empty seat. GPIPS emits nothing at
626 all before the gamefields
627 ; exist, which is what the lobby and the placement
628 phase want.
629 GFSLT: jsr SPLYR
630 bmi GFSP
631 tax
632 jmp GPIPS
633 GFSP: ldx #NSHIPS+1
634 jmp FNSPC
635 ; SPLYR -- A = the player in slot X, or $FF (N set)
636 for nobody.
637 SPLYR: txa
638 ldy BSMODE
639 beq SLP1
640 clc
641 adc #4
642 SLP1: tax
643 ldx PLOFSL,x
644 bmi SLP2
645 bsrpcnt
646 bcc SLP2
647 ldx #SFF

```

```

646 SLP2: cmp #0 ; N and Z from the
647 VALUE, not from the ; bounds compare
648 -- 1 - 2 is negative too
649 ;
650 ; GLOWER -- the two-player layout's lower rows: the
651 server's prompt in full,
652 ; each player's ships as pips, and a hint.
653 GLOWER: ldx BSREDRW
654 #4
655 and bne GLW0
656 ldx BSCLASS ; in play the prompt
657 cmp #CLPLAY
658 is empty and the hint ; is fixed: only
659 beq GLPIPS
660 the fleets can change
661 GLW0: ldx #0
662 GLW1: stx BSIDX
663 txa
664 clc
665 adc #RLOW0
666 jsr FNROWA
667 ldx BSIDX
668 a sl a
669 a sl a
670 adc BSIDX ; twelve bytes a row
671 adc BSIDX
672 clc
673 adc #GOPRMT
674 tax
675 ldy #FNTCOL
676 cpx #GOPRMT+2*FNTCOL
677 GLW2: bne GLW2
678 ldy #3-2*FNTCOL ; the third row
679 holds the last nine
680 GLW2: jsr FNRLPA
681 cpy #0
682 beq GLW3
683 tya
684 jsr FNMSC
685 ldx FNENDW
686 inx
687 cpx #3
688 bne GLW1
689 jsr GLPIPS
690 ; the hint
691 ldx #RLOW+5
692 jsr FNROWA
693 ldx # (THINTP)&FF
694 ldy # (THINTP)>>8
695 ldx BSCLASS
696 cmp #CLPLAY
697 beq GLW5
698 ldx # (THINTL)&FF
699 ldy # (THINTL)>>8
700 cmp #CLLOBBY
701 beq GLW5
702 ldx # (THINTO)&FF
703 ldy # (THINTO)>>8
704 GLW5: ldx FNSETP
705 jsr FNSTRA
706 jmp FNENDW
707 ; GLPIPS -- the two fleet rows: YOU and the enemy,
708 each with five pips.
709 GLPIPS: ldx #RLOW+3
710 jsr FNROWA
711 ldx # (TYOUS)&FF
712 ldy # (TYOUS)>>8
713 jsr FNSETP
714 ldx #0
715 jsr GPIPS
716 FNENDW
717 ldx #RLOW+4
718 jsr FNROWA
719 ldx #1
720 cpx BSRPCNT
721 bcs GLW4
722 jsr PLRECP
723 ldx #5
724 jsr FNRLPL
725 ldx #1
726 jsr GPIPS
727 GLW4: jmp FNENDW
728 ; GPIPS -- "#####" for player X's five ships, '.'
729 where one is sunk. Only
730 ; once the records carry a fleet.
731 GPIPS: ldx BSCLASS
732 cmp #CLPLAY
733 bcc GPX
734 jsr PLRECP
735 ldx #' '
736 jsr FNCHR
737 ldy (FNPTL),y
738 GP1: ldx
739 beq GP2
740 ldx #' '
741 GP3: ldx
742 GP2: ldx #'=' ; a bar; '.' is the
743 bottom font row alone ; and the strips
744 only render the top three
745 GP3: jsr FNCHR
746 iny
747 cpy #PLSHIPS+NSHIPS
748 bne GP1
749 GPX: rts
750 ;
751 ; GCLEAR -- every row this bank owns blank, every
752 slot cleared. Rows 5-14

```

```

752 ; are the playfield tables and are never touched.
753 GCLRTX is the rows,
754 GCLRPF the slots: the two passes split them.
755 GCLRPF: ldx GCLRTX
756 GCL2: stx BSIDX
757 ldx #PFM_ALL
758 ldy #0
759 jsr FNBARG
760 ldx #FB_PFCCLR
761 jsr FNBLIT
762 ldx BSIDX
763 dex
764 bpl GCL2
765 rts
766 GCLRTX: ldx #0
767 GCL1: stx BSIDX
768 ldx GCLROWS,x
769 jsr FNROWA
770 jsr FNEWOW
771 ldx BSIDX
772 inx
773 cpx #11
774 bne GCL1
775 rts
776 ; GFAIL -- the status row says the poll failed and
777 where.
778 GFAIL: ldx #RSTAT
779 jsr FNROWA
780 ldx # (TNETER)&$FF
781 ldy # (TNETER)>>8
782 jsr FNSETP
783 jsr FNSTRA
784 ldx BSSTEP
785 clc
786 adc #'0'
787 jsr FNCHR
788 ldx #' '
789 jsr FNCHR
790 ldx BSERR
791 jsr DEC3
792 jmp FNEWOW
793
794 ; DEC3 -- A as three characters, leading zeros as
795 spaces, the units digit
796 ; always. BSTMP2 is the "something has printed"
797 latch.
798 DEC3: ldy #0
799 sty BSTMP2
800 ldx #0
801 DC1: cmp #100
802 bcc DC2
803 sbc #100
804 inx
805 bne DC1
806 DC2: jsr DCDIG
807 ldx #0
808 DC3: cmp #10
809 bcc DC4
810 sbc #10
811 inx
812 bne DC3
813 DC4: jsr DCDIG
814 clc
815 adc #'0'
816 jmp FNCHR
817 DCDIG: phb
818 txa
819 bne DCD1
820 ldx BSTMP2
821 beq DCD2 ; a leading zero: a
822 space
823 ldx #'0'
824 bne DCD3
825 DCD1: clc
826 DCD2: adc #'0'
827 DCD3: inc BSTMP2
828 bne DCD3
829 DCD2: ldx #?
830 DCD3: jsr FNCHR
831 pla
832 rts
833 ;
834 -----
835 GCLROWS: DB 0, 1, 2, 3, 4, 15, 16, 17, 18, 19,
836 20
837 ; player -> slot, by layout: two players, then
838 quadrants
839 SLOTPF: DB SLTR, SLTL, SLBL, SLBR
840 DB SLBL, SLTL, SLTR, SLBR
841 ; slot -> player, by layout
842 PLOFSL: DB 1, 0, $FF, $FF
843 DB 1, 2, 0, 3
844
845 TNONE: DB " ", 0
846 TMISS: DB "MISS ", 0
847 THIT: DB "HIT ", 0
848 TSUNK: DB "SUNK ", 0
849 TYOU: DB "YOU ", 0
850 TFOE: DB "ENEMY ", 0
851 TWAITS: DB "WAIT: SHIPS", 0
852 TREADY: DB "FIRE-READY", 0
853 TWAITP: DB "READY. WAIT", 0
854 TWIN: DB "YOU WIN!", 0
855 TWINS: DB " WINS", 0
856 TOVER: DB "GAME OVER", 0
857 TYOUS: DB "YOU ", 0
858 TNETER: DB "NET ERR ", 0
859 THINTP: DB "FIRE-ATTACK", 0
860 THINTL: DB "FIRE-READY", 0
861 THINTO: DB "RESET=MENU", 0
862
863 INCLUDE "bslib.inc"
864 INCLUDE "state.inc"
865 INCLUDE "sound.inc"
866
867 END

```

SRC/LAYOUT.ASM

```

1 ; layout.asm -- the board screen, with no network at
  all.
2 ; A Flat 4K image whose playfield tables are BAKED
  INTO THE ROM at $1800+:
3 ; the cartridge serves the image's own bytes there
  at power-on, so four
4 ; composed boards are on screen before a single blit
  has run. It exists
5 ; because the board kernel is the one component of
  this port with no template
6 ; anywhere: the line-multiplexed colours, the
  abutting halves, the divider
7 ; and the 262-line frame all have to be right before
  any of them is worth
8 ; debugging over a socket.
9 ;
10 ; tools/mklayout.py writes the tables (build/
  layoutpf.inc) from a picture;
11 ; tools/pfcheck.py reads a snapshot back against the
  same picture. SELECT
12 ; flips between the four-quadrant layout and the
  two-player one; the stick
13 ; walks a cursor with FB_PFCCL, which exercises the
  cartridge's transform and
14 ; the FNBLIT wait with nothing else in the way.
15 ;
16 ; ./build.sh layout && SLOT=fujinet ./run.sh
  layout shot
17
18 CPU 6502
19 INCLUDE "vcs.inc"
20 INCLUDE "fujinet.inc"
21 INCLUDE "bsdefs.inc"
22 BSBANK EQU BANKGAM
23 BSHASINP EQU 1 ; the switches pick
  the layout
24 BSHASUJ EQU 0 ; the tables are
  baked in: no reply window
25 BSHASED EQU 0
26 BSHASNET EQU 0
27 BSHASCLS EQU 0
28 BSHASSTR EQU 1
29 BSHASRPL EQU 0
30 BSHASDEC EQU 0
31
32 ; No build/tail.inc here: this ROM assembles the
  tail body itself at the
33 ; bottom of the file, so the labels are the real
  thing.
34
35 ORG $1000
36
37 START: sei
38 cld
39 ldx #$FF
40 txx
41 lda #0
42 LCLR: sta $00,x ; $00-$7F is the
  TIA, $80-$FF is RAM
43 dex
44 bne LCLR
45 sta $00
46
47 lda #2
48 sta VBLANK
49 jsr FNARM
50 jsr FNCHR
51 beq LGOT
52 lda #CRED ; no cartridge: a
  red screen is the message
53 sta COLUBK
54 LHALT: jmp LHALT
55
56 LGOT: lda #1
57 sta BSMODE
58 lda #4
59 sta BSCURY
60 lda #3
61 sta BSCURY
62 jsr DINIT
63 jsr LTEXT
64 lda #0
65 sta VBLANK
66 jmp DFRESH
67
68 ;
69 ; The text rows: a status line, two name rows, and
  the two-player lower rows.
70 LTEXT: ldx #0
71 LT1: stx BSIDX
72 lda LROWS,x
73 jsr FNROWA
74 lda BSIDX
75 asl a
76 asl a
77 asl a
78 row asl a ; sixteen bytes a
79
80 tax
81 ldy #FNTCOL
82 LT2: lda LSTR,x
83 jsr FNCHR
84 inx
85 dey
86 bne LT2
87 jsr FNENDW
88 ldx BSIDX
89 inx
90 cpx #NLROWS
91 bne LT1
92 rts
93
94 NLROWS EQU 9
95 LROWS: DB RSTAT, RLABA, RLABB, 15, 16, 17, 18,
  19, 20
96 LSTR: DB "YOUR TURN 45 "

```

BUILD.SH

```
1 #!/usr/bin/env bash
2 # build.sh -- assemble the Atari 2600 Battleship
  client.
3 #
4 # ./build.sh           the client:    build/
  battleship.bin, 16384 bytes
5 # ./build.sh layout    the screen with no
  network: build/layout.bin
6 #
7 # The client is (N+1) x 2048 bytes: N 2K banks
  then the 2K fixed half, which
8 # is the layout vcs_set_image expects. MAME's
  vcs_cart_slot_device::call_load()
9 # accepts only 4096/8192/16384/32768, so N is 1,
  3, 7 or 15 and nothing
10 # between. Six banks are needed; the seventh is
  a stub back to bank 0.
11 #
12 # The "FUJI" claim is stamped into the fixed
  half. Without it the cartridge
13 # treats the image as a game and the mailbox
  goes dead the moment it boots.
14 #
15 # Env:
16 #   ENDPOINT=...       the server (default
  https://battleship.carr-designs.com/)
17 #   FUJI_FIRMWARE=...  firmware tree (default ~/
  Workspace/fujinet-firmware)
18 #
19 set -euo pipefail
20 cd "$(dirname "$0")"
21 #
22 FUJI_FIRMWARE="${FUJI_FIRMWARE:-$HOME/Workspace/
  fujinet-firmware}
23 VCS="$FUJI_FIRMWARE/pico/atari-2600"
24 #
25 if command -v asl >/dev/null 2>&1; then
26   AS=asl P2BIN=p2bin
27 elif [ -x "$HOME/asl/asl" ]; then
28   AS="$HOME/asl/asl" P2BIN="$HOME/asl/p2bin"
29 else
30   echo "build.sh: no Macroassembler AS found
  (tried PATH and ~/asl)" >&2
31   exit 1
32 fi
33 #
34 mkdir -p build
35 #
36 # The equates are hand-mirrored from the
  firmware header, so check them BEFORE
37 # assembling anything: a drifted address
  assembles cleanly, runs, draws a
38 # screen, and silently talks to a page that
  decodes nothing.
39 #
40 # The blit, playfield and path equates are
  passed with --extra rather than
41 # living in that tool's shared table: a name
  listed there and missing from a
42 # client's .inc is a failure, so the table can
  only hold what EVERY client
43 # has. The appkey command bytes are FUJI device
  ids from fujicommandID.h, not
44 # the mailbox header, so they are not listed.
45 python3 "$SVCS/tools/checkdefs.py" \
46   "$SVCS/firmware/include/fuji_mailbox.h" src/
  fujinet.inc \
47   --extra 'FB_TCELL=FN_BLIT_TCELL,FB_PATH=
  FN_BLIT_PATH,FB_TEXT=FN_BLIT_TEXT,FB_PFCCLR=
  FN_BLIT_PFCCLR,FB_PFLD=FN_BLIT_PFIELD,FB_PFHUL=
  FN_BLIT_PFHULL,FB_PFCCL=FN_BLIT_PFCCELL,PFROW0=
  FN_PF_ROW0,PFKLEN=FN_PF_KIND_LEN,PFM_HIT=
  FN_PFM_HIT,PFM_MID=FN_PFM_MID,PFM_AUX=
  FN_PFM_AUX,PFM_ALL=FN_PFM_ALL,PFM_CLR=
  FN_PFM_CLEAR,PFSLTS=FN_PF_SLOTS,FNBGEN=
  FN_B_BLTGEN,FH_BSL=FN_HOT_BLIT_SL,FH_BSH=
  FN_HOT_BLIT_SL,FH_BOL=FN_HOT_BLIT_DL,FH_BOH=
  FN_HOT_BLIT_DL,FH_BCNT=FN_HOT_BLIT_CNT,FH_BGO=
  FN_HOT_BLIT_GO,FP_POPCH=FN_PATH_POPCH,FP_SELO=
  FN_PATH_SELO,FP_SEL1=FN_PATH_SEL1,FP_SEL3=
  FN_PATH_SEL3,FP_SEED=FN_PATH_SEED,FP_COMMIT=
  FN_PATH_COMMIT,FNTROW=FN_T_ROWS,FNTCLH=
  FN_T_CELL_H"
48 #
49 # Regenerated every run, so a stale value cannot
  survive an environment
50 # change.
51 {
52   echo "; generated by build.sh -- do not
  edit"
53   printf 'UENDPT: DB           "%s"\n' \
54     "${ENDPOINT:-https://battleship.carr-
  designs.com/}" DB           0"
55 } > build/endpoint.inc
56 #
57 # p2bin ALWAYS emits its full -r range, so an
  over-long bank is truncated in
58 # SILENCE: the image builds, the size is right,
  checkrom passes, and the
59 # instructions past $17FF are simply gone. The
  assembler's own listing is the
60 # only thing that knows, so the build asks it.
61 bankfits() {
62   python3 tools/checkbanks.py "build/$1.lst"
63   "s2" "$1"
64 }
65 assemble() {
66   local b=$1
67   ( cd src && "$AS" -g -L -i . "$b.asm" )
68   mv "src/$b.p" "build/$b.p"
69   mv -f "src/$b.lst" "build/$b.lst" 2>/dev/
  null || true
70 }
71 #
72 #
73 stampclaim() {
74   # The fixed half is the LAST 2K, so the
  claim's offset is
75   # (size - 2048) + (FN_R_CLAIM - $1800).
76   local f=$1 size
77   size=$((stat -c%s "$f")
78   printf 'FUJI' | dd of="$f" bs=1 \
79     seek=$((size - 0x800 + 0x0710))
  conv=notrunc status=none
  echo "$f: $size bytes"
80 }
81 #
82 #
83 # ----- the layout ROM: one flat 4K
  image
84 if [ "${1:-}" = "layout" ]; then
85   # The baked tables and the baked fleet
  strips, from the picture in
86   # tools/mklayout.py. Generated here and not
  by hand: build/ is ignored, so
87   # a stale copy is the normal state of a
  fresh tree, and a stale one fails
88   # tools/pfcheck.py somewhere that looks like
  the kernel's fault.
89   python3 tools/mklayout.py > build/
  layoutpf.inc
90   assemble layout
91   bankfits layout $(($x1800))
92   "SP2BIN" build/layout.p build/layout.bin -r
  '$1800-$1FFF' -l 0
93   rm -f build/layout.p
94   stampclaim build/layout.bin
95   python3 "$SVCS/tools/checkrom.py" build/
  layout.bin
96   exit 0
97 fi
98 #
99 # ----- the client: seven banks and
  the fixed half -----
100 BANKS="bslobby bsgame bsnet bsmenu bsname
  bsplace bscomp"
101 #
102 # THE TAIL IS ASSEMBLED FIRST: it holds the
  shared transport, and the banks
103 # reach it through build/tail.inc, generated
  from the addresses the tail
104 # actually assembled to. Nothing keeps a list of
  those by hand.
105 assemble bstail
106 python3 tools/mktail.py build/bstail.lst \
107   FNRWF,FNARM,FNCHK,FNBEG,FNPB,FNPWF,FNGO,FNACK,
  \
108   FNROWA,FNCHR,FNENDR,FNSPC > build/tail.inc
109 cat build/tail.inc | tail -1 | sed 's/^; */
  tail: /'
110 #
111 parts=()
112 for b in $BANKS; do
113   assemble "$b"
114   bankfits "$b" $(($x1800))
115   "SP2BIN" "build/$b.p" "build/$b.bin" -r
  '$1800-$17FF' -l 0
116   rm -f "build/$b.p"
117   parts+=("build/$b.bin")
118 done
119 #
120 "SP2BIN" build/bstail.p build/bstail.bin -r
  '$1800-$1FFF' -l 0
121 rm -f build/bstail.p
122 #
123 cat "${parts[@]}" build/bstail.bin > build/
  battleship.bin
124 rm -f "${parts[@]}" build/bstail.bin
125 #
126 stampclaim build/battleship.bin
127 python3 "$SVCS/tools/checkrom.py" build/
  battleship.bin
128 }
```